



GeoLibre Reference

Architecture, renderers, deployment, APIs, and project documentation

Contents

Architecture	10
Overview	10
Packages	10
State flow	11
Rendering-engine model	12
3D globe view (CesiumJS)	12
DuckDB-WASM	16
Advanced Add Data workflows	18
Python sidecar	18
Offline support (PWA)	19
Container image	21
Security	22
Performance: map rendering on Linux (WebKitGTK)	24
MapLibre Renderer	26
Supported paths	26
Map, globe, and terrain	26
Basemaps and plugins	26
Architecture and license	27
Mapbox Renderer	28
Supported paths	28
Plugins	29
License and terms	33
Loading and size	33
Add Data compatibility	33
Cesium Renderer	37
Credentials	37
Supported paths	37
Scene and camera	38
Loading and architecture	38
ArcGIS Renderer	39
Nothing is bundled	39
API key	39
Supported paths	40
2D and 3D	43
Adding data	44
deck.gl layers	44
Tile archives	44
Zarr and NetCDF grids	45
Plugin controls	45

Adapted plugin panels	46
Not supported yet	46
Testing	46
License and terms	47
Self-Hosting	48
Why same-origin matters	48
Architecture	49
1. Host the data with GeoLens	49
2. Host the GeoLibre web build next to it.	50
3. Connect GeoLibre to GeoLens	56
The two deployment patterns, compared	57
Private data without GeoLens	58
Reducing outbound requests	59
Deployment checklist	60
Related pages	61
Server API	62
Conventions	62
What the reference server leaves to the operator	62
Limits	63
Visibility	64
Identity	64
Organizations	67
Groups	69
Enterprise sign-in	70
Projects	78
Personal token scopes	85
OAuth 2.0 sign-in (Authorization Code + S256 PKCE)	86
Compatibility	88
Android	89
Install	89
What works on Android vs desktop	89
Toolchain setup (one time)	90
Build	91
Signing	91
Continuous integration	92
Install / test	92
Publishing to Google Play	93
Known limitations / follow-ups	94
Open a location from another app	94
iOS	96
Install	96
What works on iOS vs desktop	96
Location permission (required)	97

Minimum iOS version	97
Toolchain setup (one time)	98
Build	99
Signing	100
Continuous integration	101
Run and test a local build	102
Publishing to the App Store	103
Known limitations / follow-ups	104
Mac App Store	105
What the MAS build removes	105
Building locally	107
CI: the mas-store.yml workflow	108
Submitting	109
Project Format	110
Desktop file opening	110
Schema	110
Plugin state	112
Legend	112
Print layout	113
Story map	113
Dashboard widgets	114
Bookmarks	114
Interaction	115
Layer object	115
Layer types	121
API	122
Credential redaction	122
Plugin API	124
Interface	124
Register a plugin	130
Add a built-in plugin (in this repository)	130
Built-in plugins	132
Example plugin	133
Read-only layer and feature queries	135
Active map click tool	136
Layer groups	136
Layer styles	137
Raster and tile layers	138
Zarr layers	141
Driving a layer's own time dimension from the Time Slider	143
Activating and deactivating other plugins	145
Custom (WebGL) layers and paint ownership	145
Right sidebar panels	146

Toolbar menus	148
Adding items to built-in menus	149
Following the app language	150
Saving credentials	151
Signing in with a session cookie	152
Floating panels	153
External plugins	153
Assistant tools	158
Map renderers	160
Collaboration	165
What syncs	165
Architecture	165
Sync protocol	166
Durable Object (workers/collab)	168
Frontend	169
Identity & permissions (MVP)	169
Chat (#754)	171
Feature flag	172
Deploying the relay (collab.geolibre.app)	172
Limitations / v2	173
Testing	173
UI Profiles	175
For users	175
For administrators	176
Deployment Capabilities	178
Not the same as UI Profiles	178
The capabilities	179
Configuring it	179
Embed API behavior	180
Related pages	181
Deployment Policy	182
Loading	182
Example	183
Field reference	184
Omitted versus empty	186
Plugin precedence	186
Precedence between sources	186
Versioning	187
Validation	187
Desktop	187
Docker	188
Internationalization	190

How it works	190
Using translations in components	191
Adding a new language	192
Keeping catalogs complete	192
Right-to-left languages	193
Keeping formatting locale-aware	193
Processing tool metadata	194
Plugin-facing surfaces	195
Known limitations	196
Python Package	197
Install	197
Quickstart	197
Two-way sync	199
Map options	200
Interactive scripting	200
API reference	201
Use in marimo	211
How it works	212
MCP server	213
Building from source	213
Rendering engines and mixed pane layouts	213
R Package	214
Installation	214
Quick start	214
Spatial data with sf	215
Remote rasters	215
Project files	215
Shiny	215
Self-hosting	216
MCP Server	217
Install	217
Run it	217
The workspace	218
Tools	218
Notes and limits	222
Driving the server from an agent	222
Under the hood	223
Renderer and pane authoring	223
Agent Skill	224
What it is for	224
What it teaches	224
Install	225
Other agent runtimes	225

Notebook Panel	226
Scripting the map from a cell	226
Driving the map from an external client (VS Code, ...)	227
Theme	228
Building the web JupyterLite site	228
Desktop server	229
Roadmap	230
v0.1: Map viewer and GeoJSON	230
v0.2: Project persistence	230
v0.3: Cloud-native formats	230
v0.4: DuckDB Spatial	230
v0.5: Advanced Add Data and plugin-backed layers	230
v0.6: Project access, web embeds, and expanded integrations	231
v0.7: Add Data expansion, identify, settings, and processing	231
v0.8: Viewer, desktop packaging, plugins, and dynamic layers	231
v0.9: Data integrations, processing, and menu reorganization	232
v1.0: Processing pipelines, external plugin system, and stable prototype	233
v1.1: Vector styling, attribute table management, and atmosphere effects	233
v1.2: New data sources, attribute analytics, routing, and platform polish	234
v1.3: Analysis depth, real-time collaboration, story maps, scripting, and an AI assistant	234
v1.4: Jupyter beside the map, spectral indices, georeferencing, and field collection	235
v1.5: Dashboards, in-browser Whitebox, map navigation history, and signed macOS installers	236
v1.6: Multi-map layouts, advanced symbology and labeling, and plugin zip install	237
v1.7: Plugin UI host API, color ramp previews, and category-browsed Whitebox tools	238
v1.8: Camera tours, live story maps, standalone HTML export, and map annotations	238
v1.9: CAD import, smarter service discovery, and a docked SQL workspace	239
v1.10: I3S scene layers, OGC API vector tiles, and local NetCDF/HDF loading	240
v2.0: A 3D globe, planetary mapping, symbology interchange, and editable source layers	240
v2.1: A data-source Browser panel, route animation, in-browser object detection, and map recording ...	241
v2.2: A styling overhaul, expression-driven fields and labels, print atlas, and browser-native conversions ..	242
v2.3: An on-map Legend panel, iOS and Google Play, the GeoLens catalog, and space-time hot spots	243
v2.4: STAC and Earthdata catalogs, a free-flight camera, a timeline for tiles and data cubes, and a live embed API	244
v2.5: QGIS and ArcGIS Pro project import, discrete global grids, hyperspectral cubes, and a self-hostable server	246
v2.6: Data and style deep links, an MCP server for AI clients, COG-backed terrain, and a design system of its own	248
v2.7: A visual Model Builder, STAC assets of every kind, a Chrome data opener, and selection by drawing ..	250
v2.8: A popup designer, quick filters, per-layer permissions, and the packages on npm	252
v2.9: A tokenless 3D globe, kiosk and classroom deployments, docked service panels, and keyless global elevation	254
v3.0: Cesium as a first-class rendering engine, dynamic 3D scenes on the globe, ArcGIS write-back, and editable SVG print layouts	255

v3.1: Mapbox and ArcGIS join the rendering engines, a live God's Eye View globe, OAuth sign-in for sharing, and a voice-driven AI Assistant	257
v3.2: Point cloud annotation, private S3 buckets, credentials in the OS keychain, and the GIS portals of every US state	258
v3.3: Line of sight, MGRS grids and coordinates, SQL Server, compare-and-restore project history, and a plugin ecosystem that docks, verifies, and extends the menus (current).	259
Plugin marketplace and registry (design)	260
Contributing	263
Ways to Contribute	263
Prerequisites	263
Setup	263
Run it locally	264
Repository layout	264
Development workflow	264
Quality checks	265
Documentation	269
Plugins and extensions	270
Backend sidecar	271
Sponsoring	271
License	271
Maintenance Notes	272
Dependency bumps that need a manual check	272
Shared dependency ranges	287
Adding a blend mode	288
Coverage floors	288
Lint warning ratchet	290
Test type-check ratchet	290
Boot bundle budget	291
Dependency updates and the audit allowlist	292
Publishing @geolibre/core and @geolibre/map	293
The bundled sidecar lockfile	293
Credentials must never reach a redistributable build	294
Generated files and cross-file sync	295
How to Cite	297
Recommended citation	297
BibTeX	297
APA	297
Citation metadata file	298
Citing dependencies	298
Acknowledgements	299
Open-source projects	299
Community contributors	300
Get involved	300

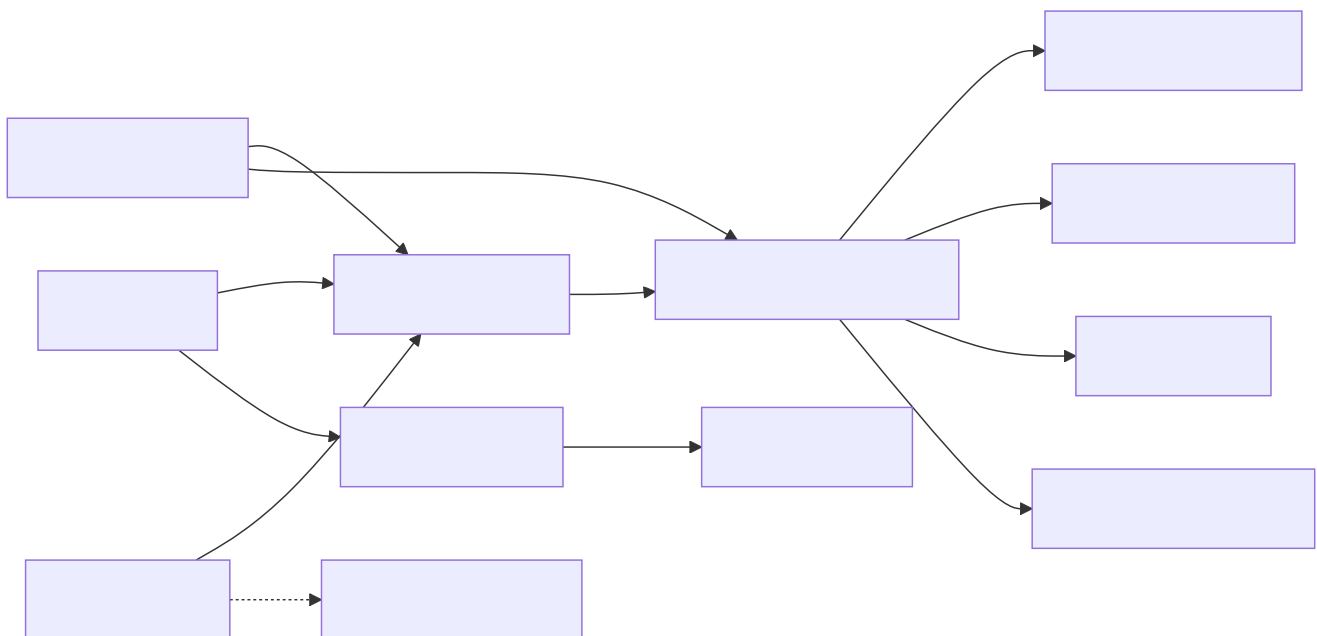
Privacy Policy	301
Summary	301
Data processed locally	301
Data sent to third-party services (optional features)	301
Plugins from the plugin registry	302
Website analytics	302
Personal information	303
Children	303
Your choices	303
Changes to this policy	303
Contact	304
Terms of Service	305
The service	305
License	305
Acceptable use	305
Google Earth Engine and other third-party services	306
Your content	306
Disclaimer of warranties	306
Limitation of liability	306
Changes to the app and these Terms	307
Contact	307

<https://geolibre.app/architecture/>

GeoLibre Architecture

Overview

GeoLibre is a free and open-source, lightweight, cloud-native GIS platform that runs in the web browser, on the desktop, on mobile, and inside Jupyter notebooks, all from a single npm workspaces monorepo. The UI is a React app that ships as a native desktop app hosted by Tauri v2 and as a browser-based web app, adapting responsively to mobile and small screens. Four engines can render a map: MapLibre GL JS (the default), Mapbox GL JS, CesiumJS, and the ArcGIS Maps SDK for JavaScript. deck.gl supplies advanced raster, point-cloud, and 3D overlays where an engine supports them. Application state lives in a Zustand store ([@geolibre/core](#)). See the user-facing [rendering-engine guide](#) and the detailed [MapLibre](#), [Mapbox](#), [Cesium](#), and [ArcGIS](#) references.



Packages

Package	Responsibility
@geolibre/core	Domain types, project JSON schema, global store
@geolibre/map	Rendering-engine lifecycles and adapters, layer sync, GeoJSON, raster, tile, MBTiles, controls, and selection styling
@geolibre/ui	Shared UI primitives (shadcn-style)
@geolibre/processing	Client-side algorithm registry

Package	Responsibility
@geolibre/ plugins	Plugin interface and built-in plugins
@geolibre/ embed	Typed, dependency-free client for the iframe embed API, published to npm
@geolibre/ collab-core	Transport-neutral collaboration relay core (wire protocol, permissions, payload validators) shared by the <code>workers/collab</code> and <code>workers/collab-node</code> relays
geolibre- desktop	Shell layout, Tauri I/O, composition

State flow

1. User adds data through the Add Data menu, the Tauri dialog, browser file picker, drag and drop, or a built-in plugin control.
2. Local vector data is parsed directly or converted to GeoJSON with DuckDB-WASM Spatial, then passed to `addGeoJsonLayer` in the store.
3. Tile, service, raster, ArcGIS, MBTiles, and plugin-backed layers create `GeoLibreLayer` records with source metadata and native MapLibre layer ids when applicable.
4. The selected `MapEngine` subscribes to `layers` and reconciles them with its SDK. MapLibre uses `MapController.syncLayers`; Mapbox, Cesium, and ArcGIS use their engine-specific adapters. The layer-control glue shared by the two Style-Spec engines lives in `packages/map/src/layer-control-host.ts`.
5. Style panel and layer panel updates change layer state, then map sync updates paint, visibility, opacity, ordering, and removal.
6. Attribute table selections update the highlighted feature source and can zoom the map to the selected feature.
7. Desktop save uses `projectFromStore` and writes `.geolibre` to disk. The earlier `.geolibre.json` name remains readable.

Every layer action replaces the `layers` array, so `useAppStore((s) => s.layers)` re-renders a component on any edit of any layer. Only subscribers that genuinely need every field of every layer (the map canvases' sync, the Layers panel) should do that. Everything else uses the narrow hooks in `packages/core/src/layer-selectors.ts`: `useLayer(id)` for one record, `useLayerIds()` for the id list, `useLayerSummaries()` for `{ id, name, type, visible, groupId }` rows, and `useLayersWhen(open)` for always-mounted dialogs and panels that need the full array only while open.

`useAppStore` is one Zustand store, but its code is split into slices under `packages/core/src/store/` (project, map view, layers, layer groups, project content, processing, libraries, session, collaboration, capabilities, and the `ui` dialog flags), composed in `packages/core/src/store.ts`. Add a new field or action to the slice that owns its data; an action that must also update another slice's fields (as `removeLayer` scrubs references everywhere) reads and writes the whole state through `set / get`. Undo history (`store/undo-history.ts`) tracks only the fields `partializeHistory` lists (layers, layer groups, basemap, story map, comments, dashboard widgets, legend). Everything else, including the

whole `ui` sub-state, the selection and the camera, never records an undo step and is never reverted by undo or redo. Secondary panes and the Print Layout also reference layers but stay untracked (panes carry their own camera and must match the untracked grid layout; the composer writes back a local copy), so each undo/redo step scrubs their references to layers the step removed.

Rendering-engine model

The Zustand store is renderer-neutral: it contains plain `GeoLibreLayer` records, preferences, and `MapViewState` values rather than objects owned by a mapping SDK. MapLibre, Mapbox, Cesium, and ArcGIS each implement the shared `MapEngine` interface for camera operations, capability checks, picking, controls, placement, extent drawing, and capture. `primaryRenderer` selects the main canvas, while each `SecondaryMapView.viewKind` selects the engine for a split pane. Changing either value unmounts the previous engine and connects the replacement to the same project state.

Capabilities are explicit rather than assumed. The shell gates commands against the active `MapEngine`; layers report engine compatibility; and plugins declare an `engines` list. This lets a project retain an unsupported layer or suspended plugin across a switch without asking every SDK to emulate every feature.

Inside `packages/map/src`, a file named for an engine (`arcgis-*`, `cesium-*`, `mapbox-*`, and MapLibre's `layer-sync`, `map-controller`, and `MapCanvas`) never imports another engine's file. Code for two engines share lives in a neutral module named for what it does (`pmtiles-archive.ts`, `protocol-tiles.ts`, `gl-style-compiler.ts`, `feature-style.ts`, and so on). The `local/no-cross-engine-imports` ESLint rule (`eslint-rules/no-cross-engine-imports.mjs`) enforces the boundary and lists the classification.

3D globe view (CesiumJS)

The default MapLibre map can be joined by — or replaced with — a 3D globe rendered with [CesiumJS](#). Cesium subscribes to the same engine-neutral store described above, whether it owns the primary canvas or one pane in a mixed-engine grid. MapLibre remains the default; integrations that paint directly through the MapLibre style API still require MapLibre.

- **Where it renders.** Two places, both drawing the same store state:
- **As a secondary pane.** Each pane in the map grid carries a 2D/3D toggle (`SecondaryMapView.viewKind`), so a globe sits beside the 2D map for comparison.
- **As the primary map** (issue #2217). **View → Rendering engine → Cesium** selects the globe through the store's `primaryRenderer`. `DesktopShell` mounts `PrimaryCesiumCanvas` in place of the previous engine's canvas, so that renderer is not merely hidden — it is unmounted and stops consuming GPU resources. Switching adds or removes no panes, and a multi-pane grid can still mix all four engines. `CesiumCanvas` distinguishes the two roles by whether it is given a `viewId`: with one it is a pane backed by a `secondaryMapViews` record (own camera, own visibility overrides); without one it is the primary map, reading and writing the shared `mapView`

directly and ignoring the `syncView` toggle (which exists to make panes follow the primary camera). Because every engine reads the same store, the camera, basemap, layers, groups, visibility, and opacity survive the switch untouched where supported.

- **What the globe cannot do (yet).** Every tool that drives a `MapController` — the map context menu, legend, comments, story map, terrain settings, the ML panels, and the MapLibre-typed plugin API — is MapLibre-only. Rather than leave them broken, `DesktopShell` does not mount them while the globe owns the primary area, and the View menu greys out the items that drive a controller (zoom, viewport history, Reset Orientation, Set View, the Google Maps/Earth hand-offs) instead of letting them silently do nothing. Renderer-neutral, store-driven surfaces (the Layers panel, Style Manager, the attribute table, processing, the Dashboard) stay available. Layer kinds the globe cannot draw stay in the project and are tagged "2D only" in the Layers panel, and come back when MapLibre does. A renderer-neutral controller interface is the follow-up milestone in #2217.
- **Enabling it.** The 2D/3D pane toggle and the Rendering engine menu are both offered whether or not a Cesium Ion token is configured — the globe draws the project basemap, which needs no token. A token adds Cesium World Terrain and supplies Ion World Imagery as the fallback for a basemap with no raster form; without one the view shows a one-line hint saying so. The token is resolved through `getCesiumIonToken()` (`@geolibre/core`), which reads `VITE_CESIUM_TOKEN / CESIUM_TOKEN` from the build **or** from a runtime override, so it can be set at build time (`CESIUM_TOKEN`; see [Optional 3D globe credentials](#)) or at runtime with no rebuild. Settings → Environment Variables has a dedicated masked **Cesium Ion token** field backed by device-local `DesktopSettings` (the OS credential store on desktop, `localStorage` on the web; never the shared project file); `useRuntimeEnvironmentVariables` projects it into `VITE_CESIUM_TOKEN` on the `window.__GEOLIBRE_RUNTIME_ENV__` global (empty values are not projected, so they cannot blank a build-time token). The shared `useCesiumIonToken()` hook re-resolves the token on the `geolibre:runtime-env-change` event and both mount sites key the globe on it, so a newly entered token takes effect without a reload.
- **Basemap.** `basemapToCesiumImagery()` (`@geolibre/core`) translates the project's `basemapStyleUrl` into a plain descriptor of what the globe should draw, and `applyBasemapImagery()` (`packages/map/src/cesium-basemap.ts`) turns that into imagery layers. The split keeps the decision engine-free and beside the basemap catalogs it reads, so the 2D and 3D renderers share one source of truth. Planetary and regional basemaps are already raster tile sets and carry straight over (TMS sources get `{y}` swapped for Cesium's `{reverseY}`), and a hybrid basemap's labels overlay rides directly above its imagery; GeoLibre's own vector styles map to a keyless raster basemap of matching tone, so a dark project basemap gives a dark globe; the blank basemap leaves a bare ellipsoid. Anything else — a provider style, a custom URL, an offline archive — has no raster form, and the renderer falls back to Ion World Imagery when a token is configured and OpenStreetMap when not. The basemap layers are inserted at the bottom of the imagery stack (index 0, the overlay at 1), which is what keeps them below the data layers `CesiumLayerSync` appends and raises to the top.
- **Lazy loading.** The whole Cesium engine (~4.8 MB) is `import()`-ed only when a globe first mounts, kept in its own build chunk (`CODE_SPLITTING_GROUPS` in `vite.config.ts`) and off the 2D boot path. The build fails if Cesium, or more than 3 MB of JS, ends up in the app entry's

static import graph (`bootBundleBudgetPlugin`). A Vite plugin (`vite-plugins/copy-cesium-assets.ts`) stages Cesium's runtime Workers/Assets/Widgets into `public/cesium/` and the canvas sets `window.CESIUM_BASE_URL` so the engine finds them.

- **Drawing.** Both engines implement `drawExtent` with a shared pointer lifecycle and geographic extents. Cesium uses a draggable entity for manual placement and ground-clamped rectangles for extraction previews. Escape, pointer cancellation, focus loss, and renderer teardown restore navigation. The raster-subset panel uses these operations on both renderers; crossing extents retain west > east so the extractor can explain that they must be split. The basemap extraction panel shares the drawing helper, while its menu remains gated on vector-style support.
- **Camera sync.** `packages/map/src/cesium-camera.ts` converts between MapLibre's Web-Mercator `MapViewState` (zoom, nadir-referenced pitch, bearing) and Cesium's camera (metric range, horizon-referenced pitch, heading), matched by **ground resolution** (metres per pixel) so the on-screen scale stays in step even when the panes differ in height. `CesiumCanvas` seeds its camera from the shared `mapView`, applies store changes to the globe, and writes the globe's own moves back — bidirectional, like the 2D panes — with a tolerance check that suppresses the `apply→moveEnd` echo so there is no jitter loop.
- **Layer sync.** `CesiumLayerSync` (`packages/map/src/cesium-layer-sync.ts`) reconciles the store's `GeoLibreLayer[]` onto the globe the way `MapController.syncLayers` does for MapLibre, reusing the same per-pane visibility overrides and group effects as `SecondaryMapCanvas`. It renders the kinds where Cesium is the natural fit — GeoJSON (a draped `GeoJsonDataSource` whose entities are styled per feature by `createFeatureStyleResolver` (`packages/map/src/feature-style.ts`, issue #2278): it evaluates the same MapLibre expressions `@geolibre/core` builds for the 2D map — single, categorized, graduated, rule-based with per-rule symbol overrides, and expression modes, simplestyle properties, proportional sizing, metre-unit strokes — with the style-spec engine, so polygon fills and outlines, line widths and arrow decorations, circle radii, classified marker sprites, fill patterns, and the layer's zoom range (a shared `DistanceDisplayCondition`) match the 2D map; points draw as circles unless the layer renders markers; a point-only layer with `pointRenderer: "cluster"` clusters through the data source's `EntityCluster` with bubbles and abbreviated counts styled like the 2D cluster layers and switched off past `clusterMaxZoom`, and a point-only layer above 50 000 features bypasses entities for one `PointPrimitiveCollection` whose primitives carry the feature reference on `id` so picking, highlighting, and filters still work (`packages/map/src/cesium-points.ts`, issue #2282; the heatmap renderer has no globe form and draws as plain circles); with field or expression labels, halos, and label zoom limits), XYZ/raster/WMTS, WMS, ArcGIS MapServer (`ArcGisMapServerImageryProvider`), and georeferenced image overlays (`SingleTileImageryProvider`) (as `ImageryLayer`s), and 3D Tiles (a `Cesium3DTileset` primitive that consumes the layer's tileset URL, request headers, and altitude offset directly, and whose features are classified by `cesium-tileset-style.ts` (issue #2290): the layer's colour expression and composed feature filter are translated from MapLibre expressions into the 3D Tiles styling language and applied as a `Cesium3DTileStyle`, so a tileset categorizes, graduates, or follows a rule tree the way an extruded vector layer does, and the layer opacity reaches it as a white multiply that fades textured tiles without tinting them — an untranslatable colour or filter leaves the tileset drawing its own colours and showing every feature rather than guessing; the attribute names the Style panel lists come from the first rendered tile, published onto

`metadata.fields` because a tileset carries its schema in the tiles rather than in the layer record; an ArcGIS I3S scene layer goes through Cesium's own `I3SDataProvider` instead of `loaders.gl`, a Gaussian-splat layer whose asset is a 3D Tiles tileset renders through the same branch because Cesium draws `KHR_gaussian_splatting` tiles natively, and a point cloud in 3D Tiles form gets `pointCloudShading` eye-dome lighting), plus COPC point clouds decoded in the browser with the `copc` package into a bounded `PointPrimitiveCollection` preview reprojected through the archive's WKT (`packages/map/src/cesium-point-cloud.ts`, issue #2285) — with live visibility/opacity, rebuild-on-source-change, and removal. Tile sources that reach the 2D map through a MapLibre custom protocol — COG tiles from the WASM tiler, raster PMTiles, local MBTiles, the desktop's native XYZ/WMS fetcher, KML super-overlays — render through `ProtocolImageryProvider` (`packages/map/src/cesium-protocol-imagery.ts`, issue #2283), a Cesium `ImageryProvider` that expands the `{z}/{x}/{y}` template the way MapLibre does, hands each tile URL to the handler in `maplibregl.config.REGISTERED_PROTOCOLS` (process-wide, so it works with no MapLibre map mounted), and decodes the bytes into an `ImageBitmap` flipped the way Cesium expects; an unregistered scheme is reported as a layer error rather than drawn blank. COG layers open the same `cog-tiler-wasm` source the raster control uses and render from the persisted `metadata.rasterState` (`cesium-cog-imagery.ts`), so a COG looks the same on both renderers; a COG restored from a project with only a desktop file path stays 2D-only until the raster control has reopened it. Zarr layers draw through zarr-cesium's `ZarrLayerProvider` (`cesium-zarr-imagery.ts`, issue #2261), fed from the same record the 2D Zarr control and the ArcGIS renderer read (the URL, a registered store or kerchunk manifest, the variable, selector, colour limits, and ramp); a Time Slider step or restyle updates the live provider and swaps in a fresh `ImageryLayer` over it rather than reopening the store, and a grid in a CRS other than EPSG:4326/3857 is reported as a layer error. zarr-cesium imports the `cesium` wrapper, which Vite aliases to `@cesium/engine` so the globe keeps one engine instance. The raster symbology (brightness, contrast, saturation, hue) maps onto `ImageryLayer`'s own adjustments for every imagery kind. Tile-backed vector layers (`vector-tiles`, vector PMTiles, vector MBTiles, and ArcGIS vector tiles with a resolved style) are draped (`cesium-drape.ts`, issue #2284): the globe runs one hidden, single-tile MapLibre map that receives the same store layers through the headless `createLayerSync` path, renders one Web Mercator tile at a time (`jumpTo` the tile centre, wait for `idle`, read the canvas back), and feeds Cesium through a `ProtocolImageryProvider`, so the Style Spec is never re-implemented and every style, filter, order, and visibility edit rebuilds the drape's imagery layer. ArcGIS vector-tile imports persist their resolved sources and prefixed style layers, allowing the headless sync to recreate them after a renderer switch or project reload and apply visibility, filters, and opacity while preserving service colors; the Style panel offers only opacity, order, zoom range, and filters for those layers rather than color editors that would not reach them. Older ArcGIS records without that resolved style remain 2D-only and must be re-added. Draped content is flat, its labels are placed per tile, and their glyphs come from a remote font host (so a local archive viewed offline drapes without text); picking on it, legacy `arcgis` records without a resolved style, EPT and plain LAS/LAZ point clouds, raw `.ply` / `.splat` splat files, and `deck.gl` viz are still skipped on the globe and render in the 2D panes; the exported `isCesiumSupportedLayerType` predicate lets the pane's layer menu — and the Layers panel, when the globe is the primary renderer — tag those "2D only". Native KML/KMZ documents

use `KmlDataSource`, preserving document styling, network links, and overlays; visibility and opacity are applied without replacing the original color properties. Local KMZ archives persist as data URLs. The Elevation Profile plugin draws native entities and samples the active terrain provider, rejecting results when that provider changes.

- **Globe interaction parity.** The primary globe publishes ground cursor coordinates and optional terrain elevation to the shared status bar, clearing them on pointer exit and teardown. Projected scene modes report a Mercator projection. Home, scene-mode, and fullscreen widgets participate in shared control positioning without remounting their DOM. In the Controls menu they answer to the existing entries: **Compass** governs the Home (reset view) button and **Globe** governs the scene-mode picker, as the globe's counterparts of the 2D reset-bearing and projection controls, and the toolbar replays those choices onto a freshly mounted globe.
- **Environment plugins.** The Sun simulation, Atmospheric Effects, Clouds, Precipitation, and Flight Simulator plugins declare Cesium alongside the 2D engines (Sun, Effects, and the Flight Simulator list MapLibre, Mapbox, and Cesium; Clouds and Precipitation also list ArcGIS) and branch on `app.getCesiumScene()` (issue #2287), a typed handle to the primary globe's namespace, widget, scene, camera, and clock that `CesiumEngine.getCesiumScene()` exposes as the globe's counterpart to `getMap()`. On the globe the Sun drives a native `SunLight`, `globe.enableLighting`, and the scene clock (the night-side depth maps onto `globe.vertexShadowDarkness`); Effects toggles the sky box and atmosphere and re-tints `SkyAtmosphere` by hue/saturation/brightness shifts from the halo settings; the Flight Simulator shares its keyboard, physics, and HUD with both 2D maps behind a camera adapter — `camera.setView` each frame with `screenSpaceCameraController.enableInputs` suspended on the globe, `setFreeCameraOptions` on Mapbox, `calculateCameraOptionsFromCameraLngLatAltRotation` on MapLibre; the weather overlays are store tile layers and render through `CesiumLayerSync` unchanged. The Spinning Globe control needs no branch: it drives the store through the control host's MapLibre facade. `DesktopShell` re-attaches these plugins after either engine mounts, so a renderer swap rebinds them the way a MapLibre re-init does.
- **Persistence.** A pane's `viewKind` is part of `SecondaryMapView` and round-trips through the `.geolibre.json` project format (`normalizeSecondaryMapView`), so a project saved with a globe pane reopens as the globe, token or not. The workspace's own choice persists as the top-level `primaryRenderer` (`normalizePrimaryRenderer`), written only when it is not the default — so a MapLibre project is byte-identical to one saved before the setting existed, and a 3D-first project reopens directly on the globe.

DuckDB-WASM

Vector file import uses DuckDB-WASM for formats that need conversion before MapLibre can render them:

```
INSTALL spatial;
LOAD spatial;
```


GeoParquet is read with DuckDB's Parquet reader after loading Spatial. Other local vector formats are passed to Spatial `ST_Read` when the WebAssembly extension can load the GDAL-backed reader. Zipped Shapefiles are parsed with `shpjs` first, then DuckDB Spatial is tried if that parser cannot read the file. KML files (and the KML inside unzipped KMZ archives) are read by an in-house parser that preserves embedded symbology, emitting `simplestyle-spec` properties (`fill` , `stroke` , `stroke-width` , and so on) so styled KML renders the way it does in Google Earth; KML the parser cannot handle falls back to the DuckDB Spatial reader, which loads the geometry without the styling. A vector layer whose features carry simplestyle properties is rendered per-feature: paint expressions read the per-feature color/width and fall back to the flat layer style.

Apache Iceberg

Iceberg tables are read on the same engine, with DuckDB's core `iceberg` extension loaded alongside Spatial (`INSTALL iceberg; LOAD iceberg;`). A table addressed by its metadata location is scanned with `iceberg_scan()` ; an Iceberg REST catalog is `ATTACH` ed under a fixed alias, enumerated with `SHOW ALL TABLES` , and the chosen table read as a qualified name. Either generated statement is only a default: the dialog pre-fills it into a SQL box, and whatever is in that box becomes the scan's source, so a `WHERE` , a join, or a projection can decide which geometries are rendered. A custom statement is checked with `cleanStatement / containsMultipleStatements` (the literal-aware helpers in `sql-workspace.ts`) before being wrapped as a sub-select, so a pasted script fails with a clear message rather than a parse error pointing at the generated wrapper.

Geometry handling is deliberately narrower than the vector-file loader's. Only columns DuckDB reports as native `GEOMETRY` are candidates: Iceberg v3 has a real geometry type — `iceberg_column_definition.cpp` maps it to `LogicalType::GEOMETRY` — so a BLOB or VARCHAR here is an ordinary attribute, and the blob/base64-WKB fallbacks that make sense for plain Parquet would only produce loads that fail inside `ST_AsGeoJSON` . **The CRS comes from the schema, not the user.** DuckDB renders a CRS-annotated column as `GEOMETRY(<crs>)` , so `icebergTransformCrs` reads it and reprojects to WGS84 through `ST_Transform` ; a column carrying no CRS parameter is `OGC:CRS84` by specification (the same constant DuckDB's extension hard-codes as `IcebergConstants::DefaultGeometryCRS`), which is already GeoJSON's lon/lat convention and passes through untransformed. Nothing about the coordinate system is persisted on the layer, so a table re-projected upstream stays correct on the next reload.

The work is split between `lib/iceberg.ts` , which holds the persisted layer config, the SQL builders, and the table-selection rule with no DuckDB dependency, and `lib/iceberg-loader.ts` , which owns the engine. The loader runs on the **SQL Workspace's** dedicated DuckDB instance rather than the shared one: an Iceberg scan reads the table's Parquet data files over HTTP, so it needs that instance's pre-spatial warm-up and its poisoned-instance recovery (see `resetSqlDatabase`).

Two consequences of Iceberg tables being large are wired into the app rather than left to the user. Every load is capped by a row limit, and the total row count — a `count(*)` over the configured source, which DuckDB can answer from the manifest metadata for an unfiltered table, though a custom statement's filters, joins, or projections have to be evaluated — is shown before anything is materialized. And the resulting layer, though refreshable on demand, is excluded from timer-driven

refresh by `supportsAutoRefresh` in `lib/layer-refresh.ts` — re-scanning a table of that size on an interval is never what the user meant, so the automatic-refresh interval is unavailable for it rather than merely defaulted off.

Advanced Add Data workflows

The Add Data surface includes native dialogs for XYZ, WMS, WFS, vector files (via the Add Vector dialog backed by `maplibre-gl-vector`), GeoJSON URLs, vector tile sources, delimited text, raster tile templates, COG and GeoTIFF rasters (via the Add Raster dialog backed by `maplibre-gl-raster`), MBTiles, ArcGIS FeatureServer or VectorTileServer layers, Apache Iceberg tables, and GPX waypoints, tracks, and routes. It also supports 3D Tiles layers, Cesium Ion assets by id (a tileset or imagery layer with `metadata.sourceKind: "cesium-ion"`, offered when the globe is the primary renderer and badged "3D only" on the 2D map; `packages/core/src/cesium-ion.ts`, issue #2290), WFS and GeoJSON URL refresh, text marker labels, and multiple DuckDB SQL query-result layers with identify, selection, and attribute table support. The Components plugin wraps `maplibre-gl-components` panels for FlatGeobuf, PMTiles, Zarr, LiDAR, and Gaussian splats, then mirrors added layers into the GeoLibre store so the Layer panel, project format, and layer control can reason about them. Additional data sources are available through the Planetary Computer and Earth Engine panels, the Overture Maps plugin, and the federal Web Services plugins. The Time Slider plugin, backed by `maplibre-gl-time-slider`, animates time series raster and vector data (COG, XYZ/WMTS, WMS-Time, and time-filtered GeoJSON) through a docked timeline, mirroring each source it adds into the GeoLibre store as an external native layer.

Local MBTiles tiles are read through a custom MapLibre protocol backed by Tauri commands. Remote rasters are fetched through the desktop backend when needed, and the local development server includes a raster proxy for selected release assets that need CORS handling.

Python sidecar

The FastAPI app in `backend/geolibre_server` backs the format Conversion tools, the raster tools, and the optional server engine for the Whitebox toolbox through a managed local processing sidecar. The desktop app starts the sidecar on demand, communicates over `127.0.0.1`, and keeps the heavier Python processing stack outside the browser bundle.

The Whitebox toolbox does **not** depend on it: its 1,000+ tools run in the browser on the `geolibre-wasm` WebAssembly runtime by default, and the sidecar is an opt-in alternative engine that can read native file paths for batch runs over a directory.

The Vector tools (Processing → GeoLibre Toolbox → Vector) run client-side with Turf.js and need no sidecar. All of the tools can optionally run on the sidecar's `/vector` endpoints, backed by GeoPandas and Shapely, for projection-aware results; the sidecar reports availability through `/vector/status`, and the dialog falls back to the client engine when the optional `vector` extra is not installed.

A third Vector engine, **Python (Pyodide)**, runs the same GeoPandas/Shapely code **in the browser** via **Pyodide** (CPython compiled to WebAssembly), so the GeoPandas path is available on the web build with no server. The geometry logic is a framework-free module, `backend/geolibre_server/geolibre_server/vector_ops.py`, that both the sidecar and the browser run — a Vite plugin (`vite-plugins/copy-vector-ops.ts`) copies it into the app bundle, and a classic Web Worker (`public/pyodide/pyodide-worker.js`) loads Pyodide from a CDN, installs `geopandas`, and calls `run_vector_tool` over a JSON-string boundary. One source of truth means the Sidecar and Pyodide engines return identical results. The Pyodide runtime is downloaded lazily on first use; the `VITE_PYODIDE_INDEX_URL` env var points it at a self-hosted mirror for offline/production deployments.

The **SQL Workspace** offers an **Apache Sedona** engine alongside DuckDB and PostGIS. It runs Sedona spatial SQL on **SedonaDB**, the single-node Rust (DataFusion + Arrow) engine, through the sidecar's `/sql` endpoints (`/sql/status`, `/sql/run`) backed by the optional `sedona` extra (`apache-sedona[db]`). In the browser — and on desktop when the extra is not installed — the same engine runs entirely client-side on **CereusDB**, a WebAssembly build of SedonaDB; the workspace prefers the sidecar when reachable and falls back to CereusDB automatically. The CereusDB bundle is large, so it is dynamically imported into its own chunk and only downloaded when a Sedona query first runs.

The sidecar has since grown past conversion. Its routers live in `backend/geolibre_server/geolibre_server/app/`: `raster.py` runs the Raster toolbox on `rasterio/numpy/contourpy`, `vector.py` backs the optional GeoPandas vector engine, `sql.py` backs the Sedona engine above, `postgis.py` and `mssql.py` load PostGIS and SQL Server tables as editable layers and write edits back, `pointcloud.py` writes point cloud annotator labels into a whole LAS/LAZ/COPC file, and `ml.py` reverse-proxies AI Segmentation to the `samgeo-api` server (SamGeo/SAM 3), which is kept out of process because SAM 3 needs a multi-gigabyte GPU Torch build. See the [Server API reference](#) for every endpoint. Leafmap and GeoAI workflows are still expected to follow the same pattern.

Offline support (PWA)

The standalone web build is an installable Progressive Web App. `vite-plugin-pwa` (configured in `apps/geolibre-desktop/vite.config.ts`) emits a web manifest plus a Workbox service worker, and `src/main.tsx` registers it next to `installStaleChunkReload` so the two coordinate. The service worker is built only for the web build; it is disabled for the Tauri desktop build (already offline via bundled assets) and the embedded Jupyter wheel (`GEOLIBRE_EMBED=1`), where `registerSW` resolves to a no-op.

Caching is split to keep the first visit light:

- **Precache (app shell).** The HTML and the JS/CSS chunks that boot the map are precached, so the shell loads with no network after the first visit. The heavy chunks below are excluded from the precache to avoid a large first-load download.
- **Runtime cache, CacheFirst.** The content-hashed build assets the precache skips (everything under `/assets/`) are cached on first use: the **MapLibre** bundle, **DuckDB-WASM and its spatial extension**, and the MapLibre feature-plugin chunks. Hashed filenames make CacheFirst safe — a redeploy mints new URLs, so a stale entry is never served as current. This is what makes

local-file workflows (DuckDB Spatial conversion) work offline after they have run online once. Self-hosting the spatial extension via `VITE_DUCKDB_SPATIAL_EXTENSION_PATH` keeps it same-origin so it is cached too. **PGLite/PostGIS** is not in this same-origin list: it is fetched from the jsDelivr CDN to keep it out of the build — ~25 MB raw, and ~22 MB of otherwise-incompressible weight in the Tauri binary — so the PostGIS SQL engine needs network on first use. A separate rule caches it thereafter (see below).

- **Runtime cache, CDN engines.** A second CacheFirst rule (`geolibre-cdn-engines`) covers the heavy engines served from jsDelivr: **Pyodide** and its wheels, **PGLite/PostGIS**, the **CereusDB** (Apache Sedona) wasm, and `gda13.js` . Their URLs embed the exact package version, so a redeploy mints new URLs and CacheFirst never serves a stale engine, and jsDelivr's permissive CORS headers mean these come back as normal (non-opaque) 200s that can be revalidated and evicted like any other entry. This is what lets the browser SQL (PostGIS), Sedona SQL, and Python features work offline **after their first online use**. When `GEOLIBRE_PGLITE_CDN=0` / `GEOLIBRE_CEREUS_CDN=0` those engines are bundled under `/assets/` instead and this rule simply never matches them; Pyodide is always CDN-loaded regardless. The rule also covers `/npm/@duckdb/` , which matches when `GEOLIBRE_DUCKDB_WASM_CDN=1` moves DuckDB off the origin (below) and when `maplibre-gl-duckdb` fetches its own engine, which it does from jsDelivr in every build.
- **Basemaps.** Tiles and styles from the CORS-friendly default hosts (OpenFreeMap, CARTO) are runtime-cached. Other remote tiles, services, and ArcGIS/WMS/WFS sources stay network-only by design and are unavailable offline.

So each of these CDN engines needs the network on **first** use, then works offline. To remove even the first-use dependency, build with `GEOLIBRE_PGLITE_CDN=0` and `GEOLIBRE_CEREUS_CDN=0` , which vendor PGLite/PostGIS and the CereusDB wasm back into the build under `/assets/` , where the same-origin rule covers them (PGLite alone re-adds ~22 MB to the Tauri binary).

To strip **all GeoLibre-controlled** external CDN references from the build output — for deployments that cannot load any resource from untrusted CDNs — use `GEOLIBRE_NO_EXTERNAL_CDN=1` . This implies `GEOLIBRE_PGLITE_CDN=0` , `GEOLIBRE_CEREUS_CDN=0` , `GEOLIBRE_GDAL_CDN=0` , and `GEOLIBRE_DUCKDB_WASM_CDN=0` , and additionally disables features whose code references external CDNs: storymap HTML export (which injects `<script>` tags from `unpkg.com`), built-in object detection models (YOLO ONNX weights from `cdn.jsdelivr.net`), Segment Everything, 3D Tiles Draco/KTX2 decoders, and GDAL export. Pyodide remains available when `VITE_PYODIDE_INDEX_URL` points to an approved mirror. Runtime `deployment.json` configures client policy; it does not change this build-time CDN packaging.

DuckDB-WASM goes the other way, because it is on the critical path for opening a local vector file and so is bundled by default. `GEOLIBRE_DUCKDB_WASM_CDN=1` moves it to jsDelivr instead — or `npm run lite:build` , which sets that flag and then *asserts* no emitted file exceeds the ceiling, so a regression fails the build rather than the upload. It is pinned to the installed version by duckdb-wasm's own `getJsDelivrBundles()` , so the fetched engine cannot drift from the loader compiled into the bundle. The flag exists for one reason: `duckdb-mvp.wasm` (~40 MB) and `duckdb-eh.wasm` (~35 MB) both exceed the **25 MiB per-asset limit on Cloudflare Pages and Workers static assets**, which rejects the upload outright, and nothing else in the build comes close (the next largest is ~22 MB). So this single

flag decides whether the web build can be hosted there at all — it takes the output from ~251 MB to ~176 MB with no file over the ceiling. GitHub Pages allows 100 MB per file and needs none of this. The flag is ignored for both targets that ship no service worker: a Tauri build, which must stay offline-capable, and an embed build (`GEOLIBRE_EMBED=1`), where the engine would otherwise be refetched every notebook session with no runtime cache behind it. Neither has the size ceiling this exists for, since a binary and a wheel are not uploaded to Cloudflare. A CDN-hosted worker script cannot be passed to `new Worker` (it must be same-origin), so that variant wraps it in a same-origin blob that `importScripts` the real one; this needs `worker-src blob:` and the CDN in `script-src`, both already in `docker/nginx.conf`. The nested `importScripts` is matched against `script-src` rather than `worker-src` — verified against that policy in Chromium and Firefox, since Firefox has historically checked worker sub-resources against `worker-src / child-src`.

Two engines have no such escape hatch. **Pyodide** is always fetched: `VITE_PYODIDE_INDEX_URL` re-points it at a mirror you host — worth doing when jsDelivr is unreachable or disallowed — but note that neither CacheFirst rule matches such a mirror, since one is scoped to `cdn.jsdelivr.net` and the other to same-origin `/assets/`, so unless the mirror is served from under `/assets/` it falls back to ordinary HTTP caching rather than the service worker. `gda13.js` — which backs the Georeferencer's client-side GeoTIFF/COG export — is never vendored at all: its wasm (~28 MB) and data (~12 MB) only ever come from the CDN, and `GEOLIBRE_GDAL_CDN=0` opts out of that rather than bundling them, leaving the loader with no paths and that export unavailable.

The **desktop** build ships no service worker, so none of this runtime caching applies there: it re-fetches every CDN-loaded engine on first use per install, unless PGLite/PostGIS and CereusDB are bundled with the flags above.

A new deploy is picked up via `registerType: "autoUpdate"`: the new service worker installs in the background and takes control (`skipWaiting` + `clientsClaim`), so its fresh precache serves subsequent requests. Workbox's default force-reload-on-activation is deliberately suppressed via `onNeedReload` in `src/main.tsx` — on the relative-base `/demo/` subpath that reload fired spuriously and discarded in-progress map state. Page recovery is delegated to `installStaleChunkReload`, which reloads on-demand only when an orphaned lazy chunk 404s (cooldown-guarded; if `sessionStorage` is blocked it skips the reload and lets the preload error surface). Precached chunks are served from cache and never 404, so the page is no longer reloaded out from under the user.

Container image

The root Dockerfile packages the browser app: a Node build stage runs the `geolibre-desktop` workspace build, then copies `apps/geolibre-desktop/dist` into nginx. The nginx config serves static assets and falls back to `index.html` for browser-entry URLs.

At startup, the entrypoint validates the selected source policy, applies nonblank environment overrides, generates nginx guards, conditionally starts the Python sidecar, and serves the generated public `/deployment.json`. The browser fetches that policy before first render, then resolves settings from policy, runtime environment and build environment, in that order. See [Deployment Policy](#) and [Self-Hosting: container policy enforcement](#).

The `Publish Container Image` workflow builds the image for pull requests and publishes it to GitHub Container Registry for pushes to `main`, version tags, and manual runs. The upstream image name is `ghcr.io/opengeos/geolibre`.

The image bundles the optional Python sidecar (uvicorn) and reverse-proxies it at `/sidecar`; it starts only when the final policy grants `processing:run` or `data:add` and `GEOLIBRE_DISABLE_SIDECAR` is not `1`. The container does not run the Tauri desktop shell, so workflows that depend on desktop filesystem access still require the installed desktop app. Client policy and nginx guards do not enforce desktop processing, browser WASM, separately exposed sidecars, or plugin execution.

Security

- The Tauri CSP pins `script-src` to version-specific jsDelivr paths and keeps `connect-src` open to `https: / http:` for user-supplied data URLs; see [Desktop CSP `script-src` allowlist](#).
- File access uses dialog-selected paths only.

Credential storage

Tokens and API keys saved in Settings (share token, Cesium Ion token, Mapbox token, ArcGIS API key, the secret fields of AI Assistant profiles), saved PostGIS connection strings, and the Share.GeoLibre OAuth refresh token live in different places per build:

- **Desktop (Tauri, macOS/Windows/Linux):** the OS credential store (macOS Keychain, Windows Credential Manager, Linux Secret Service) under the service `org.geolibre.desktop`, one entry per credential, through the `secure_store_*` commands in `src-tauri/src/secure_store.rs`. The `geolibre.desktopSettings` `localStorage` blob keeps those fields empty, and `geolibre.postgres.connectionIds` holds only the non-secret order of the `postgres.connection.<uuid>` entries. The index is written before a new entry, so a crash never leaves an unreferenced credential; a startup that finds an indexed id without a credential (its write failed) drops the id and shows the warning. `lib/credential-hydration.ts` loads them into memory before the app renders and migrates any plaintext values left in `localStorage` by an older build, removing each only after its keychain write succeeds. That load is the only read: `secure_store_get_many` answers once per page load of a webview (the reload hook in `lib.rs` reopens it), and `secure_store_seal`, which the external-plugin loader calls before importing a plugin, closes it without reading, so plugin code cannot pull saved tokens from the store (issue #2858). Writes and deletes stay open. The OAuth refresh token is stored per issuer as `share.oauth.refreshToken.<issuer>` and rotates on every refresh; the access token and the PKCE verifier stay in memory. Because the server revokes a session when a consumed refresh token is presented, the issuer is recorded in the non-secret `geolibre.share.oauth.unsavedIssuers` list before each keychain write and removed once the latest queued write for it completes. A launch that finds its issuer listed deletes the stored token and starts signed out rather than trusting a possibly consumed one. If the list itself cannot be written, the new token is not stored: a delete of the stored copy is queued and the sign-in lasts only until GeoLibre closes. After a refresh that delete is best effort; quitting before

it finishes leaves the consumed token stored, so the next launch signs out on its first refresh instead of at startup. Sign-out waits for its delete before reporting signed out, and sends the revoke request whether or not the delete succeeds. If the delete fails, Settings reports the sign-out as incomplete; the delete stays queued and is retried only with a later credential write while the app runs. After a restart the token is discarded if its issuer is on the unsaved list; if that list could not be written either, the token is restored, and it still works only if the revoke request also failed. Credentials a project file would otherwise carry (geocoding API keys `preferences.geocoding.apiKeys` and secret Environment Variables) are stored device-wide by name as `project.geocoding.apiKey.<providerId>` and `project.env.<VARIABLE>`; the non-secret `geolibre.projectCredentials.accounts` list indexes them, with the same index-first write order. Saving a project moves geocoding keys and uniquely named secret environment values to the keychain, leaving empty values in the file. Duplicate or nameless secret environment rows cannot be assigned a unique account and remain for the local keep/strip save prompt. Settings writes a value only when the user changes it. Clearing a field deletes the stored value only if the field showed that value; clearing a project-file override that differed from it drops the override and the shared value applies again. Opening a file that still carries plaintext values uses them for that session and writes nothing until the next save. `@geolibre/core`'s `project-credential-refs.ts` fills empty values from the stored ones where they are used (runtime environment, Settings); a value in the file wins. Removing or renaming a variable never deletes its stored value, since other projects on the device may use the same name. A failed keychain write likewise falls back to the keep/strip save prompt. Layer request headers are not stored by layer: a header value references a variable as `${NAME}` (for example `Authorization: Bearer ${TILES_TOKEN}`), saved as typed and resolved from the project's enabled Environment Variables when the request is made (`resolveProjectHeaderReferences`). A header whose variable is unset is not sent. Redaction keeps a header value that is only an optional scheme word plus one reference, and removes any other header value as before. Tokens that plugins save for themselves through `app.credentials` are stored as `plugin.<pluginId>.<name>` (the host adds the plugin id); the non-secret `geolibre.pluginCredentials.accounts` list indexes them with the same index-first write order, and `lib/plugin-credentials.ts` holds them in memory so plugins read synchronously. Built-in plugins still use their own localStorage keys until they move to the API.

- **Web, Jupyter embed, mobile:** localStorage, as before. Plugin credentials are `geolibre.pluginCredential.<pluginId>.<name>`. The web OAuth refresh token stays in tab-scoped sessionStorage. Project-file credentials stay in the project file, behind the keep/strip save prompt.

If the credential store is unavailable (for example, no Secret Service on a Linux session), credentials stay in memory for the session and Settings reports that the keychain cannot store project credentials. A local project save asks whether to keep remaining credentials in plaintext or strip them; it never writes them without an explicit choice.

Native HTTP trust store and mutual TLS

The desktop app issues some remote fetches (tile/URL resolution, OGC GetCapabilities, and similar) from the native Rust process rather than the WebView. That path (`guarded_http_client` in `src-tauri/src/lib.rs`) trusts the **OS/system certificate store** in addition to the bundled Mozilla roots, so a server signed by an enterprise CA installed on the machine is accepted without extra configuration.

For endpoints behind **mutual TLS (mTLS)**, the WebView `fetch` path shows an interactive OS certificate prompt; the native Rust client cannot, so point it at a client certificate with environment variables:

Variable	Purpose
<code>GEOLIBRE_HTTP_CA_CERT</code>	Path to a PEM bundle of extra CA certificate(s) to trust, on top of the OS store (for a private CA not installed system-wide).
<code>GEOLIBRE_HTTP_CLIENT_CERT</code>	Path to the client certificate to present. A <code>.pem</code> file (certificate chain plus an unencrypted PKCS#8 private key) or a PKCS#12 bundle (<code>.p12</code> / <code>.pfx</code>).
<code>GEOLIBRE_HTTP_CLIENT_CERT_PASSWORD</code>	Passphrase for a PKCS#12 client certificate. Its presence also forces the PKCS#12 code path.

A `.p12` / `.pfx` extension (or a supplied passphrase) selects PKCS#12; any other path is read as PEM. PEM identities use the default rustls backend; PKCS#12 identities use the platform native-tls backend (SChannel on Windows, Secure Transport on macOS, OpenSSL on Linux), which also reads the OS trust store. Convert a PKCS#12 export to PEM with `openssl pkcs12 -in cert.p12 -out cert.pem -nodes` if you prefer the rustls path. A set-but-empty value for any of these variables is treated as unset.

The configured client certificate is held on the shared native HTTP client and is therefore **presented to any HTTPS server that requests one** during a native fetch (tile, style, and OGC hosts a project points at), not only the endpoint the certificate was issued for. TLS sends a client certificate only when the server asks for one, so this is not an unconditional disclosure, but configure a client certificate only when the hosts the app talks to are trusted.

Performance: map rendering on Linux (WebKitGTK)

The desktop app uses the system WebView. On Linux that is **WebKitGTK**, whose WebGL/JavaScript pipeline is materially slower than the Chromium engine the browser build runs in. The most visible effect is **map panning at low zoom**:

- A blank map (no tile layer) pans at a steady 60 FPS at any zoom.
- With any tile layer (vector **or** raster XYZ), FPS collapses to single digits **while tiles are loading**, then snaps back to 60 once loading stops, at the same zoom. Low zoom only makes it constant because panning across the whole world loads tiles continuously and the cache never settles.

The cost is WebKitGTK processing each newly-loaded tile on the main thread: the GPU upload of its texture (raster) or vertex buffers (vector) via synchronous WebGL calls, plus the subsequent fade-in repaint frames. Vector tile parsing and bucket building run in MapLibre's Web Workers, so they are not the bottleneck here. Each tile-integration render cycle measured ~125 ms wall-clock in WebKitGTK versus a few ms in Chromium. The gap is in WebKitGTK's WebGL implementation and compositor pipeline (driver command-stream flush, TextureMapper GL surface composition), not solely its JavaScriptCore JS engine. This is a WebView-engine limitation, not a bug in GeoLibre, and it does not affect the browser build or (untested) the macOS/Windows WebViews.

Ruled out during diagnosis (so future investigation does not repeat them): software rendering (the GPU is used, Intel i915 confirmed), GPU saturation (the render engine stays ~20% idle), the Tauri IPC file read (~126 ms for a 22 MB GeoJSON), `JSON.parse` (~36 ms), KWin compositor latency, `renderWorldCopies`, the globe vs. mercator projection, and `preserveDrawingBuffer`.

To reproduce the measurement: temporarily log MapLibre `render` events, `dataloading` (tile) events, and a `requestAnimationFrame` counter once per second; FPS tracks tile-load count inversely. The quickest check is this frame-rate meter pasted into the WebView devtools console, then pan while watching the logged FPS:

```
let f = 0,
    t = performance.now();
(function loop(n) {
  f++;
  if (n - t >= 1000) {
    console.log("FPS", f);
    f = 0;
    t = n;
  }
  requestAnimationFrame(loop);
})(t);
```

Mitigations (reduce *how many* tiles load during a pan, since per-tile cost is fixed by the engine) are not yet implemented: a larger `maxTileCacheSize`, 512px raster tiles instead of 256px, and `fadeDuration: 0`, ideally gated to WebKitGTK so the Chromium-based builds keep full fidelity.

<https://geolibre.app/maplibre-renderer/>

MapLibre renderer

Choose **View** → **Rendering engine** → **MapLibre** to use GeoLibre's default renderer. MapLibre GL JS is bundled with every GeoLibre distribution, requires no account or access token, and is available for both the primary map and every split pane.

MapLibre has the broadest feature and plugin compatibility in GeoLibre. New projects use it unless `primaryRenderer` says otherwise, and an omitted `viewKind` in a secondary pane also means MapLibre.

Supported paths

- GeoJSON and imported vector data with single, categorized, graduated, rule-based, expression, proportional, heatmap, cluster, pattern, label, and extrusion styling.
- Raster and image sources, including XYZ, WMS, WMTS, COG/GeoTIFF, image and video overlays, raster PMTiles, and desktop MBTiles.
- Vector tiles and vector PMTiles or MBTiles, including imported Mapbox styles and ArcGIS vector-tile services.
- 3D Tiles, I3S, LiDAR and COPC point clouds, Gaussian splats, 3D models, Zarr, NetCDF/HDF, and deck.gl visualization layers through GeoLibre's specialized renderers and overlays.
- Feature picking, selection, editing, measurement, extent drawing, draggable placement, print and screenshot capture, and the full MapLibre-oriented plugin surface.

See [Supported Data Formats](#) and [Features](#) for the complete user-facing lists. Individual data sources can still have platform, credential, or network requirements even though the MapLibre engine itself does not.

Map, globe, and terrain

MapLibre supports Mercator and globe projections. **Controls** → **Globe** switches the projection, while **Controls** → **Terrain** adds elevation to either view. The terrain source can be GeoLibre's global source or a compatible local or remote COG DEM. This globe projection remains a MapLibre style map; choose the [Cesium renderer](#) for a native 3D globe and Cesium-native scene formats.

Basemaps and plugins

The shared Basemaps panel provides keyless styles as well as optional provider styles configured through **Settings** → **Environment Variables**. Provider keys belong to those services and are not a requirement of MapLibre itself.

Plugins default to MapLibre support when their manifest omits `engines`. MapLibre-specific plugins can access the native map through `app.getMap()`; portable plugins should instead use the renderer-neutral or shared Style-Spec APIs described in the [Plugin API](#).

Architecture and license

`MapController` is the MapLibre implementation of `MapEngine`. `packages/map/src/layer-sync.ts` reconciles project layer records with native sources and style layers, while specialized controls register protocols or shared overlays for formats that MapLibre does not draw directly. The reusable headless surface is published as `@geolibre/map`.

MapLibre GL JS and GeoLibre are open source under permissive licenses. See the [architecture](#) for the shared engine model and [Acknowledgements](#) for project and license links.

<https://geolibre.app/mapbox-renderer/>

Mapbox renderer

Choose **View → Rendering engine → Mapbox** to render a project with Mapbox GL JS. The engine is also available from the rendering-engine menu in each split pane. MapLibre remains the default. Projects save the primary and secondary renderer choices, and the Python and iframe APIs accept `mapbox` as a renderer name.

Paste your token into **Settings → Environment Variables → Mapbox token** and click **Save Settings**. Like the Cesium token, it is stored on this device, outside the project file. Existing enabled Mapbox environment-variable rows move into this field when settings are saved; Cancel leaves them unchanged. Alternatively, launch the development server with `MAPBOX_TOKEN` in its environment. Token changes recreate Mapbox maps. A missing token displays setup instructions; map loading errors redact access tokens. Use a public Mapbox token appropriate for your application. Mapbox use is associated with that token's account and is subject to Mapbox's terms and usage pricing.

New projects use Mapbox Standard by default for the Mapbox renderer. Open the shared **Basemaps** panel from the Layers panel or Add Data to select Mapbox styles (including Streets), public styles, or stacked raster basemaps. The separate floating Mapbox selector has been removed.

A style selected in the Basemaps panel is saved as `preferences.map.mapboxStyleUrl` while Mapbox is active, leaving the shared MapLibre/Cesium background unchanged. All Mapbox panes share that choice. Previously saved projects keep their selected styles. Provider credentials come from Environment variables or the basemap control's API keys panel.

Supported paths

- Native GeoJSON, including the vector importer's materialized data, with point, line, polygon, extrusion, label, data-driven color, opacity, and filter styles.
- Remote vector PMTiles archives through Mapbox GL JS's native archive reader.
- LiDAR (LAS/LAZ, COPC) and standard 3D Tiles through deck.gl overlays.
- Deck.gl Layers built in **Add Data → Deck.gl Layer** (every kind in the builder, including the 3D Model scenegraph kind behind **Add Data → 3D Model**), plus the 3D Z-value and feature-diagram rendering of ordinary vector layers, through the same shared interleaved deck.gl overlay.
- DuckDB query layers from **Add Data → DuckDB**, drawn by the panel's own deck.gl overlay.
- Zarr layers from **Add Data → Zarr Layer**, STAC Zarr assets, and NetCDF/HDF or Kerchunk cubes with a time axis, drawn by `@carbonplan/zarr-layer` — a `CustomLayerInterface` implementation that targets Mapbox GL as well as MapLibre (globe and Mercator), added by the same Zarr control. Zarr Layer and STAC adds keep the current projection; local NetCDF/HDF cubes and Kerchunk references draw untiled in Web Mercator, so adding one switches the map to Mercator, as on MapLibre. A remote NetCDF/HDF URL renders its selected slice as an image overlay instead.

- HTTP(S) raster tiles (XYZ, WMS and WMTS), vector tiles with named source layers, and georeferenced image/video sources.
- Shared layer/group visibility, opacity and ordering; synchronized or independent split-view cameras; Mercator/globe projection and Mapbox terrain.
- Feature picking, selection highlighting, extent drawing, draggable placement, and engine-level image capture.
- The same default on-map controls as MapLibre, governed by the same **Controls** menu: fullscreen, the reset pitch & bearing compass beneath it, a globe/Mercator toggle, the scale bar (following the project's scale unit) and attribution, with navigation and geolocate available but off by default. Mapbox GL JS has no globe control of its own, so the engine mounts a stand-in (`packages/map/src/mapbox-globe-control.ts`) that mirrors MapLibre's button markup; clicking it persists the projection into project preferences as on MapLibre. Terrain is a scene setting here (the Controls menu toggles Mapbox terrain directly, without a button, as on Cesium). The attribution control cannot be hidden, Mapbox draws its own logo, and the Maptoolkit logo is MapLibre-only.
- The on-map layer control (`maplibre-gl-layer-control`), with the same per-layer visibility, opacity, zoom-to and style-editor round trip to the store as on MapLibre. The control only needs the shared style API, so both 2D engines drive it through one host (`packages/map/src/layer-control-host.ts`); a `mapbox://` style cannot be fetched by the control, so the engine seeds it with the loaded style's own layers to tell basemap from project layers. Toggle it from **Controls** → **Layer Control**, as on MapLibre. Split panes never mount a second control.

Plugins

Plugins declare the renderers they support (`engines`); the Plugins menu and command palette grey out the rest. `app.getMap()` stays MapLibre-only, so a Mapbox-capable plugin reads the map through `getStyleMap(app)` (falls back to `app.getMapboxMap()`) and stays on the Style Spec surface both engines share — see [Supporting the Mapbox renderer](#) for the rules, the audit that enforces them, and how plugin-drawn layers are adopted. The vector import panel uses the existing store-based geometry bridge.

Available on Mapbox (the September 2026 plugin audit, each verified in a browser against an authenticated Mapbox map):

- **Layer Control, Basemaps, Deck.gl Layer** and **Components** (as before).
- **Web Services** — FEMA NFHL, NASA Earthdata (GIBS), US EPA EnviroAtlas, USGS National Map, USGS NLDI, Vantor, Planet Open Data, Earthdata GIS, OpenAerialMap, ArcGIS Hub, Socrata, CKAN, STAC Catalogs, Portolan, Source Cooperative, Natural Earth, Hugging Face, Esri Wayback, and GeoLens. The docked panels mount on the Mapbox map; the raster layers their controls create are adopted by the engine under the controls' own ids (one copy, store visibility/opacity/removal apply, rebuilt after a basemap swap). GeoLens *private* rasters need a per-request API key that only MapLibre's `setTransformRequest` can inject; the panel reports that they need the MapLibre renderer, while public rasters and vector data work.

- **Gridlines** and the **DGGS** grids (H3, S2, A5, DGGRID, DGGAL, OLC, Geohash, Tilecode), including cell labels and click identification. Mapbox Standard's root style carries no symbol layer to borrow a font from, so labels use Mapbox's `Open Sans Regular` / `Arial Unicode MS Regular` glyphs.
- **Time Slider** for XYZ, WMS, GeoJSON and TiTiler-served COG sources. A COG bound to the `gpu / wasm` engines (MapLibre-only tile protocols) is re-added through TiTiler; a mosaic manifest is dropped with a console warning. The pixel time series tool picks and marks its points on Mapbox too (the engine's click events and projected DOM markers), and Identify reads a pixel's band values, as on MapLibre.
- **NetCDF** sample markers, the 3D cube's "current view" and "draw" extents, and the COG spectral-profile click work on either 2D engine.
- **Timelapse**, including recording the Mapbox canvas to video.
- **Elevation Profile**, **USGS LiDAR** (the 3DEP index raster is adopted natively; point clouds already drew through deck.gl), and **Mapillary** (coverage vector tiles are plugin-owned native layers mirrored from the store).
- **Elements** (annotations) and **Dimensions**. MapLibre's `Marker` cannot be added to a mapbox-gl map, so pins, sticky notes and image cards are placed by an engine-neutral marker (`annotation-marker.ts`): MapLibre's own marker on MapLibre, a DOM element repositioned through `project()` on Mapbox.
- **Route Animation** (marker, trail and arrow image through the shared style API; rebinds across a renderer swap). The panel's layer picker lists inline GeoJSON line layers on Mapbox; layers whose geometry only lives in a map source (Add Vector Layer's GeoJSON mode) need MapLibre's `getData()`.
- **Clouds** and **Precipitation** (store tile layers; the frame scrub goes through the store on Mapbox rather than the instant `setTiles` shortcut).
- **Geo Editor**, including Sketches, in-place geometry editing of a store layer and loading map-view features into the editor. Geoman keeps its engine calls behind one map adapter, and the three members that build MapLibre objects (the `Marker` behind vertex handles and the cursor marker, the `LngLatBounds` behind the cut tool's query box, and the promise-style `loadImage` for the default marker icon) are swapped for mapbox-gl's on the Mapbox map (`geo-editor-mapbox.ts`); the toolbar's rotate and feature-properties popups come from `maplibre-gl-geo-editor`'s `createPopup` option, fed mapbox-gl's `Popup`. Text markers use Mapbox's `Open Sans Regular` glyphs when the style has no font to borrow.
- **Atmospheric Effects** (its overlay canvases mount in the Mapbox canvas container; the control container is lifted above them, as on MapLibre) and **Sun** (canvas night mask, raster layer and `setLight` all apply to mapbox-gl).
- **Flight Simulator**. mapbox-gl kept the free camera MapLibre dropped, so the Mapbox adapter places the eye directly — a `MercatorCoordinate` carrying the altitude plus `setPitchBearing` — where MapLibre converts through `calculateCameraOptionsFromCameraLngLatAltRotation`. Terrain, the suspended interaction handlers, the widened pitch ceiling and the exit view all behave as on MapLibre; `setCenterClampedToGround` has no counterpart and needs none, because the free camera positions the eye rather than the map center. One setting does nothing here: mapbox-gl 3 has no camera roll axis at all (its free camera orientation is

documented as representable with only pitch and bearing), so **Bank the horizon in turns** leaves the horizon level — the aircraft still banks, and a bank still turns it. The panel says so while the Mapbox renderer is primary.

- **Street View** (Google and Mapillary). Everything the upstream control touches is on the shared surface except the location marker: MapLibre's `Marker` reads `map._camera.transform` on every position update, so it threw on the first map click. `maplibre-gl-streetview` 0.8.0 takes a `createMarker` factory, and the plugin feeds it mapbox-gl's own `Marker` on a Mapbox host — the control still owns the marker element and its direction arrow, and only the positioning changes engine. A renderer swap rebuilds the control, so the marker follows whichever engine is primary.
- **Layer Swipe** for native style layers. The control drives both maps only through the surface the two engines share, so the one map it constructed itself — the clipped comparison pane, until now always a MapLibre one — comes from `maplibre-gl-swipe`'s `createMap` option (added in 0.13.0), fed mapbox-gl's `Map`. Two Mapbox specifics come with it: the pane is handed the access token explicitly (mapbox-gl reads its token from a global the app never sets, so a second map built without it renders nothing), and the basemap grouping is seeded with `basemapLayerIds` because a `mapbox://` style URL cannot be fetched — the same reason the layer control seeds its own. The panel lists each row by the name the Layers panel shows rather than by the style layer id it drives, because the engine publishes the same style-layer-id-to-name bridge MapController does (`packages/map/src/layer-labels.ts`); without it a row read `geolibre-mapbox-<id>-geojson-fill`. The deck.gl **raster provider stays MapLibre-only**: COG and `maplibre-gl-raster` layers can draw on the primary Mapbox map through its deck.gl adapter, but the swipe provider cannot mirror them into the comparison pane. The swipe panel therefore omits those layers. One known defect: changing the basemap while the swipe is active leaves the previous comparison pane — and the map inside it, a live WebGL context — orphaned on the Mapbox canvas. The swipe itself keeps working against the new basemap. The same sequence on MapLibre leaves one pane, so it sits in the Mapbox control lifecycle rather than the plugin; tracked in #2430.
- **GeoAgent**. Almost every tool already sits on the shared Style Spec surface; four did not, and each broke differently — `add_marker` built MapLibre's `Marker / Popup`, `set_projection` wrote `{ type }` (Mapbox takes a name string), `get_map_state` read `projection.type`, and `run_maplibre_script` handed user-authored code the wrong namespace. That matters more here than in a control that simply fails to mount: an agent run breaks mid-way, after it has already changed the map. `maplibre-gl-geoagent` 0.6.1 takes the engine as one option (`mapEngine`) and all four follow it, so the plugin names the host's engine once (`geoagent-map-engine.ts`) and hands over the whole mapbox-gl namespace — `run_maplibre_script` passes it straight to the script it runs. Agent overlays reach the Layers panel as plugin-owned rows the engine adopts, as on MapLibre. `set_sky / clear_sky` stay MapLibre-only: mapbox-gl has no `setSky` (it draws sky through a style layer), and the tool reports that instead of failing silently.
- **Overture Maps**. mapbox-gl 3.30+ reads `.pmtiles` archives itself, through a tile provider it fetches from `api.mapbox.com` (allowlisted in the desktop and web CSPs), so the plugin hands `maplibre-gl-overture-maps` its `nativePmtiles` option and the control adds plain https archive URLs instead of registering MapLibre's `pmtiles://` protocol; the inspection popup comes from the control's `createPopup` option, fed mapbox-gl's `Popup`. The Layers-panel mirrors are plugin-

owned rows (the engine neither compiles nor repaints them), so release, visibility, opacity, color and export all work as on MapLibre. The Style panel's 3D extrusion of the buildings theme is a MapLibre layer-sync feature and stays MapLibre-only.

No plugin is held back by a MapLibre-internal dependency of its own. What remains MapLibre-only is a map feature rather than a plugin.

Two engine changes came with the port and apply to every plugin: a store layer added while a Mapbox source is still loading is now synced when that source finishes (previously it waited for `idle`, which a map with an animated canvas source never reaches), and the shared paint builders scale a plugin-owned native layer's opacity by the store opacity instead of replacing it.

The offline (local PMTiles) basemap is MapLibre-only as well: its `pmtiles://` source protocol is not registered with Mapbox, so a Mapbox pane whose project basemap is an offline archive falls back to the default basemap (with a console warning). Pick a Mapbox style from the shared Basemaps panel instead. MapLibre custom protocols, tiled/streamed vector imports beyond the bridge's materialization limits, custom COG terrain, and other plugin-owned layers require additional adapters. **Custom terrain sources** are one of those with UI of its own: mapbox-gl has no `raster-dem` source a COG can back, so the Terrain settings dialog's "Terrain source" section (a COG URL, a local file, or a raster layer already on the map) is hidden on this renderer and explains why. Mapbox's own global terrain and the vertical-exaggeration slider still work.

Layers drawn with deck.gl need none: `@deck.gl/mapbox` targets Mapbox GL JS natively, so the engine reports `capabilities.deckOverlay` and the shared interleaved overlay binds to the Mapbox map through `app.getMapboxMap()`. Visible unsupported layers report an error on the map instead of being silently omitted. The heatmap and clustered point renderers compile to native Mapbox layers, with a clustered layer's authored filters applied to its data before clustering, as on MapLibre. The Style panel's symbology compiles to native Mapbox layers as on MapLibre: marker and KML icons, Geo Editor text markers, fill patterns and line decorations (generated sprites supplied through `styleimagemissing`), the inverted fill and the geometry generator (companion GeoJSON sources), the flat fill below a zoom-stepped extrusion, and attribute labels (de-duplicated labels read an aggregated companion source; the data-defined size, color, opacity, visibility and priority expressions apply). Layer blend modes are not reproduced, and the Style panel says so on a layer that sets one. Large GeoJSON (over 50,000 features) keeps mapbox-gl's own GeoJSON source: MapLibre serves such layers as vector tiles through a geojson-vt protocol, and mapbox-gl has no `addProtocol` hook, but its source already tiles the data in a worker with geojson-vt. With 200,000 points, adding the layer took about 2.1 s on Mapbox against 1.5 s on MapLibre, a restyle 0.9 s against 0.8 s, and a data edit 2.1 s against 1.2 s, with a shorter longest main-thread stall on Mapbox (0.8 s against 0.85 s).

Mapbox Standard is loaded as a local style import with a shared opacity setting. The Background card fades its land and water colors, labels (including ocean labels), 3D objects, and atmosphere while preserving project layers and Standard's configuration.

License and terms

GeoLibre itself is MIT licensed, but the Mapbox renderer depends on [Mapbox GL JS v3](#) (`mapbox-gl 3.32.0` at the time of writing), which is **not** open source. Mapbox GL JS v3 is distributed under the [Mapbox Terms of Service](#) and its [license](#); it requires an active Mapbox account, may only be used with an access token from that account and with the relevant Mapbox products, and its terms restrict altering the SDK's billing, accounting and data-collection code. Usage-based billing and Mapbox's attribution requirements depend on how the Mapbox services are used under your account and the applicable terms; consult those terms before enabling the renderer in a product. The SDK is a runtime dependency of `@geolibre/map`, so npm consumers of that package and the desktop and web distributions receive it even when MapLibre stays the active renderer, but no Mapbox code runs (and no Mapbox service is contacted) until a Mapbox pane is opened.

Loading and size

Mapbox's JavaScript and CSS are imported only when a Mapbox pane mounts. The production build gives them a separate chunk and excludes them from the PWA's initial precache. MapLibre startup therefore does not download the Mapbox engine. Desktop/web distribution artifacts still include it. With Mapbox GL JS 3.30.0, the engine and stylesheet add approximately 1.91 MB raw, or 532 KB with gzip; this excludes map tiles and other service responses.

Add Data compatibility

The Add Data menu waits for Mapbox to finish loading before accepting an import, so early clicks cannot lose a panel-opening request.

The Add Data menu and command palette withhold loaders that require an unimplemented MapLibre protocol or custom render pass. These entries are visible but disabled in the menu with a Mapbox compatibility hint: MBTiles and Gaussian Splatting. Cesium Ion and CZML scene loaders remain Cesium-only; KML / KMZ opens on every renderer, going through the host KML importer (the drag-and-drop path) off the globe.

Deck.gl Layer, 3D Model, and DuckDB are enabled: the first two render through the shared interleaved deck.gl overlay (as 3D Tiles already did), the third through the DuckDB panel's own deck.gl overlay. Like every deck.gl overlay they hold the map in the Mercator projection while their layers are shown, so a globe view snaps to Mercator when one is added. Deck.gl Layer data is stored inline in the project, as on MapLibre. The DuckDB panel remounts on the live map after a renderer swap; results that were only cached in the panel are redrawn when it reopens, and a project-restored query layer needs its query re-run, exactly as on MapLibre.

FlatGeobuf uses the shared vector importer on Mapbox. ArcGIS vector-tile services retain their resolved tile sources, service styles, classification filters, visibility, and opacity. STAC supports catalog browsing, extent search, bbox drawing, footprints, and selection on both MapLibre and Mapbox; remote vector PMTiles and Zarr assets can be added on either.

NetCDF/HDF files and directly readable remote files render a selected plane as an image on both engines; cubes with a time axis, and Kerchunk references, go through the Zarr renderer on Mapbox exactly as on MapLibre.

Browser validation

The September 2026 audit opened every one of the 37 Add Data entries that was enabled, checked the existing disabled entries, and exercised the public data paths below with an authenticated Mapbox map. A mounted panel alone is not a successful import; the table records the level of verification. Backend and service restrictions are included explicitly.

Panel	Result
Vector	US states GeoJSON: 52 features imported and rendered
Raster	Public DEM GeoTIFF: GPU raster displayed
Delimited Text	US cities CSV: 109 points imported and rendered
CAD	US states DXF: 58 entities discovered; EPSG:5070 import exercised
File Geodatabase	Panel opens; its local GDAL/sidecar workflow requires Desktop
Geotagged Photos	EXIF sample JPEG: one located photo imported
GPX	Fells Loop: 86 waypoints and one route; both native sources created
Encoded Polyline	Precision-5 sample: one line imported and rendered
MBTiles	Disabled: local custom protocol has no Mapbox adapter
OSM PBF	Monaco extract: 4,249 points, 4,002 lines, and 2,341 polygons imported and displayed
XYZ	USGS imagery sample: native raster source mounted
WMS	USGS NAIP sample: native raster source mounted
CSW Catalog	Open Canada catalog searched; Manitoba Economic Regions imported as eight GeoJSON features
WFS	MapServer continents service imported; the GeoServer sample was blocked by its remote service
WMTS	EOX Sentinel-2 cloudless sample: native raster source mounted

Panel	Result
OGC API - Features	pygeoapi lakes sample: 25 features imported and rendered
OGC Vector Tiles	PDOK BGT sample: native vector-tile source mounted
ArcGIS	4,186 city features rendered; Santa Monica parcels rendered using all seven service style layers
GeorSS	USGS daily earthquake feed: 34 features imported (the live count changes)
STAC	Earth Search connected; 20 Sentinel-2 search footprints added
Video	Mapbox coastal video sample: native video source mounted
Deck.gl	Scatterplot sample (Manhattan points) rendered through the shared deck.gl overlay; visibility and opacity follow the layer store
GeoParquet	US states: 52 features imported and rendered
FlatGeobuf	Countries: 179 features imported through the shared vector bridge
PMTiles	Remote vector archives use native Mapbox sources; Tilezen's nine source layers and Mapbox's earthquake archive rendered
Zarr	CarbonPlan climate sample added through the panel: the custom layer mounts on the Mapbox map and loads its pyramid (6 levels, band/month axes). Checked without a paintable token, so pixel output was not confirmed; the renderer's Mapbox support is upstream's
NetCDF / HDF	Air-temperature file: selected time slice added as a native image
LiDAR	Autzen COPC rendered (10,653,336 archive points); the small PDAL COPC fixture loads 1,065 points
Gaussian Splattin g	Panel opens; custom rendering unsupported and entry disabled
3D Tiles	AGI headquarters tileset renders through deck.gl; altitude placement, visibility and restoration have regression coverage
Cesium Ion	Disabled: Cesium-only
CZML	Disabled: Cesium-only
KML / KMZ	Imported through the host KML importer as GeoJSON, ground-overlay, and model layers, the same path a dropped file takes
3D Model	Shanghai sample model placed through the scenegraph builder
DuckDB	NYC sample database queried; the result layer rendered and survived a MapLibre → Mapbox renderer swap

Panel	Result
Postgre SQL	Panel explains its Desktop/Martin requirement; no database connection tested
Apache Iceberg	Panel opens; no table/catalog connection supplied for an import

An opt-in regression suite repeats the FlatGeobuf, ArcGIS vector-tile, STAC, and menu-boundary checks in light and dark themes:

```
# Supply a public token in the environment before running.
npm exec -- playwright test e2e/mapbox-add-data.spec.ts
```

The tests skip when `MAPBOX_TOKEN` is absent. They use live public services; remote-service availability is part of these integration checks.

PMTiles and 3D adapters

The PMTiles panel keeps its archive discovery and source-layer selection UI, but Mapbox imports go through the layer store. Each selected source layer has an independent Mapbox source and editable style. This path supports remote HTTP(S) **vector** archives whose URL path ends in `.pmtiles`; raster archives, local files, and the offline PMTiles basemap remain unsupported and receive a load error.

LiDAR uses the existing point-cloud loader and its separate deck.gl canvas. Standard 3D Tiles use the shared interleaved overlay, retaining the same project source records as MapLibre. Both require Mercator. Tileset altitude offsets move the geometry and traversal bounds together, so lowered tiles remain visible when zooming in or reopening a saved project. The 3D Tiles panel reports loading and fetch errors; visibility, opacity and removal follow the layer store.

The opt-in `e2e/mapbox-archives-3d.spec.ts` uses public PMTiles, COPC and AGI tileset URLs and checks save/reopen in light and dark themes. Set `MAPBOX_TOKEN` at runtime, then run it with `npm playwright test e2e/mapbox-archives-3d.spec.ts`. Google Photorealistic and authenticated I3S services require their own credentials; those services and every 3D Tiles extension are not covered by this test.

<https://geolibre.app/cesium-renderer/>

Cesium renderer

Choose **View** → **Rendering engine** → **Cesium** to render the primary workspace as a native CesiumJS globe. Cesium is also available from the rendering-engine menu in every split pane, so a project can place a globe beside any of the other three engines. Primary and pane choices are saved with the project.

Credentials

The globe works without a Cesium ion token: it drapes the project basemap as its base imagery, uses keyless Terrarium elevation (or a configured COG) for terrain, and shows a hint that links to the token setting. A token adds Cesium World Terrain, Ion World Imagery as the fallback for a basemap with no raster form, and access to ion-hosted assets. The hosted web version bundles a demo token; the desktop and mobile apps need your own for those features.

Enter the token under **Settings** → **Environment Variables** → **Cesium Ion token**. It is stored on the current device rather than in the project file. Developers can alternatively set `CESIUM_TOKEN` or `VITE_CESIUM_TOKEN` at build time. See [Optional 3D globe credentials](#).

Supported paths

- Styled GeoJSON, including categorized, graduated, rule-based, expression, proportional, marker, cluster, label, pattern, extrusion, and Z-aware rendering.
- XYZ, WMS, WMTS, COG, raster PMTiles, local MBTiles, image overlays, and other tile sources connected through GeoLibre's protocol bridge.
- Vector tiles, vector PMTiles, and vector MBTiles draped through a hidden MapLibre renderer so their Style Specification output is preserved.
- Native 3D Tiles, Cesium ion tilesets and imagery, CZML, KML/KMZ, I3S scene layers, supported point clouds, and Gaussian-splat tilesets.
- Globe-native picking and highlighting, extent drawing, draggable placement, screenshots, print layouts, video and camera tours, terrain-aware elevation profiles, and environment effects.

The Layers panel retains unsupported content rather than deleting it. Such a layer is marked as unavailable for the active engine and returns when the project is switched to a compatible renderer. Plugins follow the same model: only plugins whose `engines` list contains `cesium` remain active.

Scene and camera

The globe provides 3D, 2D, and Columbus scene modes, terrain, native globe lighting, and a camera translated to and from GeoLibre's shared longitude, latitude, zoom, bearing, and pitch state. Mixed panes synchronize by ground resolution rather than raw zoom, keeping their apparent scales aligned even when panes have different sizes.

Loading and architecture

Cesium's JavaScript is loaded only when the first Cesium canvas mounts. Its Workers, Assets, and Widgets are staged with the application and resolved from `CESIUM_BASE_URL` ; choosing another engine unmounts the Cesium canvas and releases its rendering resources.

`CesiumEngine` implements the shared `MapEngine` interface, and `CesiumLayerSync` translates project layers into Cesium data sources, imagery layers, and primitives. See [3D globe view \(CesiumJS\)](#) for the detailed implementation, compatibility boundaries, camera conversion, and persistence model.

CesiumJS is open source under the Apache 2.0 license. Cesium ion is a separate hosted service with its own account terms and usage limits.

<https://geolibre.app/arcgis-renderer/>

ArcGIS renderer

Choose **View** → **Rendering engine** → **ArcGIS** to render a project with the [ArcGIS Maps SDK for JavaScript](#). The engine is also available from the rendering-engine menu in each split pane. MapLibre remains the default. Projects save the primary and secondary renderer choices, and the Python and iframe APIs accept `arcgis` as a renderer name.

Nothing is bundled

The SDK is **not** part of GeoLibre's build. `@arcgis/core` is 84 MB unpacked across 18,810 files, and bundling it would grow the web build, the Python wheel and the desktop installers by tens of megabytes. Instead the engine imports the modules it needs from Esri's versioned ES-module CDN the first time an ArcGIS pane mounts (`https://js.arcgis.com/<version>/@arcgis/core/...`), the same way the PostGIS and Sedona SQL engines are fetched from jsDelivr. Only the adapter code in `packages/map/src/arcgis-*.ts` ships with GeoLibre.

Consequences:

- The first ArcGIS pane needs network access, on the desktop too. The service worker caches the SDK modules and stylesheet after first use (`geolibre-arcgis-sdk` in `vite.config.ts`), so the SDK itself boots offline afterwards like the other CDN-loaded engines. Esri basemaps, tile services and other remote layers still need the network, as on every renderer.
- The SDK version is pinned in `packages/map/src/arcgis-sdk.ts` (`ARCgis_SDK_VERSION`). Bumping it is a deliberate change; see [Maintenance](#).
- Esri documents the ES-module CDN as a prototyping path and logs "Only use ES modules from ArcGIS CDN for testing" once per session; its supported production path is an npm build, which is exactly what this integration avoids for size. The first load pulls a few hundred small modules, so the first ArcGIS pane of a session takes longer to appear than a Mapbox one.
- The content-security policies of the desktop app (`tauri.conf.json`) and the Docker image (`docker/nginx.conf`) allow-list `https://js.arcgis.com/` in `script-src` and `font-src` (the SDK's icon and text fonts). The SDK's stylesheet is fetched as text and inlined, since neither policy allows external stylesheets.

API key

The renderer works without a key. New projects then show keyless **Esri World Imagery** as the ArcGIS basemap; a basemap you pick from the shared **Basemaps** panel is translated into tiles the SDK can draw (the same translation the 3D globe uses), with World Imagery as the fallback for a style that has no tiled form. Your layers draw through the SDK's own layer classes.

An **ArcGIS API key** adds Esri's basemap styles. Create one in your ArcGIS account (ArcGIS Online or ArcGIS Location Platform) under **Content** → **New item** → **Developer credentials** → **API key credentials** with the **Basemaps** privilege, then paste it into **Settings** → **Environment Variables** → **ArcGIS API key** and click **Save Settings**. Like the Mapbox and Cesium tokens, it is stored on this device, outside the project file. Alternatively, launch the development server with `ARCGIS_API_KEY` in its environment. Key changes recreate ArcGIS maps. Requests made with the key are metered against its account and subject to Esri's terms; basemap requests have a generous free allotment.

With a key, new projects use **ArcGIS Streets** for the ArcGIS renderer. The **Change background** picker offers Esri's Streets, Navigation, Topographic, Light Gray, Dark Gray, Imagery, Imagery (no labels), Oceans, Outdoor and OpenStreetMap styles when ArcGIS is active and a key is configured. Selecting one preserves the camera. The choice is saved as `preferences.map.arcgisBasemap` (an Esri basemap style id such as `arcgis/streets`, `arcgis/imagery` or `osm/standard`); selecting a basemap from the shared **Basemaps** panel while ArcGIS is active clears it, so the pane follows the shared MapLibre/Cesium basemap again. Without a key the override draws Esri World Imagery instead.

Supported paths

- Native GeoJSON, including the vector importer's materialized data, FlatGeobuf and GeoParquet imports, and ArcGIS feature layers already loaded into the project. Point, line and polygon symbology, fill and stroke opacity, circle radius, labels (field or expression, size, colour, halo, placement, offset, rotation, case transform), and the data-driven colour modes — categorized, graduated, rule-based, expression and simplestyle — all render. The SDK has no MapLibre Style Spec, so the engine evaluates the same MapLibre expressions `@geolibre/core` builds for the 2D map *per feature* with the style-spec engine and bakes the answers into the features; the layer's renderer is a unique-value renderer over the resulting symbol keys. Layer filters, quick filters, the time slider's filter and the embed filter are applied the same way before features reach the SDK. Zoom-dependent expressions (metre-unit strokes, per-rule zoom ranges) are re-evaluated when the integer zoom changes.
- The symbology pack draws through companion layers built from the filtered features: the **inverted fill** as a world mask with the polygons cut out (clamped to the Web Mercator latitude limit, as on Mapbox), the **geometry generator** (centroids, bounding boxes, convex hulls, buffers) in its own fill, stroke and proportional circle size, and **line decorations** as a CIM marker line that repeats the shape at the style's spacing and turns arrows to follow the line (a flat `MapView` only; the SDK draws no CIM line symbols in a scene). **Label de-duplication** labels the aggregated points from a label-only layer. The label size, colour, opacity and visibility expressions are evaluated per feature and grouped into one label class per resolved size and colour, since the SDK's label symbols are not data-driven (a continuous ramp is binned into a few dozen classes). The layer's opacity also applies to its labels, so an opacity expression cannot make a label more opaque than its layer. Companion layers are not identified or selected.
- HTTP(S) raster tiles (XYZ, WMTS tile templates), WMS (the GetMap template is split into the SDK's `WMSLayer` description), and vector tiles with named source layers (drawn by the SDK's `VectorTileLayer` from the same style layers the Mapbox engine compiles). Markers, line

decorations and fill patterns are GeoLibre's generated images, baked into a sprite sheet the SDK reads through a request interceptor; labels use Arial from Esri's public glyph service, so they need the network. Past a source's maximum zoom the SDK magnifies its last tiles, patterns and decoration spacing with them, where MapLibre keeps them crisp. A raster template `WebTileLayer` cannot express — a `{bbox-epsg-3857}` request, a TMS scheme, `{-y}` or `{quadkey}`, a 512 px tile size, a source `minzoom / maxzoom`, or several templates — draws through a custom tile layer that requests each tile as MapLibre would, cropping 512 px tiles and overzooming past `maxzoom`. A template on a registered MapLibre protocol (the desktop app's CORS-exempt `geolibre-wms://` WMS fetcher, say) is asked of that protocol's handler, as the globe does.

- **Add Data → ArcGIS Layer** stores cached MapServer and ImageServer services as tile templates and dynamic ones (sublayers, a rendering rule, no usable cache) as `/export / / exportImage` bounding-box templates, both drawn as above. FeatureServer layers, and any service recorded as `type: "arcgis"` (hand-authored or Python records), draw through the SDK's own `FeatureLayer`, `TileLayer`, `MapImageLayer` and `ImageryLayer`; a single MapServer sublayer (`MapServer/2`) is a `FeatureLayer` too. A service layer draws with its layer style, as on the 2D map; its identified features can be highlighted, and its features read back as GeoJSON (paged past the service's per-request record limit, up to 50 requests). A FeatureServer layer's filters (quick filters, the expression filter, the time and embed filters) become the service's `SQL definitionExpression`; a filter with no SQL form is reported in the map's banner and the service draws unfiltered.
- Georeferenced images, placed by their four corners through a control-point georeference, so rotated and skewed fits land where they do on MapLibre.
- Shared layer/group visibility, opacity and ordering; synchronized or independent split-view cameras; the project's zoom and bounds constraints.
- The Style panel's raster brightness, contrast, saturation and hue sliders, as the SDK's CSS-filter `effect` on raster layers, and every layer's blend mode as the SDK's `blendMode` (`add` is the SDK's `plus`). A `SceneView` ignores the raster effects, and blends only tiled and imagery layers.
- Geo Editor text markers and annotation text, drawn as text at their points in their own colours.
- Feature picking (click identify with the layer's popup template and field visibility; **Identify visible layers** groups every layer's hits, including WMS GetFeatureInfo, Time Slider pixels, COG/NetCDF pixels and DuckDB layers), hover map tips (the layer's hover fields, picked from the store's features as on Mapbox), geotagged-photo popups on a click without the Identify tool, selection highlighting, extent drawing, draggable placement, and engine-level image capture. **Layers → Select features** (single, rectangle, polygon, freehand and radius) runs the same gestures as on MapLibre on the primary map. The status bar's pointer elevation reads the ground under the cursor from the scene's elevation surface, or (with consent) the same remote lookup the other maps use.
- **GPS Tracking** (the position marker, accuracy circle and track, following the position), **Add comment**, the Georeferencer's **Link on map**, and the Pixel Time Series and NetCDF sample markers work through the engine's render surface and click events.
- Plugin controls hosted on the map receive MapLibre's `move`, `zoom`, `rotate`, `click` and `mousemove` events, so readouts such as View State and the minimap follow the camera. Story chapter layer fades run over the chapter's transition.

- **Search places** flies to places and coordinates with a temporary marker, and frames H3 cells with a filled outline. Clearing the search removes its highlight without removing a selection made elsewhere.
- Point heatmaps use the shared color ramp, radius, intensity and optional weight field in both 2D and 3D. Esri's density kernel differs from MapLibre's, so the visual density can differ; SceneView caps the radius at 112 points. Clustering uses native count labels and the configured radius and maximum zoom in 2D. Built-in and custom SVG fill patterns also render in 2D, with per-feature fill opacity and independent outlines.
- The built-in controls the **Controls** menu governs, as the SDK's own widgets: fullscreen, compass (resets rotation), zoom (navigation) and locate (geolocate), plus a globe/Mercator toggle and terrain (see [2D and 3D](#)). The scale bar is the 2D map's own: it measures around the view's centre with the active body's radius, in metric, imperial or nautical units, on a flat map and in a scene. Attribution is drawn by the view itself (`attributionVisible`); Esri requires it and it cannot be hidden.
- The **Print Layout** atlas frames each page by driving the view's camera north up, with the fit MapLibre's `fitBounds` computes, and waits for the view to finish drawing before the capture. The mask outside the current feature is painted onto the capture rather than drawn on the live map, and a fixed scale is corrected up to three times, aiming for 0.5 % of the requested one, since the SDK rounds the zoom it is given.
- **Controls → Layer Control** toggles the native ArcGIS layer list, enabled by default on the primary map like the shared plugin. Its visibility toggles update the project and the sidebar, and sidebar changes update the list. Mixed-geometry GeoJSON records have one entry for all their parts. Temporary search and selection highlights are omitted. Split panes keep the control hidden by default and retain their independent layer visibility.
- Plugins that only add store layers: **ArcGIS Hub**, **Socrata** and **CKAN** search, the **OSM Downloader**, **Source Cooperative**, **Hugging Face**, **Earthdata GIS**, and the **Weather** overlays (Clouds and Precipitation).
- Plugins built on a MapLibre control, through the map the host hands its controls (see [Plugin controls](#)): the **Web Services** catalogs (FEMA NFHL, NASA Earthdata, US EPA EnviroAtlas, USGS National Map), **Historical Imagery** (Esri Wayback), the **DGGS** grids (H3, S2, A5, DGGRID, DGGAL, OLC, Geohash, Tilecode) and the **Graticule**, and the imagery catalogs with footprints (STAC, OpenAerialMap, Vantor, Fields of the World, Satellite Embeddings, IGN LiDAR HD, GeoLens). Footprint clicks and hover, selection outlines and box drawing work; a GeoLens raster that needs an API key still requires MapLibre.
- **Controls** menu entries and plugins whose plugin needs a MapLibre, Mapbox or Cesium map (Atmospheric Effects, Sun, Route Animation, Flight Simulator, Directions, Reverse Geocode, the Geo Editor, Annotations, Swipe, Street View, Timelapse and others) are greyed out with the reason while ArcGIS is the primary renderer.
- If the SDK cannot be loaded from the CDN, the map's banner offers **Retry**, which reloads the page: the browser keeps a failed module import for the life of the page, so only a reload can fetch it again.

2D and 3D

The SDK draws flat maps and 3D scenes through two different view classes, so the ArcGIS pane picks one from the project's map preferences and rebuilds the view when the choice changes. The camera (centre, zoom, bearing and pitch) carries over.

Projection	Terrain	View
Globe	off or on	<code>SceneView</code> , global (a 3D globe)
Mercator	on	<code>SceneView</code> , local (a flat 3D map)
Mercator	off	<code>MapView</code> (a flat 2D map)

New projects use the globe projection, so an ArcGIS pane opens as a globe. The globe button under the compass switches projection (as on MapLibre, a split pane's button only switches that pane), and **Controls** → **Terrain** turns terrain on or off. The 3D modules (`views/SceneView` and the elevation layers, close to a megabyte) are not part of the first load: a flat ArcGIS map fetches them in the background once the page is idle, so the first switch to a globe does not wait on the network. While a new view loads, the previous one stays on screen and is swapped out once the new view's basemap has drawn; data layers and terrain finish loading on the new view.

In a scene:

- The camera tilts (right-drag, or the project's saved pitch), limited by the project's maximum pitch. The status bar shows the camera's altitude.
- Lighting follows the camera (the SDK's virtual lighting), so the whole visible map is lit. The SDK's default simulated sun would leave part of the globe on the night side.
- Terrain drapes the map over Esri's [World Elevation](#) service. It needs no API key. **Controls** → **Terrain exaggeration** scales the heights.
- **Controls** → **Terrain exaggeration** also accepts a local or remote COG DEM in EPSG:3857 or EPSG:4326, using the same reader as the other engines. The local file stays on the device. The source and exaggeration survive switches between flat maps, local scenes and the globe during the session; they are not saved in the project. Missing DEM pixels and areas outside the COG use zero metres. **Use global terrain** restores Esri's World Elevation.
- Polygon layers whose style extrudes (the Style panel's **3D extrusion**) draw as extruded 3D shapes with the same height and colour as MapLibre's fill-extrusion: the height property times the height scale (or the advanced height expression) is the top and the base height the bottom, and the colour follows the layer's categorized, graduated or rule-based symbology, the advanced colour expression, or the extrusion colour. On a 2D `MapView` they stay flat fills.
- Identify, selection highlighting, extent drawing and capture work as in 2D. The project's zoom range becomes the scene's altitude range on a globe.
- **3D (Z values)** places vector coordinates at their absolute altitude, with the configured vertical scale and offset. Selection highlights use the same transformed coordinates. Source data stays unchanged.

Adding data

Files dropped onto the map, the host importers behind **Add Data → FlatGeobuf Layer / GeoParquet Layer / KML / KMZ / Delimited Text**, and the **XYZ, WMS, WMTS** and **ArcGIS Layer** dialogs all work on the ArcGIS map. The **Vector Layer** panel uses the shared store bridge for bounded vector imports; large streaming GeoParquet still requires MapLibre. **Raster Layer** opens a host dialog for a local GeoTIFF or HTTP(S) URL.

GeoTIFF/COG files and URLs render through the existing WebAssembly COG tiler and a native ArcGIS tile layer in 2D and 3D. Saved RGB bands, continuous color ramps, stretch, gamma, nodata and opacity are honored; edit them in the Style panel. Browser files last for the session; desktop local paths can be reopened on the same device. GPU-only classified/custom color ramps still require the MapLibre raster control.

deck.gl layers

Add Data → Deck.gl Layer / 3D Model works on the primary flat map and local 3D scene. The shared overlay renders saved visualizations and models, follows visibility, opacity and ordering among deck.gl layers, and releases its GPU resources on a renderer switch. The shared overlay is one native SDK layer; deck.gl layers cannot be interleaved individually with native ArcGIS layers. Native feature Z rendering stays owned by ArcGIS.

The adapter uses the MIT-licensed compositor from `@deck.gl/arcgis 9.4.0` with CDN-loaded SDK classes, avoiding a bundled `@arcgis/core` dependency. Local 3D rendering uses the upstream experimental camera approximation: it can drift at extreme camera angles and does not share the terrain depth buffer. Global scenes and secondary panes do not host the overlay; the menus and layer badges reflect that restriction. Switch back to a flat map or local scene to restore the layers.

Tile archives

Add Data → PMTiles loads remote vector (MVT) and raster archives directly through the shared PMTiles reader. Native vector tiles retain polygon, line and point styles; raster archives use native tiled imagery and resample their last native level when zooming in. Existing in-memory archives in the shared registry also work. **Add Data → MBTiles** uses the desktop file reader for both vector and raster tiles. MBTiles still requires the desktop app.

The vector adapter owns a request interceptor per layer and removes it when the layer is replaced or removed. Tiles are read on demand, including cancellation; synthetic tile addresses never go to the network. MLT encoding and archive text labels are not supported. The adapter reads PMTiles zoom limits from the archive header, including for older projects that omit those limits.

Each vector source layer uses a separate native `VectorTileLayer` so its style and visibility can be controlled independently. The `PMTiles` reader is shared, but the SDK decodes tiles separately for each native layer. Archives with many source layers therefore use more decoding work and memory than the shared MapLibre source; enable only the layers needed for the current view.

Zarr and NetCDF grids

Add Data → Zarr loads numeric Zarr v2/v3 variables through native tiled imagery. The reader supports regularly spaced, one-dimensional spatial axes, ascending or descending latitude, 0–360° longitude, CF scale/offset and fill values, and integer selectors for other dimensions. The Time Slider uses the same selector path. Projected grids require a CRS or proj4 definition through the import API. Curvilinear coordinates and automatic multiscale selection are not supported; a pyramid level can be selected by its variable path. The plugin `queryZarrLayer` API (point values and region statistics) is not yet supported for native ArcGIS grids.

Add Data → NetCDF uses the existing file dialog. Image slices render as native image overlays; kerchunk-backed grids share the tiled Zarr reader. Reference manifests are preserved in the layer source for project restoration. Registered local Zarr stores remain session-local. Reads return bounded windows and retain at most 32 MiB of compressed data per layer; coarse views of large untiled arrays can still require many chunk requests.

Plugin controls

A plugin control mounted on the ArcGIS map receives a MapLibre-shaped map. Camera, projection, pointer events and the DOM go through the view. Its style calls (`addSource` , `addLayer` , `setPaintProperty` , `getStyle` , ...) record into a style that is never drawn by the SDK directly; instead:

- A layer the plugin mirrors into the GeoLibre store (a Web Services WMS layer, footprints registered with `registerExternalNativeLayer`) is drawn by the engine from that store record, like any other layer.
- Any other GeoJSON layer (a grid, a selection outline, a draw preview) is drawn as graphics in one overlay above the project layers. Fill, line, circle and text layers are drawn, with filters, zoom ranges and data-driven or zoom expressions evaluated per feature as MapLibre evaluates them. Icons, fill patterns, extrusions and text along lines are not drawn, and labels have no collision handling.
- Layer-scoped events (`map.on("click", layerId, ...)` , `mouseenter` , `mouseleave`) and a point `queryRenderedFeatures` answer from whichever of those two draws the layer. A box query answers empty.
- `dragPan.disable()` stops the view panning, so a control's own box or line drawing receives the drag. Feature state, images and the other interaction handlers are inert.

A plugin that draws through custom layers, canvas sources, `Marker / Popup` , raster or vector-tile sources it does not mirror, or MapLibre-only APIs still declares no ArcGIS support. Plugins read this map through `getControlMap` in `packages/plugins/src/plugins/style-map.ts` .

Adapted plugin panels

LiDAR, **DuckDB** and ordinary **3D Tiles** render through deck.gl on the primary flat map or local scene. Google Photorealistic and I3S tiles still require another renderer. The global globe and secondary panes do not host these overlays. LiDAR keeps the existing COPC/EPT streaming and styling controls; its terrain toggle delegates to the host terrain setting. Saved URL LiDAR and 3D Tiles layers restore when the view is rebuilt. Browser-local point cloud files and cached DuckDB query results retain their existing session lifetime; reopen the source/query when necessary. Scene overlays have the same experimental alignment and depth limitations described above.

Not supported yet

- Gaussian splats and Cesium-only sources, STAC search results and video overlays. **Add Data** greys these out while ArcGIS is the primary renderer, and the layer panels badge such layers **No ArcGIS**. A layer a plugin draws on MapLibre only (Planetary Computer, say) is named in the map's banner too.
- An Esri vector tile service's own icon and label layers (its stored style keeps no sprite or glyphs), in any view.
- Navigation limits differ slightly from MapLibre's: a view that settles past the zoom range or the restricted bounds eases back inside them (on a flat map the wheel also stops at the zoom limits), the bounds hold the view's centre rather than the whole viewport, and the SDK always wraps around the antimeridian, so turning off world copies only clamps the views the app applies.
- Measure and the geometry editor draw through the MapLibre/Mapbox style API; the Controls menu greys Measure out, and editing a layer's geometry says the renderer does not support it.
- Diagrams and the label priority expression (the SDK has no per-feature label priority); clustering, fill patterns, blend modes and line decorations in a `SceneView`; extrusion on a flat map. The Style panel names whichever of these a layer turns on.
- Arbitrary MapLibre custom layers and rendering APIs still require adapters. The primary view hosts DOM controls with navigation methods; Vector, LiDAR, DuckDB and 3D Tiles have explicit rendering bridges. Layer Control delegates to ArcGIS's native layer list.
- Video overlays. The SDK does not support clustering or picture-fill patterns in `SceneView`: scenes retain individual point symbols and solid polygon fills. Heatmap labels are also unsupported in scenes. Markers (built-in shapes, custom SVG, KML icons) do draw, as picture symbols baked from the same sprites MapLibre uses.

Testing

`tests/arcgis-layers.test.ts` covers the layer compiler and basemap planner, `tests/arcgis-engine.test.ts` drives the engine against a fake SDK, and `tests/arcgis-renderer.test.ts` covers the project format, settings and loader boundaries. None of them touch the network. `e2e/arcgis-renderer.spec.ts` is the opt-in browser check against Esri's real CDN: set `ARCGIS_API_KEY` for the full

suite, or `ARCGIS_E2E=1` for keyless coordinate and H3 search coverage. With `ARCGIS_E2E=1`, `e2e/arcgis-offline.spec.ts` also verifies a fresh keyless boot under the production Tauri CSP, cached SDK/inline-data startup with the browser offline, and a visible error when the CDN is unavailable on first use. The CSP test runs in Chromium with the exact policy header; it does not replace native webview testing on each desktop platform.

License and terms

The ArcGIS Maps SDK for JavaScript is distributed by Esri under its own [terms of use](#), not an open-source license. GeoLibre does not redistribute it; the SDK is fetched from Esri's CDN by the user's browser at runtime, and use of Esri's basemaps and location services is governed by the account the API key belongs to.

<https://geolibre.app/self-hosting/>

Self-Hosting & Private Data

GeoLibre is a client-side application: the map, the layers, and the analysis all run in the browser or in the desktop shell, and nothing is uploaded to a GeoLibre service unless you explicitly use an optional online feature (see [Privacy Policy](#)). That makes it a good fit for organizations that cannot put their data on a public host at all: Indigenous and local communities managing their own monitoring data, health and humanitarian programs, protected species records, or any deployment where the data must stay on infrastructure the organization controls.

What GeoLibre does **not** do is host your data. It reads data from wherever you point it. So a private deployment is really two questions:

1. **Where do the datasets and projects live, and who is allowed to read them?**
2. **How does GeoLibre reach them from the browser?**

This page answers both. The short version:

Recommended setup

Host your data with **GeoLens**, a self-hosted spatial catalog (FastAPI + PostGIS) with accounts, per-dataset permissions, and OGC/STAC APIs. Host the **GeoLibre web build** next to it on the *same origin*, behind the *same* authentication layer. Then use GeoLibre's built-in **GeoLens plugin** to search the catalog and add datasets to the map. Every byte is served from your own origin to an authenticated browser and to nowhere else, credentials are sent only to that origin, and there is no CORS to configure.

Why same-origin matters

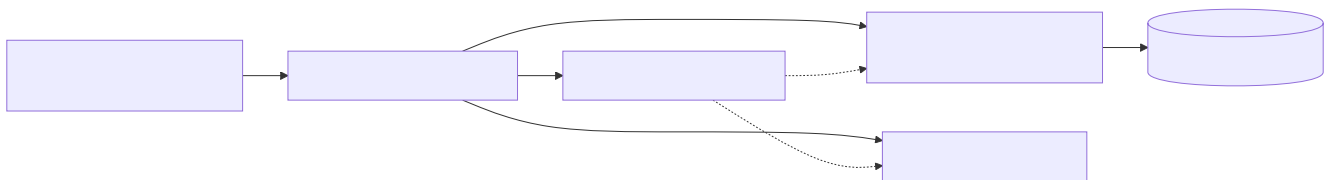
Everything GeoLibre fetches is fetched by the browser, so the browser's rules apply:

- **Cookies.** GeoLibre's requests use the browser default credentials mode (`same-origin`). A session cookie set by your SSO layer is sent automatically when the data URL is on the **same origin** as the app, and is **not** sent when it is on a different origin. So a cookie-gated dataset works out of the box on one origin, and cross-origin it fails whatever the server does, because the request arrives with no cookie for the server to check. Same-origin is necessary but not sufficient: the cookie's own `Path` still has to cover the URL being requested, so set it to `Path=/'` for a layout that spreads the app, the API, and the project files across sibling paths.
- **CORS.** Same-origin requests need no CORS headers at all. Cross-origin ones need the data host to allow your GeoLibre origin explicitly.
- **Tile requests.** MapLibre issues raster and vector tile requests itself. Those follow the same cookie and CORS rules, which is why a tile endpoint gated by a session cookie only works when it is same-origin.

- `?url= projects`. A `.geolibre.json` loaded through the `?url= deep link` is fetched the same way, so a project file behind your SSO layer loads when the app is served from that same origin, and fails with a network/CORS error when it is not.
- **Content Security Policy.** The Docker image allows `https:` in `connect-src`, plus loopback for local development, so a self-hosted data server reached through it must be over **HTTPS** (plain `http://` works only on `localhost / 127.0.0.1`). The desktop app additionally allows plain `http:` to any host, so it can reach a self-hosted Ollama, SamGeo, or other service on your local network without HTTPS (issue #2620).

Putting GeoLibre and the data on one origin turns all five of these from configuration problems into non-problems.

Architecture



One hostname, one login, three paths. GeoLibre at `/gis` reads the catalog at `/api`, the tiles at `/api/tiles/...`, and any saved projects at `/projects/...`, all with the visitor's own session.

1. Host the data with GeoLens

GeoLens is an open-source, self-hosted geospatial data catalog and map builder built on FastAPI, PostGIS, React, and MapLibre. It is the piece GeoLibre deliberately does not try to be, and the two line up well:

GeoLens provides	GeoLibre consumes it as
Dataset search (fuzzy plus optional semantic ranking over pgvector)	The catalog list in the GeoLens plugin panel
Signed XYZ vector tiles (MVT)	A vector tile layer, token refreshed automatically
OGC API Features (<code>/api/collections/{id}/items</code>)	A GeoJSON layer with full attributes
STAC 1.0 and server-rendered raster tiles	A raster layer
Per-dataset permissions, accounts, OAuth 2.0 / OIDC SSO, per-user API keys	The identity that decides what the plugin can see
Uploads of Shapefile, GeoPackage, GeoJSON, GeoParquet, CSV, XLSX, GeoTIFF/COG	Nothing to convert by hand

Your data stays in standard PostGIS and open formats, so nothing about this choice is one-way.

Install it with Docker Compose on the same VM:

```
curl -fsSL https://getgeolens.com/install.sh | sh
```

Then set at least these in its `.env` before exposing it (see the [GeoLens configuration reference](#)):

Variable	Why it matters here
<code>PUBLIC_APP_URL</code> / <code>PUBLIC_API_URL</code>	The browser-facing URLs. These are what you type into the GeoLibre plugin, so they must be the public HTTPS ones, not the Compose service names.
<code>CORS_ALLOWED_ORIGINS</code>	Only needed if GeoLibre is served from a different origin. List that origin exactly.
<code>ENVIRONMENT=production</code>	Hides the API docs endpoints and hardens OAuth cookies.
<code>JWT_SECRET_KEY</code>	Signing secret for sessions.

Upload your datasets and keep them private: GeoLens uses role-based access control with per-dataset permissions. The GeoLibre plugin can then read them two ways, and which one applies depends on how your deployment authorizes API requests. On a same-origin deployment the plugin's calls carry the visitor's GeoLens session automatically. Otherwise, create a per-user API key and paste it into the plugin panel.

2. Host the GeoLibre web build next to it

The published image serves the production web build with nginx:

```
docker run -d --name geolibre -p 8081:80 \
  -e GEOLIBRE_SHARE_URL=off \
  ghcr.io/opengeos/geolibre:latest
```

For a deployment under a path such as `/gis`, bake the base path in at build time and strip the prefix in your proxy:

```
docker build --build-arg GEOLIBRE_APP_BASE=/gis/ -t geolibre-private .
```

The recommended deployment contract is the runtime `deployment.json` policy. On Docker, `GEOLIBRE_DEPLOYMENT_FILE` selects a read-only source file (for example `/etc/geolibre/deployment.json`); startup strictly validates it, then applies nonblank environment overrides field by field and writes generated policy at `/usr/share/nginx/html/deployment.json`, served at `/deployment.json`. Never mount the source over that generated output. Without a source or overrides, startup generates `{"version":1}`. The client resolves policy, runtime environment, then build environment. Docker nginx enforces only its generated final policy; build-only grants do not configure its guards.

Variable	Recommended value	Effect
GEOLIBRE_DEPLOYMENT_FILE	unset, or mounted source policy	Validated at boot; generated public file is <code>/usr/share/nginx/html/deployment.json</code> , served at <code>/deployment.json</code> .
GEOLIBRE_CAPABILITIES	unset, or capabilities to grant	Runtime field override; <code>none</code> grants none. Unknown names stop boot. Final generated policy controls nginx sidecar guards and sidecar startup.
GEOLIBRE_AI_URL / GEOLIBRE_AI_PROXY_URL / GEOLIBRE_AI_PROXY_TOKEN	unset, or approved proxy trio	<code>GEOLIBRE_AI_URL=/ai</code> overrides policy to enable AI. The proxy URL must be an HTTPS origin without credentials/path/query/fragment; token only <code>[A-Za-z0-9._-]</code> . Missing or invalid configuration stops boot. Keep upstream URL and token private.
VITE_GEOLIBRE_CAPABILITIES (legacy build arg; still honoured)	unset, or capabilities to grant	Client fallback only; does not configure nginx guards.
GEOLIBRE_AI_MODEL	optional, with <code>GEOLIBRE_AI_URL</code>	Overrides policy model only when <code>GEOLIBRE_AI_URL</code> is set.
GEOLIBRE_COLLAB_URL	unset, or your own relay	Unset leaves live collaboration dark. Set it to a <code>wss://</code> relay you run if you want multiplayer editing without the hosted relay.
GEOLIBRE_GEOLENS_URL	unset for the image default, <code>same-origin</code> , your GeoLens URL, or <code>off</code>	Pre-fills and automatically connects the GeoLens plugin. The stock image defaults to the browser origin, which suits a GeoLens API co-located behind the same reverse proxy. A URL must be a server root without a query or fragment. <code>off</code> keeps the panel idle until the user chooses a server. The last successful server is remembered per browser.
GEOLIBRE_AUTH_USER / GEOLIBRE_AUTH_PASSWORD	set, for a quick single credential	nginx Basic Auth over the app and the <code>/sidecar</code> API. One shared credential, not accounts. Use a real auth proxy for multi-user or SSO.
GEOLIBRE_CLERK_PUBLISHABLE_KEY	unset, or a Clerk publishable key	Unset keeps the app public and does not load Clerk. A key requires individual users to sign in before the web interface renders; protect server APIs separately.
GEOLIBRE_CLERK_WAITLIST	unset, or <code>1</code> alongside a Clerk key	Adds Clerk's waitlist form to the sign-in screen, so visitors can request access and you approve each one from the Clerk Dashboard. Leave unset for an invite-only ("restricted") instance, where nothing would act on a request.
GEOLIBRE_AUTH0_DOMAIN / GEOLIBRE_AUTH0_CLIENT_ID	unset, or both, if you use Auth0 instead of Clerk	The same sign-in gate backed by Auth0 Universal Login. Both are required together, and the container refuses to start if Clerk is configured as well — pick one provider.
GEOLIBRE_CONVERSION_ROOTS	<code>/data</code> (the image default)	Confines every sidecar read and write to the mounted directory.

Variable	Recommended value	Effect
<code>GEOLIBRE_POSTGIS_HOSTS</code>	unset unless needed	The sidecar's PostGIS endpoints refuse every destination until this names the allowed databases, so a caller cannot aim them at hosts only the container can reach.
<code>GEOLIBRE_MSSQL_HOSTS</code>	unset unless needed	Enables SQL Server endpoints only for the listed hosts/IPs and optional ports. Requires a derived image with the <code>mssql</code> extra and separately installed ODBC Driver 18; named instances require a host-only entry.
<code>GEOLIBRE_MSSQL_ALLOW_MANAGED_IDENTITY</code>	unset unless needed	Managed identity authentication is disabled, because it signs in as the container's own Azure identity for every caller sharing the sidecar. Set to <code>1</code> only when the sidecar is single-tenant and its identity should back database access. <code>GEOLIBRE_MSSQL_DESKTOP_AUTH=1</code> , which also enables Windows and interactive Entra sign-in, implies this opt-in, so never set it on a shared sidecar.
<code>GEOLIBRE_DISABLE_SIDECAR</code>	<code>1</code> if you do not need it	Skips uvicorn regardless of capabilities; every <code>/sidecar/</code> request returns HTTP 403 with an <code>application/json</code> detail response.
<code>GEOLIBRE_EMBED_ORIGINS</code>	unset, or the exact host page origin	Off by default, so a framed deployment cannot be driven by whoever frames it.
<code>GEOLIBRE_NO_EXTERNAL_CDN</code> (build arg)	<code>1</code> for restricted deployments	Strips GeoLibre's own references to external CDNs (<code>unpkg.com</code> , <code>cdn.jsdelivr.net</code>) from the build output. Features whose assets are only available from a CDN are disabled or degraded: storymap HTML export, built-in object detection models, ONNX WASM, 3D Tiles Draco/KTX2 decoders, and gdal3.js export. Pyodide is not hard-disabled — the flag drops only its default index URL, so setting <code>VITE_PYODIDE_INDEX_URL</code> to an approved mirror keeps it working. Also forces <code>GEOLIBRE_PGLITE_CDN=0</code> , <code>GEOLIBRE_CEREUS_CDN=0</code> , <code>GEOLIBRE_GDAL_CDN=0</code> , and <code>GEOLIBRE_DUCKDB_WASM_CDN=0</code> — so PGLite/PostGIS, CereusDB, and DuckDB-WASM stay available , vendored into the build under <code>/assets/</code> (at a larger build size) rather than fetched. Note that some third-party packages (DuckDB-WASM, loaders.gl, maplibre-gl-3d-tiles) carry their own internal CDN URLs that this flag cannot remove; see architecture.md for the details. Intended for deployments that cannot reference untrusted external CDNs (e.g. enterprise environments with strict CSP requirements).
<code>GEOLIBRE_APP_NAME</code>	optional, e.g. <code>Acme Maps</code>	Replaces "GeoLibre" at the start of the toolbar and in the browser tab title. Whitespace runs collapse to one space and the name is capped at 60 characters. <code>VITE_GEOLIBRE_APP_NAME</code> is the equivalent build arg.
<code>VITE_WELCOME_DISABLED=1</code> (build arg)	optional	Skips the first-launch wizard for every visitor.

See [Getting Started](#) for the full list.

Before exposing the image publicly

`docker/nginx.conf` allows `http://localhost:*` and `http://127.0.0.1:*` in `connect-src` so local development can load data from a dev server on another port. On a public host that lets the served JavaScript probe each visitor's loopback interface. Drop those allowances from the CSP for a public deployment.

Container policy enforcement

The container uses the final deployment policy, after environment overrides, to guard these route families at nginx. Each row is an **any-of** check: at least one listed capability must be granted.

Sidecar route prefix	Required capability (any of)
<code>/sidecar/whitebox</code>	<code>processing:run</code>
<code>/sidecar/raster</code>	<code>processing:run</code>
<code>/sidecar/vector</code>	<code>processing:run</code>
<code>/sidecar/pointcloud</code>	<code>processing:run</code>
<code>/sidecar/ml</code>	<code>processing:run</code>
<code>/sidecar/sql</code>	<code>processing:run</code>
<code>/sidecar/postgis</code>	<code>data:add</code>
<code>/sidecar/mssql</code>	<code>data:add</code>
<code>/sidecar/conversion</code>	<code>processing:run</code> or <code>data:add</code>

A denied route returns HTTP 403 with an `application/json` body containing `detail`, rather than reaching the sidecar. Omitting `capabilities` grants all capabilities. If the final grant contains neither `processing:run` nor `data:add`, unicorn is not started and all `/sidecar/` requests return that JSON 403 response. `GEOLIBRE_DISABLE_SIDE CAR=1` does the same regardless of the grant.

When unicorn runs, `/health`, `/algorithms`, `/shutdown`, `/run`, `/docs`, `/redoc`, and `/openapi.json` remain unguarded by capability grants. This is a route-family policy, not per-method authorization. Guards use nginx's normalized URI, including duplicate-slash merging and percent-decoded prefixes.

The final policy also controls `/ai`: `ai.enabled: false`, or an absent `ai.enabled`, disables the proxy. Enabling it requires all three values: `GEOLIBRE_AI_URL=/ai`, `GEOLIBRE_AI_PROXY_URL=https://<approved-origin>`, and `GEOLIBRE_AI_PROXY_TOKEN=<instance-token>`. The URL must be an HTTPS origin without credentials, path, query or fragment; the token may contain only `[A-Za-z0-9._-]`. Missing or invalid configuration stops boot. `GEOLIBRE_AI_URL=/ai` overrides a mounted false value; upstream URL/token alone do not enable AI. `GEOLIBRE_AI_MODEL` overrides policy only when `GEOLIBRE_AI_URL` is set. Keep proxy credentials, sidecar token, trusted proxies, Basic Auth and CSP private; only public policy belongs in the generated file.

Container guards are not general authorization

These restrictions apply only to the bundled container's nginx routes. They do not enforce capabilities on browser WASM engines, desktop processing, or separately exposed services. Keep the sidecar and AI upstream private behind nginx, retain filesystem confinement, and use Basic Auth or a real auth proxy for user authentication. Client-side hiding alone is not an authorization boundary.

Putting both behind one auth layer

Basic Auth is fine for a single shared credential. For accounts or SSO, put [oauth2-proxy](#), [Authelia](#), or Cloudflare Access in front of the *unmodified* image, and route both applications under one hostname. A minimal Caddy example:

```
maps.example.org {
    # Everything is behind the same forward-auth / SSO layer.
    forward_auth authelia:9091 {
        uri /api/verify?rd=https://auth.example.org
        copy_headers Remote-User Remote-Groups Remote-Email
    }

    handle_path /gis/* {
        reverse_proxy geolibre:80
    }

    handle_path /projects/* {
        root * /srv/projects
        file_server
    }

    # Catch-all: Geolens's own entry point, which serves its UI and routes
    # /api to its API service internally, so one upstream covers both. Check
    # your Geolens deployment: if it exposes the API on a separate address,
    # give it its own route above this block, and match on `handle` rather than
    # `handle_path` so the `/api` prefix survives (the API expects it).
    handle {
        reverse_proxy geolens:8080
    }
}
```

With that in place:

```
https://maps.example.org/gis/?url=https://maps.example.org/projects/watershed.geolibre.json
```

opens a private project for an authenticated user and returns the login redirect for anyone else. No CORS headers, no tokens in URLs, and nothing served to anyone the SSO layer has not admitted.

Two layers of authentication, not one

The forward-auth layer decides who reaches the origin. It does not, by itself, tell GeoLens who the visitor is: GeoLens applies its own accounts and per-dataset permissions, and the GeoLibre plugin sends only what the browser attaches (a same-origin cookie) plus an `X-API-Key` header if you gave it a key. So the visitor needs a GeoLens identity too. Either configure GeoLens to consume your provider (it supports OAuth 2.0 / OIDC, so point it at the same identity provider and a visitor who signs in gets a GeoLens session cookie on this origin), or have each user paste a per-user API key into the plugin panel. If neither is true, the app loads and public datasets appear while private ones stay invisible, which is a confusing failure worth ruling out first. `/projects` and other static files behind the proxy are unaffected: they are gated by the forward-auth layer alone.

`url=` must be absolute

The project deep link is validated as an absolute `http(s)` URL, so `?url=/projects/watershed.geolibre.json` is ignored. Write the full URL, as above. It is still same-origin, so the session cookie is still sent.

Passing the signed-in user to the projects server

If you also run the `projects server` (`geolibre-server` in `docker-compose.yml`), the same proxy can sign users in to it, so Share and the Project Gallery need no separate GeoLibre password. The server trusts the proxy's `Remote-User` and `Remote-Email` headers on its sign-in (consent) page once `GEOLIBRE_PROXY_AUTH=true`, and only on connections from an address listed in `GEOLIBRE_TRUSTED_PROXIES`:

1. Set `GEOLIBRE_PROXY_AUTH=true` and set `GEOLIBRE_TRUSTED_PROXIES` to the address the projects server sees the proxy connect from: a comma-separated list of IPs or CIDRs, such as the proxy container's IP or a Docker network that contains only the proxy and the server. Every other peer's identity headers are ignored. `GEOLIBRE_TRUSTED_PROXIES` alone only trusts the proxy's `X-Forwarded-For`.
2. Bind the projects server so only the proxy can reach it. The Compose file publishes it on `127.0.0.1` only; drop that port entirely when the proxy runs in the same Compose network.
3. The proxy **must strip identity headers sent by the client**. Otherwise anyone who reaches a route where the proxy does not overwrite them can claim to be any user.

The projects server answers on `/api/*` like GeoLens does, so give it its own hostname and point the web image's `GEOLIBRE_SHARE_URL` (and the server's `GEOLIBRE_PUBLIC_URL`) at it. Only the consent page goes through forward auth: the app calls the token endpoint and the API cross-origin with bearer tokens, and a login redirect there would break them.

```
projects.example.org {
  # Drop client-sent identity headers before anything else runs. Caddy orders
  # request_header ahead of handle, so this precedes the forward_auth below.
  request_header -Remote-User
  request_header -Remote-Email
```

```
# The consent page: forward auth sets the headers for the signed-in user.
handle /oauth/authorize {
    forward_auth authelia:9091 {
        uri /api/verify?rd=https://auth.example.org
        copy_headers Remote-User Remote-Email
    }
    reverse_proxy geolibre-server:8000
}

# `handle`, not `handle_path`: the server expects the /oauth and /api prefixes.
handle /oauth/* {
    reverse_proxy geolibre-server:8000
}

handle /api/* {
    reverse_proxy geolibre-server:8000
}
}
```

Signing in from the app then opens a consent page that names the proxy user (Signed in through your organization's proxy as ...) and asks only for the device label. The first sign-in creates the account. If your proxy uses other header names (oauth2-proxy sends `X-Forwarded-User` and `X-Forwarded-Email`), set `GEOLIBRE_PROXY_USER_HEADER` and `GEOLIBRE_PROXY_EMAIL_HEADER` and strip those names instead. See [Trusted-header proxy sign-in](#) for the full contract.

The nginx Basic Auth built into the web image (`GEOLIBRE_AUTH_USER` / `GEOLIBRE_AUTH_PASSWORD`) is not a substitute: it is a single shared credential, so it cannot tell the projects server who the visitor is.

3. Connect GeoLibre to GeoLens

The **GeoLens** plugin is built in. There is nothing to install, no marketplace step, and no bundled drop-in to deploy: open the **Plugins** menu and choose **GeoLens**.

In the panel:

1. Pick a server from the list or type your own base URL, for example `https://maps.example.org`. Give the server root, not the `/api` path: the plugin appends `/api/...` itself. A trailing slash is trimmed for you, and a bare hostname is read as `https://`.
2. Leave **API key** empty for public datasets, or paste a GeoLens API key to see and read private ones. On a same-origin deployment the request also carries the visitor's GeoLens session cookie, so if your deployment authorizes API requests by session, private datasets are visible with no key at all.
3. Search the catalog and add a dataset. Vector datasets can be added as **vector tiles** (fast, scale to large tables) or as **GeoJSON** through OGC API Features (full attributes, editable, subject to a feature limit you can set in the panel). Raster datasets are added as server-rendered raster tiles.
4. Each result carries a **Metadata** link back to its page in GeoLens.

Once a dataset is on the map it is an ordinary GeoLibre layer: style it, classify it, open the [attribute table](#), run [processing tools](#) and [spatial SQL](#) against it, and publish it into a [Story Map](#) or an [embedded map](#).

Editing back to GeoLens

A dataset added as GeoJSON can be redrawn with the GeoEditor or retyped in the attribute table, and the plugin's **Edits** section writes the changes back to the GeoLens dataset feature by feature. This requires credentials **and** a server with dataset editing enabled (`enable_dataset_editing` , read at connect time from `/api/settings/feature-flags/`). When it is off, the plugin shows saving as unavailable rather than offering a write that would fail halfway through.

How credentials are handled

This is the part that matters for data sovereignty:

- The API key is held **in memory for the session only**. It is not written to `localStorage` , and it is not written into the project file.
- A saved `.geolibre.json` records only the GeoLens **server URL** and **dataset id** for each layer. Sending that project to someone else does not send them the data or the key.
- On reopening a project, layers from **public** datasets re-mint their signed tile token automatically and render. Layers from **private** datasets stay blank until you reconnect in the plugin panel with a key you are entitled to.
- Signed vector tile tokens are short-lived HMAC tokens minted per dataset, and the plugin refreshes them before they expire.
- A private raster's API key is attached to **exactly** that raster's tile URL prefix and to nothing else, so a basemap, another plugin's tiles, or a second GeoLens server on the same origin never sees it.
- Deactivating the plugin drops every registered key, so a private layer stops rendering. The credential does not outlive the plugin.

The two deployment patterns, compared

These are the two options raised in [discussion #1807](#), and they behave quite differently.

Pattern A: GeoLibre on the community server (recommended)

Serve the web build from the same VM and the same origin as the data, behind the same SSO layer, as described above.

Property	Result
Session cookies reach the data	Yes
CORS configuration needed	None
Private <code>?url=</code> projects	Work
Private tiles, COGs, GeoParquet, FlatGeobuf on your own host	Work
Data or tokens reaching a third party	None
Cost	You run and update one more container

This is the pattern to choose when the requirement is that the data must reach nobody but the users your own auth layer admits, and no third-party service along the way.

Pattern B: hosted GeoLibre, private data elsewhere

A user opens `https://web.geolibre.app/?url=https://community.example.org/project.geolibre.json`.

The project fetch and every layer fetch is now cross-origin, so:

- **Session cookies are not sent.** A URL gated by an SSO cookie returns the login page (or a redirect), and GeoLibre reports a fetch or parse error. There is no setting that changes this.
- **CORS must allow** `https://web.geolibre.app` on the data host, including any custom request header you rely on.
- **Credentials must therefore travel in the URL or in a header.** In practice that means a **signed, expiring URL** (an S3/MinIO presigned URL, a GeoLens signed tile URL, or your own HMAC scheme), because MapLibre's own tile requests cannot carry a custom header that GeoLibre has not been told about. The GeoLens plugin is the exception on the header path: it sends `X-Api-Key` on its API calls, and attaches it to that dataset's raster tiles through MapLibre's `transformRequest` hook.
- **Tokens in URLs leak** into browser history, proxy logs, referrer headers, and anything the user copies and pastes. Keep expiries short and scope each token to one dataset.
- Your data host still sees requests from the user's browser, and the third-party app origin still serves the code that reads it. If your governance rules say the data may not be handled by software served from outside the community server, this pattern does not satisfy them even though the bytes go directly from your server to the user's browser.

Pattern B is a reasonable fit for **semi-public** data (a published map with a capability URL) and a poor fit for **sensitive** data. When in doubt, use pattern A: the web build is a directory of static files, so hosting it costs very little.

Private data without GeoLens

GeoLens is the recommended path because it adds a catalog, accounts, and permissions. If you already have data serving infrastructure, these all work against a private, authenticated, same-origin host:

- **Cloud-native files.** COG, PMTiles, FlatGeobuf, GeoParquet, and Zarr are read with HTTP range requests, so any static file server behind your auth layer works. Serve them from the same origin and the session cookie is sent with every range request.
- **OGC services.** WMS, WMTS, and OGC API Features endpoints on your own server are added from the Add Data menu.

- **PostGIS.** On the desktop app, **Add Data → PostgreSQL/PostGIS** serves tiles straight out of a database through a local [Martin](#) tile server, which is the closest analogue to pointing QGIS at a database. This is desktop only, and not available in the sandboxed Mac App Store build (see [Downloads](#)).
- **MBTiles and local files.** The desktop app reads local MBTiles, rasters, and vector files with no server at all, which is the simplest possible private workflow: download from the community server, analyze offline.
- **The sidecar.** If you enable the bundled Python sidecar, keep `GEOLIBRE_CONVERSION_ROOTS` pointed at exactly the directory you mounted, and leave `GEOLIBRE_POSTGIS_HOSTS` and `GEOLIBRE_MSSQL_HOSTS` unset unless you need those endpoints; SQL Server also requires a derived image with pyodbc and ODBC Driver 18 installed.

Reducing outbound requests

A private deployment usually also wants to minimize what the browser fetches from the public internet:

Feature	Default	How to keep it internal
Basemaps	OpenFreeMap / CARTO tiles	Use the Basemaps plugin's custom style URL and serve your own style plus a PMTiles basemap from your server, or use a blank background. Any HTTPS host is already allowed by the CSP; a plain <code>http://</code> host needs a CSP change.
Geocoding	Public Nominatim	Point it at a self-hosted Nominatim or Pelias (see Data Integrations).
Routing and isochrones	Public FOSSGIS Valhalla (<code>valhalla1.openstreetmap.de</code>)	Set <code>VITE_ROUTING_ENDPOINT</code> to your own Valhalla server. This covers Processing → GeoLibre Toolbox → Network and the Drive time / Walk time quick actions . It is read at build time, or per browser from Settings → Environment variables; the Docker image has no build arg or runtime variable for it. An HTTPS endpoint needs no CSP change.
Point elevation readout	Public Open-Meteo elevation API, whenever 3D terrain has no sample for the point	Leave the readout off (it is off by default), or decline the consent prompt GeoLibre shows before the first remote lookup — that is the gate the resolver checks. Enabling 3D terrain makes the remote call rare but does not rule it out, since a point terrain cannot answer still falls through.

Feature	Default	How to keep it internal
Python (Pyodide) vector engine	Loads Pyodide from jsDelivr	Set <code>VITE_PYODIDE_INDEX_URL</code> to a mirrored copy of the Pyodide distribution (at build time, or per browser in Settings → Environment variables). Serve the mirror from your own origin, or add it to the CSP <code>script-src</code> , which pins only the jsDelivr Pyodide path.
AI assistant	Off unless configured	Leave <code>GEOLIBRE_AI_URL</code> unset, or route it through your own proxy.
Project sharing	<code>share.geolibre.app</code>	<code>GEOLIBRE_SHARE_URL=off</code> , or your own projects server .
Collaboration	Off unless configured	Leave <code>GEOLIBRE_COLLAB_URL</code> unset, or run <code>workers/collab-node</code> yourself.
Telemetry	None	GeoLibre collects no analytics or usage data. The hosted sites (geolibre.app, web.geolibre.app) run Google Analytics, but nothing you deploy does. The Docker image cannot turn it on: the <code>Dockerfile</code> declares no build argument for it, and the container's CSP does not allow <code>googletagmanager.com</code> . A build from source can, by setting <code>VITE_GEOLIBRE_GA_MEASUREMENT_ID</code> for <code>npm run build</code> with your own measurement ID; there is no runtime variable that enables it. See Privacy Policy .

Deployment checklist

- [] GeoLens (or your own data host) running on the community VM, datasets private by default.
- [] GeoLibre web build served from the **same origin**, under a path such as `/gis`, built with the matching `GEOLIBRE_APP_BASE`.
- [] One TLS-terminating reverse proxy with your SSO layer in front of both.
- [] `GEOLIBRE_SHARE_URL=off` (or your own server) so nothing can be published externally by accident.
- [] Review the generated public `/deployment.json`; exercise denied sidecar routes and confirm they return 403.
- [] `GEOLIBRE_CONVERSION_ROOTS` confined, `GEOLIBRE_POSTGIS_HOSTS` and `GEOLIBRE_MSSQL_HOSTS` unset unless required, sidecar disabled if unused.
- [] Loopback `connect-src` allowances removed from the CSP.
- [] Basemap, geocoding, and Pyodide sources pointed at internal hosts if the deployment must not reach the public internet.

- [] A test with a **logged-out** browser: the app, the catalog, the tiles, and the `?url=` project should all be unreachable.

Related pages

- [Getting Started: Run with Docker](#)
- [Server API](#) for self-hosting projects, accounts, and sharing
- [Collaboration](#) for the self-hostable relay
- [Data Integrations](#)
- [Embedding & Sharing](#)
- [Privacy Policy](#)

<https://geolibre.app/server-api/>

GeoLibre projects and identity API

This document defines version 1 of the HTTP contract used by GeoLibre's Project Gallery and **Project → Share** flow. A compatible server may use any implementation or storage engine. The reference implementation lives in `backend/geolibre_server_api`.

Conventions

- The base URL is configured with `GEOLIBRE_SHARE_URL` at container runtime (or `VITE_GEOLIBRE_SHARE_URL` at build time).
- JSON request and response bodies use `application/json` and camel-case keys.
- Dates are UTC ISO 8601 strings.
- Authenticated endpoints accept a personal API token or OAuth access token in `Authorization: Bearer <token>`.
- Error responses are JSON objects with an `error` string. `401` means a missing, invalid, or expired token; `403` means the authenticated principal lacks permission; `404` deliberately covers both a missing project and a project the caller may not discover; `409` is a uniqueness conflict; `422` is malformed input; and `429` is rate limiting.
- Public and unlisted raw project bodies (latest and versioned) use `Cache-Control: public, no-cache` with a strong `ETag`; `If-None-Match` with a matching tag returns `304`. Caches may store the body but must revalidate it on every use, so a visibility change takes effect at the next fetch. Responses containing private, organization, or group-protected content must use `Cache-Control: private, no-store`, including metadata listings.
- Cross-origin web deployments must allow `Authorization` and `Content-Type` from the GeoLibre web origin. On self-hosted Tauri installations using browser fetch, allow `tauri://localhost` and/or `http://tauri.localhost` explicitly. The shipped desktop HTTPS share origin uses native HTTP for authenticated requests; that transport does not depend on CORS.

What the reference server leaves to the operator

Deployment protections remain the operator's responsibility:

- **Personal-token lifecycle.** Omitting `expiresInDays` preserves the v1 delete-only lifecycle and creates a non-expiring token. Require an explicit 1–365 day lifetime where bounded credentials are needed, and revoke or rotate delete-only and legacy tokens operationally.
- **Rate limiting.** The OAuth consent flow caps pending interactions per browser binding, but a fresh cookie bypasses that cap; the reference server has no general request limiter. Before enabling OAuth publicly, enforce per-client-IP limits at the ingress on **GET and POST** `/oauth/authorize`, `GET /oauth/sso/callback`, `POST /oauth/token`, `POST /api/auth/token`, `POST /api/accounts`, and `POST /api/account/password`. The last four POSTs include password or token operations; consent login and the PAT/account routes run scrypt, and the single sign-on

callback calls the organization's identity provider. Add per-username limits where the ingress can safely parse credentials. Every public path to the API must go through this limiter: Compose binds the API host port to loopback by default. A root-issuer nginx deployment can put this zone in its `http` context and the location in its TLS issuer `server` context:

```
# http context
limit_req_zone $binary_remote_addr zone=geolibre_auth:10m rate=12r/m;

# TLS issuer server context; proxy other API routes separately.
location ~ ^/(oauth/(authorize|token|sso/callback)|api/(auth/token|accounts|account/password))
$ {
    limit_req zone=geolibre_auth burst=6 nodelay;
    limit_req_status 429;
    client_max_body_size 16k;
    access_log off;
    proxy_pass http://127.0.0.1:8000;
    proxy_set_header Host $http_host;
}
```

Preserve the original Host authority, including any port, or OAuth host binding rejects the request. Route the issuer's exact discovery URL and other API paths to the same backend; for a path-prefixed issuer, apply the limit to its externally visible prefix and strip that prefix when proxying. Suppress authorization request query strings, callback `Location` headers, and callback request query strings at the web ingress in proxy, WAF, and load-balancer logs. - **A request-size limit.** The server rejects an oversized *declared* `Content-Length` before reading the body, but a chunked or HTTP/2 request declares no length and is parsed in full before the per-route limit applies. Cap request size at the proxy as well.

The reference implementation is a correctness baseline, not a hardened deployment.

Limits

Field	Limit
project title (derived from the uploaded project)	100 Unicode code points
username	3–39 lowercase ASCII letters, digits, or hyphens, not starting or ending with a hyphen
slug	1–100 lowercase ASCII letters, digits, or hyphens
description	2,000 Unicode code points
tags	20 tags, 40 Unicode code points each
project document	50 MiB UTF-8 JSON
thumbnail	5 MiB; PNG, JPEG, or WebP
limit	default 24, maximum 100

Servers may configure a smaller upload limit, but must return `413` and an `error` explaining that limit. The reference server reads its document and thumbnail limits from `GEOLIBRE_MAX_PROJECT_BYTES` and `GEOLIBRE_MAX_THUMBNAIL_BYTES`.

Visibility

- `public`: discoverable in the public listing and readable without auth.
- `unlisted`: omitted from public listings, but readable by anyone holding its URL. It appears in the owner's authenticated listing.
- `private`: an individually owned project is readable only by its owner unless explicitly shared with a group. An organization-owned private project is readable by organization administrators and by its creator while that creator remains an administrator, publisher, or member. Other organization members and viewers need an explicit group share. Raw and thumbnail URLs require the same Bearer token as the metadata endpoint.
- `organization`: readable by every signed-in member of the owning organization. It is omitted from public listings and its raw/thumbnail responses are always `Cache-Control: private, no-store` so removing a member revokes a known URL immediately after their client revalidates it.

Changing visibility affects every version immediately. A raw URL is therefore not a capability URL for a private project.

Identity

POST /api/accounts

Creates an account and returns a personal API token once. This endpoint may be disabled when an installation delegates identity to an external provider. `name`, `scopes`, and `expiresInDays` are optional. New tokens default to all three project scopes. Omitting `expiresInDays` preserves the v1 delete-only token lifecycle (the token does not expire); the accepted explicit lifetime is 1–365 days. An unknown or empty `scopes` list returns `400 {"error": "invalid_scope"}`; an `expiresInDays` outside 1–365 returns `400 {"error": "invalid_request"}`. `email` is optional, trimmed and normalized to lowercase, validated, and unique when present. A password shorter than 8 characters or a malformed username is `422`; a taken username or email is `409`.

```
{
  "username": "ada",
  "password": "correct horse battery staple",
  "email": "ada@example.org",
  "name": "GeoLibre desktop",
  "scopes": ["read:projects", "write:projects"],
  "expiresInDays": 30
}
```

Response `201`:

```
{
  "account": {"id": "uuid", "username": "ada", "email": "ada@example.org", "createdAt": "2026-08-03T12:00:00Z"},
  "token": "secret-token",
  "tokenId": "uuid",
  "scopes": ["read:projects", "write:projects"],
  "expiresAt": "2026-09-02T12:00:00Z"
}
```

POST /api/auth/token

Exchanges account credentials for a personal API token. It accepts the same optional policy fields (but not `email`) and returns the same shape as account creation. Tokens are opaque and stored only as SHA-256 digests.

Status	error
401	invalid username or password
401	account temporarily locked (organization lockout policy)
403	password expired (change it with <code>POST /api/account/password</code>)
403	single sign-on required (an organization of the account disallows built-in accounts)

GET /api/account **and** PATCH /api/account

GET /api/account returns `{"account": <account>}` for any valid credential.

PATCH requires `write:projects`. `{"email": "ada@example.org"}` sets the signed-in account's validated, normalized email; `{"email": null}` clears it. A duplicate email is `409`. The response is `{"account": <account>}` and uses `Cache-Control: private, no-store`.

DELETE /api/auth/token

Revokes the presented Bearer token: a personal token is deleted, and an OAuth access token revokes its whole session family. Response: `204`.

GET /api/users/me

Returns the account, effective credential scopes, and OAuth session ID. `sessionId` is `null` for personal tokens. A management grant reports only `["manage:sessions"]`, not the project's scopes.

```
{
  "user": {"id": "uuid", "username": "ada", "email": "ada@example.org", "createdAt": "2026-08-03T12:00:00Z"},
  "sessionId": "oauth-session-uuid",
  "scopes": ["read:projects", "write:projects", "share:public"]
}
```

An identity provider may create accounts without a username. Project creation for such an account must return `400` with an error containing the stable, case-insensitive sentinel text `username required`. Existing clients recognize that phrase and direct the user to account settings.

Session and personal-token management

These routes require a separate OAuth Bearer grant whose **only** scope is `manage:sessions`. A project OAuth grant, even for the same account, and every personal API token receive `403 insufficient_scope`. The client first resolves the account ID and project `sessionId` using its project credential, then requests fresh management consent and compares the management account ID before listing anything. Management credentials must not be persisted or used for project/gallery calls.

GET `/api/auth/sessions?limit=50&offset=0¤tSessionId=<project-session-id>` returns `{"sessions": [<session>], "limit": 50, "offset": 0, "total": 1}`. `limit` is 1–100; `offset` is nonnegative. If supplied, `currentSessionId` must name an active project session owned by the management account, otherwise the response is `404`. The server marks exactly that entry `current: true`. Only active, unexpired project OAuth sessions and personal API tokens appear; short-lived management grants never appear. Rows sort by creation time newest first, then ID for ties. Each row contains a public UUID, not a token:

```
{
  "id": "uuid",
  "kind": "oauth",
  "clientId": "geolibre-desktop",
  "label": "GeoLibre Desktop",
  "scopes": ["read:projects", "write:projects", "share:public"],
  "createdAt": "2026-08-03T12:00:00Z",
  "lastUsedAt": null,
  "expiresAt": "2026-09-02T12:00:00Z",
  "current": true,
  "legacy": false
}
```

Personal tokens use `kind: "personal-token"`, `clientId: null`, `current: false`, and may have `expiresAt: null`. Old tokens without a policy are backfilled on listing, marked `legacy: true`, and remain valid until revoked. Management responses use `Cache-Control: private, no-store`.

DELETE `/api/auth/sessions/{id}` returns `204` for an owned project OAuth family or personal token, including one already revoked. Management grants are not addressable through this endpoint; unknown, foreign, and management-only IDs return the same `404`. Revocation invalidates the family, not just one access token. If the ID is the current project session, the client must immediately clear its local project credential and protected Gallery/remote-edit state; it must not keep a stale session UI. A pasted personal token remains a separate credential and is not silently replaced by the OAuth grant.

POST `/api/auth/sessions/revoke-others` accepts `{"currentSessionId": "<project-session-id>"}` and returns `204`. It atomically revokes every other OAuth family and **every** personal token for this account, while keeping both the owned active project session named in the request and the calling management grant. Missing, foreign, expired, revoked, or management-only IDs are `404` without

partial revocation. Clients should confirm this destructive action and warn that scripts and CI using personal tokens will stop working. Refresh rotation and bulk revocation serialize on the session rows on PostgreSQL.

Organizations

Organization and group routes use the same scopes as project routes: every `GET` (memberships, members, invitations, galleries, a confined group's thumbnail, and `GET /api/projects?shared_with_me=true`) requires `read:projects`, and every mutation (creating organizations or groups, changing settings or membership, issuing, revoking, or accepting invitations, joining, deciding join requests, moderating, and thumbnails) requires `write:projects`. Reading a non-public project reached through an organization or group, like a private one, also requires `read:projects`.

`POST /api/organizations` creates an organization and makes the caller its first `administrator`. The body contains `slug`, `name`, `publicSharingPolicy` (`yes`, `publishers`, or `no`), `defaultVisibility`, and optional `categories`. The slug is globally unique. When a project is created in the organization without a `visibility`, the server applies `defaultVisibility`. A `public` default requires `publicSharingPolicy yes`; creating or patching an organization into any other combination returns 422.

Organization roles are:

- `administrator`: manage settings and membership, and mutate any organization-owned project.
- `publisher`: create organization content and publish publicly when policy is `publishers` or `yes`.
- `member`: create organization content and share within the organization; may publish only when policy is `yes`.
- `viewer`: read organization-visible content only.

A publisher or member who creates organization content may manage that content while they retain that organization role. Administrators may manage every organization project. Demotion to viewer or removal from the organization immediately removes the creator's management permission; the project remains owned by the organization rather than becoming orphaned.

The same rule governs private reads: administrators and active creators can read private organization projects they can manage. Membership alone does not grant a publisher, member, or viewer access to somebody else's private project.

An administrator can also move a project *into* the organization with `POST /api/projects/{id}/transfers ({"organizationId": "..."} )`, which applies immediately because the caller already manages the recipient (see [Transfers](#)).

Routes:

- `GET /api/organizations/mine` lists memberships and each caller's `role`.

- GET `/api/organizations/{id}` returns settings to a member.
- PATCH `/api/organizations/{id}` changes `name`, `publicSharingPolicy`, `defaultVisibility`, or `categories`; administrator only. Tightening `publicSharingPolicy` to `publishers` or `no` changes every `public` organization project whose creator could not publish it under the new policy (by their current role; creators no longer in the organization included) to `organization` visibility, and logs a `visibility_change` activity for each.
- DELETE `/api/organizations/{id}` deletes the organization, its organization-owned projects and their stored objects, its groups, members, and invitations; administrator only. Members' personal projects are kept. If any organization project is delete-protected, it refuses the entire deletion with `409` until that project's protection is turned off.
- GET `/api/organizations/{id}/members` lists members.
- PUT `/api/organizations/{id}/members` adds or updates `{"username":"ada","role":"member"}`; administrator only. Lowering a role re-applies the public sharing policy: `public` projects whose creator can no longer publish them become `organization`.
- DELETE `/api/organizations/{id}/members/{username}` removes a member (administrator only), and `{username}=me` leaves. The last administrator cannot be removed, leave, or be demoted (`409`). Neither can the break-glass administrator of the organization's identity provider (`422` account is the organization's break-glass administrator); clear `breakGlassUsername` on the provider first. Projects the leaver created stay owned by the organization, and their `public` ones become `organization` unless the policy is `yes`. The leaver's memberships and pending invitations in the organization's groups are removed. Groups they own pass to the administrator who removed them; a member who owns one must transfer it before leaving (`409`).
- POST `/api/organizations/{id}/invitations` creates a pending invitation for exactly one `username` or `email`; GET on the same path lists pending, accepted, and revoked invitations. Issuance and listing are administrator only. The creation response alone includes the opaque token.
- DELETE `/api/organizations/{id}/invitations/{invitationId}` changes a pending invitation to `revoked`; administrator only.
- POST `/api/organizations/invitations/{token}/accept` requires sign-in, verifies the account's username or email, adds the member with the invited role, and changes the invitation to `accepted`.
- GET `/api/organizations/{id}/projects` returns the organization gallery. A non-administrator sees public and organization-visible projects plus private projects separately shared with one of their groups.

Supplying `organizationId` on project creation or patch transfers the project to organization ownership. Its `username` is then `null`, every organization administrator can manage it, and raw routes use `/org/{organizationSlug}/{projectSlug}[.geolibre.json]`. Only an administrator of the owning organization may change or clear `organizationId` on an organization-owned project (`403` otherwise); clearing it returns the project to its creator's individual account. A patch that changes `organizationId` re-checks the project's group targets as a create in the new organization would, whether they are sent in `groupIds` or kept from before; send `groupIds` to replace targets that no longer qualify. The public sharing policy is enforced on create and patch, including direct API

requests. Servers retain a nullable creator identity separately from ownership. New projects record their creating account whether ownership is individual or organizational; organization ownership remains authoritative, and the creator identity does not populate `username` or create an individual project URL.

Groups

`POST /api/groups` creates a standalone or organization-associated group. The body contains `name`, optional `description` and `organizationId`, `joinPolicy` (`invite`, `request`, or `open`), and `sharedUpdate`. An organization-associated group admits only members of that organization: adding, inviting by username, joining, accepting an invitation, or approving a request for anyone else returns 403. `sharedUpdate` is fixed at creation and cannot be patched; `name`, `description`, and `joinPolicy` are settings. An optional PNG, JPEG, or WebP thumbnail uses `PUT / GET / DELETE /api/groups/{id}/thumbnail`.

Group roles are `owner`, `manager`, and `member`. Exactly one accepted member is the owner. An owner transfers ownership by assigning `owner` through `PUT /api/groups/{id}/members`; the prior owner becomes a manager atomically. Managers can add/remove ordinary members, invite, decide join requests, and remove projects from the group. Only the owner can manage managers or transfer ownership, and an owner cannot leave until ownership is transferred.

Routes:

- `GET /api/groups/mine` lists accepted memberships; `GET /api/groups/{id}` returns group detail to a signed-in caller. `PATCH /api/groups/{id}` changes `name`, `description`, or `joinPolicy`; owner or manager only.
- `DELETE /api/groups/{id}` deletes the group with its memberships, invitations, and thumbnail; owner only. Projects shared with the group are kept and lose only that group target.
- `GET /api/groups/{id}/members` lists accepted members. Owners/managers also see pending join requests.
- `PUT /api/groups/{id}/members` adds or changes a member using `username` and `role`; `DELETE /api/groups/{id}/members/{username}` removes one, and `{username}=me` leaves.
- `POST /api/groups/{id}/invitations` creates a pending invitation for exactly one `username` or `email`. The creation response includes its opaque token; manager listings (`GET` on the same path) omit the token and retain pending, accepted, and revoked rows. `DELETE .../invitations/{invitationId}` changes a pending invitation to `revoked`, and `POST /api/groups/invitations/{token}/accept` changes it to `accepted` while adding the signed-in target account.
- `POST /api/groups/{id}/join` immediately joins an open group, creates a pending request for a request group, and rejects an invite-only group.
`POST /api/groups/{id}/members/{username}/decide` with decision `accept` or `reject` moderates a pending request.
- `GET /api/groups/{id}/projects` lists targeted projects. `DELETE /api/groups/{id}/projects/{projectId}` removes that target without deleting the project.

Project create and patch requests accept `groupIds`. The caller must be an accepted member of every target. For an organization-owned project, a non-administrator may target only that organization's groups. A member can read a private project targeted to their group and can update its content only if that group's immutable `sharedUpdate` value is true. Removing the membership or target revokes access on the next request; protected raw and thumbnail responses are never shared or persistently cached.

Invitation tokens are bearer credentials. For both organization and group invitations, servers must store only a SHA-256 digest, return the raw token only from the creation call, and hash the path token before acceptance lookup. Accepted and revoked tokens cannot be reused.

Group thumbnails follow the group's join policy. An `open` group's thumbnail is public and may use `Cache-Control: public, max-age=3600`. For `invite` and `request` groups, only accepted members may fetch the thumbnail and every successful response uses `Cache-Control: private, no-store`; non-members receive `404`. This prevents a stable public thumbnail URL from disclosing content from a membership-confined group.

Enterprise sign-in

Organization security policy

Organization administrators can set a security policy for their members.

- `GET /api/organizations/{id}/security-policy (read:projects)` returns `{"securityPolicy": {...}}` with `idleTimeoutSeconds`, `absoluteSessionSeconds`, `adminReauthSeconds`, `adminIpAllowlist` (list), `passwordMinLength`, `passwordMinClasses`, `passwordMaxAgeDays`, `lockoutThreshold`, and `lockoutSeconds`. Unset values are `null`; with no policy every value is `null` and the allowlist is empty.
- `PUT /api/organizations/{id}/security-policy (write:projects)` replaces the whole policy (an omitted field becomes `null`) and returns the `GET` shape.

Both require an organization administrator and respond with `Cache-Control: private, no-store`.

Bounds:

Field	Allowed
<code>idleTimeoutSeconds</code>	300–2592000
<code>absoluteSessionSeconds</code>	900–31536000
<code>adminReauthSeconds</code>	60–86400
<code>passwordMinLength</code>	8–128
<code>passwordMinClasses</code>	1–4 (lowercase, uppercase, digits, symbols)
<code>passwordMaxAgeDays</code>	1–3650
<code>lockoutThreshold</code>	3–100
<code>lockoutSeconds</code>	60–86400

Field	Allowed
adminIpAllowlist	up to 50 IP addresses or CIDR networks

Out-of-range values are a 422. Other 422 errors: adminIpAllowlist entries must be IP addresses or networks, lockoutThreshold and lockoutSeconds must be set together, and adminIpAllowlist must include your current address (so an administrator cannot lock themselves out).

When an account belongs to several organizations, the strictest value wins: the shortest idle timeout, absolute session lifetime, password age, and lockout threshold; the longest minimum length, class count, and lockout duration. Password length is never below 8.

- **Idle and absolute session limits** apply to OAuth sessions and personal tokens. An OAuth session idle longer than idleTimeoutSeconds, or older than absoluteSessionSeconds since its sign-in, is revoked: Bearer use returns 401 invalid or expired token and refresh returns 400 invalid_grant. A personal token is measured from its creation and last use and is revoked the same way. New OAuth families never outlive the absolute limit.
- **Lockout:** lockoutThreshold consecutive wrong passwords lock the account for lockoutSeconds; a successful sign-in resets the count.
- **Password rotation:** a password older than passwordMaxAgeDays is rejected at sign-in until changed with POST /api/account/password. For accounts created before this feature, the age counts from their first sign-in after the upgrade.
- **Administrator re-authentication:** when the organization sets adminReauthSeconds, administrator mutations (organization settings, members, invitations, security policy) from a credential whose sign-in is older than that return 401 {"error": "reauthentication_required"} with WWW-Authenticate: Bearer error="insufficient_user_authentication", max_age="<seconds>" (RFC 9470). Sign in again to continue. Reads are not affected.
- **Administrator IP allowlist:** when adminIpAllowlist is non-empty, every administrator route of that organization from an address outside it returns 403 administrative access is not allowed from this network. The client address is the direct peer unless the peer is listed in GEOLIBRE_TRUSTED_PROXIES (comma-separated IPs or CIDRs); then the rightmost untrusted X-Forwarded-For entry is used.

POST /api/account/password

Unauthenticated. Body: {"username": "ada", "currentPassword": "...", "newPassword": "..."} (new password up to 1024 characters). Works even when the current password has expired. Response: 204.

Status	error
401	invalid username or password
401	account temporarily locked
403	single sign-on required
422	new password must differ from the current password

Status	error
422	password must be at least <n> characters
422	password must use at least <n> of: lowercase, uppercase, digits, symbols

Organization identity provider

An organization administrator can connect one OpenID Connect provider (Entra ID, Okta, Google, Keycloak, ADFS 2016+, ...) to the organization.

- GET `/api/organizations/{id}/identity-provider` (`read:projects`) returns `{"identityProvider": {...}}`, or 404 identity provider not configured.
- PUT `/api/organizations/{id}/identity-provider` (`write:projects`) creates or replaces it and returns the GET shape.
- DELETE `/api/organizations/{id}/identity-provider` (`write:projects`) removes it. Response: 204, also when none is configured.

All three require an organization administrator (the organization's IP allowlist applies, and re-authentication applies to PUT and DELETE) and respond with `Cache-Control: private, no-store`. PUT body:

```
{
  "issuer": "https://login.example.org/realms/acme",
  "clientId": "geolibre",
  "clientSecret": "...",
  "tokenEndpointAuthMethod": "client_secret_basic",
  "scopes": ["openid", "email", "profile"],
  "usernameClaim": "preferred_username",
  "emailClaim": "email",
  "groupsClaim": "groups",
  "defaultRole": "member",
  "roleMappings": [{"value": "gis-admins", "role": "publisher"}],
  "groupMappings": [{"value": "gis-admins", "groupId": "group-uuid"}],
  "requireMfa": false,
  "allowBuiltinAccounts": true,
  "breakGlassUsername": null,
  "enabled": true
}
```

Field	Rules
issuer	Required, up to 512 characters, <code>https://</code> without query or fragment. Compared exactly with the ID token's <code>iss</code> , never normalized.
clientId	Required, 1-255 characters.
clientSecret	1-512 characters. Required when creating; omitted or <code>null</code> on update keeps the stored secret. Never returned.
authorizationEndpoint, tokenEndpoint, jwksUri	All three (each <code>https://</code>) or none. When none, they are read from the issuer's discovery document.
tokenEndpointAuthMethod	<code>client_secret_basic</code> (default) or <code>client_secret_post</code> .

Field	Rules
scopes	Up to 20 simple scope tokens, including openid. Default openid, email, profile.
usernameClaim, emailClaim, groupsClaim	Claim names, up to 64 characters. Defaults preferred_username, email, groups; groupsClaim may be null.
defaultRole	Organization role when no role mapping matches. Default member.
roleMappings	Up to 100 {"value", "role"} entries.
groupMappings	Up to 100 {"value", "groupId"} entries; each group must belong to this organization.
requireMfa	Require "mfa" in the ID token's amr. Default false. Providers that list only the individual factors (for example ["pwd", "otp"]) need a claim mapper that adds "mfa", or every sign-in is rejected.
allowBuiltinAccounts	false disables password sign-in for the organization's members. Default true.
breakGlassUsername	A current administrator of this organization who keeps password sign-in. Required when allowBuiltinAccounts is false.
enabled	A disabled provider is neither offered nor accepted. Default true.

The GET shape echoes the settings (scopes as a list) plus protocol ("oidc"), the stored endpoints, clientSecretSet: true instead of the secret, redirectUri, and updatedAt. Register redirectUri (<issuer>/oauth/sso/callback of this server; null when OAuth is not configured) at the identity provider for a confidential client using the authorization code flow with S256 PKCE.

422 errors: issuer must be an https URL without query or fragment, clientSecret is required, set all three endpoints or none, identity provider endpoints must be https URLs, scopes must include openid and use simple scope tokens, group mapping must name a group in this organization, break-glass account must be an organization administrator, a break-glass administrator is required when built-in accounts are disallowed, and identity provider discovery failed. Out-of-range values are a generic 422.

- **Discovery:** without explicit endpoints, PUT fetches <issuer>/well-known/openid-configuration. Its issuer must equal the configured issuer exactly and it must name https:// authorization, token, and JWKS endpoints; a network error, non-200 status, a body over 1 MiB, or invalid JSON also fails discovery. The endpoints are stored; discovery runs again only on the next PUT. Signing keys are fetched from jwksUri on the first sign-in and cached; changing issuer or jwksUri clears the cache. Once the provider has linked federated identities, PUT cannot change its issuer or jwksUri; it returns 409 issuer or JWKS endpoint cannot change while federated identities are linked. This prevents a replacement authority from reusing sub values to resolve existing accounts. Account migration or re-linking is not automatic; do not delete and recreate the provider as a workaround, since deletion removes the identity links. A sign-in validated against old settings is rejected if those settings changed before identity linking; PUT and sign-in serialize on the provider row in PostgreSQL. A PUT racing provider removal or replacement returns HTTP 409 with message identity provider was removed or replaced; try again.

- **Internal addresses (SSRF protection):** any signed-in user can create an organization and choose its provider URLs, so the server connects to an identity provider only on public addresses. Each host is resolved once and every loopback, private, link-local, CGNAT (`100.64.0.0/10`), multicast, reserved, or unspecified address (including their IPv4-mapped, NAT64, 6to4, and Teredo forms) is dropped; a host left with none fails like a network error, so discovery answers `identity provider discovery failed` and sign-in is rejected. The connection goes to the checked address, while TLS still verifies the certificate against the URL's hostname. To use a provider on an internal network, list its networks in `GEOLIBRE_OIDC_ALLOWED_NETWORKS` (comma-separated IPs or CIDRs; an invalid entry fails startup).
- **Accounts:** the first sign-in of a provider subject (`sub`) creates an account linked to it. Its username comes from `usernameClaim` (or else `emailClaim`): lowercased, cut at `@`, other characters replaced with `-`, with a `-2 ... -99` suffix when taken. When nothing fits, the account has no username (see the `username required` sentinel under `GET /api/users/me`). Its email is set only when `email_verified` is `true` and no other account uses the address. A sign-in is never linked to an existing account by email, so existing members who move to single sign-on get a new account. Federated accounts have no password.
- **Mappings,** applied at every sign-in from the `groupsClaim` value (a string or a list of strings): the role is the highest-ranked `role` among matching `roleMappings`, else `defaultRole`. A new member receives it; an existing member's role follows it only when `roleMappings` is non-empty, and neither the organization's last administrator nor its break-glass administrator is ever demoted. A lowered role re-applies the public sharing policy as `PUT /api/organizations/{id}/members` does: `public` projects the member can no longer publish become `organization`. For each mapped group, the account becomes a `member` when a matching value is present and loses a plain `member` row when none is; owner and manager rows are never changed.
- **Built-in accounts:** with `allowBuiltinAccounts: false`, a correct password for any member of the organization other than the break-glass administrator is rejected: `403 single sign-on required` from `POST /api/auth/token` and `POST /api/account/password`, and `Your organization requires single sign-on. Use "Sign in with your organization".` on the consent page. The break-glass account keeps password sign-in only while it remains an administrator of the organization; lockout still applies to it. While it is the break-glass account it cannot be demoted, removed, or leave (`422`); clear `breakGlassUsername` first.
- **Deletion** removes the provider, its account links, and pending sign-in redirects. Accounts and memberships remain, but accounts created through single sign-on have no way to sign in. A provider configured later links subjects afresh, so their next sign-in creates new accounts; remove the old ones.
- **Secret storage:** `clientSecret` is stored unencrypted in the database, at the same trust level as the rest of its contents.

Single sign-on on the consent page

When any organization has an enabled identity provider, the consent page adds a second form: an `Organization` field (the organization slug) and a `Sign in with your organization` button (`decision=sso` , with the same `interaction` , `csrf` , and `label` fields). Its `POST` answers `303` to that organization's authorization endpoint with `state` , `nonce` , an S256 PKCE challenge, and `max_age` when the organization's security policy sets `adminReauthSeconds` . An unknown slug or a disabled provider re-renders the consent page with `Single sign-on is not configured for that organization` . Browsers apply `form-action` to that redirect, so while single sign-on is offered the consent page's `Content-Security-Policy` adds `https:` to `form-action` .

`GET /oauth/sso/callback` receives the provider's response. The `state` must be live and unused, the interaction undecided and unexpired, and the request must carry the browser-binding cookie of the browser that started consent.

- `error` from the provider: `303` to the client's callback with `error=access_denied` .
- Otherwise the server redeems the `code` at the token endpoint with the client secret and PKCE verifier and validates the ID token: an RS256, PS256, or ES256 signature from the provider's JWKS (an unknown key id refetches the key set at most once a minute), `iss` , `aud` (plus `azp` when there are several audiences), `exp` and `iat` with 60 seconds of leeway, `nonce` , `sub` , `amr` when `requireMfa` is set, and `auth_time` when `max_age` was sent. Success approves the interaction exactly like a password sign-in: `303` to the client's callback with `code` , `state` , and `iss` . The OAuth session's sign-in time is the ID token's `auth_time` when present.
- Any other failure, including a reused `state` , returns a `400` page with `invalid_request: single sign-on response rejected` . The reason is only logged (`oidc sign-in rejected: <reason>`).

Calls to the identity provider give up when connecting or any read stalls for 10 seconds, or once the whole response has taken longer than 10 seconds. They never follow redirects, ignore proxy environment variables, and stop reading at 1 MiB. Set `GEOLIBRE_OIDC_CA_BUNDLE` to also trust a provider whose certificate is issued by a private CA; the public CAs stay trusted.

Trusted-header proxy sign-in

With `GEOLIBRE_PROXY_AUTH=true` (or `1 / yes`), behind an identity-aware proxy listed in `GEOLIBRE_TRUSTED_PROXIES` , the consent page trusts the proxy's user header (`GEOLIBRE_PROXY_USER_HEADER` , default `Remote-User`) and optional email header (`GEOLIBRE_PROXY_EMAIL_HEADER` , default `Remote-Email`). Without `GEOLIBRE_PROXY_AUTH` , `GEOLIBRE_TRUSTED_PROXIES` only trusts `X-Forwarded-For` and identity headers are never read. When the direct peer is a trusted proxy and the user header is present, the page shows `Signed in through your organization's proxy as <user>` instead of the password and single sign-on forms, and `Allow` approves the interaction for the account linked to that user. The first sign-in creates the account: the username is derived as for single sign-on, and the email is set when it is valid and unused. Organization mappings and the built-in account switch do not apply to proxy identities. An empty user, one over 255 characters, or one containing control characters returns a `400` page with

`invalid_request: invalid proxy identity`; an account that cannot be created returns `invalid_request: proxy sign-in failed`. Identity headers from any other peer are never read, so the proxy must strip client-sent identity headers and be the only network path to the API.

SCIM 2.0

An organization's identity provider can provision its users and groups with SCIM 2.0 (RFC 7643/7644). An organization administrator mints a token for it:

- `POST /api/organizations/{id}/scim-tokens (write:projects)`, body `{"label": "Entra ID"}` (1–100 characters), returns `201 {"token": "...", "scimToken": {...}, "baseUrl": "<public URL>/scim/v2/<organization id>"}`. The raw token is shown only here; the server stores its digest.
- `GET /api/organizations/{id}/scim-tokens (read:projects)` returns `{"scimTokens": [...]}`, newest first.
- `DELETE /api/organizations/{id}/scim-tokens/{tokenId} (write:projects)` revokes the token. Response: `204`; an unknown token, or one of another organization, is `404 SCIM token not found`.

A token object is `{"id", "label", "createdAt", "lastUsedAt", "revokedAt"}` (`lastUsedAt` is updated at most once a minute). The routes require an organization administrator (with the organization's IP allowlist, and re-authentication for `POST` and `DELETE`) and respond with `Cache-Control: private, no-store`.

Configure the identity provider with `baseUrl` as the tenant/SCIM URL and the token as a Bearer secret. Every SCIM request needs `Authorization: Bearer <token>` for that organization; a missing, revoked, or other organization's token is `401` with `WWW-Authenticate: Bearer`. Tokens also stop working when their creator is deactivated or ceases to be an active organization administrator. SCIM requests are machine-to-machine, so the organization's administrator IP allowlist and re-authentication window do not apply to them; revoke a token to cut off a provider.

Request bodies are `application/scim+json` (or `application/json`); responses are `application/scim+json`. Errors use the SCIM error message: `{"schemas": ["urn:ietf:params:scim:api:messages:2.0:Error"], "status": "409", "detail": "userName already exists", "scimType": "uniqueness"}`.

Endpoint	Notes
<code>GET /ServiceProviderConfig, /ResourceTypes, /Schemas</code>	Discovery. PATCH and filtering are supported (up to 100 results); bulk, sort, ETags, and password changes are not.
<code>GET /Users</code>	<code>startIndex</code> (default 1) and <code>count</code> (0–100, default 100), ordered by creation. <code>filter</code> only <code>userName eq "..."</code> (case-insensitive) or <code>externalId eq "..."</code> ; anything else is <code>400 invalidFilter</code> .

Endpoint	Notes
POST /Users	Creates an account, or adopts the existing SSO account when this organization's enabled identity provider has a unique matching lowercased username claim or verified email claim. Password accounts, proxy identities, accounts managed by another organization, and ambiguous matches are not adopted. <code>userName</code> (1–255 characters, stored lowercased) is required and unique per organization (409 uniqueness).
GET / PUT / PATCH / DELETE / Users/{id}	<code>id</code> is the account id. PUT replaces <code>userName</code> , <code>externalId</code> , <code>displayName</code> , <code>emails</code> , and (when present) <code>active</code> .
GET /Groups	<code>filter</code> only <code>displayName eq "..."</code> (case-insensitive) or <code>externalId eq "..."</code> .
POST /Groups	Creates an organization group owned by the token's creator, with <code>join_policy</code> <code>invite</code> . Every member must be a user provisioned in this organization (400 invalidValue).
GET / PUT / PATCH / DELETE / Groups/{id}	PUT replaces <code>displayName</code> , <code>externalId</code> , and the member set. DELETE deletes the group.

A user resource carries `id`, `userName`, `externalId`, `displayName`, `active`, `emails` (the primary address only), and `meta`. Other attributes are accepted and ignored. A provisioned account has no password and no email address of its own: `emails` is kept for the SCIM representation only. Its username is derived from `userName` the way single sign-on derives it, and it joins the organization with the identity provider's `defaultRole` (`member` without a provider). A group resource's `members` lists only plain accepted members; the owner and managers are never listed or changed by SCIM. Only users and groups created through SCIM are visible to it (404 resource not found otherwise).

SCIM identity claims are refreshed on every OIDC sign-in. An email claim is eligible for matching only when the provider asserts `email_verified: true`. When a previously signed-in account has no SCIM resource and a matching SCIM resource belongs to an unused SCIM-created account, the next sign-in reconciles the SCIM resource onto the real account, transfers only member-role entries in this organization's SCIM groups (without duplicates), then deactivates the orphan and removes its organization membership and group-member rows. The orphan account record is retained. A deactivated SCIM resource remains deactivated after reconciliation.

Deleting a managed SCIM user deactivates the account and revokes its OAuth and personal-token credentials, but retains its federated identity. Reprovisioning the same username adopts that deactivated account and reactivates it without restoring old credentials; an authorization code approved before deactivation cannot be exchanged afterward.

PATCH bodies need the `urn:ietf:params:scim:api:messages:2.0:PatchOp` schema and an `Operations` list (else 400 `invalidSyntax`). `op` is case-insensitive. For users, `add` and `replace` accept the paths `active`, `userName`, `externalId`, `displayName`, `emails`, and `emails[type eq "work"].value`, or no path with an object of attributes; unknown paths are ignored. `remove` accepts only `externalId` and `displayName`. Other operations are 400 `unsupported patch operation` (`invalidSyntax`). For groups, `add` takes `members` (a list of `{"value": "<user id>"}`), `displayName`,

or `externalId`; `remove` takes `members[value eq "<user id>"]`, `members` with a value list (or no value to remove every member), or `externalId`; `replace` takes `displayName`, `externalId`, `members`, or no path with an object of those. A successful PATCH returns `200` with the resource.

Entra ID: `active` may be the strings `"True"` / `"False"`, operation names may be capitalized (`Replace`), and Entra's extra attribute paths are ignored, so its default attribute mappings work unchanged. Entra soft-deletes by setting `active` to `false`, and later sends `DELETE`.

Deactivation. Setting `active` to `false` (`PUT` or `PATCH`), or `DELETE`, depends on who manages the account:

- An account this organization manages (created by its SCIM or its single sign-on) is deactivated: every OAuth session and personal token is revoked in the same transaction, Bearer use returns `401 invalid or expired token`, refresh returns `400 invalid_grant`, password sign-in fails as an invalid password, single sign-on returns a `403` page with `This account has been deactivated.`, and trusted-proxy sign-in shows the same message on the consent page.
- Any other account only loses its membership in this organization and its membership (except group ownership) in the organization's groups; the account itself and its other organizations are untouched. Access ends on the next request because authorization reads memberships live. Its SCIM `active` reads `false` while it is not a member.
- Setting `active` back to `true` reactivates a managed account and re-adds the organization membership if missing. Credentials revoked by the deactivation stay revoked; the user signs in again.
- `DELETE /Users/{id}` deactivates as above, removes the organization and group memberships, and forgets the SCIM user. Removing the organization's only administrator, by deactivation of an account it does not manage or by `DELETE`, is `409 cannot remove the last organization administrator (mutability)`; removing the identity provider's break-glass account is `409 cannot remove the organization's break-glass administrator (mutability)`. Clear `breakGlassUsername` on the provider first.
- Removing a membership applies the organization's public sharing policy the same way leaving through the members API does: the removed account's public organization projects become organization-only unless the policy is `yes`.

Single sign-on link: the first single sign-on of a subject in an organization with SCIM users links to the provisioned account whose `userName` equals the ID token's `usernameClaim` (lowercased), or failing that its `emailClaim`, instead of creating a new account. Provision `userName` as the value the provider puts in that claim (for Entra ID, the UPN in `preferred_username`).

Projects

Project representation

```
{
  "id": "uuid",
  "username": "ada",
  "slug": "wetlands",
```

```

"title": "Wetlands",
"description": "",
"visibility": "public",
"canEdit": true,
"organization": {"id": "uuid", "slug": "watershed-lab", "name": "Watershed Lab"},
"groupIds": ["group-uuid"],
"thumbnailUrl": "/api/projects/uuid/thumbnail",
"views": 12,
"forkCount": 0,
"versionCount": 1,
"featured": false,
"deleteProtected": false,
"role": "edit",
"expiresAt": null,
"hasPassword": false,
"createdAt": "2026-08-03T12:00:00Z",
"updatedAt": "2026-08-03T12:00:00Z",
"tags": [],
"rawJsonUrl": "https://example.org/ada/wetlands.geolibre.json",
"projectUrl": "https://example.org/ada/wetlands",
"viewerUrl": "https://example.org/?project=https%3A%2F%2Fexample.org%2Fada%2Fwetlands.geolibre.json"
}

```

`organization` is non-null whenever the project is organization-owned, regardless of visibility. `groupIds` is an array of group identifiers the project is shared with (empty array when none). Authenticated project, listing, create, and update responses include `canEdit`, computed by the server for that caller. It is true for an individual owner, an organization administrator, an active organization creator, or a member of a targeted group whose `sharedUpdate` setting is true. Clients must use this value instead of reconstructing authorization from roles. Anonymous responses omit it. Because authenticated public responses vary by caller, they use `Cache-Control: private, no-store`. Unknown fields must be ignored by consumers.

`deleteProtected` is the owner's per-project "prevent deletion" switch. It is `false` by default and present in every project representation, anonymous ones included. While it is `true`, `DELETE /api/projects/{id}` is refused (see below).

POST /api/projects

Requires auth. Creates a project and its first immutable version.

```

{
  "filename": "Wetlands.geolibre.json",
  "content": "{\"version\":\"1.0\", ...}",
  "visibility": "public",
  "organizationId": "org-uuid",
  "groupIds": ["group-uuid-1", "group-uuid-2"]
}

```

`content` is a string containing a valid GeoLibre project JSON document. `filename` supplies a fallback title/slug; the project document's non-empty title is authoritative. `visibility` is optional and is `public`, `unlisted`, `private`, or `organization`. `organizationId` is required when `visibility` is `organization`. `groupIds` is an optional array of group identifiers; the caller must be a member of

every listed group, and for an organization project a non-administrator may list only that organization's groups. When `visibility` is omitted, the organization's `defaultVisibility` applies, or `private` for a personal project.

Optional share-link settings (only for `public` or `unlisted` projects; any other visibility answers 422 when one is set): `role` (`view`, `comment`, or `edit`; default `edit`), `expiresIn` (`24h`, `7d`, `30d`, or `never`), and `password`. They are echoed in every project representation as `role`, `expiresAt` (ISO timestamp or `null`), and `hasPassword`. `role` is metadata for viewers; the server enforces only the expiry and the password. Once `expiresAt` has passed, anyone but a manager of the project gets 410 (`share link expired`) from every read route. While a password is set, those readers get 401 (`share password required`) until they unlock the link with `POST /{username}/{slug}/access` (or `POST /org/{organization}/{slug}/access` for an organization project) with `{"password": "..."}.` That returns `{"content": "<project JSON>", "role": "view"}` with `Cache-Control: private, no-store`. After 10 wrong passwords within 5 minutes from one client address, the route answers 429 for that project, even for the right password. The count is kept in memory, per server process.

The version-list route (`GET /api/projects/{id}/versions`) takes no password, so while a password is set it answers 401 to everyone except a manager of the project.

GET /api/projects

Returns a page in newest-updated-first order:

```
{"projects": [], "limit": 24, "offset": 0, "total": 0}
```

Query parameters:

- `limit`: integer page size.
- `offset`: non-negative number of matching records to skip.
- `featured=true`: return featured projects only.
- `mine=true`: return the caller's own projects, including unlisted and private ones. Requires auth; without a valid token this is 401.
- `shared_with_me=true`: return organization-visible projects from the caller's organizations, organization public projects, manageable private/unlisted organization projects, and projects explicitly targeted to their groups. Requires auth and cannot be combined with `mine=true`.
- `shared_source=organizations|groups`: with `shared_with_me=true`, restrict the query before pagination and counting. `organizations` includes public and organization-visible projects in the caller's organizations plus private/unlisted projects manageable as an administrator or active creator. `groups` includes projects explicitly targeted to an accepted group membership. Using this parameter without `shared_with_me=true` is 422.

Only public projects are returned unless `mine=true` or `shared_with_me=true` is set. An Authorization header does not broaden a public listing by itself. Invalid pagination or combining both private listing modes is 422.

GET /api/users/{username}/projects

Returns `{"projects": [...]}` owned by `{username}`, in newest-updated-first order. Auth is optional and decides the breadth of the result: when the token identifies `{username}`, the listing includes their unlisted and private projects; every other caller, authenticated or not, sees only that user's public projects. The current client first resolves its username through `GET /api/users/me`, then calls this route.

The route accepts `limit` (1-100, default 24) and `offset` (default 0).

A non-owner therefore gets a filtered `200`, not a `403` — the listing narrows rather than refusing, which keeps a user's existence from being probed through the status code.

GET /api/projects/{id}

Returns `{"project": <project>}` if visible to the caller.

GET /api/projects/{id}/versions

Requires auth and read access to the project. Returns newest first:

```
{"versions": [{"number": 3, "createdAt": "2026-08-03T12:00:00Z", "url": "https://example.org/api/projects/uuid/versions/3"}]}
```

Protected project history responses use `Cache-Control: private, no-store`. The existing `GET /api/projects/{id}/versions/{version}` route continues to return the immutable project document itself.

PATCH /api/projects/{id}

Requires ownership, or organization administrator / active organization creator access for organization-owned projects. Accepted fields are `title`, `description`, `visibility`, `tags`, `organizationId`, `groupIds`, and `deleteProtected`. Response: `{"project": <project>}`. An explicit null for `visibility`, `organizationId`, or `deleteProtected` where the field is non-nullable is refused with `422` rather than failing at commit.

PUT /api/projects/{id}/content

Requires ownership, organization administrator or active organization creator access for organization-owned projects, or membership in a targeted shared-update group. Creates a new immutable version.

```
{"content": "{\"version\": \"1.0\", ...}", "expectedVersion": 3}
```

`expectedVersion` is optional. When provided and it does not match the current latest version, the write still succeeds under last-write-wins and the `201` response includes a `warning` string containing the stable phrase `version conflict`. A matching or omitted version has no `warning` field.

Response 201: {"project": <project>, "version": <positive integer>}.

DELETE /api/projects/{id}

Requires ownership. Deletes metadata and stored objects. Response: 204. When the project's `deleteProtected` is `true`, the request is refused with 409 and {"error": "project is delete-protected; turn off deleteProtected before deleting it"}. The phrase `delete-protected` is stable: clients match on it to explain the refusal. Turning the switch off with `PATCH /api/projects/{id} {"deleteProtected": false}` unblocks the delete.

Deleting a project also removes its pending transfers and its redirect rows.

GET /api/shares

Requires `read:projects`. Returns {"shares": [<project>, ...]}, newest-updated first: the projects the caller manages whose `visibility` is `public` or `unlisted` (organization-visible projects are not link shares). Each entry is a project representation plus `projectSlug`. A share's `id` is its project id. Expired links stay listed so they can be revoked.

DELETE /api/shares/{id}

Requires `write:projects` and management of the project. Revokes the share: the project becomes `private` and its `role`, `expiry`, and `password` are reset. The project, its versions, and its group shares are kept. Response: 204; 403 when the caller does not manage the project; 404 when the project is unknown, already private, or organization-visible.

GET /api/projects/{id}/activity

Requires ownership. Returns the project's activity log, newest first, capped at 100 entries:

```
{
  "activity": [
    {
      "id": "...",
      "action": "visibility_change",
      "actorId": "...",
      "details": {
        "before": "private",
        "after": "public",
        "createdAt": "..."
      }
    },
    {
      "id": "...",
      "action": "open",
      "actorId": null,
      "details": {
        "date": "2026-08-21",
        "count": 40,
        "createdAt": "..."
      }
    }
  ]
}
```

Actions and their `details`: `version_save` (`version`), `fork` (`forked_project_id`), `visibility_change` (`before`, `after`), `transfer` (`from`, `to` — `"org:<slug>"` or a username), `fetch` of the raw JSON (`version`) and `open` of the project page. `actorId` is the acting account, or `null` for an anonymous visitor. Anonymous `open` and `fetch` events are **never stored per visitor**: they are aggregated into one row per action and UTC day carrying a `count`, and no IP address or other visitor fingerprint is recorded. Rows are pruned after `GEOLIBRE_ACTIVITY_RETENTION_DAYS` (default 90) the next time the project logs an event. The log is visible only to the owner and never appears in listings.

DELETE /api/projects/{id}/activity

Requires ownership. Deletes every activity row for the project. Response: 204 .

POST /api/projects/{id}/forks

Requires auth. Creates a new project owned by the caller from the visible source's latest content. The request body is **optional**: {"visibility": ...} selects the fork's visibility, and omitting the body entirely (the common "fork this project" call) must behave as {"visibility": "private"} rather than returning 422 . Responds 201 with {"project": <project>} . The source forkCount increases atomically. A fork of an organization project that is not public or unlisted stays owned by that organization, so the caller's role and the organization's public sharing policy apply to it exactly as on create.

Raw project and website-compatible routes

- GET /{username}/{slug}.geolibre.json returns the latest project document with Content-Type: application/json .
- GET /api/projects/{id}/versions/{version} returns an immutable historical document.
- GET /{username}/{slug} may return an HTML project page or redirect to the configured GeoLibre viewer. It is the projectUrl advertised by the API.
- Organization-owned equivalents are GET /org/{organizationSlug}/{slug}.geolibre.json and GET /org/{organizationSlug}/{slug} .

Every successful read of the latest raw document may increment views ; servers must not count failed or unauthorized reads.

When a project was moved by a transfer, the address it vacated answers 301 Moved Permanently to the project's new raw JSON (for the .geolibre.json route) or new page URL (for the page route). The redirect is followed only when the caller can already see the target project, so a private or organization-only project's new address is not disclosed to others: an anonymous request to a vacated private address is 404 , not 301 . Authorized redirects to private or organization-only projects use Cache-Control: private, no-store so the old path cannot retain a previously authorized destination after sign-out. Public and unlisted redirects use Cache-Control: public, no-cache so a later transfer or visibility change revalidates the destination.

Transfers

A project can be handed to another user, who must accept, or to an organization the caller administers, which applies immediately. The project keeps its id , views , forkCount , version history, and activity; only its namespace and slug change. Every transfer clears the project's group shares (they were grants to the previous audience) and records a permanent redirect for the address it vacates.

POST `/api/projects/{id}/transfers` requires ownership, or administrator membership for an organization-owned project, plus `write:projects`. An active organization creator who is not an administrator cannot transfer its property out. The body takes exactly one of `username` or `organizationId`, and an optional `slug`:

```
{"username": "bob", "slug": "wetlands"}
```

- A **user** target creates a `pending` transfer. Nothing moves until that user accepts, so the project keeps its current owner and address in the meantime. Responds `201` with `{"transfer": <transfer>, "project": <project>}`.
- An **organization** target requires that the caller administers the organization and is applied immediately (the transfer is stored as `accepted`). It responds `201` with `{"transfer": <transfer>, "project": <project>}`, the project already moved.

Refusals use stable phrases so clients can explain them:

Stat us	error	Cause
409	slug already exists for the new owner	The destination namespace already uses the requested slug; retry with another <code>slug</code> .
409	a transfer is already pending for this project	One pending transfer per project.
404	user not found	No account has that username.
422	provide exactly one of <code>username</code> or <code>organizationId</code>	Both or neither target was given.
422	project already belongs to that owner	The destination is already the owner.

Receiving and managing:

- GET `/api/transfers/incoming` lists the caller's pending offers, newest first: `{"transfers": [...]}`.
- GET `/api/transfers/outgoing` lists the pending transfers the caller started: `{"transfers": [...]}`.
- POST `/api/transfers/{id}/accept` (recipient only) moves the project. Its body is optional: `{"slug": "..."} overrides the destination slug, which is how a recipient resolves a slug conflict (409 slug already exists for the new owner). Responds 200 with {"project": <project>, "transfer": <transfer>}`. If the initiator no longer manages the project, or another request resolved the offer first, the response is `409 transfer is no longer valid`.
- POST `/api/transfers/{id}/decline` (recipient only) responds `204`.
- DELETE `/api/transfers/{id}` cancels a pending transfer (the initiator, or anyone who can still manage the project) and responds `204`.

A transfer can change visibility: a project that was `public` becomes `organization` when it moves into an organization whose `publicSharingPolicy` is not `yes`, and a project that was `organization` becomes `private` when it moves to an individual. Both changes are recorded as `visibility_change` activity.

A vacated `<username>/<slug>` (or `/org/<slug>/<slug>`) address stays **reserved** while its redirect exists: a later upload of the same title in that namespace receives a `-2` suffix rather than taking over the old link.

Both listing routes use `Cache-Control: private, no-store`.

A transfer is returned as:

```
{
  "id": "uuid",
  "projectId": "uuid",
  "projectTitle": "Wetlands",
  "projectSlug": "wetlands",
  "fromUsername": "ada",
  "toUsername": "bob",
  "toOrganization": null,
  "slug": "wetlands",
  "status": "pending",
  "createdAt": "2026-09-30T12:00:00Z",
  "resolvedAt": null
}
```

`toUsername` is `null` for an organization transfer, where `toOrganization` is `{"id", "slug", "name"}` instead. `status` is `pending`, `accepted`, `declined`, or `cancelled`. `projectSlug` is the project's slug *at the time of the response*; `slug` is the slug it will take at its destination.

Thumbnails

`PUT /api/projects/{id}/thumbnail` requires ownership and accepts the image bytes with their image content type. `GET /api/projects/{id}/thumbnail` follows project visibility. `DELETE` removes it. Upload and delete responses are `204`.

Personal token scopes

Scope	Grants
<code>read:projects</code>	List and open the caller's own projects, including unlisted/private projects; read organization/group memberships, galleries, and projects shared with the caller
<code>write:projects</code>	Create, update, delete, and fork projects the caller may manage or shared-update; create and manage organizations, groups, memberships, and invitations; change the account email
<code>share:public</code>	Create a public project or raise a project's visibility to public

New personal tokens require a nonempty subset of these scopes. Omitting `scopes` preserves the historical project permissions for existing clients. Omitting `expiresInDays` keeps the token valid until revoked (the v1 delete-only lifecycle); set `expiresInDays` to 1–365 to mint an expiring token. Tokens that predate the policy table are upgraded on first use with all three project scopes, no expiry, and a legacy marker.

A valid credential missing a required scope receives 403 with `{"error": "insufficient_scope", "requiredScope": "<scope>"}` and `WWW-Authenticate: Bearer error="insufficient_scope"`. Missing credentials use the `Bearer` challenge; malformed, unknown, revoked, and expired credentials use `Bearer error="invalid_token"`.

OAuth 2.0 sign-in (Authorization Code + S256 PKCE)

The reference server implements Authorization Code with PKCE (`S256` only) for public clients; no client secret is accepted. Registrations are exact and startup-validated through `GEOLIBRE_OAUTH_CLIENTS`. The only supported client IDs are `geolibre-web` and `geolibre-desktop`. Empty or unset configuration disables every OAuth route without changing personal-token startup behavior.

The issuer is `GEOLIBRE_PUBLIC_URL`. When OAuth is enabled it must be an absolute HTTPS URL. Loopback HTTP is allowed only for `localhost` or `127.0.0.1` with an explicit port. The request `Host` header, including its port, must match the issuer authority.

Discovery

`GET /.well-known/oauth-authorization-server` returns RFC 8414 metadata. For an issuer with path `/services/projects`, the route is `/.well-known/oauth-authorization-server/services/projects`. The document advertises the authorization, token, and revocation endpoints; authorization code and refresh grants; `S256`; the three project scopes; and `manage:sessions` (OAuth-only).

Authorization and consent

`GET /oauth/authorize` accepts one value each for `response_type=code`, `client_id`, exact `redirect_uri`, nonempty `scope`, `state`, `code_challenge`, and `code_challenge_method=S256`; `device_label` is optional. State is 16–512 URL-safe characters. The S256 challenge is the 43-character unpadded base64url SHA-256 value.

Duplicate authorization parameters, unknown clients, unregistered redirects, and state values longer than 512 characters return a local error page without a `Location` header. Other authorization errors redirect to the already validated callback with `error`, `iss`, and the exact `state` value when supplied.

`POST /oauth/authorize` submits the server-owned consent form. It requires the browser-binding cookie, CSRF value, same-origin `Origin` or `Referer`, and account credentials. Approval returns 303 to the exact callback with a single-use code, `state`, and `iss`; cancellation returns `access_denied`.

Authorization codes expire after 60 seconds by default (`GEOLIBRE_OAUTH_CODE_TTL_SECONDS`). Production HTTPS uses a host-only `Secure` browser-binding cookie; permitted loopback HTTP development uses a host-only non-`Secure` cookie so Safari can submit the consent form.

Web redirects must be absolute HTTPS URLs ending in `/oauth-callback.html` . Explicit-port loopback HTTP is allowed for development. Desktop redirects must be exactly `org.geolibre.desktop:/oauth/callback` . The installed Tauri desktop app opens consent in the system browser and receives that URI through the OS protocol handler (on macOS, Windows, and Linux), not an inbound HTTP listener. It accepts a callback only for a live, matching state and issuer. A callback that cold-launches an app with no pending verifier cannot complete sign-in: the user must restart consent. No authorization code or token belongs in a diagnostic log or a persisted project.

Token exchange and rotation

`POST /oauth/token` accepts form-urlencoded bodies up to 16 KiB:

- `grant_type=authorization_code` requires `client_id` , `code` , `redirect_uri` , and a 43–128 character `code_verifier` .
- `grant_type=refresh_token` requires `client_id` and `refresh_token` . Optional `scope` must be the same scope set as the original grant; ordering does not matter.

Success returns:

```
{
  "access_token": "opaque",
  "token_type": "Bearer",
  "expires_in": 600,
  "refresh_token": "opaque",
  "scope": "read:projects write:projects"
}
```

Request `manage:sessions` **alone** for a fresh step-up consent. Combining it with project scopes is `invalid_scope` . Its success response has `"scope":"manage:sessions"` and `"expires_in":300` (or less if the server enforces a shorter access lifetime), but **no** `refresh_token` . The server never creates a refresh row for this grant, rejects refresh attempts, and caps its access token and family at five minutes. A client must hold the management token only in memory and discard it when session management closes, the project session changes, or the grant expires. A `401` on a management request requires a new consent; it must not sign the project session out.

Access tokens expire after 600 seconds by default (`GEOLIBRE_OAUTH_ACCESS_TTL_SECONDS`) and never outlive their family. Refresh tokens are single-use and rotate on every use. Reusing a consumed refresh token revokes the entire family, including tokens minted by the successful rotation. A family expires at issuance plus the configured refresh TTL (`GEOLIBRE_OAUTH_REFRESH_TTL_SECONDS` , 30 days by default); rotation never extends it.

An enabled server deletes bounded batches of expired interactions, access tokens, and families at startup, during OAuth requests, and every five minutes while running. Consumed refresh generations stay until the family expires so replay detection remains effective.

`POST /oauth/revoke` accepts `client_id`, `token`, and optional advisory `token_type_hint`. A matching access or refresh token revokes its entire family. Unknown, already-revoked, and wrong-client tokens all return the same empty `200`.

OAuth failures use `invalid_request`, `invalid_client`, `invalid_grant`, `invalid_scope`, `unsupported_grant_type`, or `unsupported_token_type`. Token and revocation responses are `no-store`. Raw codes and tokens are returned once; the database stores only SHA-256 digests.

Project OAuth access tokens use the same project scope matrix as personal tokens. `manage:sessions` authorizes only the session-management routes documented above. It is exclusive to OAuth consent, never available to personal tokens; it grants no project read or write access. `admin:org` remains reserved and is rejected.

Compatibility

The API is additive within version 1. Implementations must not repurpose fields or narrow visibility rules. New optional fields and endpoints may be added. Breaking changes require a new `/api/v2` namespace. The conformance baseline is the frontend tests for `share-geolibre.ts` and `share-gallery.ts`, plus the reference server's API tests.

<https://geolibre.app/android/>

Android

GeoLibre runs as a native Android app built from the same React codebase via **Tauri v2 mobile** — no separate app. The webview UI is bundled in the APK, so the app shell works offline; map tiles and the heavier engines are fetched on demand (same as the desktop build).

Install

GeoLibre is published on [Google Play](#). The Play build is signed and updates automatically:

[Get GeoLibre on Google Play](#)

Prefer to sideload? Every [release](#) attaches signed, per-ABI APKs — see [Downloads](#). The rest of this page is for developers building the app themselves.

What works on Android vs desktop

The Android build ships the full map workspace, Add Data, the Whitebox geoprocessing toolbox (1,000+ tools, run in the browser on WebAssembly), the Vector tools (Turf.js / in-browser GeoPandas via Pyodide), the SQL Workspace (DuckDB-WASM and the in-browser PGlite/PostGIS engine), the Python Console (Pyodide), geocoding, statistics, the AI assistant, story maps, and plugins.

Tools that depend on a **local desktop process** are hidden on mobile, because Android has no Python sidecar or local helper binaries:

- Processing → GeoLibre Toolbox → **Conversion, AI Segmentation**, and the sidecar-backed **Raster** tools (all need the Python sidecar; the Raster submenu keeps only the client-side **Download Global DEM** tool)
- Add Data → **PostgreSQL** (served by the local Martin tile server) and **SQL Server**

These are gated by a user-agent `isMobile()` check, so the Processing menu, the Add Data menu, and the Layer panel's add-data group never offer them. Everything else runs client-side.

The database sources have one entry point that check does not cover: the Browser panel keeps its **Databases** section on every platform for discovery, so its `+` still opens the connection dialog. The dialogs gate on `isDesktopRuntime() ( isTauri() && !isMobile() )` rather than on `isTauri()` alone, so on Android they show the "requires GeoLibre Desktop" notice and disable Connect instead of calling a helper that cannot exist (GeoLibre#2091).

Toolchain setup (one time)

You need the Android SDK + NDK, a JDK (17 or 21 — newer JDKs can break the Android Gradle Plugin), and the Rust Android targets. The cleanest, sudo-free layout keeps everything under a user-writable SDK at `~/Android/Sdk`.

```
# 1. JDK 17/21 (or reuse Android Studio's bundled JBR at /opt/android-studio/jbr)
export JAVA_HOME=/path/to/jdk-21

# 2. Android SDK components (sdkmanager ships with Android Studio cmdline-tools)
export ANDROID_HOME="$HOME/Android/Sdk"
yes | sdkmanager --sdk_root="$ANDROID_HOME" --licenses
sdkmanager --sdk_root="$ANDROID_HOME" \
  "platform-tools" "platforms;android-36" \
  "build-tools;36.0.0" "ndk;27.3.13750724"
export NDK_HOME="$ANDROID_HOME/ndk/27.3.13750724" # Tauri needs NDK_HOME

# 3. Rust + the four Android targets (install rustup if you don't have it)
rustup target add aarch64-linux-android armv7-linux-androideabi \
  i686-linux-android x86_64-linux-android
```

NDK **r27 (LTS)** is the supported line for Tauri v2. Add the four `export` s to your shell profile so every session has them.

API **36** (Android 16), not 34: the Tauri v2.11 Android template generates `compileSdk = 36 / targetSdk = 36`, so Gradle needs the matching platform installed. It is also Google Play's floor — from **2026-08-31** new apps and updates must target API 36 to be accepted.

16 KB page sizes

Google Play rejects apps targeting Android 15+ whose native libraries are not aligned for 16 KB memory pages, and such libraries fail to load on 16 KB devices. NDK r28+ does this by default; **r27 does not**, so the flags are passed explicitly. Export this before an Android build (CI sets it at workflow level in `.github/workflows/android.yml`):

```
export RUSTFLAGS="-C link-arg=-Wl,-z,max-page-size=16384 -C link-arg=-Wl,-z,common-page-size=16384"
```

It has to be the `RUSTFLAGS` **environment variable**. Putting the same flags in `target.<triple>.rustflags` in a `.cargo/config.toml` does *not* work: the Tauri CLI sets `RUSTFLAGS` itself when it invokes cargo for Android, and an env `RUSTFLAGS` overrides the config file outright. The config-file form is silently ignored — it parses, it builds, and it ships 4 KB-aligned libraries that Play rejects. Tauri appends to an inherited value, so exporting it works.

Check the result on a built APK — the bytes Play actually receives:

```
unzip -o -q app-arm64-release-unsigned.apk 'lib/**/*.so' -d /tmp/apkcheck
"$NDK_HOME/toolchains/llvm/prebuilt/linux-x86_64/bin/llvm-objdump" -p \
  /tmp/apkcheck/lib/**/*.so | awk '$1 == "LOAD" { print $NF }' | sort -u
# every value must be 2**14 or greater; 2**12 means the flags did not apply
```

Use `llvm-objdump -p` rather than `readelf -l`: `readelf` wraps each LOAD across two lines, so it is easy to parse the wrong column and read an address as an alignment. CI runs this same check over every packaged `.so` and fails on a regression.

Build

```
cd apps/geolibre-desktop
npx tauri android init
npx tauri android build --apk --split-per-abi
npx tauri android build --aab
```

generate src-tauri/gen/android (once)
release APKs, one per ABI (~40 MB each)
universal AAB for Google Play

- `gen/android` is generated (git-ignored) and regenerated on demand.
- Build **release**, not `--debug`: the stripped, size-optimized Cargo profile makes each APK ~40 MB; a debug build is ~200 MB (unstripped `.so` with debuginfo).
- `--split-per-abi` emits one APK per architecture instead of a single ~150 MB universal APK. Install the `arm64` one on real phones.
- Output: `src-tauri/gen/android/app/build/outputs/apk/<abi>/release/app-<abi>-release-unsigned.apk`, where `<abi>` is Tauri's short name — `arm64`, `arm`, `x86`, `x86_64` — *not* the Android ABI directory name (`arm64-v8a`, `armeabi-v7a`) that appears inside the APK under `lib/`.
- Sideload/GitHub-release path: the per-ABI **APKs**.
- Google Play path: the universal **AAB** (Play generates per-device splits from it). Don't `--split-per-abi` the AAB — Play wants the one bundle.

The app is named **GeoLibre** on Android (the desktop build is "GeoLibre Desktop") and uses the package id `org.geolibre.app`, both set via `src-tauri/tauri.android.conf.json`, which also drops the Python backend from the Android bundle.

The Android id is overridden there rather than in `tauri.conf.json` on purpose: `identifier` is shared by every platform and also determines the macOS bundle ID, the Linux AppStream id, and the webview data directory. Changing it globally would orphan existing desktop users' settings and break the Linux/COPR/Homebrew packaging, all of which still key off `org.geolibre.desktop`.

Signing

Release APKs are unsigned. To install one, sign it (a debug key is fine for testing; use a real key for distribution):

```
BT="$ANDROID_HOME/build-tools/36.0.0"
KS="$HOME/.android/debug.keystore" # auto-created by Android tooling; or make your own
# -P 16, not -p: -p only guarantees 4 KB, and Play requires the .so to sit on a
# 16 KB boundary inside the zip. Same flag CI uses.
"$BT/zipalign" -P 16 -f 4 app-arm64-release-unsigned.apk aligned.apk
"$BT/apksigner" sign --ks "$KS" --ks-pass pass:android \
  --ks-key-alias androiddebugkey --key-pass pass:android \
```

```
--out geolibre-arm64.apk aligned.apk
"$BT/apksigner" verify geolibre-arm64.apk
```

The example signs the arm64 APK. Substitute the ABI you need throughout — `app-x86_64-release-unsigned.apk` → `geolibre-x86_64.apk`, and likewise for `armeabi-v7a` / `x86`. The emulator section below installs `geolibre-x86_64.apk`, which is this same walkthrough with `arm64` swapped for `x86_64`.

For a real upload/release key:

```
keytool -genkeypair -v -keystore upload.jks -alias upload -keyalg RSA \
-keysize 2048 -validity 10000
```

Continuous integration

`.github/workflows/android.yml` builds **signed**, per-ABI release APKs on each published GitHub release (and on demand via the "Run workflow" button) and uploads them as the `geolibre-android-release-apks` artifact. On a published release the APKs are **also attached to that release** as downloadable assets — but only when they carry a real release signature; debug-signed APKs stay a CI artifact and the run logs a warning saying so. It signs with your release keystore when these repository secrets are set, and otherwise falls back to a throwaway debug key so the artifact is still installable for testing:

- `ANDROID_KEYSTORE_BASE64` — `base64 -w0 upload.jks`
- `ANDROID_KEYSTORE_PASSWORD`
- `ANDROID_KEY_ALIAS`
- `ANDROID_KEY_PASSWORD`

It also builds a universal **AAB** and uploads it as the separate `geolibre-android-play-aab` artifact — but *only* on runs that have the real release keystore, since Play rejects a debug-signed bundle. Without the keystore the AAB build is skipped entirely rather than built and discarded. On a published release the `geolibre-android.aab` is attached to the release alongside the APKs, so the exact bundle submitted for a tag stays recoverable after the CI artifact expires.

Sideload? Take an `.apk`. The `.aab` is the Google Play upload format and cannot be installed on a device.

Install / test

On a phone

1. Enable **Developer options** (tap Build number 7×) and **USB debugging**.
2. Sideload the signed APK:

```
adb install -r geolibre-arm64.apk
```

Or copy the APK to the phone and tap it (allow "install unknown apps").

For live development with hot reload, connect the device and run `npm run tauri android dev`.

On an emulator

```
sdkmanager --sdk_root="$ANDROID_HOME" \
  "emulator" "system-images;android-36;google_apis_playstore;x86_64"
avdmanager create avd -n geolibre \
  -k "system-images;android-36;google_apis_playstore;x86_64" -d pixel_7
emulator -avd geolibre
# x86_64 APK to match the x86_64 system image – the emulator can translate the
# arm64 build, but far slower, and it would not exercise the x86_64 libraries.
adb install -r geolibre-x86_64.apk
```

If you ever rebuild with a **different** signing key, uninstall the old copy first (`adb uninstall org.geolibre.app`) — Android rejects updates whose signature changed. This also applies when moving between a sideloaded APK and the Play build: Play App Signing re-signs with Google's key, so the two are not upgrade-compatible.

Publishing to Google Play

GeoLibre is live on Play as [org.geolibre.app](https://play.google.com/store/apps/details?id=org.geolibre.app); this section records the onboarding for reference and for anyone publishing a fork. Only step 3 recurs per release: build the AAB with the upload key and bump the `versionCode`. The build side is covered by the CI workflow above; the rest is one-time Play Console onboarding.

1. **Developer account** (\$25, one-time). Register as an **organization** rather than a personal account if you can: personal accounts created after 2023-11-13 must run a closed test with **12 opted-in testers for 14 consecutive days** before they can apply for production access. Organization accounts are exempt.
2. **Play App Signing**. Upload `upload.jks` as the *upload* key; Google holds the actual app signing key and re-signs each bundle. The repository's `ANDROID_KEYSTORE_*` secrets are that upload key — keep the keystore backed up, since losing it requires a Play support reset.
3. **Upload the AAB** from the `geolibre-android-play-aab` CI artifact, or from the release's `geolibre-android.aab` asset once that artifact has expired. The `versionCode` is derived from the version in `tauri.conf.json` and must increase on every upload.
4. **Store listing assets**: 512×512 icon, a **1024×500 feature graphic**, and at least two phone screenshots. Add 7-inch and 10-inch tablet screenshots too — Play down-ranks apps without them, and a GIS workspace is genuinely tablet-appropriate.
5. **Privacy policy URL** — point at the published [privacy policy](#).

6. **Data safety form.** Declare each network destination honestly: geocoding, the AI assistant, basemap/tile fetches, and Google OAuth for Earth Engine. Note which are *transmitted* versus *collected* — GeoLibre does not operate a backend that retains user data, but the form asks per-purpose.
7. **Content rating** questionnaire and target audience.

Keep *Known limitations* below in mind for each update: several Add Data paths are still inert on Android. A user tapping one and getting nothing is a one-star review, so gate them on mobile the way the sidecar tools already are via `isMobile()`.

Known limitations / follow-ups

- Local-file sources (MBTiles, local rasters, project files) assume real filesystem paths; Android scoped storage returns content URIs, so those flows need adapting before they work natively.
- The **Download Offline Area** tool relies on a service worker, which the Tauri builds (desktop and Android) don't use — it's a PWA feature. Native offline basemap caching (bundled/downloaded MBTiles/PMTiles) is a future enhancement.
- Earth Engine OAuth uses a desktop loopback/multi-window flow; a mobile deep-link redirect is future work.

Open a location from another app

GeoLibre handles Android `ACTION_VIEW` intents with the `geo:` scheme, both when launching the app and while it is already open. Supported forms include:

- `geo:40.7128,-74.006`
- `geo:40.7128,-74.006?z=12`
- `geo:0,0?q=40.7128,-74.006(New%20York)&z=12`

Coordinates use latitude, longitude order. Numeric `q` coordinates override the URI's placeholder coordinates; address-only queries are not supported. An optional third coordinate in a direct URI is altitude in meters; it is validated and ignored when positioning the map. Zoom defaults to 14 and must be between 0 and 24. Invalid locations are ignored. A received location moves the map without replacing the current project's layers.

The Tauri deep-link plugin generates the Android intent filter from `tauri.android.conf.json`, so it also applies when CI regenerates `gen/android`. To check both a cold launch and a running app on a connected device, run this command twice, panning the map between invocations:

```
adb shell am start -a android.intent.action.VIEW -d 'geo:0,0?q=40.7128,-74.006&z=12'
org.geolibre.app
```


Web links can open the same location with `https://geolibre.app/?`

`lat=40.7128&lon=-74.006&zoom=12` or the compact form

`https://geolibre.app/?12/40.7128/-74.006`. A coordinate-only launch takes precedence over saved startup settings and skips onboarding. An explicit project or data link retains its existing camera/loading behavior when combined with coordinate parameters.

<https://geolibre.app/ios/>

iOS

GeoLibre runs as a native iOS app built from the same React codebase via **Tauri v2 mobile** — no separate app. The webview UI (WKWebView) is bundled in the app, so the shell works offline; map tiles and the heavier engines are fetched on demand (same as the desktop and Android builds).

Install

GeoLibre is published on the [App Store](#) for iPhone and iPad. The Store build is signed and updates automatically:

Get GeoLibre on the App Store

GeoLibre does not publish a sideloadable `.ipa` for end users; the App Store listing, and TestFlight for beta builds, are how it is distributed. Each release *does* carry a `GeoLibre_<version>_ios_app-store.ipa` asset, but unlike the Android APKs that is the App Store **submission** package — archived next to the tag so a submission stays reproducible — and it cannot be installed on a device. The rest of this page is for developers building the app themselves, which has to happen on a Mac (iOS cannot be cross-compiled from Linux).

The signing and archiving pipeline behind that listing runs two ways, and either produces a submittable `.ipa` — bundle id `org.geolibre.app`, signed by *Apple Distribution*, `get-task-allow=false`, an embedded App Store provisioning profile, the merged location usage string, and the web assets embedded in the binary:

- **Locally** on a Mac (Xcode 26.6 / iOS SDK 26.5): `tauri ios init` → unsigned archive → `xcodebuild -exportArchive`.
- **In CI**, on a `macos-15` runner (Xcode 26.3): it imports the certificate and profile from the `APPLE_IOS_*` secrets, archives, exports under manual signing, verifies the signature, and publishes the `geolibre-ios-ipa` artifact.

What works on iOS vs desktop

Same split as Android. The iOS build ships the full map workspace, Add Data, the Vector tools (Turf.js / in-browser GeoPandas via Pyodide), the SQL Workspace (DuckDB-WASM and PGlite/PostGIS), the Python Console (Pyodide), geocoding, statistics, the AI assistant, story maps, and plugins.

Tools that depend on a **local desktop process** are hidden on mobile because iOS has no Python sidecar or local helper binaries and its sandbox forbids spawning subprocesses:

- Processing → GeoLibre Toolbox → **Conversion, AI Segmentation**, and the sidecar-backed **Raster** tools (all need the Python sidecar; the Raster submenu keeps only the client-side **Download Global DEM** tool). The Whitebox geoprocessing toolbox runs in WebAssembly, needs no sidecar, and stays available.
- Add Data → **PostgreSQL** (served by the local Martin tile server) and **SQL Server**

These are gated by a user-agent `isMobile()` check (which already matches iPhone/iPad), so the Processing menu, the Add Data menu, and the Layer panel's add-data group never offer them. Everything else runs client-side.

The database sources have one entry point that check does not cover: the Browser panel keeps its **Databases** section on every platform for discovery, so its + still opens the connection dialog. The dialogs gate on `isDesktopRuntime()` (`isTauri()` && `!isMobile()`) rather than on `isTauri()` alone, so on iOS they show the "requires GeoLibre Desktop" notice and disable Connect instead of calling a helper that cannot exist (GeoLibre#2091).

Unlike Android, the App Store build also hides **Earth Engine**: signing in from the packaged app needs a loopback OAuth listener, which the Apple App Store builds compile out (`src-tauri/src/lib.rs`) so the app binds no listening socket.

Location permission (required)

This is the one place iOS differs sharply from Android. Android's geolocation plugin declares the runtime permission and the OS shows a generic dialog; **iOS terminates the app the instant it requests location if no usage-description string is present**. GeoLibre supplies it in `src-tauri/Info.ios.plist`:

```
<key>NSLocationWhenInUseUsageDescription</key>
<string>GeoLibre uses your location to center the map, capture GPS points during Field Collect
ion, and record GPS tracks.</string>
```

Tauri merges `Info.ios.plist` into the generated `gen/apple/geolibre-desktop_iOS/Info.plist` at build time. `gen/apple` is git-ignored, so this file is the durable home for the string — the same reason Android's manifest permissions come from the plugin rather than a hand-edited, regenerated manifest. It covers all three location consumers: Field Collection, GPS Tracking, and the Controls → GeoLocate map control. After a build, confirm the key survived the merge (see *Build* below).

Minimum iOS version

GeoLibre targets **iOS 15.0**, set once in `apps/geolibre-desktop/src-tauri/tauri.ios.conf.json`:

```
"bundle": { "iOS": { "minimumSystemVersion": "15.0" } }
```

Tauri templates that value into the generated `gen/apple/project.yml` as XcodeGen's `deploymentTarget.iOS`, which Xcode turns into `IPHONEOS_DEPLOYMENT_TARGET` and finally the app's `MinimumOSVersion`. There is nothing to hand-edit in the generated project, and nothing in `Info.ios.plist`: Xcode writes the plist key from the build setting.

Leave the key out and the **Tauri CLI's own default applies**, which is `14.0`. That is below Apple's floor: starting Spring 2027, App Store Connect refuses any upload with a `MinimumOSVersion` under 15.0. Until then it accepts the build and sends a warning email afterward (`ITMS-90068`), which is how version 2.5.0 build 7 was found to have shipped at 14.0. That build is the one that was approved, so the live App Store listing still reads *iOS 14.0 or later*; the listing picks up 15.0 with the next release.

Because `gen/apple` is generated, a change here only takes effect once the project is regenerated. CI regenerates on every run, so it picks the value up automatically; locally, regenerate it yourself (paths relative to `apps/geolibre-desktop`, as everywhere else in this document):

```
cd apps/geolibre-desktop
rm -rf src-tauri/gen/apple
npx tauri ios init

# What init generated:
grep -A2 'deploymentTarget:' src-tauri/gen/apple/project.yml

# What a build actually shipped. `MinimumOSVersion` exists only in the *built*
# app, since Xcode writes it from the build setting; it is not in the generated
# source Info.plist. So this needs an archive, which `init` alone does not
# produce.
npx tauri ios build --no-sign
/usr/libexec/PlistBuddy -c 'Print :MinimumOSVersion' \
  src-tauri/gen/apple/build/geolibre-desktop_iOS.xcarchive/Products/Applications/GeoLibre.app/
Info.plist
```

The CI job asserts this in two places (see *Continuous integration*): the configured value against Apple's floor and against the generated `project.yml` right after `ios init`, and then the exported `.ipa`'s `MinimumOSVersion`, which is the copy App Store Connect actually reads.

Raising the floor further is a product decision, not a technical one. iOS 15 is the last release for the iPhone 6s/7 and iPad Air 2 generation, so 16.0 would drop those devices in exchange for a newer WebKit baseline.

Toolchain setup (one time)

You need a **Mac** with **Xcode** (from the App Store; open it once to accept the license and install the iOS platform), the command-line tools, CocoaPods, and the Rust iOS targets.

```
xcode-select --install          # command-line tools (if not already)
sudo xcodebuild -license accept
brew install cocoapods          # or `gem install cocoapods`

# Rust device + simulator targets (install rustup first if needed)
rustup target add aarch64-apple-ios aarch64-apple-ios-sim x86_64-apple-ios
```

`aarch64-apple-ios` is real devices and the App Store `.ipa`; `aarch64-apple-ios-sim` is the simulator on Apple Silicon Macs.

Build

```
cd apps/geolibre-desktop
npx tauri ios init           # generate src-tauri/gen/apple (once)
npx tauri ios dev            # run in the simulator / on a tethered device
npx tauri ios build --no-sign # unsigned .ipa + .xcarchive (no Apple account needed)
```

`npx tauri ios build --export-method app-store-connect` **does not work as-is**. The project `tauri ios init` generates bakes `CODE_SIGN_IDENTITY = "iPhone Developer"` into *both* the debug and release configurations, so Xcode resolves **development** signing even when the export method is App Store. Automatic development signing then demands a registered device and fails with:

```
error: Your team has no devices from which to generate a provisioning profile.
error: No profiles for 'org.geolibre.app' were found: Xcode couldn't find any
      iOS App Development provisioning profiles matching 'org.geolibre.app'.
```

Overriding `CODE_SIGN_IDENTITY="Apple Distribution"` does not help either — Xcode rejects it as *"automatically signed for development, but a conflicting code signing identity ... has been manually specified."*

Split the archive from the export instead. Archive **unsigned**, then let the export step do the distribution signing — this needs no registered device, and Xcode creates the Apple Distribution certificate, registers the App ID, and generates the App Store profile on the fly:

```
cd apps/geolibre-desktop
npx tauri ios build --no-sign # → gen/apple/build/geolibre-desktop_iOS.xcarchive

G=src-tauri/gen/apple
cat > "$G/ExportOptions-appstore.plist" <<'PLIST'
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/
PropertyList-1.0.dtd">
<plist version="1.0"><dict>
  <key>method</key><string>app-store-connect</string>
  <key>teamID</key><string>YOUR_TEAM_ID</string>
  <key>signingStyle</key><string>automatic</string>
  <key>uploadSymbols</key><true/>
</dict></plist>
PLIST

xcodebuild -exportArchive -allowProvisioningUpdates \
  -archivePath "$G/build/geolibre-desktop_iOS.xcarchive" \
  -exportOptionsPlist "$G/ExportOptions-appstore.plist" \
  -exportPath "$G/build/appstore"
```

This produces a genuinely submittable `build/appstore/GeoLibre.ipa`: signed by `Apple Distribution`, `get-task-allow=false`, entitlement `<TEAM>.org.geolibre.app`, and an embedded *iOS Team Store Provisioning Profile*. Verify with `codesign -dvvv` before uploading.

Note the certificate Xcode creates this way is **Cloud Managed**, so it does *not* appear in `security find-identity -v -p codesigning`. An empty identity list is not evidence the signing failed — check the `.ipa` itself.

- `gen/apple` is generated (git-ignored) and regenerated on demand. `init` applies `tauri.ios.conf.json` (bundle id, deployment target, drops the Python backend); `Info.ios.plist` (the location string) is merged later, by `tauri ios build / dev`, as noted below.
- The app is named **GeoLibre** on iOS (the desktop build is "GeoLibre Desktop") and uses the bundle id `org.geolibre.app`, both set via `src-tauri/tauri.ios.conf.json` — the same override pattern and reasoning as Android (`identifier` in `tauri.conf.json` stays `org.geolibre.desktop` so it keeps keying desktop settings and the Linux/macOS packaging).
- Verify the location string landed after a build:

```
/usr/libexec/PlistBuddy -c 'Print :NSLocationWhenInUseUsageDescription' \
src-tauri/gen/apple/geolibre-desktop_iOS/Info.plist
```

The merge happens during `tauri ios build / dev`, **not** during `ios init` — straight after `init` the generated plist does not contain the key yet, and that is expected. Note also that the Xcode project, target, and plist directory are named from the **Cargo package** (`geolibre-desktop` in `src-tauri/Cargo.toml`), not from `productName`. The user-visible app name and bundle id still come from `tauri.ios.conf.json` — they land as `PRODUCT_NAME: GeoLibre` and `PRODUCT_BUNDLE_IDENTIFIER: org.geolibre.app` in the generated `project.yml`.

Signing

Every build that runs on a real device or reaches the App Store must be signed — there is no debug-keystore shortcut like Android's. You need:

1. An **Apple Developer Program** membership (\$99/year).
2. A **signing certificate** (Apple Distribution for App Store; Apple Development for device testing), exported from Keychain as a `.p12`.
3. A **provisioning profile** for the `org.geolibre.app` app id.
4. Your **Team ID** (App Store Connect → Membership).

Locally, opening `gen/apple/geolibre-desktop.xcodeproj` in Xcode once and enabling "Automatically manage signing" with your team is the simplest path. For CI, the identity is imported from secrets (below).

The local and CI identities are not interchangeable. The export recipe above lets Xcode create a **Cloud Managed** Apple Distribution certificate and an Xcode-managed profile. Neither can be fed to CI: a cloud-managed identity has no exportable private key, so there is no `.p12` to base64, and CI's manual-signing export rejects an Xcode-managed profile outright ("*is Xcode managed, but signing settings require a manually managed profile*"). For CI you must create a **separate, manually managed** pair in the Developer portal — an Apple Distribution certificate from a CSR you generate in Keychain Access (so you hold the private key and can export the `.p12`), plus an App Store provisioning profile for `org.geolibre.app` bound to it. Holding both a cloud-managed and a manual distribution certificate at once is fine and normal.

Continuous integration

`.github/workflows/ios.yml` runs on `macos-15` on each published GitHub release (and on demand via "Run workflow"). Because iOS can't be cross-compiled from Linux, this is the only mobile workflow that needs a macOS runner. The image is not free to choose — see the Xcode floor below.

- **With Apple signing secrets set**, it imports the identity into a throwaway keychain, archives **unsigned**, exports a signed `.ipa` under manual signing, verifies the bundle id, the signature, the build number shape and the `MinimumOSVersion`, and uploads it as the `geolibre-ios-ipa` artifact — plus, on a published release, as the `GeoLibre_<version>_ios_app-store.ipa` release asset, which outlives the artifact's 14-day retention:
 - `APPLE_IOS_CERTIFICATE_BASE64` — `base64 -i dist.p12`
 - `APPLE_IOS_CERTIFICATE_PASSWORD` — only if the `.p12` is password-protected
 - `APPLE_IOS_PROVISIONING_PROFILE_BASE64` — `base64 -i profile.mobileprovision`
 - `APPLE_TEAM_ID` — **reused** from the existing macOS/Homebrew signing secrets; the Team ID is account-wide.
- **Without them**, it falls back to a no-signing **compile check** (`cargo build --lib --release --target aarch64-apple-ios`) so CI still catches iOS build breakage; it just can't produce an installable `.ipa`. A release run in that state attaches nothing and logs a warning saying which secrets were missing, rather than failing the release.

Reusing the macOS/Homebrew signing secrets?

Only the **Team ID**. The repo's release workflow already signs and notarizes the macOS build for the Homebrew cask, but its `APPLE_CERTIFICATE` is a **Developer ID Application** certificate — that type signs macOS apps distributed *outside* the App Store and **cannot sign an iOS app**. iOS needs its own **Apple Distribution** certificate and a **provisioning profile** (Developer ID distribution has neither), so those are the `APPLE_IOS_*` secrets above, deliberately named apart from the macOS ones to avoid feeding in the wrong cert/password. The good news: they're created under the **same paid Apple Developer account** at no extra cost — add an Apple Distribution certificate and an App Store provisioning profile for `org.geolibre.app` in the Developer portal, and reuse the existing `APPLE_TEAM_ID`.

Why the job archives and exports as two steps. It archives with `npx tauri ios build --no-sign` and then signs in a separate `xcodebuild -exportArchive` step using **manual** signing, for the reason given under *Build*: `tauri ios build --export-method` cannot sign this project, because the generated project bakes in `CODE_SIGN_IDENTITY = "iPhone Developer"` and automatic signing therefore hunts for a development profile no matter what the export method says.

This has been verified on a runner — a `workflow_dispatch` run with `export_method: app-store-connect` completed green, reporting `IPA ... bundle id org.geolibre.app`, signed by Apple Distribution and publishing the `geolibre-ios-ipa` artifact. A build (version 2.5.0, build 7) has since been through App Store review and is live. What remains unexercised is the **release-triggered** entry point: the job has only ever been started manually.

The runner's Xcode sets a hard floor. `tauri ios init` generates an Xcode project in **object format 77**, which only **Xcode 16+** can open. On `macos-14` the archive failed immediately, before compiling anything:

```
xcodebuild: error: Unable to load workspace '../geolibre-desktop.xcodeproj/project.xcworkspace/'.
Reason: The project 'geolibre-desktop' cannot be opened because it is in a future Xcode project file format (77).
```

The job therefore runs on `macos-15` and explicitly selects the newest Xcode on the image rather than trusting its default, failing early with a clear message if that is older than 16. Note this floor comes from the **Tauri CLI's** generated project, not from anything in this repo — re-check it whenever the Tauri CLI is bumped.

Separately, App Store Connect rejects uploads built against an SDK below its current floor. That failure is *silent* at build time — an older-but-openable Xcode still produces an `.ipa` and only fails at upload. The Xcode version and iOS SDK list are printed into the run log so a rejected upload can be diagnosed from the run instead of guessed at.

The `workflow_dispatch export_method` input picks the export path (`app-store-connect` for TestFlight/App Store, `release-testing` for ad-hoc registered devices, `debugging` for development).

Run and test a local build

- **Simulator:** `npx tauri ios dev` and pick a simulator, or open the Xcode project and Run. No paid account needed for the simulator.
- **Your own device:** tether it, open `gen/apple/geolibre-desktop.xcodeproj` in Xcode, select the device, and Run (a free Apple ID allows 7-day device signing).
- **Testers:** distribute a signed build through **TestFlight** (upload the `.ipa` via Xcode Organizer or Transporter, then invite testers in App Store Connect).

Publishing to the App Store

GeoLibre is **live on the App Store**, so the one-time onboarding below is finished; each step it covers is marked (*Done.*) and only needs revisiting when something it declares changes. Every subsequent release repeats step 3, upload a build and submit it for review, under the standing constraint in step 2. The build side is covered by the CI workflow; the rest is App Store Connect.

1. **App record.** (*Done.*) In App Store Connect, create a new app with the bundle id `org.geolibre.app` (register the app id in the Developer portal first). GeoLibre's record is Apple ID `6796039674`, listed under Productivity.
2. **Minimum Functionality (Guideline 4.2).** Apple rejects apps that are "a repackaged website." GeoLibre passes because it's a bundled native app with real device integration — GPS, offline-capable map workspace, local file handling — not a wrapper that loads a remote URL. Keep it that way: ship the web assets in the binary (the default here), don't point the webview at `geolibre.app`.
3. **Upload** the `.ipa` (via Transporter or Xcode Organizer) to a TestFlight build, then submit that build for App Store review. Take it from the `geolibre-ios-ipa` CI artifact, or — for an older tag whose artifact has expired — from that release's `GeoLibre_<version>_ios_app-store.ipa` asset.

Build numbers. App Store Connect consumes a `CFBundleVersion` permanently per marketing version, and each upload must also sort above the last one consumed, so every upload needs a fresh, higher value. CI stamps the archive's `CFBundleVersion` with `$GITHUB_RUN_NUMBER` — an integer that only ever rises — while `CFBundleShortVersionString` keeps tracking `tauri.conf.json` and stays the version users see.

Re-running an existing run appends `$GITHUB_RUN_ATTEMPT` (`42` becomes `42.2`), because the run number itself is fixed for a run's lifetime: without the suffix, re-running a run whose IPA was already uploaded would regenerate the same build number. The one hand-uploaded 2.4.0 build used `2.4.0`, which a bare run number of 3 or more already beats under Apple's dotted-integer comparison.

This is deliberately **not** done with `tauri ios build --build-number`: that flag *appends* to the version, producing e.g. `2.4.0.42`. A `CFBundleVersion` may hold at most **three** period-separated integers, so App Store Connect rejects a four-component value at upload (`ITMS-90060`) — after the whole build has run. The verify step asserts the shape for that reason.

Uploading by hand (no CI) means setting the build number yourself if the marketing version has been uploaded before. 4. **Store listing.** (*Done.*) Icon (already generated under `src-tauri/icons/ios`), screenshots for the required device sizes (6.7" and 6.5" iPhone, plus 12.9" iPad — a GIS workspace is genuinely iPad-appropriate), description, keywords. 5. **Privacy.** (*Done.*) Fill the **App Privacy** questionnaire honestly — declare each network destination (geocoding, the AI assistant, basemap/tile fetches, Google OAuth for Earth Engine) and that location is used *when in use* and not collected by a backend. Point the privacy policy URL at the published [privacy policy](#). 6. **Age rating and category.** (*Done.*) GeoLibre is listed under **Productivity**. 7. **Export compliance.** (*Done.*) Already declared in `src-tauri/Info.ios.plist`:

```
<key>ITSAppUsesNonExemptEncryption</key>
<false/>
```

App Store Connect asks about encryption on *every* upload, and a build with the question unanswered cannot be submitted or sent to testers ("Missing Compliance"). Declaring it in the plist answers it once, at build time.

`false` asserts that GeoLibre's use of encryption stays limited to what Apple treats as exempt — HTTPS/TLS via the system libraries for tiles, geocoding, the AI assistant, cloud catalogs, and the collaboration relay — and that the app ships no cryptography of its own. That holds today, but it is a **legal declaration**, not a build flag: revisit it if that ever changes.

Builds uploaded before this key was added (the first 2.4.0 upload) still need the question answered by hand in App Store Connect.

Known limitations / follow-ups

Most mirror Android:

- Local-file sources (MBTiles, local rasters, project files) assume real filesystem paths; iOS hands apps sandboxed URLs via the document picker, so those flows need adapting before they work natively.
- The **Download Offline Area** tool relies on a service worker, which the Tauri builds don't use — it's a PWA feature. Native offline basemap caching is future work.
- Earth Engine is hidden in the App Store build because its OAuth uses a desktop loopback flow; a mobile deep-link redirect (an iOS URL scheme / universal link) is future work.
- iPadOS multitasking (Split View / Stage Manager) hasn't been tuned; the responsive layout should adapt, but verify on a real iPad before release.

<https://geolibre.app/mac-app-store/>

Mac App Store

GeoLibre Desktop is published on the [Mac App Store](#):

[Get GeoLibre Desktop on the Mac App Store](#)

For the user-facing comparison of the two macOS builds, see [Downloads](#). This page documents how the Store build is produced and why it differs.

GeoLibre Desktop ships on macOS two ways:

- **Developer ID** (the default): the notarized `.dmg` published by `release.yml` on every GitHub release and distributed through the website and Homebrew. Full feature set, hardened runtime, no App Sandbox.
- **Mac App Store (MAS)**: a sandboxed variant built by the `mas-store.yml` workflow (or `npm run tauri:build:mas` locally). This page documents that variant.

The App Store imposes two hard rules the default build cannot satisfy:

1. **App Sandbox is mandatory.** The app is confined to its container plus files the user explicitly picks in an open dialog. Spawning downloaded, unsigned executables is impossible.
2. **Guideline 2.5.2:** the app may not download or execute code that changes its features. The default build bootstraps `uv`, a Python environment, and the martin tile server at runtime, all of which are downloads of executable code.

So the MAS build compiles those services out instead of shipping them broken.

What the MAS build removes

The `mas` cargo feature (`apps/geolibre-desktop/src-tauri`) replaces these Tauri commands with stubs, and `GEOLIBRE_MAS_BUILD=1` hides the matching UI:

- **Python sidecar** (Whitebox server engine, Conversion/Raster sidecar engines, PostGIS endpoints, SamGeo segmentation, SedonaDB SQL).
- **Jupyter server** (the Notebook panel falls back to the bundled JupyterLite, which the web build already uses; Pyodide executes inside WebKit, which App Review permits).
- **martin tile server** and its GitHub binary download. Add Data hides the sources with no client-side engine: **PostgreSQL** (martin), plus **SQL Server** and **File Geodatabase** (both sidecar-backed).
- **External plugin installation** (zip archives and registry installs). Built-in and bundled drop-in plugins keep working.
- **In-app update checks** (`GEOLIBRE_STORE_BUILD=1` , same as the Microsoft Store MSIX build): a Store app updates only through the Store.

- **Earth Engine sign-in** (Processing → Earth Engine, and the GeoAgent plugin's Earth Engine overlay). Unlike the items above this is not a sandbox limit but an *entitlement* one — see below.

Earth Engine and the `network.server` entitlement

Earth Engine sign-in uses Google's OAuth **loopback-redirect** flow for native apps: `src-tauri/src/earth_engine_oauth.rs` binds a listener on `127.0.0.1`, opens the consent page in the system browser, and accepts the browser's inbound redirect to receive the token. Accepting an inbound connection requires `com.apple.security.network.server`.

App Review **rejected GeoLibre Desktop 2.4.0** over exactly this (guideline 2.4.5, submission `76036eca-dc21-45d3-873e-39b015d37036`): an automated analysis flags the server entitlement when it cannot find matching functionality, and the bind happens lazily inside a Tauri command, so nothing static points at it.

Rather than defend the entitlement in App Review Information, both Apple targets drop the feature: the module is gated `#[cfg(not(any(feature = "mas", target_os = "ios")))]` and replaced with stub commands that bind nothing, so **neither the Mac App Store build nor the iOS app ever opens a listening socket**. The entitlement is gone from `mas/Entitlements.mas.plist.template`, and `mas-store.yml` fails the build if it reappears in the signature. (iOS has no entitlements file — those sandbox keys are macOS-only — but it ships through the same review pipeline, so it gets the same treatment.)

On the UI side, `isEarthEngineAvailable()` (`packages/plugins/src/plugins/earth-engine-auth.ts`) hides the Processing menu item and the command-palette entry, and `authenticateEarthEngine` throws early if anything else reaches it. It detects the macOS build from the `__GEOLIBRE_MAS_BUILD__` define and iOS from the user agent, and only applies inside the packaged app — the **web build keeps Earth Engine everywhere**, including Safari on iOS, because a browser uses Google's popup/redirect flow and binds nothing.

If a future feature genuinely needs to accept an inbound connection, re-adding the entitlement is not sufficient on its own: describe the functionality in **App Review Information** before submitting, or the automated check rejects it again.

Share.GeoLibre sign-in uses a custom URI, not a listener

The MAS desktop build retains Share.GeoLibre project sign-in and Settings → Environment → session management. Its PKCE consent opens the system browser; the exact registered callback `org.geolibre.desktop:/oauth/callback` returns through the installed app's OS URI handler. No socket binds, so this does **not** require `com.apple.security.network.server`. The app starts the callback listener before accepting a sign-in; a callback that launches a cold process has no pending verifier and shows a restart-sign-in message rather than exchanging its code. A late callback from a timed-out or cancelled consent is ignored so it cannot interrupt an immediate retry. A fresh management consent lasts at most five minutes, has no refresh token, and stays in memory only. The desktop project's refresh token is kept in the OS keychain, so the sign-in survives a restart until the user signs out. Pasted personal API tokens remain a separate optional fallback.

For the shipped `https://share.geolibre.app` origin, authenticated desktop requests use native HTTP with redirects disabled so a redirected Bearer header cannot cross hosts. Self-hosted share origins use browser fetch and must allow the Tauri webview origin in CORS; the MAS sandbox's `network.client` entitlement permits those outgoing connections. Neither path adds an inbound listener or changes the `network.server` ban.

Everything client-side is unchanged and fully functional: MapLibre/deck.gl rendering, Add Data for local and remote files, DuckDB-WASM vector reading, Whitebox WASM tools, Turf/Pyodide vector tools, browser-engine conversions, client raster tools, the SQL Workspace on CereusDB/PGlite, the Python console, onnxruntime-web detection/segment-everything, geocoding, and collaboration.

Known limitations under the sandbox

- **Loose shapefiles:** the sandbox only grants access to the files the user picked, so the automatic read of `.dbf` / `.prj` / `.shx` siblings next to a picked `.shp` is denied. The MAS build compensates by matching companions out of the same dialog selection, so **multi-select every shapefile part** (`.shp`, `.dbf`, `.prj`, ...) or load a **zipped shapefile**. A `.shp` picked alone loads without attributes or CRS. This also applies when a saved project restores a loose `.shp` layer: re-add it or use a zip.
- **Environment variables:** the "read API keys from the shell environment" convenience returns nothing, because a sandboxed app does not inherit a login shell environment. Enter keys in Settings instead.
- **Reopening projects that reference local files:** the sandbox grant for a picked file ends with the process unless the app stores a security-scoped bookmark, which GeoLibre does not do yet. After a relaunch, a project layer backed by a local path outside the container fails to restore its data; re-add the file to reload it. Adopting security-scoped bookmarks (e.g. via `tauri-plugin-persisted-scope`'s macOS bookmark support) is the planned fix.

Building locally

Unlike the other desktop builds, this one **embeds the JupyterLite site**, so the `jupyter lite` CLI has to be installed before you build:

```
pip install -r apps/geolibre-desktop/jupyterlite/requirements.txt
```

Without it the build fails with an explicit error rather than producing an app whose Notebook panel loads the wrong thing (Tauri answers a missing asset with `index.html`, so a MAS build with no JupyterLite renders a second copy of GeoLibre inside the panel).

```
# One-time: render the entitlements with your team id.
APPLE_TEAM_ID=XXXXXXXXX scripts/render-mas-entitlements.sh

# Put a Mac App Store provisioning profile for org.geolibre.desktop at
# apps/geolibre-desktop/src-tauri/mas/embedded.provisionprofile
# (App Store Connect -> Profiles -> macOS App Development/Distribution).

APPLE_SIGNING_IDENTITY="Apple Distribution: Your Name (XXXXXXXXXX)" \
```

```
npm run tauri:build:mas -- --target universal-apple-darwin

# Wrap the .app into a Store-submittable installer package.
xcrun productbuild \
  --sign "3rd Party Mac Developer Installer: Your Name (XXXXXXXXXX)" \
  --component "apps/geolibre-desktop/src-tauri/target/universal-apple-darwin/release/bundle/
macos/GeoLibre Desktop.app" \
  /Applications GeoLibre-Desktop-mas.pkg
```

`npm run tauri:build:mas` wires everything else: the `mas` cargo feature, the `tauri.mas.conf.json` overlay (App Sandbox entitlements, embedded provisioning profile, no bundled sidecar resources, hardened runtime off), and the `GEOLIBRE_MAS_BUILD / GEOLIBRE_STORE_BUILD` frontend flags. It refuses to run with `--native-duckdb` (native DuckDB loads its spatial extension as unsigned native code at runtime).

The sandboxed app can be smoke-tested by running the built `GeoLibre Desktop.app` directly; `codesign -d --entitlements - --xml` on the bundle should show `com.apple.security.app-sandbox`, and must **not** show `com.apple.security.network.server`.

For an OAuth release smoke, **install** the signed `.app` before testing the URI association. Configure the issuer with the `geolibre-desktop` client and the exact callback above, then use Settings → Environment to sign in via the system browser. Verify a matching callback completes only while consent is pending, a cold callback asks for a restart, a second management consent lists sessions, and revoking the current project session signs the app out. Repeat the URI roundtrip with installed Windows/MSIX and Linux packages on their respective OS runners; compiling a package on macOS does not verify their protocol associations. Check packaged logs for redacted callback query parameters, and check the app signature again for the missing `com.apple.security.network.server` entitlement.

CI: the `mas-store.yml` workflow

Runs on each **published GitHub release** (matching `release.yml`, `android.yml` and `ios.yml`) and on demand via the "Run workflow" button: it builds the universal sandboxed app, signs it, verifies the sandbox entitlement, produces the signed `.pkg`, and uploads it as the `geolibre-mas-pkg` artifact. On a release run it also attaches that `.pkg` to the release, so the exact submitted bytes for a tag remain available after the CI artifact's retention window. It never uploads to App Store Connect — that step stays manual (see *Submitting* below).

A MAS-signed `.pkg` can only be installed through the App Store; downloading it from the releases page gets you a package that will not install. It is attached as a submission archive, not as a macOS download. Point users at the App Store listing, the Homebrew cask, or the Developer ID `.dmg` from `release.yml`.

A missing-secrets run is handled by the `secrets-gate` job: on a `workflow_dispatch` it is a hard error (you asked for a `.pkg` and cannot get one), while on a release it logs a warning and skips the macOS job so an otherwise-good release does not go red.

Required repository secrets:

Secret	Contents
APPLE_MAS_CERTIFICATE	base64 <code>.p12</code> containing both the "Apple Distribution" and "3rd Party Mac Developer Installer" certificates with private keys
APPLE_MAS_CERTIFICATE_PASSWORD	password of that <code>.p12</code>
APPLE_MAS_SIGNING_IDENTITY	Apple Distribution: Name (TEAMID)
APPLE_MAS_INSTALLER_IDENTITY	3rd Party Mac Developer Installer: Name (TEAMID)
APPLE_MAS_PROVISIONING_PROFILE	base64 Mac App Store <code>.provisionprofile</code> for <code>org.geolibre.desktop</code>
APPLE_TEAM_ID	already configured for <code>release.yml</code>

Creating the inputs (Apple Developer account required):

1. In the developer portal, create an **App ID** for `org.geolibre.desktop` (macOS, no extra capabilities).
2. Create the two certificates ("Apple Distribution" and "3rd Party Mac Developer Installer") via Keychain Access CSRs, then export both together from Keychain Access as one `.p12`: `base64 -i certs.p12 | pbcopy`.
3. Create a **Mac App Store Connect** provisioning profile for that App ID and the distribution certificate: `base64 -i profile.provisionprofile | pbcopy`.

Submitting

1. Create the app record in App Store Connect for `org.geolibre.desktop` (the same team as the iOS `org.geolibre.app` record; the two are separate apps). This is already done for GeoLibre Desktop, App Store ID `6796848769` (the numeric listing identifier App Store Connect labels "Apple ID" on the App Information page), so later releases start at step 2.
2. Download the `.pkg` — from the `geolibre-mas-pkg` artifact of the release's workflow run, or from the release's own `GeoLibre.Desktop_<version>_universal_mas.pkg` asset once the artifact has expired — and upload it with the **Transporter** app (or `xcrun altool --upload-app -f <pkg> -t macos`).
3. Fill in screenshots, description, and the privacy questionnaire (see `docs/privacy.md`; the app makes no tracking calls).
4. In the review notes, mention that processing engines run client-side as WebAssembly inside the webview and that the app downloads only data (map tiles, user-requested datasets), never code.

Version numbers come from `tauri.conf.json` (`CFBundleShortVersionString`); every upload needs a version or build number higher than the last one in App Store Connect.

<https://geolibre.app/project-format/>

GeoLibre Project Format

Projects are saved as `.geolibre` files. GeoLibre continues to open the earlier `.geolibre.json` name, but the single extension lets desktop installers associate projects with GeoLibre without taking ownership of every JSON file on the system.

Desktop file opening

Desktop installers register `.geolibre` as a GeoLibre Project document. Opening one from the file manager launches GeoLibre, or brings the existing GeoLibre window forward, and loads the project. The desktop executable also accepts both supported names on the command line:

```
geolibre-desktop /path/to/watershed.geolibre
geolibre-desktop /path/to/legacy.geolibre.json
```

Operating systems see only `.json` at the end of a `.geolibre.json` name. Those legacy files continue to open from inside GeoLibre and from the command line, but renaming one to `.geolibre` is required for file-manager association. The file contents do not change.

Schema

Field	Type	Description
<code>version</code>	string	Format version: the app writes <code>0.2.0</code> ; older files and the Python package write <code>0.1.0</code>
<code>name</code>	string	Project display name
<code>mapView</code>	object	<code>center</code> , <code>zoom</code> , <code>bearing</code> , <code>pitch</code> , optional <code>bbox</code>
<code>basemapStyleUrl</code>	string	MapLibre style JSON URL, or an empty string for a blank background
<code>basemapVisible</code>	boolean	Whether the Background layer is visible
<code>basemapOpacity</code>	number	Background layer opacity from <code>0</code> to <code>1</code>
<code>blankBackgroundColor</code>	string null	Optional custom colour for the Blank background; <code>null</code> uses the theme default
<code>layers</code>	array	Layer definitions (see below)
<code>selectedLayerId</code>	string null	Optional layer selected in the Layers panel when the project was saved; <code>null</code> restores no active layer
<code>layerGroups</code>	array	Optional Layers-panel folders (<code>id</code> , <code>name</code> , <code>parentId</code> , <code>collapsed</code> , <code>visible</code> , <code>opacity</code>); omitted when empty

Field	Type	Description
<code>styles</code>	object	Map of layer id → <code>LayerStyle</code>
<code>preferences</code>	object	Project map preferences, environment variables, and geocoding settings (see Credential redaction)
<code>plugins</code>	object	Optional external plugin manifest URLs, active plugin IDs, plugin map-control positions, and plugin settings
<code>legend</code>	object	Optional Print Layout legend customizations (title, grouping, ordering, per-item rename/hide)
<code>printLayout</code>	object	Optional Print Layout composer settings (title, page size, orientation, blocks, atlas); omitted when default
<code>storymap</code>	object	Optional scroll-driven story map (chapters and presentation settings); omitted when there are no chapters
<code>models</code>	array	Optional saved processing models (Model Builder pipelines)
<code>processingHistory</code>	array	Optional record of processing tool runs (Processing History)
<code>widgets</code>	array	Optional Dashboard panel chart widgets (see below); omitted when there are none
<code>dashboardColumns</code>	number	Optional Dashboard widget-grid column count (1-6, default 2); omitted when default
<code>mapLayout</code>	object	Optional multi-map grid (<code>rows</code> , <code>cols</code> , <code>syncView</code>); omitted for a single map
<code>secondaryMapView</code>	array	Optional panes past the primary one, each with its own camera and layer visibility; omitted for a single map
<code>primaryMapLabel</code>	string	Optional label for the primary pane of a multi-map grid
<code>bookmarks</code>	array	Optional saved map views from the Bookmarks panel (see below); omitted when there are none
<code>bookmarkGroups</code>	array	Optional folders the bookmarks are organized into; omitted when there are none
<code>styleLibrary</code>	array	Optional project-scoped Style Manager entries (name, tags, kind, <code>LayerStyle</code> subset); omitted when empty
<code>primaryRenderer</code>	string	Optional engine for the primary map area: <code>"maplibre"</code> (2D, the default), <code>"mapbox"</code> (Mapbox GL JS), <code>"arcgis"</code> (ArcGIS Maps SDK for JavaScript) or <code>"cesium"</code> (3D globe); omitted when default
<code>comments</code>	array	Optional review comments anchored to map points or features, with replies
<code>interaction</code>	object	Optional startup Identify target and control visibility (see below); omitted by default
<code>metadata</code>	object	Free-form project metadata

Plugin state

```
{
  "manifestUrls": ["https://example.com/plugins/example-plugin/plugin.json"],
  "activePluginIds": ["maplibre-layer-control", "maplibre-gl-swipe"],
  "mapControlPositions": {
    "maplibre-layer-control": "top-right",
    "maplibre-gl-swipe": "top-left"
  },
  "settings": {
    "maplibre-gl-swipe": {
      "orientation": "vertical",
      "position": 50,
      "collapsed": false,
      "active": true,
      "leftLayers": ["layer-a"],
      "rightLayers": ["layer-b"]
    }
  }
}
```

Projects without a `plugins` section open with the built-in default plugin state.

Legend

The Print Layout legend is always derived from the visible layers' symbology; the `legend` object stores only the user's edits layered on top, so customizations survive layer additions and removals.

```
{
  "title": "Legend",
  "groupByLayer": true,
  "order": ["layer-b", "layer-a"],
  "overrides": {
    "layer-a": { "label": "Roads" },
    "layer-b::0": { "label": "Low" },
    "layer-b::1": { "hidden": true }
  }
}
```

- `title` — heading drawn above the legend entries.
- `groupByLayer` — when `true`, graduated/categorized classes are grouped under a per-layer heading; when `false`, classes are listed flat.
- `order` — top-level entry order by layer id (top-first); layers not listed keep their default order after the listed ones.
- `overrides` — per-item `label` and `hidden` edits keyed by a stable item key: a layer id for a whole entry, or `${layerId}::${index}` for an individual class within a graduated/categorized entry.

Projects without a `legend` section open with the default legend (auto-generated from the layers, titled "Legend").

Print layout

The Print Layout composer (Project -> Print Layout) belongs to the project, so the page it composes reopens as it was saved. The section is written only once a setting differs from the defaults, so a project that never opened the composer carries no `printLayout` key.

```
{
  "title": "Dentists by region",
  "subtitle": "2026",
  "paperSize": "a3",
  "orientation": "portrait",
  "pageMargin": "normal",
  "showLegend": true,
  "showScaleBar": true,
  "showNorthArrow": true,
  "showDataTable": true,
  "tableLayerId": "layer-a",
  "tableColumns": ["name", "count"],
  "atlasEnabled": false
}
```

- Title block: `title`, `subtitle`, `titlePlacement`, `titleAlign`. A blank `title` follows the project name rather than freezing a copy of it.
- Page: `paperSize` (`a4`, `a3`, `letter`, `legal`, `tabloid`, `fullhd`, `hd`, `uhd4k`, `square`, `custom`), `orientation`, `customWidth` / `customHeight` / `customUnit` for `custom`, `pageMargin`, and the page/map frame colours.
- Elements: the `show*` toggles for legend, scale bar, north arrow, footer, date, attribution, colorbar, custom legend and info block, with their per-element settings.
- Data blocks: `tableLayerId` / `chartLayerId` name a project layer; a block whose layer is missing from the project opens cleared rather than blank.
- Atlas: `atlasEnabled` plus the coverage layer, extent mode, sorting, filter and filename pattern for the map series.

Unknown or malformed values fall back to the default for that field, so a hand-edited file never leaves the composer in an unusable state. Per-session state (the captured map image, the current atlas page, the dialog's panel widths) is deliberately not stored.

Story map

A story map turns the project into a scroll-driven narrative. Each chapter captures a camera view plus text, and can fade project layers in or out on enter/exit. The section is omitted entirely when the project has no chapters.

```
{
  "title": "A Tour of Three Cities",
  "subtitle": "Built with GeoLibre",
  "byline": "By the GeoLibre team",
  "footer": "Source: OpenStreetMap",
  "theme": "dark",
  "showMarkers": true,
  "markerColor": "#3fb1ce",
}
```

```

"inset": false,
"insetPosition": "bottom-right",
"chapters": [
  {
    "id": "intro",
    "title": "San Francisco",
    "description": "A hilly city on the tip of a peninsula. <em>HTML allowed.</em>",
    "image": "https://example.com/sf.jpg",
    "alignment": "left",
    "hidden": false,
    "location": { "center": [-122.4194, 37.7749], "zoom": 11, "pitch": 45, "bearing": 0 },
    "mapAnimation": "flyTo",
    "rotateAnimation": false,
    "onChapterEnter": [{ "layerId": "layer-a", "opacity": 1, "duration": 2000 }],
    "onChapterExit": [{ "layerId": "layer-a", "opacity": 0 }]
  }
]
}

```

`alignment` is one of `left`, `center`, `right`, `full`; `mapAnimation` is `flyTo`, `easeTo`, or `jumpTo`. Layer opacity changes reference project layer ids. Build and present story maps from **Project** → **Story Map**, or export a self-contained HTML page for static hosting.

Dashboard widgets

```

{
  "widgets": [
    { "id": "w1", "layerId": "layer-a", "type": "histogram", "field": "pop", "bins": 12 },
    { "id": "w2", "layerId": "layer-a", "type": "bar", "category": "kind", "aggregation": "sum", "valueField": "pop", "title": "Population by kind" }
  ]
}

```

Each widget binds a chart to a layer's attributes. `type` is one of `histogram`, `scatter`, `bar`, `line`, `box`, `pie`, `indicator`, `selector`, or `list`. Which other keys apply depends on the type: `field` (histogram/line/box, and indicator unless it counts), `xField` / `yField` (scatter), `category` + `aggregation` + `valueField` (bar/pie), `bins` (histogram), `indicatorAggregation` (`count` / `sum` / `mean` / `min` / `max` / `median`) + `prefix` + `suffix` (indicator), `category` + `multiple` (selector), and `listFields` + `sortBy` + `sortDir` + `limit` (list). Bar aggregation is `count` / `sum` / `mean`; pie is `count` / `sum` only. `title` is an optional label and `color` an optional hex (`#rgb` / `#rrggbb`) for the chart's marks (the series color for single-series charts; the base of a monochromatic ramp for bar/pie). Unused keys are ignored. The Dashboard panel (Tools → Dashboard, or the **Dashboard** button in the attribute table) also stores `dashboardColumns`, the widget-grid column count (1-6, default 2), at the top level of the project. Charts read from GeoJSON-backed vector layers and DuckDB query layers; widgets bound to a missing or non-attribute layer are shown as empty.

Bookmarks

```

{
  "bookmarkGroups": [{ "id": "hoods", "name": "Neighborhoods", "collapsed": false }],
  "bookmarks": [
    {

```

```

    "id": "downtown",
    "name": "Downtown",
    "lng": -83.92,
    "lat": 35.96,
    "zoom": 14,
    "pitch": 30,
    "bearing": 340,
    "createdAt": 1700000000000,
    "groupId": "hoods",
    "extra": { "visibleLayerIds": ["cities"] }
  }
]
}

```

Saved views from the Bookmarks panel (Controls → Bookmark), in panel order. `lng` / `lat` (-180..180 / -90..90), `zoom` (0-24), `pitch` (0-85) and `bearing` (wrapped into 0-360) are the camera; `createdAt` is milliseconds since the epoch. `groupId` files a bookmark in one of `bookmarkGroups` (folders do not nest); a `groupId` that names no folder is dropped on load. `extra` is state captured with the view: `visibleLayerIds` lists the layers to show when the bookmark is opened, and the others are hidden. Bookmarks with a missing or out-of-range camera, or a duplicate `id`, are dropped on load. Before these fields existed, bookmarks lived only in the browser's local storage.

Interaction

```

{
  "interaction": {
    "identify": ["layer-a", "layer-b"],
    "controls": { "search": true, "globe": false }
  }
}

```

Applied when the project opens, so a map shared from a notebook starts the way it was set up there. The Python package writes it from `Map.set_identify` and `Map.show_control` / `Map.hide_control` in `save_project`, `to_project` and `to_html`. `identify` is a layer id, "all" for every visible queryable layer, a list of layer ids to identify only those, or `null` for off; ids that name no layer are skipped. `controls` maps a toolbar panel (`bookmark`, `search`, `measure`, `minimap`, `print`) or built-in map control (`navigation`, `fullscreen`, `compass`, `geolocate`, `globe`, `scale`, `attribution`, `logo`) to whether it starts shown; other names are ignored. The app writes the block back unchanged when it saves the project; turning Identify or a control on or off in the app does not edit it.

Layer object

```

{
  "id": "uuid",
  "name": "My Layer",
  "type": "geojson",
  "source": { "type": "geojson" },
  "visible": true,
  "opacity": 1,
  "style": {
    "minZoom": 0,
    "maxZoom": 24,

```



```

    "fillColor": "#3b82f6",
    "strokeColor": "#1e40af",
    "strokeWidth": 2,
    "strokeWidthUnit": "pixels",
    "fillOpacity": 0.6,
    "circleRadius": 6,
    "rasterBrightnessMin": 0,
    "rasterBrightnessMax": 1,
    "rasterSaturation": 0,
    "rasterContrast": 0,
    "rasterHueRotate": 0
  },
  "metadata": {},
  "geojson": { "type": "FeatureCollection", "features": [] },
  "sourcePath": "/path/to/file.geojson",
  "editorTracking": {
    "enabled": false,
    "createdByField": "created_by",
    "createdAtField": "created_at",
    "editedByField": "edited_by",
    "editedAtField": "edited_at"
  },
  "capabilities": {
    "query": true,
    "create": true,
    "update": true,
    "delete": true,
    "export": true
  }
}

```

A layer can define an optional `capabilities` set (`query` , `create` , `update` , `delete` , `export`) to explicitly declare allowed user and session actions. Each flag gates only the actions it names: `query` the identify/attribute-table and feature-selection paths, `create` / `update` / `delete` the feature writes, and `export` the paths that copy the layer's data out (Export, Save to Layer Library, Export selection). Renaming a layer or removing it from the project is not a feature edit and stays available. Omitted capabilities default to the behavior inferred from the layer's source kind. When `export` is `false` , project sharing/publishing strips embedded GeoJSON for that layer.

`export: false` strips the feature data the project file itself carries — the layer's inline `geojson` and its `metadata.embeddedGeoJSON` snapshot. It does not remove the layer's `source` or `connection` , so a layer that fetches its data live (WFS, ArcGIS FeatureServer, a remote GeoJSON or tile URL) is still shared with a reachable URL and re-fetches the same data when the project is opened. For that class of layer the flag hides the copy in the file, not the data itself.

`capabilities` describes the layer, and the app enforces it in the UI; it is not an access-control mechanism. The PostGIS sidecar refuses writes that contradict the capabilities the client sends with a save, which keeps an in-app edit from contradicting the layer's own configuration, but the sidecar has no independent record of a table's capabilities and cannot tell one caller from another. Use database grants for restrictions that must hold against an untrusted client.

For WFS GetFeature and GeoJSON URL layers, `metadata.refresh` can persist an optional auto-refresh interval. `intervalMs` can be any positive interval in milliseconds:

```

{
  "metadata": {
    "refresh": { "enabled": true, "intervalMs": 60000 }
  }
}

```

```
}
}
```

Manual refresh uses the same saved source URL without requiring this metadata.

Reloadable layers also persist a top-level `connection` record, which is the primary source of truth for the refresh cadence and carries the durable synchronization status shown in the Layers panel.

`metadata.refresh` is kept as a legacy fallback for projects saved before `connection` existed:

```
{
  "connection": {
    "layerId": "layer-1",
    "interval": 60,
    "lastSyncedAt": "2026-08-01T14:20:00.000Z",
    "lastError": null,
    "onFailure": "keep-last"
  }
}
```

`interval` is the automatic refresh cadence in **seconds**, or `null` for manual synchronization only. `lastSyncedAt` records the most recent successful synchronization and `lastError` the most recent failure (cleared on the next success). `onFailure` decides whether a failed synchronization retains the last good data (`"keep-last"` , the default) or discards it (`"clear"`).

SQL Server layers may also carry top-level `mssqlWritebackPending: true` when a write committed but its reread failed, or the write outcome could not be confirmed. This flag survives project save/load and blocks another Save Edits until a successful manual Refresh restores generated keys and the read baseline. Recovery failure preserves the layer's features even with `onFailure: "clear"` . Successful recovery removes the flag; the prior baseline metadata remains unchanged until that read succeeds.

For local-file vector layers on the desktop app, `metadata.watch` can persist a "watch this file for changes" toggle. When enabled, the desktop app registers a filesystem watcher that reloads the layer's features from `sourcePath` whenever the file changes on disk:

```
{
  "metadata": {
    "watch": { "enabled": true }
  }
}
```

The key is omitted when watching is off, and it has no effect off the desktop host (the browser cannot watch a local filesystem path).

A layer may carry persistent attribute joins (Layer properties → Joins): live left joins that materialize columns from another layer's attribute table — typically a geometry-less table added via Delimited Text with no coordinate fields — into this layer's feature properties, matched on a key field:

```
{
  "joins": [
    {
      "id": "uuid",
      "joinLayerId": "other-layer-uuid",

```

```

    "targetField": "name",
    "joinField": "state_name",
    "fields": ["pop_2025", "median_income"],
    "prefix": "census_",
    "enabled": true,
    "addedFields": ["census_pop_2025", "census_median_income"],
    "stats": {
      "matchedCount": 50,
      "unmatchedTargetCount": 2,
      "unmatchedJoinCount": 2
    }
  }
]
}

```

`fields` (subset to bring over; omitted = every field except the key), `prefix`, and `enabled` are optional. `addedFields` and `stats` are engine bookkeeping, rewritten on every apply: `addedFields` lists the output columns the join added (so re-applying can strip them first, keeping the operation idempotent), and `stats` records the last match counts shown in the Joins UI. Joins re-resolve against the loaded layer set on project open, so a saved copy of the joined output self-heals if the join table changed.

A vector layer may carry **quick filters** (Layer properties → Quick filters): data-driven filter controls that narrow what the layer draws without anyone writing a MapLibre expression. What persists is the *control state*, never the compiled output, so a saved filter can always be reopened and changed:

```

{
  "quickFilters": [
    {
      "id": "uuid",
      "field": "state",
      "kind": "categorical",
      "values": ["OR", "WA"]
    },
    { "id": "uuid", "field": "pop", "kind": "range", "min": 200000, "max": null },
    {
      "id": "uuid",
      "field": "founded",
      "kind": "date",
      "dateKind": "iso",
      "start": "1840-01-01",
      "end": "1900-12-31"
    },
    { "id": "uuid", "field": "name", "kind": "text", "operator": "contains", "text": "port" }
  ]
}

```

`kind` picks the control and the comparison: `categorical` (checkboxes over `values`), `range` (inclusive numeric `min` / `max`, either side `null` for an open bound), `date` (inclusive `YYYY-MM-DD` `start` / `end`), and `text` (case-insensitive `contains` / `startsWith` / `equals` against text). `dateKind` says how the field stores its timestamps — `iso` (the default, comparing the leading `YYYY-MM-DD` of ISO text), `epochMs`, or `epochS` — and `enabled: false` mutes a control without discarding what it was answered with. A control with nothing chosen places no constraint, so an emptied selection shows every feature rather than none.

A vector layer may also carry a persistent boolean MapLibre expression filter. This is authored from **Select by Expression** → **Filter layer** and hides non-matching features without changing or copying the source data:

```
{
  "filterExpression": [ ">=", ["get", "population"], 100000 ]
}
```

The expression is saved with the project and remains active until it is cleared from the Select by Expression panel or from the layer's **Clear filters** action. It combines with Quick Filters and the transient filters described below.

What is stored is a plain MapLibre expression, so the Expression Builder's @ variables (@project_name, @layer_name, @feature_count, @map_zoom, @map_scale) are resolved to literal values at the moment the filter is applied and do not track the map afterwards. Use the MapLibre ["zoom"] operator instead when the filter should follow the current zoom.

A point layer using the cluster renderer is worth noting, because MapLibre clusters at the source, before the renderer evaluates any filter. GeoLibre therefore narrows a clustered layer's source data by the persistent expression filter and Quick Filters, so its bubbles and counts follow both. A Time Slider window, a rule filter, and an embed `setFilter` remain per-feature render filters and cannot change an already-built cluster, so counts stay unnarrowed by those while clustering is on.

The compiled filter is combined with the transient `timeFilter` and `embedFilter` and with the rule-based renderer's hide-unmatched filter under a single `all`, so a host page's `setFilter`, a Time Slider window, and a user's quick filter narrow the layer together instead of replacing one another. Quick filters are also shown in the read-only `layout=viewer` chrome: filtering is a way of reading a shared map, not of editing it.

When a `geojson` layer enables `style.simpleStyleEnabled`, individual features may override the layer style with `simplestyle-spec` properties (`stroke`, `fill`, `stroke-width`, `fill-opacity`, ...). GeoLibre also honors a per-feature `text-color` on text-marker points (used by the Annotations layer), falling back to `style.textColor` when a feature does not set it.

A vector layer may carry a `popup` block (Style panel → Popup): the design for what a viewer sees when they click or hover a feature. It decides which fields appear, in what order, under what labels and value formatting, what titles the popup, and whether a hover tooltip follows the pointer:

```
{
  "popup": {
    "click": true,
    "hover": true,
    "titleField": "name",
    "showFeatureId": false,
    "maxWidth": 640,
    "imageHeight": 420,
    "fields": [
      {
        "field": "pop_max",
        "label": "Population",
        "kind": "number",
        "format": { "thousands": true, "suffix": " people" },
      }
    ]
  }
}
```

```

    "hover": true
  },
  { "field": "region", "label": "Region" },
  { "field": "homepage", "kind": "link", "format": { "linkLabel": "City website" } }
]
}
}

```

Every key is optional, and **a layer with no popup block keeps the historical behavior**: the layer name as the heading, then every property as a row. `click` defaults to `true` (`false` suppresses the Identify popup); `hover` defaults to `false`. `fields` narrows and orders what is shown — an absent or empty list means every visible field, in the data's own order — and a listed field the feature does not carry is skipped rather than printed empty. Each field's `kind` is one of `auto` (the untyped rendering: sanitized KML `description` markup and inline `data:image/*;base64` values as thumbnails), `text`, `number`, `date`, `link`, or `image`, and `format` carries `decimals`, `thousands`, `dateFormat` (`date` | `datetime` | `time` | `iso` | `year`), `prefix`, `suffix`, and `linkLabel`. `hover: true` on a field puts it in the tooltip's short subset; a tooltip with no flagged field shows the title alone, or nothing when the title is just the layer name.

`maxWidth` (288-1200) is how wide the click popup may draw and `imageHeight` (40-1200) how tall an `image` field's thumbnail may draw inside it, both in CSS pixels; each is clamped to its range and `maxWidth` is capped by the viewport besides, so neither can produce a popup that blankets the map. Absent, the popup keeps its default width and image height. A thumbnail keeps its aspect ratio, so `imageHeight` alone does little for a landscape photo without `maxWidth` to match.

`titleField` leads the popup with a feature's own value instead of the layer name; `titleExpression` (a MapLibre expression source) wins over it, and both fall back to the layer name when they produce nothing. `bodyExpression` replaces the whole body — the field rows and the `id` row with them — with a sentence built from the feature's properties, so `showFeatureId` has no effect while one is set.

`fieldVisibility` stays authoritative: a field marked `"hidden"` or `"excluded"` never reaches a popup, even when the `popup` block names it — the popup design selects from what is visible, it cannot re-expose what the author hid. That holds for the expressions too: `titleExpression` and `bodyExpression` are evaluated against the visible properties only, so a `["get", ...]` cannot pull back a hidden column or one of GeoLibre's internal ones. Raster pixel identify goes through a different path and ignores `popup`.

Descriptive metadata

A layer may carry a `descriptiveMetadata` block: the catalog description edited in the layer's **Metadata** dialog (layer menu → Metadata). It documents the data and changes nothing about how the layer renders. It is separate from the layer's `metadata` record, which holds internal state the renderers and plugins key off.

```

{
  "descriptiveMetadata": {
    "title": "Rivers of Tennessee",
    "abstract": "Major rivers digitized from 1:24k topographic maps.",
    "keywords": ["hydrology", "rivers"],
    "license": "CC-BY-4.0",
  }
}

```

```

    "attribution": "© Tennessee GIS",
    "contact": { "name": "Ada Lovelace", "email": "ada@example.org", "organization": "TN
GIS" },
    "lineage": "Digitized in 2019; generalized with Douglas-Peucker (10 m).",
    "temporalExtent": { "start": "2019-01-01", "end": "2019-12-31" },
    "links": [{ "href": "https://example.org/rivers", "rel": "about", "title": "Project
page" }]
  }
}

```

Every member is optional and every value is a string (or a list of them). `license` is an [SPDX identifier](#) or free text. `temporalExtent` bounds are ISO 8601 dates (YYYY-MM-DD) or date-times; either may be left out for an open interval. A link needs an absolute `href`; `rel` is a link relation (`related` when omitted) and `title` a label. On load, blank members, blank or duplicate (case-insensitive) keywords, and links with no `href` are dropped, and a block left with nothing in it is removed — an empty `descriptiveMetadata` is never written. Malformed values (an email without a domain, an impossible date) are kept so the dialog can show and fix them; the dialog refuses to save them.

The dialog's **Export as STAC Item** writes the block as a [STAC 1.0](#) Item: the layer id as `id`, the layer's WGS84 extent as `geometry / bbox`, `title` (the layer name when unset), `abstract` as `description`, `keywords`, `license`, the contact as a `producer` `provider`, `lineage` as `processing:lineage` (with the processing extension declared), the temporal extent as `datetime` or `start_datetime / end_datetime`, the links, and a `data` asset for a remote source URL (a `tiles` asset when it is a `{z}/{x}/{y}` tile template). A license that is not an SPDX-style identifier is exported as `proprietary` with the text in `geolibre:license`; the attribution goes to `geolibre:attribution`. A layer with no temporal extent is stamped with the export time, since STAC requires a `datetime`. Exporting a vector layer to GeoParquet writes the block as JSON into the file's Parquet key-value metadata under `geolibre:metadata`, beside the `geo` key.

Layer types

Type	Status
<code>geojson</code>	Supported for imported files and GeoJSON URLs
<code>xyz</code>	Supported for raster tile templates
<code>wms</code>	Supported as tiled WMS GetMap layers
<code>wmts</code>	Supported for WMTS raster tile templates
<code>raster</code>	Supported for raster tile templates; with <code>source.ionAssetId</code> and <code>metadata.sourceKind: "cesium-ion"</code> , a Cesium Ion imagery asset the 3D globe loads with the app's Ion token (3D only)
<code>vector-tiles</code>	Supported for MapLibre vector tile sources
<code>mbtiles</code>	Supported in the desktop app through a local MapLibre protocol
<code>arcgis</code>	Supported for ArcGIS VectorTileServer layers (FeatureServer layers are saved as <code>geojson</code> , and MapServer/ImageServer layers as <code>raster</code>)
<code>pmtiles</code>	Supported through the Components plugin

Type	Status
cog	Supported for COG and GeoTIFF raster layers
flatgeobuf	Supported through the Components plugin and imported as GeoJSON when loaded as a local vector file
zarr	Supported through the Components plugin
lidar	Supported through the Components plugin
gaussian-splat	Supported through the Components plugin
geoparquet	Imported as GeoJSON via DuckDB-WASM
duckdb-query	Supported for SQL query-result layers
deckgl-viz	Supported through the Deck.gl Layer plugin
video	Supported for georeferenced video overlays
image	Supported for georeferenced image overlays (KML ground overlays, the Georeferencer, NetCDF grids)
3d-tiles	Supported through the <code>maplibre-gl-3d-tiles</code> package; with <code>source.ionAssetId</code> and <code>metadata.sourceKind: "cesium-ion"</code> , a Cesium Ion tileset the 3D globe loads with the app's Ion token (3D only)

API

```
import {
  createEmptyProject,
  parseProject,
  redactCredentials,
  serializeProject,
} from "@geolibre/core";
```

Credential redaction

A local project may contain credentials needed to restore authenticated data, including layer request headers, geocoding API keys, environment variables, and plugin settings. Any project leaving the local workspace must pass through `redactCredentials(project)` first. GeoLibre applies this invariant to Share, standalone HTML export, embed snapshots, and collaboration snapshots. On the desktop app, geocoding API keys and uniquely named secret environment variables move to the OS keychain rather than the file; see [Credential storage](#). Duplicate or nameless secret environment rows cannot be stored under a unique account, so Local Save and Save As offer the same keep/strip prompt used when the keychain is unavailable or the web build contains project credentials.

A layer `source.requestHeaders` value may reference an environment variable as `${NAME}`, e.g. `"Authorization": "Bearer ${TILES_TOKEN}"`. The reference is saved as written and resolved from the enabled `environmentVariables` rows when the request is made; a header whose variable is unset or empty is not sent. A value that is only an optional scheme word followed by one reference survives redaction; any other header value is removed.

Each `preferences.environmentVariables` row may carry `"secret": false`. Rows without it are secrets: redaction removes them, and the desktop app stores their values in the keychain when they have a unique, nonempty name. Rows marked `"secret": false` are ordinary values: they stay in the file and survive Share and export.

Saved model and processing-history parameter bags do not currently accept credentials and are treated as structural project content. If a future processing tool accepts credentials, those fields must be added to the central redaction registry and traversal.

The redaction pass removes credential-bearing fields and authentication parameters in URLs while preserving non-secret broker references. External plugin settings are arbitrary, so they are omitted from egress snapshots by default. Recipients retain the plugin manifest and activation metadata, but must configure their own settings.

<https://geolibre.app/plugin-api/>

GeoLibre Plugin API

Interface

```
import type { Feature, FeatureCollection, Geometry } from "geojson";
import type { IControl } from "maplibre-gl";
import type * as Proj4 from "proj4";

export type GeoLibreMapControlPosition =
  | "top-left"
  | "top-right"
  | "bottom-left"
  | "bottom-right";

export type GeoLibreBuiltInMapControl =
  | "navigation"
  | "fullscreen"
  | "geolocate"
  | "globe"
  | "terrain"
  | "scale"
  | "attribution"
  | "logo"
  | "maptoolkit-logo"
  | "layer-control";

export interface GeoLibrePlugin {
  id: string;
  name: string;
  version: string;
  activeByDefault?: boolean;
  /**
   * Renderers this plugin supports. Defaults to `["maplibre"]`.
   * Engine-neutral plugins (e.g. catalog/service browsers that only write to
   * the GeoLibre store) or plugins with multi-engine adapters declare
   * `["maplibre", "cesium"]`; plugins that stay on the Style Spec surface the
   * two 2D engines share declare `["maplibre", "mapbox"]` (see "Supporting the
   * Mapbox renderer" below). The Plugins menu gates options against the active
   * renderer.
   */
  engines?: ("maplibre" | "mapbox" | "cesium" | "arcgis")[];
  /** Plugins in the same group cannot be active at the same time. */
  exclusiveGroup?: string;
  /** At least one name is required for handleUrlParameters to be called. */
  urlParameterNames?: string[];
  // Return (or resolve) false to refuse activation; the host then rolls back
  // the optimistic active state.
  activate: (app: GeoLibreAppAPI) => boolean | void | Promise<boolean | void>;
  deactivate: (app: GeoLibreAppAPI) => void;
  handleUrlParameters?: (
    app: GeoLibreAppAPI,
    params: URLSearchParams
  ) => void | Promise<void>;
  getMapControlPosition?: () => GeoLibreMapControlPosition;
  setMapControlPosition?: (
    app: GeoLibreAppAPI,
    position: GeoLibreMapControlPosition
  ) => boolean | void;
  getProjectState?: () => unknown;
  applyProjectState?: (app: GeoLibreAppAPI, state: unknown) => boolean | void;
  /**
```

```

    * The plugin saves its own panel open/collapsed state through
    * getProjectState/applyProjectState, so the project-restore pass must not
    * collapse its control.
    */
    restoresPanelCollapseState?: boolean;
  /**
    * The plugin's project state is project data, not a preference: every
    * project load calls applyProjectState, with undefined when the file has no
    * state for it, so the previous project's data is not carried over.
    */
    clearsStateOnProjectLoad?: boolean;
  /**
    * A workspace tool, not part of a project (e.g. the S3 Browser panel):
    * project loads and map swaps leave it running, and it is never saved in
    * the project's active plugins. Only for plugins with no map controls.
    */
    sessionScoped?: boolean;
}

// Resolved by app.getDeckGL(): GeoLibre's own deck.gl modules, so a plugin
// renders on the host's single instance instead of bundling its own copy.
export interface GeolibreDeckGL {
  core: typeof import("@deck.gl/core");
  layers: typeof import("@deck.gl/layers");
  aggregationLayers: typeof import("@deck.gl/aggregation-layers");
  geolayers: typeof import("@deck.gl/geo-layers");
  meshLayers: typeof import("@deck.gl/mesh-layers");
  mapbox: typeof import("@deck.gl/mapbox");
}

export interface GeolibreLayerSummary {
  id: string;
  name: string;
  type: string;
  visible: boolean;
  opacity: number;
}

export interface GeolibreLayerGroupSummary {
  id: string;
  name: string;
  parentId: string | null; // null for a group at the panel root
  visible: boolean;
  opacity: number;
  collapsed: boolean;
}

export interface GeolibreSelection {
  layerId: string | null;
  features: Feature<Geometry | null>[];
}

export type GeolibreActiveMapTool = "identify" | "feature-selection" | null;

export interface GeolibreRasterWindowOptions {
  bounds: [number, number, number, number]; // WGS84 [west, south, east, north]
  width?: number; // sample grid, default 32
  height?: number;
  band?: number; // 1-based, default 1
  signal?: AbortSignal;
}

export interface GeolibreRasterWindowReading {
  values: number[]; // row-major, width * height, nodata included
  width: number;
  height: number;
  band: number;
  nodata: number | null;
}

```

```

    overviewLevel: number;
}

// An excerpt: the full interface, with every optional method, is in
// packages/plugins/src/types.ts.
export interface GeoLibreAppAPI {
  setBasemap: (styleUrl: string) => void;
  addGeoJsonLayer: (
    name: string,
    data: FeatureCollection,
    sourcePath?: string
  ) => string;
  listLayers?: () => GeoLibreLayerSummary[];
  getLayerFeatures?: (layerId: string) => Feature<Geometry | null>[];
  // Dress a layer with an SLD, QML or Mapbox GL style. See "Layer styles".
  importLayerStyle?: (
    layerId: string,
    text: string
  ) => GeoLibreImportLayerStyleResult;
  getSelectedFeatures?: () => Feature<Geometry | null>[];
  getSelectedLayerId?: () => string | null;
  // Sample a raster layer over a geographic window. See "Sampling raster
  // values" below.
  readRasterWindow?: (
    layerId: string,
    options: GeoLibreRasterWindowOptions
  ) => Promise<GeoLibreRasterWindowReading | null>;
  getDrawnFeatures?: () => Feature<Geometry | null>[];
  onSelectionChange?: (
    callback: (selection: GeoLibreSelection) => void
  ) => () => void;
  getActiveMapTool?: () => GeoLibreActiveMapTool;
  onActiveMapToolChange?: (
    callback: (tool: GeoLibreActiveMapTool) => void
  ) => () => void;
  // Native raster/tile layers (see "Raster and tile layers" below). Each
  // returns the new layer's id and the layer appears in the Layers panel and
  // persists with the project, like addGeoJsonLayer does for vector data.
  addTileLayer?: (
    name: string,
    url: string,
    options?: GeoLibreTileLayerOptions
  ) => string;
  addWmtsLayer?: (
    name: string,
    url: string,
    options?: GeoLibreTileLayerOptions
  ) => string;
  addWmsLayer?: (name: string, options: GeoLibreWmsLayerOptions) => string;
  addWfsLayer?: (name: string, options: GeoLibreWfsLayerOptions) => Promise<string>;
  // Native client-side COG (reads the GeoTIFF directly; band/rescale/colormap/
  // nodata controls). Resolves with the new layer's id (see "Raster and tile
  // layers" below).
  addCogLayer?: (
    name: string,
    url: string,
    options?: GeoLibreCogLayerOptions
  ) => Promise<string>;
  // Zarr through the host's own @carbonplan/zarr-layer instance, with
  // crs/proj4 reprojection (see "Zarr layers" below).
  addZarrLayer?: (
    name: string,
    url: string,
    options: GeoLibreZarrLayerOptions
  ) => Promise<string>;
  setZarrLayerSelector?: (
    layerId: string,
    selector: Record<string, number | string>
  )

```

```

) => Promise<boolean>;
// Click-to-value / region statistics on a Zarr layer (see "Zarr layers").
queryZarrLayer?: (
  layerId: string,
  geometry: GeoLibreZarrQueryGeometry,
  selector?: GeoLibreZarrQuerySelector,
  options?: GeoLibreZarrQueryOptions
) => Promise<GeoLibreZarrQueryResult | null>;
// Register a layer the plugin added to the map itself, so it appears in the
// Layers panel (see "Custom (WebGL) layers and paint ownership" below).
registerExternalNativeLayer?: (
  layer: GeoLibreExternalNativeLayerRegistration
) => void;
unregisterExternalNativeLayer?: (id: string) => void;
// Layers-panel groups (folders). See "Layer groups" below.
addLayerGroup?: (name?: string, layerIds?: string[]) => string;
listLayerGroups?: () => GeoLibreLayerGroupSummary[];
moveLayersToGroup?: (layerIds: string[], groupId: string | null) => void;
moveLayerGroupToGroup?: (id: string, parentId: string | null) => void;
removeLayerGroup?: (id: string) => void;
// Time Slider adapters (see "Driving a layer's own time dimension" below).
registerTemporalLayer?: (
  layerId: string,
  adapter: TemporalLayerAdapter,
  options?: { bind?: boolean }
) => () => void;
unregisterTemporalLayer?: (layerId: string) => void;
// See "Activating and deactivating other plugins" below.
activatePlugin?: (pluginId: string, state?: unknown) => Promise<boolean>;
deactivatePlugin?: (pluginId: string) => boolean;
getActiveBasemap: () => string;
onBasemapChange: (callback: (styleUrl: string) => void) => () => void;
fetchArrayBuffer?: (url: string) => Promise<ArrayBuffer>;
// Desktop only: fetch through the native client (see "Signing in with a session cookie").
nativeFetch?: typeof globalThis.fetch;
fitBounds?: (bounds: [number, number, number, number]) => void;
// The extent the primary map currently shows, [west, south, east, north] in
// degrees, on either renderer. The engine-neutral replacement for
// getMap()?.getBounds() – see "Reading the viewport" below.
getViewBounds?: () => [number, number, number, number] | null;
getMap?: () => import("maplibre-gl").Map | null;
// The active primary renderer, including while its canvas is being replaced.
getMapRenderer?: () => "maplibre" | "mapbox" | "cesium" | "arcgis";
// The native mapbox-gl map, only while Mapbox is the primary renderer; null
// otherwise. Built-in plugins read the 2D map through getStyleMap(app), which
// falls back to this when getMap() is null – see "Supporting the Mapbox
// renderer" below.
getMapboxMap?: () => import("mapbox-gl").Map | null;
// The mapbox-gl namespace and the access token, only while Mapbox is the
// primary renderer, for a plugin that must construct Mapbox classes or a
// second Mapbox map.
getMapboxGl?: () => typeof import("mapbox-gl").default | null;
getMapboxAccessToken?: () => string | null;
// The primary ArcGIS MapView or SceneView, or null on another renderer.
// The shared deck overlay hosts flat maps and local scenes only.
getArcgisView?: () => ReturnType<
  import("@geolibre/map").ArcgisEngine["getView"]
>;
// The MapLibre-shaped map controls receive on ArcGIS, or null on another
// renderer. Its style calls are recorded, not drawn by the SDK: the host
// draws the store layers they mirror and GeoJSON overlays itself (see
// docs/arcgis-renderer.md, "Plugin controls"). Built-in plugins read it
// through getControlMap(app) in style-map.ts.
getArcgisControlMap?: () => import("maplibre-gl").Map | null;
// The primary Cesium globe's scene (namespace, widget, scene, camera, clock,
// canvas, readView), or null when the primary map is not a globe. The globe's
// counterpart to getMap for plugins that declare engines: ["maplibre", "cesium"].
getCesiumScene?: () => import("@geolibre/map").CesiumSceneHandle | null;

```

```

addMapControl: (
  control: IControl,
  position?: GeoLibreMapControlPosition
) => boolean;
removeMapControl: (control: IControl) => void;
// Note: showing the "terrain" control (visible: true) also switches 3D
// terrain on, mirroring the Controls menu so the user doesn't have to click
// the control button as a second step. Hiding it turns terrain back off.
setBuiltInMapControlVisible: (
  control: GeoLibreBuiltInMapControl,
  visible: boolean
) => boolean;
getBuiltInMapControlPosition: (
  control: GeoLibreBuiltInMapControl
) => GeoLibreMapControlPosition;
setBuiltInMapControlPosition: (
  control: GeoLibreBuiltInMapControl,
  position: GeoLibreMapControlPosition
) => boolean;
getDeckGL?: () => Promise<GeoLibreDeckGL>;
getProj4?: () => Promise<typeof Proj4>;
// Right-sidebar panels (see "Right sidebar panels" below).
registerRightPanel?: (panel: GeoLibreRightPanelRegistration) => () => void;
unregisterRightPanel?: (id: string) => void;
openRightPanel?: (id: string) => boolean;
collapseRightPanel?: (id: string) => void;
closeRightPanel?: (id: string) => void;
getActiveRightPanel?: () => string | null;
setActiveRightPanelDock?: (dock: GeoLibreRightPanelDock) => void;
getActiveRightPanelDock?: () => GeoLibreRightPanelDock | null;
// Localization (see "Following the app language" below).
getLocale?: () => string;
onLocaleChange?: (listener: (locale: string) => void) => () => void;
translate?: (
  key: string,
  defaultValue: string,
  params?: Record<string, string | number>
) => string;
registerTranslations?: (
  resources: Record<string, Record<string, string>>
) => void;
// Credential storage (see "Saving credentials" below).
credentials?: GeoLibrePluginCredentials;
// Top toolbar menus (see "Toolbar menus" below).
registerToolbarMenu?: (menu: GeoLibreToolbarMenu) => () => void;
unregisterToolbarMenu?: (id: string) => void;
// Items in the built-in menus (see "Adding items to built-in menus" below).
registerMenuContribution?: (
  contribution: GeoLibreMenuContribution
) => () => void;
unregisterMenuContribution?: (id: string) => void;
// Floating panels (see "Floating panels" below).
registerFloatingPanel?: (
  panel: GeoLibreFloatingPanelRegistration
) => () => void;
unregisterFloatingPanel?: (id: string) => void;
openFloatingPanel?: (id: string) => boolean;
closeFloatingPanel?: (id: string) => void;
getOpenFloatingPanels?: () => string[];
// AI Assistant tools and guidance (see "Assistant tools" below).
registerAssistantTool?: (tool: Tool) => () => void;
registerAssistantToolSpec?: (spec: AssistantToolSpec) => () => void;
registerAssistantGuidance?: (text: string) => () => void;
}

// A getter re-reads the label on every render (see "Toolbar menus" below).
export type GeoLibreToolbarLabel = string | (() => string);

```

```

export interface GeolibreToolbarMenu {
  id: string;
  label: GeolibreToolbarLabel;
  icon?: string; // URL or data: URI
  items: GeolibreToolbarMenuItem[];
}

export type GeolibreToolbarMenuItem =
  | {
    type?: "action";
    id: string;
    label: GeolibreToolbarLabel;
    icon?: string;
    disabled?: boolean;
    onSelect: () => void;
  }
  | {
    type: "submenu";
    id: string;
    label: GeolibreToolbarLabel;
    icon?: string;
    items: GeolibreToolbarMenuItem[];
  }
  | { type: "separator"; id?: string };

export interface GeolibreMenuContribution {
  id: string;
  menu: "addData" | "processing" | "controls";
  items: GeolibreToolbarMenuItem[];
}

export interface GeolibreFloatingPanelRegistration {
  id: string;
  // A getter makes the title re-localize live on language changes: it is
  // re-evaluated on every getFloatingPanel() call, so it picks up the current
  // language without re-registering the panel. Caveat: the registry itself
  // does not subscribe to i18n events, so the getter is only re-run when a
  // consumer re-reads the panel. In practice every host component that renders
  // the title also calls useTranslation(), whose languageChanged re-render
  // re-reads the panel as a side effect; a host that reads the title without
  // that subscription would show a stale title after a language switch until
  // the next registry mutation, and must re-read the panel itself on language
  // change. A plain string is frozen at registration time.
  title: string | (() => string);
  icon?: string; // URL or data: URI
  defaultWidth?: number;
  defaultHeight?: number; // omitted: the card sizes to its content
  position?: GeolibreMapControlPosition; // corner it first opens at, default "top-left"
  render: (container: HTMLElement) => void | (() => void);
  onOpen?: () => void;
  onClose?: () => void;
}

export type GeolibreRightPanelDock =
  | "left-of-layers" // left of the Layers panel
  | "right-of-layers" // between the Layers panel and the map
  | "left-of-style" // between the map and the Style panel
  | "right-of-style" // right of the Style panel
  | "replace-style" // share the Style sidebar's single rail (shared-rail mode, default)
  | "replace-layers"; // share the Layers sidebar's single rail (shared-rail mode)

export interface GeolibreRightPanelRegistration {
  id: string;
  // A getter makes the title re-localize live on language changes: it is
  // re-evaluated on every getRightPanel() call, so it picks up the current
  // language without re-registering the panel. Caveat: the registry itself
  // does not subscribe to i18n events, so the getter is only re-run when a
  // consumer re-reads the panel. In practice every host component that renders

```



```
// the title also calls useTranslation(), whose languageChanged re-render
// re-reads the panel as a side effect; a host that reads the title without
// that subscription would show a stale title after a language switch until
// the next registry mutation, and must re-read the panel itself on language
// change. A plain string is frozen at registration time.
title: string | (() => string);
/** Initial dock position; "replace-style" (default). */
dock?: GeoLibreRightPanelDock;
/** Optional rail icon: a URL or data: URI rendered as an image. */
icon?: string;
/** Preferred expanded width in px (desktop only; host-clamped). */
defaultWidth?: number;
/** Deactivate the owning plugin when the user closes the panel. */
deactivatePluginOnClose?: boolean;
/** Fill the panel body with your own DOM. May return a cleanup function. */
render: (container: HTMLElement) => void | (() => void);
onOpen?: () => void;
onCollapse?: () => void;
onClose?: () => void;
/** Called only on an explicit close, not when another panel displaces it. */
onExplicitClose?: () => void;
}
```

Register a plugin

```
import { PluginManager } from "@geolibre/plugins";

const manager = new PluginManager();
manager.register(myPlugin);
manager.activate("my-plugin", appApi);
```

Add a built-in plugin (in this repository)

Built-in plugins ship with GeoLibre itself. Most plugins do **not** need to be built in — see [External plugins](#) to ship one without forking GeoLibre. To add one to this repository:

1. Create a plugin file in `packages/plugins/src/plugins/`.

```
import type { GeoLibreAppAPI, GeoLibrePlugin } from "../types";

export const myPlugin: GeoLibrePlugin = {
  id: "my-plugin",
  name: "My Plugin",
  version: "0.1.0",
  activate: (app: GeoLibreAppAPI) => {
    app.setBasemap("https://example.com/style.json");
  },
  deactivate: () => {
    /* basemap remains until the user changes it */
  },
};
```

Basemap plugins deliberately leave the style they applied in place on deactivate — the built-in `osm-basemap` and `carto-light` plugins do the same, since reverting the map under the user would be more surprising than keeping what they last selected. Plugins that add controls, listeners, or layers must undo them in `deactivate()`.

1. Export it from `packages/plugins/src/index.ts`.

```
export { myPlugin } from "../plugins/my-plugin";
```

1. Register it in `apps/geolibre-desktop/src/hooks/usePlugins.ts` by adding it to the `BUILT_IN_PLUGINS` array, which is passed to `manager.registerAll`.

```
import { myPlugin } from "@geolibre/plugins";

const BUILT_IN_PLUGINS: GeolibrePlugin[] = [
  maplibreLayerControlPlugin,
  maplibreGeoEditorPlugin,
  // ...
  myPlugin,
];
manager.registerAll(BUILT_IN_PLUGINS);
```

For a MapLibre control plugin, add the package dependency, then call `app.addMapControl(control, "top-left")` in `activate()` and `app.removeMapControl(control)` in `deactivate()`. If the control's npm package ships its own stylesheet, import that stylesheet in `apps/geolibre-desktop/src/main.tsx`, alongside the existing `maplibre-gl-*/style.css` imports. That is only for a dependency's own CSS — any app-specific fixes on top of it belong in `index.css`, as described under [Styling third-party controls](#) below.

Built-in MapLibre controls such as Navigation, Fullscreen, Geolocate, Globe, Terrain, Scale, Attribution, and Logo are toggled from the desktop app's Controls menu, which also opens Search, a standalone place search panel backed by the Components plugin. Keep project-specific controls such as Layer Control and Components in the Plugins menu when they use the plugin API or need plugin lifecycle behavior.

The Components plugin wraps `maplibre-gl-components` controls and wires their layer events into the GeoLibre store. It provides Add Data shortcuts for FlatGeobuf, PMTiles, Zarr, LiDAR, and Gaussian splats, while raster COG and GeoTIFF layers can also be added through the standard Add Raster Layer dialog.

Styling third-party controls

If a third-party MapLibre control needs app-specific styling fixes, add scoped overrides in `apps/geolibre-desktop/src/index.css` instead of editing files in `node_modules`. Keep selectors limited to the plugin's control class. For example, GeoEditor toolbar buttons need a local override because MapLibre's default control button CSS can override their flex centering:

```
.geo-editor-control .geo-editor-tool-button {
  align-items: center;
  display: flex !important;
```

```

    justify-content: center;
    line-height: 0;
    padding: 0;
  }

  .geo-editor-control .geo-editor-tool-button svg {
    display: block;
    flex: 0 0 auto;
    margin: 0;
  }

```

Run `npm run build` and `pre-commit run --files <changed files>` before submitting the change. If you also touched pages under `docs/`, build the site — CI runs `zensical build --strict`, so a broken link or a page missing from the `mkdocs.yml` `nav` fails the build. See [Contributing](#) for both gates in full.

Built-in plugins

A selection of the built-in plugins:

ID	Description
<code>osm-basemap</code>	OpenFreeMap Liberty style
<code>carto-light</code>	CARTO Positron GL style
<code>maplibre-gl-basemap-control</code>	Adds a MapLibre basemap picker
<code>maplibre-gl-components</code>	Adds the MapLibre Components control grid and panels for FlatGeobuf, COG, PMTiles, Zarr, LiDAR, and Gaussian splats
<code>maplibre-gl-geo-editor</code>	Adds GeoEditor drawing controls
<code>maplibre-gl-dimensions</code>	Adds Dimension tools (linear/angular CAD-style dimension lines, with optional vertex snapping)
<code>maplibre-gl-geoagent</code>	Adds GeoAgent map assistant controls
<code>maplibre-gl-usgs-lidar</code>	Searches and streams USGS 3DEP LiDAR point clouds
<code>geolibre-ign-lidar-hd</code>	Searches IGN LiDAR HD tile coverage (WFS) and downloads point cloud (COPC LAZ) files. Its live-network test is opt-in via <code>RUN_LIVE_TESTS</code> (see <code>tests/ign-lidar-hd.test.ts</code>)
<code>maplibre-gl-streetview</code>	Adds street view controls
<code>maplibre-gl-swipe</code>	Adds map swipe controls

Example plugin

```
import type { GeoLibreAppAPI, GeoLibrePlugin } from "@geolibre/plugins";

export const myPlugin: GeoLibrePlugin = {
  id: "my-plugin",
  name: "My Plugin",
  version: "0.1.0",
  activate(app: GeoLibreAppAPI) {
    app.setBasemap("https://example.com/style.json");
  },
  deactivate() {
    // Clean up controls, listeners, and plugin state here.
  },
};
```

Map control plugins can optionally expose `getMapControlPosition()` and `setMapControlPosition()` so the desktop Plugins menu can move the control between map corners. Position-aware plugins should remove and recreate or re-add their control when the position changes.

Plugins with serializable runtime settings can expose `getProjectState()` and `applyProjectState()` so GeoLibre can save and restore those settings in the project file. A wrapper should use these hooks to adapt upstream control APIs such as `getState()` without requiring every upstream package to implement a GeoLibre-specific interface.

Plugins that render with `deck.gl` should call `app.getDeckGL()` (returns a promise) to obtain GeoLibre's own `deck.gl` modules — `core`, `layers`, `aggregationLayers`, `geoLayers`, `meshLayers`, and `mapbox` (use `mapbox.MapboxOverlay` for interleaved MapLibre rendering). Render on the host's single `deck.gl` instance rather than bundling a second copy: `deck.gl` and `luma.gl` throw on a version mismatch and share global singletons, so a bundled copy fails to render. Call it with optional chaining (`app.getDeckGL?.()`) since a host variant may not ship `deck.gl`.

Shared projection library

External plugins can use `app.getProj4()` to resolve the host's proj4 module namespace. Its `default` export is the callable library:

```
import type * as Proj4 from "proj4";

const module: typeof Proj4 | undefined = await app.getProj4?.();
if (!module) throw new Error("This plugin requires a host with getProj4 support.");
const proj4 = module.default;
proj4.defs("my-plugin:utm32n", "+proj=utm +zone=32 +datum=WGS84 +units=m +no_defs");
const [longitude, latitude] = proj4("my-plugin:utm32n", "EPSG:4326", [500000, 0]);
```

This returns longitude 9 and latitude 0. See the [proj4 API](#) for coordinate conversion and CRS definitions. The mutable definitions registry is shared with the host (including ArcGIS/Zarr reprojection) and all plugins. Register only plugin-prefixed names, as above; do not register or redefine `EPSG:*` names or names belonging to the host or other plugins. A conflicting definition can

silently corrupt subsequently created transforms. These naming rules are a plugin-author responsibility, not an isolation boundary enforced by this API. Unknown EPSG definitions are not fetched automatically.

For one-off conversions, pass the definition directly instead of modifying the shared registry:

```
const [longitude, latitude] = proj4(
  "+proj=utm +zone=32 +datum=WGS84 +units=m +no_defs",
  "EPSG:4326",
  [500000, 0],
);
```

Older or variant hosts may omit `getProj4`; plugins must handle that absence. Shared access does not change `addGeoJsonLayer`'s WGS84 longitude/latitude requirement: reproject source coordinates before adding the layer. Omit proj4's runtime import and dependency from the plugin bundle when using this API; type-only references, such as the namespace import above, need no runtime copy.

Plugins can also declare URL query parameters and handle them when GeoLibre opens. URL parameter handlers run after the map is ready, external plugins are loaded, and project plugin state has been restored. GeoLibre calls handlers for plugins whose declared parameter names are present in the URL, and it suppresses repeated handling of the same URL context for the same plugin. If a matching plugin is registered (installed) but inactive, GeoLibre first attempts to activate it via `PluginManager.activate`; the handler runs only if activation succeeds (an `activate()` that returns `false` or throws leaves the plugin inactive and skips dispatch). Parameter names are case-sensitive, as URL query parameters are: declaring `exampleGeoJson` will not match `?ExampleGeoJson=...`.

```
import type { GeoLibreAppAPI, GeoLibrePlugin } from "@geolibre/plugins";

export const plugin: GeoLibrePlugin = {
  id: "example-url-loader",
  name: "Example URL Loader",
  version: "0.1.0",
  urlParameterNames: ["exampleGeoJson"],
  activate() {
    // Set up controls or plugin state here.
  },
  deactivate() {
    // Clean up controls, listeners, and plugin state here.
  },
  async handleUrlParameters(app: GeoLibreAppAPI, params: URLSearchParams) {
    for (const dataUrl of params.getAll("exampleGeoJson")) {
      // URL parameter values are attacker-controlled: only fetch HTTPS URLs
      // and verify the origin is one you trust before loading. Parsing the
      // value rejects malformed URLs, and the protocol check blocks
      // non-HTTPS schemes (file://, data:, http://); neither protects
      // against SSRF to loopback or private-network addresses.
      let parsedUrl: URL;
      try {
        parsedUrl = new URL(dataUrl);
      } catch {
        continue;
      }
      if (parsedUrl.protocol !== "https:") continue;
      const response = await fetch(parsedUrl.href);
      if (!response.ok) continue;
      app.addGeoJsonLayer("Example URL layer", await response.json(), dataUrl);
    }
  }
}
```

```
    },
  };
```

Validate URL parameter values before acting on them. Anyone can craft a link to GeoLibre, so handlers that fetch a parameter value should reject unexpected schemes (`file://` , `data:` , plain `http://`) and only contact origins they trust.

For example:

```
https://web.geolibre.app/?url=https://example.com/
project.geolibre.json&exampleGeoJson=https://example.com/data.geojson
```

A URL parameter activates only an already-registered (installed) plugin that owns it; it never loads a plugin from the URL. For external plugins, include the plugin manifest URL in the project `plugins` state (so the plugin is registered) before relying on its URL handler — the matching parameter then activates and dispatches it even if it is not in the active set.

Read-only layer and feature queries

External plugins can inspect the current layer list, a layer's GeoJSON features, the current feature selection, and features in GeoEditor's Sketches layers. The methods are optional so the same plugin remains compatible with older hosts.

```
const layers = app.listLayers?.() ?? [];
const selectedLayerId = app.getSelectedLayerId?.() ?? null;
const selectedFeatures = app.getSelectedFeatures?.() ?? [];

if (selectedLayerId && app.getLayerFeatures) {
  const layerFeatures = app.getLayerFeatures(selectedLayerId);
  console.log(layers, layerFeatures, selectedFeatures);
}

const drawnFeatures = app.getDrawnFeatures?.() ?? [];
```

`getSelectedFeatures` returns every selected feature in the selected layer, not only the most recently selected feature. Features without a GeoJSON `id` are matched using their zero-based array index converted to a string. An empty selection returns an empty array. `getLayerFeatures` throws when the layer id is unknown and returns an empty array for a layer that has no GeoJSON features.

Selection subscriptions fire after the selected layer or selected feature-id array changes. Keep and call the returned unsubscribe function during plugin deactivation:

```
const unsubscribe = app.onSelectionChange?.(({ layerId, features }) => {
  console.log(layerId, features);
});

// In deactivate or another cleanup path:
unsubscribe?.();
```

These methods are a read-only query surface: calling them does not change the GeoLibre store. Plugins must also treat returned GeoJSON features as read-only and use host APIs such as `addGeoJsonLayer` when they need to add data.

Active map click tool

Plugins that handle map clicks or hover can avoid presenting their own interaction while a GeoLibre map tool owns the pointer. The optional live getter and subscription cover Identify and feature selection across map renderers; they do not depend on the cursor style or saved project snapshot.

```
const currentTool = app.getActiveMapTool?.() ?? null;
if (currentTool !== null) closePluginPopup();
const unsubscribe = app.onActiveMapToolChange?.((tool) => {
  if (tool !== null) closePluginPopup();
});

function handleMapClick(event: unknown) {
  if ((app.getActiveMapTool?.() ?? null) !== null) return;
  openPluginPopup(event);
}

function handleMapHover(event: unknown) {
  if ((app.getActiveMapTool?.() ?? null) !== null) return;
  showPluginTooltip(event);
}

// In deactivate or another cleanup path:
unsubscribe?.();
```

`getActiveMapTool` returns `"identify"` while Identify is enabled, `"feature-selection"` while a selection gesture owns map clicks, and `null` otherwise. Feature selection takes precedence if both states briefly overlap. Changing the Identify target does not emit a tool change. Subscribers receive only changes after registration, not an initial callback; read the getter to initialize plugin state. Keep and call the returned unsubscribe function on deactivation. Replacing one active selection gesture with another keeps `"feature-selection"` active without an intermediate `null`.

Both methods are optional so plugins remain compatible with older hosts.

Layer groups

A plugin that adds several related layers can put them in a Layers-panel group (a folder) instead of leaving them loose at the panel root, and can nest one group inside another the same way a user can by hand.

```
// Create a folder, optionally moving existing layers into it. Returns its id.
const basins = app.addLayerGroup?.("Basins", [catchmentLayerId]) ?? null;

// Append to a folder you created earlier rather than creating a second one
// with the same name. A null group id lifts the layers back to the root.
app.moveLayersToGroup?.([outletLayerId], basins);

// Nest a folder inside another one. A null parent lifts it back to the root.
```



```
const subBasins = app.addLayerGroup?.("Sub-basins");
if (basins && subBasins) app.moveLayerGroupToGroup?.(subBasins, basins);

// Remove the folder without removing the layers inside it.
if (subBasins) app.removeLayerGroup?.(subBasins);
```

`moveLayerGroupToGroup` is the group-of-groups counterpart of `moveLayersToGroup`: the same reparenting the Layers panel's own "Move to group" menu performs. It is a no-op when either id is unknown, when the group is already in that parent, and when the move would make a group its own ancestor — the host refuses the cycle rather than corrupting the tree, so a plugin does not have to walk the parent chain itself.

`addLayerGroup` is the only source of group ids for folders a plugin creates. To address a folder it did not create — one the user made, or one another plugin made — read the tree first:

```
const groups = app.listLayerGroups?.() ?? [];
const existing = groups.find((group) => group.name === "Basins");
const roots = groups.filter((group) => group.parentId === null);
```

`listLayerGroups` returns every group with `parentId` set to the enclosing group's id, or `null` at the root, so the flat array describes the whole folder tree. It is the store's own group order, which is not the panel's: the panel re-orders a group after its parent for display, while a reparent leaves the array alone. Like the other read-only queries, calling it does not change the store.

Group visibility and opacity are **combined** with each child layer's own: a hidden group hides its children on the map without touching their individual `visible` flags, and group opacity multiplies into each child's. `removeLayerGroup` through this API removes only the folder, never its contents: the layers it held move to the panel root, and any groups nested inside it are reparented to the removed folder's own parent.

These methods are typed optional for forward-compatibility with host variants, so call them with optional chaining.

Layer styles

`importLayerStyle` applies a style written in another format to a layer, like the Layers panel's "Import style": an OGC SLD, a QGIS QML or a Mapbox GL style JSON, detected from the content. The style is merged over the layer's current one, so fields it does not describe keep their values, and it is saved with the project. Like the Layers panel, it styles only GeoJSON and vector-tile layers; it accepts any such layer id, not only the plugin's own, and throws for an unknown one.

A plugin that adds features from a web service can dress them as the service does. A GeoServer, for example, returns a layer's SLD for WMS `GetStyles`:

```
const layerId = app.addGeoJsonLayer("Land cover", features);
const sld = await(
  await fetch(
    `${wmsUrl}?service=WMS&version=1.1.1&request=GetStyles&layers=M5:L4`
  )
).text();
```

```
const result = app.importLayerStyle?.(layerId, sld);
if (result && !result.ok)
  console.warn(`Style not applied (${result.reason})`, result.warnings);
```

`result.warnings` lists what the style asked for that GeoLibre could not represent. On failure, `reason` is `invalid` when the text is not a style in any format read, `no-match` when it parsed but describes no symbology the layer can wear, `unsupported-layer` when the layer is not a vector layer; the layer is left untouched.

Raster and tile layers

`addGeoJsonLayer` registers vector data as a native layer. For raster and tile data there are three matching helpers — `addTileLayer` (XYZ), `addWmtsLayer` (WMTS), and `addWmsLayer` (WMS). Each returns the new layer's id, and the layer appears in the Layers panel with full opacity, reorder, and styling support and persists with the project, so a plugin no longer has to call `getMap().addSource()/addLayer()` directly (which leaves the layer invisible to GeoLibre's layer store).

```
export interface GeolibreTileLayerOptions {
  tileSize?: number; // default 256
  attribution?: string;
  bounds?: [number, number, number, number]; // [west, south, east, north] in WGS84
  minzoom?: number;
  maxzoom?: number;
  scheme?: "xyz" | "tms"; // default "xyz"
  visible?: boolean; // default true
  opacity?: number; // default 1
  beforeLayerId?: string; // insert beneath this layer
  metadata?: Record<string, unknown>; // provenance merged into layer.metadata
}

export interface GeolibreWmsLayerOptions extends GeolibreTileLayerOptions {
  url: string; // WMS GetMap endpoint
  layers: string; // comma-separated layer name(s)
  styles?: string;
  format?: string; // default "image/png"
  transparent?: boolean; // default true
  version?: string; // "1.1.1" (default) or "1.3.0" (sends CRS instead of SRS)
  crs?: string; // "EPSG:3857" (default), "EPSG:4326", "CRS:84" (1.3.0 only), any
  "EPSG:<code>"
  queryable?: boolean; // false when the capabilities mark the layers queryable="0": identify
  skips it
}

export interface GeolibreWfsLayerOptions {
  url: string; // WFS GetFeature endpoint
  typeName: string; // advertised feature type
  version?: string; // defaults to "2.0.0"
  bbox?: [number, number, number, number]; // [west, south, east, north] in WGS84
  maxFeatures?: number; // positive integer, default 1000; sent as count (WFS 2.x) or
  maxFeatures
  metadata?: Record<string, unknown>; // provenance merged into layer.metadata
}

export interface GeolibreCogLayerOptions {
  engine?: "maplibre-gl-raster" | "cog-tiler-wasm" | "titiler" | "auto"; // see below
  bands?: string; // "1" (single band) or "1,2,3" (RGB)
  colormap?: string; // named colormap for a single-band COG, e.g. "terrain"
  rescaleMin?: number;
  rescaleMax?: number;
```

```

nodata?: number; // pixel value rendered transparent
opacity?: number; // default 1
beforeLayerId?: string;
zoomTo?: boolean; // fit the map to the COG once loaded (default true)
}

```

```

// XYZ tiles (e.g. an imagery, topo, or DEM hillshade endpoint).
app.addTileLayer?.(
  "LINZ Aerial Imagery",
  "https://tiles.example.nz/aerial/{z}/{x}/{y}.png",
  {
    attribution: "Sourced from LINZ. CC BY 4.0",
    maxzoom: 22,
    bounds: [166.0, -47.5, 178.6, -34.0],
  }
);

// WMTS tile URL template.
app.addWmtsLayer?.(
  "LINZ Topo50",
  "https://tiles.example.nz/topo50/{z}/{x}/{y}.png"
);

// WMS – pass the request parameters; the host builds the GetMap tile URL.
app.addWmsLayer?.("LINZ Coverage", {
  url: "https://wms.example.nz/wms",
  layers: "coverage",
  transparent: true,
});

// A WMS that does not offer EPSG:3857: request a CRS it lists instead.
app.addWmsLayer?.("Cadastral parcels", {
  url: "https://wms.example.it/wms",
  layers: "parcels",
  version: "1.3.0",
  crs: "EPSG:6706",
});

// The capabilities mark the layer queryable="0": identify skips it.
app.addWmsLayer?.("Buildings", {
  url: "https://wms.example.it/wms",
  layers: "buildings",
  queryable: false,
});

// A layer found in a catalogue: keep its provenance with it.
app.addWmsLayer?.("Bathymetry", {
  url: "https://wms.example.org/wms",
  layers: "bathymetry",
  metadata: {
    catalogRecordId: "rndt:abc-123",
    catalogRecordUrl: "https://catalog.example.org/records/abc-123",
    publisher: "Example Hydrographic Office",
    license: "CC BY 4.0",
  },
});

// COG – read the GeoTIFF directly (client-side), with raster controls.
const cogId = await app.addCogLayer?.(
  "LINZ DEM",
  "https://cog.example.nz/dem.tif",
  { colormap: "terrain", nodata: -9999 }
);

```

`addTileLayer`, `addWmtsLayer`, `addWmsLayer`, and `addWfsLayer` take an optional `metadata` object, merged into the new layer's `metadata`. Use it to leave a catalogue layer's provenance with it (the record id, a link to its metadata page, the publisher, the licence): it is shown in the layer's Metadata dialog and saved with the project. It must be a plain, JSON-serializable object, or the call throws (`addWfsLayer` rejects). GeoLibre's own keys win over a field of the same name, and credential-named fields (`token`, `apiKey`, ...) are stripped when the project is shared or exported. The Metadata dialog also shows the service address and request fields GeoLibre keeps on a service or tile layer's `source` (for WMS: `url`, `layers`, `styles`, `format`, `version`, `crs`, and `queryable` when false), with credentials removed.

WFS layers use the host's GetFeature loader, including GeoJSON/GML fallback, reprojection, desktop native HTTP, and refresh. `addWfsLayer` resolves with the new layer id and rejects if loading fails or the service returns no features. Saved projects normally keep the request URL rather than embedding the downloaded collection, and reopening fetches it again. Exception: if saving strips credentials from the URL, the fetched collection is embedded so the layer remains visible without storing the secret; it is not refetched from the sanitized URL. The optional `bbox` is WGS84 [`west`, `south`, `east`, `north`] and must not cross the antimeridian (`west` must not exceed `east` — a Pacific-spanning box throws); `maxFeatures` (default 1,000) caps the request and is stored in the GetFeature URL, so refreshes and projects that retain the URL use the same limit.

`addWfsLayer` requires an absolute HTTP(S) URL and sends its request through the same host-managed WFS path Add Data uses: the desktop app fetches through the native HTTP client (bypassing CORS) and the web build through its development proxy. Credentials in the URL are never written to diagnostics or a saved project: the native diagnostics log records userinfo and credential query values (including AWS `x-amz-*` parameters) as `[redacted]`, and a save strips the URL's userinfo and credential parameters while keeping the fetched collection embedded so it stays visible without persisting the secret.

The example uses a hostname, which the desktop app and the Vite development server accept. Any destination your plugin chooses is fetched from the user's own browser or desktop network position, so treat the URL as untrusted input and do not embed credentials you would not want a shared project or diagnostics log to reveal.

```
const layerId = await app.addWfsLayer?.("Roads", {
  url: "https://services.example.org/geoserver/wfs",
  typeName: "transport:roads",
  bbox: [10, 40, 12, 42],
  maxFeatures: 5000,
});
```

`options.engine` picks the renderer (`"maplibre-gl-raster"` for the GPU/deck.gl path, `"cog-tiler-wasm"` for the WebAssembly tiler, `"titiler"` for a TiTiler server). Unlike the other options it is **not per layer**: the raster control holds one engine for every raster it manages, so naming one re-renders the rasters already on the map. Pass `"auto"` to leave whatever the control is on alone; omit it and the GPU renderer is used. The GPU renderer requires a Mercator projection, so a plugin that expects to work on the globe should ask for `"cog-tiler-wasm"`.

`addWmsLayer` requests Web Mercator tiles unless `crs` names another CRS, for a server whose capabilities do not list EPSG:3857. Pick one the layer lists, preferably geographic (`EPSG:4326` , `EPSG:4258` , `EPSG:6706` , or `CRS:84` with `version: "1.3.0"`), otherwise a projected `EPSG:<code>` such as `EPSG:25832` . Only the desktop app redraws those tiles into Web Mercator: the web build still sends the Web Mercator BBOX, so such a layer stays blank there. An unsupported value, or `CRS:84` with WMS 1.1.1, throws.

`addTileLayer` / `addWmtsLayer` / `addWmsLayer` expect **pre-rendered tiles** (e.g. a COG already served through a tiler such as titiler as an XYZ endpoint). `addCogLayer` is different: it loads the **GeoTIFF itself** and renders it client-side, exposing band selection, rescale, colormap, and nodata in the raster panel. It is async (it fetches the file's header), so it returns a `Promise<string>` and rejects if the COG cannot be read.

The helpers are typed optional for forward-compatibility with host variants, so call them with optional chaining (`app.addTileLayer?.(...)`).

Desktop (Tauri) note: The desktop app enforces a Content Security Policy. Its `connect-src` allows any `https:` or `http:` host, so tile and data requests from a plugin reach their server, but `script-src` only allows the app itself, `blob:` URLs and a short list of version-pinned CDN paths. A plugin that injects a `<script>` or `import()` s a module from another host is blocked; bundle that code into the plugin instead. A bundled (first-party) plugin can add the path to `script-src` in `apps/geolibre-desktop/src-tauri/tauri.conf.json` (see [Desktop CSP script-src allowlist](#)). The web build is unaffected.

Zarr layers

`addZarrLayer` renders a Zarr store (Zarr v2/v3 over HTTP) through **GeoLibre's own** `@carbonplan/zarr-layer` instance and mirrors the result into the Layers panel. It is the Zarr counterpart of `addCogLayer` . Stores that are not read from a URL — a kerchunk-backed cloud NetCDF, an Icechunk repository, a folder on disk — reach the same renderer through an internal `store` option that this API does not expose, so they are added by GeoLibre's own panels rather than by a plugin.

Do not bundle `@carbonplan/zarr-layer` in a plugin: a second copy ships a duplicate numcodecs WASM payload, and adding the renderer's layer yourself with `getMap().addLayer()` produces a MapLibre **custom** layer, which has no paint properties for the Style panel to drive.

```
export interface GeolibreZarrLayerOptions {
  variable: string; // array to render (required)
  selector?: Record<string, number | string>; // non-spatial dims, e.g. { time: 0 }
  clim?: [number, number];
  colormap?: string | string[]; // named ramp ("viridis") or hex stops
  opacity?: number;
  zarrVersion?: 2 | 3;
  crs?: string; // e.g. "EPSG:32633"
  proj4?: string; // for a CRS with no built-in
  bounds?: [number, number, number, number]; // [xMin, yMin, xMax, yMax] in the store's CRS
  spatialDimensions?: { lat?: string; lon?: string };
```

```

headers?: Record<string, string>; // authenticated stores
beforeLayerId?: string;
}

// A projected national grid: crs/proj4 are forwarded to the renderer, which
// reprojects on the GPU. Without them the store is read as WGS84 and lands in
// the wrong place.
const layerId = await app.addZarrLayer?.(
  "seNorge tmax",
  "https://example.no/senorge.zarr",
  {
    variable: "tmax",
    selector: { time: 0 },
    clim: [-30, 30],
    colormap: "viridis",
    crs: "EPSG:32633",
  }
);

// Drive a time slider without rebuilding the layer: the renderer keeps the
// chunks it already fetched. (`addZarrLayer` is optional, so guard the id.)
if (layerId) {
  await app.setZarrLayerSelector?.(layerId, { time: 12 });
}

```

`addZarrLayer` is headless: it does not open the Zarr panel (the user can still open it from **Add Data** → **Zarr Layer** to tweak colormap and color limits). It resolves with the new layer's id once the layer is registered, and rejects when `variable` is missing or the store cannot be read. The layer supports visibility, opacity, ordering, and removal from the Layers panel like any other layer.

`selector` picks a slice by coordinate **value**, not by index: on a `month` axis of 1-12, December is `{ month: 12 }`. The panel's Selector (JSON) field means the same thing, and editing it now re-slices the layers already on the map.

Click-to-value and region statistics

`queryZarrLayer` reads the layer's values under a GeoJSON geometry: a `Point` for Identify, a `Polygon` / `MultiPolygon` for zonal statistics. It is the read counterpart of `setZarrLayerSelector`, reaching the same live renderer by layer id.

```

export type GeoLibreZarrQueryGeometry =
  | { type: "Point"; coordinates: [number, number] } // WGS84 lng/lat
  | { type: "Polygon"; coordinates: number[][][] }
  | { type: "MultiPolygon"; coordinates: number[][][][] };

// Dimension values to read instead of the slice on screen. A list per dimension
// (e.g. { month: [1, 7] }) nests the returned values by that dimension.
export type GeoLibreZarrQuerySelector = Record<
  string,
  | number
  | string
  | number[]
  | string[]
  | {
    selected: number | string | number[] | string[];
    type?: "index" | "value";
  }
>;

```

```
export interface GeolibreZarrQueryOptions {
  signal?: AbortSignal; // cancel a query the user moved past
  includeSpatialCoordinates?: boolean; // default true
}

// { [variable]: values, dimensions, coordinates }
export interface GeolibreZarrQueryResult {
  [variable: string]: unknown;
  dimensions: string[];
  coordinates: { [key: string]: (number | string)[] };
}

// Identify: read the value under a map click, for the slice on screen.
map.on("click", async (event) => {
  const { lng, lat } = event.lngLat;
  const result = await app.queryZarrLayer?.(layerId, {
    type: "Point",
    coordinates: [lng, lat],
  });
  const [value] = (result?.sst as number[]) ?? [];
  showReadout(value); // undefined outside the store's grid
});

// Zonal statistics: every pixel inside a polygon, on another time slice.
const region = await app.queryZarrLayer?.(layerId, aoi.geometry, { time: 12 });
```

The renderer already holds the store's grid, so it does the reprojection and fill-value masking: pass a WGS84 `[lng, lat]` straight from a map click rather than opening the store again with your own zarrita point read. Note the returned `coordinates` are in the store's **source** CRS (Web Mercator meters for EPSG:3857, degrees for EPSG:4326, source units for a custom-proj4 dataset), not WGS84.

Values come back **empty** rather than as an error for a geometry outside the store's grid, and for a layer whose first chunks have not loaded yet, so a query fired immediately after `addZarrLayer` resolves can be empty even though the id is live. `queryZarrLayer` itself resolves to `null` when no live Zarr layer has that id, and an aborted query **rejects** (an abort error), so guard the call if you cancel on every new click.

A `selector` passed here only scopes the read: the layer keeps rendering the slice it was on, so an Identify readout for another time does not disturb the map. Use `setZarrLayerSelector` when you do want the display to move.

The panel can also open a store from a folder on disk, via a **Browse folder** button next to the Zarr URL. It appears in the desktop app and in browsers with the File System Access API (Chromium), since reading a folder needs a filesystem API the plugin cannot supply; elsewhere the panel is unchanged. A local cube binds to the Time Slider like a remote one — its CF `units` are read out of the folder, because its recorded URL is an identifier rather than an address.

Driving a layer's own time dimension from the Time Slider

The Time Slider understands three kinds of temporal layer. Two are built in: a **vector** layer filtered by a timestamp property, and a **raster time series** of dated sources the dock steps between. The third is for a layer that is *one store* with time as an **internal dimension** — a Zarr data cube, or a plugin's own frame-based layer — where the timeline picks a slice rather than a source.

That third kind is expressed as a **temporal adapter**:

```
export interface TemporalLayerAdapter {
  getTimeValues: () => ReadonlyArray<Date | number |
string>; // the time coordinate, in index order
  setTime: (date: Date) => void | Promise<void>; // apply a date to the layer
  dimension?: string; // the axis name, default "time"
  granularity?: "hour" | "day" | "month" | "year"; // stepping unit; derived from the axis
span when omitted
  displayUnits?: ("hour" | "day" | "month" | "year")[]; // units offered by the slider's
granularity controls
}
```

The slider owns the snapping: it reads `getTimeValues()`, finds the index nearest the current date, and calls `setTime()` with **that slice's own date**, skipping a tick that lands on the slice already showing. So a daily cube under a month-stepping timeline costs one chunk fetch per changed index, not one per tick. Calls are throttled while the handle is dragged, with the trailing edge always applied. Accepted value forms are `Date`s, epoch seconds/milliseconds, bare calendar years, and date strings.

A Zarr layer added with `addZarrLayer` **registers its own adapter** for its `time` axis (decoding CF units such as "days since 1970-01-01" from the store's metadata), so it needs no call here. Use `registerTemporalLayer` for a custom layer you render yourself:

```
const times = frames.map((frame) => frame.timestamp); // Date[] | number[] | string[]

const detach = app.registerTemporalLayer?.(
  layerId,
  {
    dimension: "time",
    getTimeValues: () => times,
    setTime: (date) => showFrame(nearestFrame(date)),
  },
  { bind: true }
);
```

Registering makes the layer **bindable**: the Layers panel's row menu gains "Bind time dimension to Time Slider". Passing `{ bind: true }` binds it immediately and opens the dock, which is usually what a plugin that just loaded a cube wants. The binding is persisted on `layer.metadata.timeBinding` (as `{ kind: "selector", dimension, min, max, granularity }`) so it survives a project round-trip; the time values themselves are not persisted, because they belong to the data store — re-register the adapter when the layer is recreated and the timeline picks it back up.

Call the returned function, or `app.unregisterTemporalLayer?.(layerId)`, to drop the adapter. Removing the layer does it too.

A bound cube shares the timeline with any vector bindings and dated overlays: the track spans the union of their extents, and the widest dataset sets the stepping granularity.

Closing the dock again. A dock that a binding opened closes itself once the last temporal layer is gone, so it never lingers over a map with no timeline. Removing the bound layer is enough; if your plugin keeps the layer and only drops its binding, clear `layer.metadata.timeBinding` as well as unregistering the adapter. A dock the *user* opened from the Plugins menu is left alone, and so is one that still has raster sources of its own or a KML `<TimeSpan>` overlay to drive.

Activating and deactivating other plugins

```
await app.activatePlugin?.("maplibre-gl-time-slider");
app.deactivatePlugin?.("maplibre-gl-time-slider");
```

`activatePlugin` takes an optional second argument, a project-state patch applied once the target is active; it resolves false when the plugin is unavailable, refuses to activate, or rejects the state.

`deactivatePlugin` is its counterpart and returns true when the plugin ended up inactive.

Neither may target the **calling** plugin: `activatePlugin` on yourself is meaningless, and deactivating yourself would unmount the code still on the stack. Both return false in that case.

Custom (WebGL) layers and paint ownership

`registerExternalNativeLayer` mirrors a layer the plugin added to the map itself into GeoLibre's layer store, so it appears in the Layers panel and persists with the project:

```
export interface GeoLibreExternalNativeLayerRegistration {
  id: string;
  name: string;
  type?: GeoLibreLayer["type"]; // closest built-in type, e.g. "raster"
  nativeLayerIds: string[]; // the MapLibre layer id(s) you added
  source?: Record<string, unknown>;
  sourceIds?: string[];
  sourceId?: string;
  geojson?: FeatureCollection; // for vector layers
  beforeId?: string;
  opacity?: number;
  style?: Partial<LayerStyle>;
  metadata?: Record<string, unknown>;
  sourcePath?: string;
  paintMode?: "geolibre" | "plugin"; // see below
  paintBridge?: {
    // see below
    setOpacity?: (opacity: number) => void;
    setVisibility?: (visible: boolean) => void;
  };
}
```

A plugin that renders with its own MapLibre `CustomLayerInterface` — a WebGL layer that draws its own pixels — registers it the same way:

```
const layer = new MyWebGLLayer({ id: "my-layer" });
app.getMap?.()?.addLayer(layer);

app.registerExternalNativeLayer?.({
```

```

id: "my-layer",
name: "My WebGL layer",
type: "raster",
nativeLayerIds: ["my-layer"],
// The layer has no MapLibre paint properties, so GeoLibre must not offer
// paint editors that cannot reach it.
paintMode: "plugin",
// Optional: keep the Opacity sliders live by forwarding them to the layer.
paintBridge: {
  setOpacity: (opacity) => layer.setOpacity(opacity),
  setVisibility: (visible) => layer.setActive(visible),
},
});

```

- `paintMode: "plugin"` tells the Style panel that the plugin paints the layer. It then shows only the controls that actually apply — insert-below, zoom range, and (with a bridge) opacity — instead of raster brightness/saturation/contrast/hue sliders that silently do nothing. Visibility, reordering, and removal keep working from the Layers panel; MapLibre honors `visibility` and the zoom range on a custom layer.
- `paintBridge` supplies the setters GeoLibre calls when the user changes opacity or visibility. Supplying `setOpacity` keeps the Opacity slider in both the Layers and Style panels; omitting it hides the slider rather than leaving an inert one. Supplying a bridge implies `paintMode: "plugin"`.
- The setters are called only when the value changes, not on every layer sync, and they are held outside the layer record (functions cannot be serialized into a project file). Re-register the layer after a project reload to restore the bridge, and call `unregisterExternalNativeLayer(id)` from `deactivate`.

For Zarr specifically, prefer `addZarrLayer` above: the host's renderer already integrates with the panels, so no custom layer or bridge is needed.

Right sidebar panels

A plugin can register a native right-sidebar panel that docks beside the built-in Style panel and behaves like a first-class part of the workspace, instead of emulating one with a fixed overlay. The host renders the panel chrome (a header with collapse and close buttons, a collapsible rail, and a resize handle); the plugin owns only the body.

```

export const myPlugin: GeoLibrePlugin = {
  id: "my-workbench",
  name: "Workbench",
  version: "0.1.0",
  activate(app) {
    // Register once, then open. registerRightPanel returns an unregister fn.
    this._unregister = app.registerRightPanel?.({
      id: "my-workbench",
      title: "Workbench",
      defaultWidth: 360,
      render(container) {
        const button = document.createElement("button");
        button.textContent = "Run analysis";
        container.appendChild(button);
        // Optional cleanup, run when the panel closes or is unregistered.
        return () => button.remove();
      }
    });
  }
};

```

```

    },
    onOpen() {},
    onCollapse() {},
    onClose() {},
  });
  app.openRightPanel?.("my-workbench");
},
deactivate(app) {
  app.closeRightPanel?.("my-workbench");
  this._unregister?();
},
} as GeoLibrePlugin & { _unregister?: () => void };

```

Notes:

- `render(container)` is called once with an empty element you fill with plain DOM. An external plugin cannot share GeoLibre's React instance, so the contract is DOM, not a React node. The container stays mounted across collapse, so any state in your DOM persists; the returned cleanup runs on close or unregister.
- Only one plugin panel is active at a time. The built-in panel on the side the plugin panel is docked (Layers on the left, Style on the right) collapses to its rail while the plugin panel is expanded next to it, and restores when the plugin panel moves to the other side, collapses to its own rail, or closes.
- `openRightPanel(id)` makes the panel active and expanded (it also expands a collapsed panel); `collapseRightPanel(id)` collapses it to its rail without closing; `closeRightPanel(id)` releases the workspace; `getActiveRightPanel()` returns the active id or `null`.
- The panel is a flex sibling of the map, so opening it shrinks the map view (the map keeps filling the remaining space); no manual map padding is required.
- **Dock position:** a panel docks at one of four positions (left to right): `left-of-layers`, `right-of-layers` (between Layers and the map), `left-of-style` (between the map and Style), or `right-of-style` (the default). Set `dock` on the registration to choose the initial position. The user steps the panel between positions at runtime with the two move buttons in the panel header (disabled at the ends), and a plugin can set it directly with `app.setActiveRightPanelDock?.(...)`. The position resets to the panel's declared `dock` when it closes or another panel opens.
- **Shared-rail modes** (`replace-style` / `replace-layers`): two non-positional docks for workbench-style plugins that want to feel like a first-class sidebar workspace rather than a second rail beside Style (right) or Layers (left). Register with `dock: "replace-style"` (or `"replace-layers"`) and the host shows a single rail on that edge listing both your panel and the built-in panel; selecting one expands it while the other stays as a rail entry. The two are mutually exclusive, so the user never sees two adjacent rails. The built-in panel starts collapsed so the workbench reads as the active workspace, and the user can expand it (which collapses the workbench) at any time. Everything else (chrome, resize, collapse, close, lifecycle hooks) is unchanged.
- **Switching modes at runtime:** the modes are not exclusive choices baked in at registration. In a positional dock the panel header shows a **merge** button that joins the shared rail on its current side — a layers-side panel (`left-of-layers` / `right-of-layers`) joins the Layers rail, a style-side panel the Style rail. In a shared rail it shows a **detach** button that pops the panel back out to a movable positional panel on the same side (`right-of-layers` / `right-of-style`),

where the left/right move buttons return. A plugin can drive the same switch with `app.setActiveRightPanelDock?.("replace-style" | "replace-layers" | "right-of-style" | ...)`. The shared rails are not part of the left/right *step* sequence (the arrows only walk the four positional docks); merge/detach is the way in and out.

- These methods are typed optional for forward-compatibility with host variants that have no right sidebar, so call them with optional chaining (`app.registerRightPanel?.(...)`).

Toolbar menus

A plugin can add its own top-level menu button to the GeoLibre banner (beside Project / Edit / View / Plugins), with nested submenus and action items. Register the menu in `activate` and unregister it in `deactivate`:

```
const unregister = app.registerToolbarMenu?({
  id: "my-plugin-menu",
  label: "Workbench",
  items: [
    {
      id: "open",
      label: "Open workbench",
      onSelect: () => app.openRightPanel?.("my-workbench"),
    },
    {
      type: "submenu",
      id: "tools",
      label: "Tools",
      items: [
        {
          id: "qa",
          label: "Data QA",
          onSelect: () => app.openFloatingPanel?.("my-qa"),
        },
      ],
    },
    { type: "separator" },
    { id: "about", label: "About", disabled: false, onSelect: () => {} },
  ],
});
```

Each item is an **action** (`onSelect`, the default when `type` is omitted), a **submenu** (nested `items`), or a **separator**. Items typically open a right panel or a floating panel, but `onSelect` can run anything. Re-registering the same `id` replaces the menu, so you can rebuild it as your plugin's state changes.

Menus from **external plugins** (loaded from a zip, a manifest URL, or a bundled drop-in) render at the end of the banner, after the Help menu, so third-party menus sit together past the built-in menus. Menus from built-in plugins render beside the built-in menus. The host decides placement from the menu's owning plugin, so you do not need to do anything special.

The user decides whether an external plugin's menu occupies the banner. Each installed plugin that has registered a menu gets a **Show menu in toolbar** switch in its Plugins → Installed entry; turning it off moves the plugin's menus into that entry as submenus, with the same items. The choice is

remembered per plugin on the device. Built-in plugin menus always stay in the banner. Since your menu may end up nested a level deeper, keep its top level short and consider [adding items to a built-in menu](#) instead when you only have a few actions.

Every `label` (the menu button's, a submenu trigger's, an action's) accepts a **getter function** as well as a plain string, the same way panel titles do:

```
app.registerToolbarMenu?.({
  id: "my-plugin-menu",
  label: () =>
    app.translate?.("plugin.my-plugin.menu", "Workbench") ?? "Workbench",
  items: [
    {
      id: "open",
      label: () =>
        app.translate?.("plugin.my-plugin.open", "Open workbench") ??
        "Open workbench",
      onSelect: () => app.openRightPanel?.("my-workbench"),
    },
  ],
});
```

The host re-reads every label each time it renders the menu tree, and it re-renders on a language change, so a getter follows the app language without your plugin re-registering its menu. A plain string is frozen at registration time. A getter that throws or returns nothing usable degrades to the item's id path and warns once, so a broken label cannot make the menu disappear.

Adding items to built-in menus

Most plugins do not need a top-level menu of their own. Like a QGIS plugin that adds itself to the Vector or Raster menu, a plugin can add items to one of GeoLibre's built-in menus instead:

```
const unregister = app.registerMenuContribution?.({
  id: "my-plugin-processing",
  menu: "processing", // "addData" | "processing" | "controls"
  items: [
    {
      id: "run",
      label: "Run analysis...",
      onSelect: () => app.openRightPanel?.("my-workbench"),
    },
  ],
});
```

The host always nests the items under a submenu named after your plugin, placed after a separator at the end of the target menu, and sorts plugin submenus alphabetically. A plugin cannot insert loose items between the built-in entries or reorder them. Several contributions from the same plugin to the same menu share one submenu, with a separator between them. Contributions with no items are not shown.

Items use the same shape as [toolbar menus](#): actions, submenus and separators, with optional icons and label getters that follow the app language. Register in `activate` and call the returned function (or `app.unregisterMenuContribution?.(id)`) in `deactivate`; re-registering the same `id` replaces the

contribution. Ids are global, like toolbar menu ids, so prefix them with your plugin id; the host warns when one plugin's id replaces another's, and `unregisterMenuContribution` only removes contributions your own plugin registered. The host also drops a plugin's contributions when it is deactivated or its activation fails, so a missed disposer cannot leave stale items behind. A `menu` the host does not recognize (for example, one added in a newer GeoLibre) logs a warning and is ignored rather than failing your activation. Like toolbar menus, contributions are hidden in the read-only viewer and when a deployment does not grant `plugins:install`.

Following the app language

The GeoLibre UI is translated with `react-i18next`, but a plugin renders its panels as plain DOM and cannot use the host's React hooks. Four methods bridge that gap:

```
// The active catalog code ("en", "zh", "pt-BR", ...).
const locale = app.getLocale?(). ?? "en";

// Resolve a key, falling back to your own English text when no catalog has it.
const title =
  app.translate?.("plugin.my-plugin.title", "Workbench") ?? "Workbench";
const label = app.translate?.("plugin.my-plugin.count", "{{n}} features", {
  n: 3,
});

// Re-render when the user switches language. Returns an unsubscribe function –
// call it from `deactivate`, or the listener keeps re-rendering DOM you no
// longer own.
const stop = app.onLocaleChange?.((next) => renderPanel(container, next));

// Ship your own translations (call once, from `activate`). Flat dotted keys per
// locale; `translate` then resolves them in that language.
app.registerTranslations?.({
  de: {
    "plugin.my-plugin.title": "Werkbank",
    "plugin.my-plugin.count": "{{n}} Objekte",
  },
  fr: {
    "plugin.my-plugin.title": "Atelier",
    "plugin.my-plugin.count": "{{n}} entités",
  },
});
```

Conventions:

- **Always pass your own English text as** `defaultValue`. It is what renders in a language you ship no translation for, and on a host that predates these methods, so translations are an upgrade, not a prerequisite.
- **Namespace your keys by plugin id** (`plugin.<your-id>.<something>`) so they cannot collide with the host's own keys. `registerTranslations` enforces this: it accepts only string values under `plugin.<id>` and drops (with a console warning) anything else.
- **The host's catalogs win.** A key GeoLibre's bundled catalogs already define keeps the host's text, and `registerTranslations` never overwrites an existing entry. Registrations last for the session; registering the same key again is a no-op, so do it once rather than on every activation.

- Plural keys work as they do in the host: register `plugin.my-plugin.items_one / plugin.my-plugin.items_other` (and the extra forms languages such as Russian or Arabic need) and pass `{ count } in params`.
- These methods are typed optional like the rest of the API, so call them with optional chaining and keep a literal fallback.
- Panel titles and toolbar labels take getters precisely so they can call `app.translate?.()` and stay current; use those rather than re-registering on every language change. DOM you build once (buttons, hints, status lines) should be re-labelled from an `onLocaleChange` listener.

Built-in plugins (in `packages/plugins`) follow the same contract with two helpers exported from `@geolibre/plugins`:

- `createPluginTranslator(app, pluginId)` returns `tr(key, english, params?)`, which resolves `plugin.<pluginId>.<key>` and interpolates the English fallback itself when the host has no `translate`. A key starting with `@` is absolute, for reusing a host key (`tr("@common.cancel", "Cancel")`). `app` may be a getter, for plugins that keep the API in a module-level variable.
- `pluginDisplayTitle(app, pluginId, name)` returns a panel-title getter that reads `toolbar.plugin.<pluginId>`, the plugin's name as the Plugins menu shows it, so the dock header and the menu entry stay in the same language.

Their strings live in GeoLibre's bundled catalogs (`apps/geolibre-desktop/src/i18n/locales/*.json`) rather than in `registerTranslations`, and the catalog test requires every locale to carry them (see `docs/i18n.md`).

Saving credentials

A plugin that needs to remember a token or API key should use `app.credentials` instead of writing its own `localStorage` key. On the desktop app the values live in the OS credential store (macOS Keychain, Windows Credential Manager, Linux Secret Service); on the web build, the Jupyter embed and mobile they live in `localStorage`.

```
const apiKey = app.credentials?.get("api-key") ?? ""; // "" when nothing is saved
const persisted = app.credentials?.set("api-key", value); // "" deletes
const where = app.credentials?.location(); // "keychain" | "browser", for UI copy
```

- **Names** are 1-64 characters of letters, digits, `_` and `-`; anything else throws a `TypeError`.
- **Scoping**: the host adds your plugin id, so the value is stored as `plugin.<your-id>.<name>`. You never pass the id.
- **Reads are synchronous**. The desktop app loads every saved value at startup, before any plugin runs.
- **set returns** `true` when the value is persisted and `false` when it lasts only for this session (a failed `localStorage` write, or an unavailable keychain on desktop). Desktop keychain write failures also raise the app's credential-storage warning, so you do not need your own.

- **This is storage, not isolation.** Plugins run as trusted code in the app window, so a plugin can still read another plugin's values from memory or by wrapping `fetch`. The id prefix keeps well-behaved plugins apart; it is not a security boundary.
- **The desktop credential store cannot be read from a plugin.** The app reads every saved credential in one native call during startup, before any external plugin loads, and the store then refuses further reads until the window reloads (the loader also closes reads before importing a plugin, in case that read never ran). A plugin calling the store's Tauri command directly gets an error instead of the saved tokens. This narrows what a plugin can reach; it does not isolate plugins. Values the app has already loaded into memory (Settings tokens, other plugins' `app.credentials` values) remain reachable from the shared JavaScript realm, writes to the store are not restricted, and on the web, Jupyter and mobile builds the values sit in `localStorage`. Install only plugins you trust.
- **Never put secrets in `getProjectState`.** Plugin settings are saved in project files and shared or exported with them.
- **Built-in plugins use it too.** The Hugging Face, Mapillary and God's Eye View tokens moved from their own `localStorage` keys to `app.credentials`; the host migrates an existing plaintext value on first launch and removes it only after the new write succeeds.

Signing in with a session cookie

A service that signs users in with a session cookie does not work from the desktop app's webview. The page runs at `tauri://localhost`, so a cookie the service sets is third-party, and WebKit (macOS, Linux) drops it: the login request returns 200 and the next request returns 401. On the desktop, `app.nativeFetch` sends requests through the app's native HTTP client instead. It has the `fetch` signature, takes any method and body, is not subject to CORS, and keeps cookies in a jar that lasts the app session.

```
const http = app.nativeFetch ?? fetch; // undefined in the browser and Jupyter builds
await http(`${server}/api/v1/auth/access-token`, {
  method: "POST",
  body: new URLSearchParams({ username: email, password }),
});
const user = await (await http(`${server}/api/v1/users/current`)).json(); // sends the cookie
```

- **Cookies stay native.** Set-Cookie headers are not exposed to JavaScript, and `credentials` has no effect: every request sends whatever the jar holds for that host.
- **One jar for the session.** It is shared by every plugin and cleared when the app restarts, so a plugin should be ready to sign in again.
- **Limits:** request and response bodies up to 64 MiB, a 120 second timeout, and no link-local or cloud-metadata addresses. `AbortSignal` cancels the native request.

Floating panels

A floating panel is a draggable, closeable card the host overlays on the map, opening at the corner named by `position` (top-left by default). Unlike a dockable right panel (one active panel docked at a fixed position), several floating panels can be open at once and they do not shrink the map. The render contract is the same plain-DOM `render(container)` as right panels.

```
const unregister = app.registerFloatingPanel?>({
  id: "my-qa",
  title: "Data QA",
  defaultWidth: 300,
  render(container) {
    container.textContent =
      "Rendered by the plugin via registerFloatingPanel().";
    return () => {
      // optional cleanup, run on close/unregister
    };
  },
});

app.openFloatingPanel?("my-qa"); // open (or bring to front)
app.closeFloatingPanel?("my-qa"); // close
app.getOpenFloatingPanels?(); // -> string[] of open ids, stacking order
```

Use a right panel for a primary, persistent workspace and a floating panel for an ancillary tool or dashboard the user positions over the map. As with the other surfaces, call these methods with optional chaining since they are typed optional.

External plugins

Use the [GeoLibre plugin template](#) as the recommended starting point for external plugin development. The template includes a MapLibre control wrapper, a `plugin.json` manifest, a GeoLibre plugin entry point, and a `package:geolibre` script that builds the zip layout GeoLibre Desktop expects.

GeoLibre Desktop loads external plugins from the app data `plugins/` directory at startup. External plugins are trusted code; whether they load is also controlled by the client-side [deployment plugin policy](#). The policy is bypassable and provides neither signing nor sandboxing. External plugins can be installed as:

- A `.zip` file with a root `plugin.json`.
- An unpacked directory with a root `plugin.json`.
- A HTTPS `plugin.json` manifest URL.

The fastest way to install a `.zip` is **Manage Plugins > Settings > Install from file**: pick a packaged plugin archive and GeoLibre validates it (parsing `plugin.json`, enforcing the manifest rules, and checking the entry/style are present and within the size limit) before installing it. The plugin loads immediately and persists; reinstalling the same id replaces the previous copy and reloads the updated version. Persistence differs by build:

- **Desktop** copies the archive into the app data `plugins/` directory as `<plugin-id>.zip`, where the startup scan re-loads it.
- **Web** unpacks the archive in the browser and stores the bundle in IndexedDB, replaying it on the next visit. Web-installed plugins are listed under **Install from file** with an uninstall control (the desktop copies live on disk and are managed there).

The Plugins settings section can also add local development directories outside the app data folder. Each configured directory can contain plugin zips, unpacked plugin bundle folders, or be a single unpacked plugin bundle itself. Configured development directories are scanned before the app data `plugins/` directory, so a development copy can override an installed external plugin with the same ID. Built-in plugins still take precedence over all external plugins.

For the web app, use manifest URLs or **Install from file** (above). Manifest URLs: GeoLibre fetches the manifest, resolves `entry` and `style` relative to the manifest URL, then loads the bundled ESM entry. Browser loading requires HTTPS except for `localhost` and depends on the host allowing CORS. Install-from-file unpacks the uploaded zip in the browser (no network or CORS) and persists it in IndexedDB. Both paths execute the bundled ESM entry the same way (a `blob: import()`, allowed by the web build's `script-src`), so external plugins remain trusted code regardless of how they were installed.

Bundled plugins (baked into the build)

To ship an external plugin as part of GeoLibre — loaded automatically, with no Settings entry and no manifest URL — drop its built bundle into the Vite public directory, one folder per plugin id:

```
apps/geolibre-desktop/public/plugins/example-plugin/  
  plugin.json  
  dist/index.js  
  dist/style.css
```

This is the **same content a manifest URL would serve**. A drop-in is all that is required — no source edits per plugin. The `bundledPlugins()` Vite plugin (`apps/geolibre-desktop/vite-plugins/bundled-plugins.ts`) scans `public/plugins/` at build and dev-server start, exposes the discovered manifest paths through the `virtual:bundled-plugins` module, and `usePlugins.ts` loads them through the normal external-plugin path (`fetch` → `blob import` → `register`). Discovery happens at build time, so restart the dev server or rebuild after adding, updating, or removing a plugin folder.

The same folder serves **both** the web and desktop builds: the desktop app bundles the identical frontend (`frontendDist` in `tauri.conf.json`) and serves it from `tauri://localhost`, which is same-origin and allowed by the desktop CSP (`connect-src 'self'`, `script-src ... blob:`). Bundled

manifest URLs are injected at load time rather than stored in Settings. Bundled plugins are eligible to load without being listed in `allowed` or permitting sideload, but an id in `blocked` still prevents loading. They are deduplicated by plugin id against any user/project plugin of the same id.

Private plugins should be git-ignored under `public/plugins/` (see that folder's `.gitignore`) and copied in at build/deploy time (for example in CI before `npm run build`, or by a plugin repo's own install script) so their code stays out of GeoLibre's history. The discovery code is generic and committed; only the plugin payload is excluded.

A bundled drop-in's `plugin.json` may additionally set `"activeByDefault": true` to activate the plugin on startup. Saved plugin state still wins: a loaded project or persisted state carrying `activePluginIds` overrides the default. The flag is honored **only** for bundled drop-ins; it does not bypass a blocked plugin policy.

If instead you want a plugin compiled into the main JS bundle (no `plugin.json`, no fetch), register it as a built-in plugin (see [Add a built-in plugin](#)).

```
{
  "id": "example-plugin",
  "name": "Example Plugin",
  "version": "0.1.0",
  "entry": "dist/index.js",
  "description": "Optional short description",
  "style": "dist/style.css",
  "engines": ["maplibre", "cesium"]
}
```

The `entry` file must export a `GeoLibrePlugin` as either the default export or a named `plugin` export. The exported plugin `id`, `name`, and `version` must match `plugin.json`. The entry must be a self-contained `.js` or `.mjs` bundle because relative module imports inside the zip are not resolved by this first loader. The optional `engines` array declares which of GeoLibre's four map renderers the plugin supports (`"maplibre" | "mapbox" | "cesium" | "arcgis"`, defaulting to `["maplibre"]`). Plugins supporting the native globe add `"cesium"`; plugins that stay on the Style Spec surface can add `"mapbox"` (see [Supporting the Mapbox renderer](#)); and plugins with an ArcGIS-native adapter can add `"arcgis"` (see the [ArcGIS renderer](#)). The host suspends a plugin when the selected engine is not in this list and restores it when a compatible engine becomes active.

External plugin entries are executed with `import(URL.createObjectURL(...))`, which is why the desktop CSP in `tauri.conf.json` includes `blob:` in `script-src`. Removing `blob:` from `script-src` breaks external plugin loading. Combined with `'unsafe-eval'`, this means code that can create a blob URL can execute scripts, which is acceptable because external plugins are trusted local files installed by the user.

Because plugins run as trusted code in the host document, they can read `window.__GEOLIBRE_RUNTIME_ENV__`, the runtime environment map. On the desktop app this map includes the AI Assistant's [OS-environment keys](#) (the allowlisted provider variables read from the user's shell), not only the values typed into Settings → Environment Variables. Treat any credential reachable through the app's environment as visible to installed plugins, and only install plugins you trust.

Plugin project settings are treated as sensitive at the project egress boundary. GeoLibre keeps them in a trusted local save only when the user explicitly chooses to retain credentials, and removes the entire `plugins.settings` object from shares, standalone HTML exports, embed snapshots, and collaboration snapshots. The exception is state a plugin's registry entry declares publishable (see `publishableSettings` under [Plugin marketplace](#)), which is kept and still scrubbed for credentials. Store portable, non-secret identifiers such as broker references in layer source or metadata instead when recipients need them.

Manifest paths must be relative zip paths with forward slashes, no leading slash, no backslashes, and no `..` segments. External plugins cannot set `activeByDefault` on the exported plugin object, and the manifest-level flag is honored only for bundled drop-ins (see "Bundled plugins" above); saved project state can still reactivate an external plugin by ID after the zip is loaded.

The optional `style` CSS is injected globally into the host document, not scoped to the plugin. Plugin authors are responsible for scoping their selectors (for example with a plugin-specific class prefix) so broad rules do not restyle the rest of the app. Injected CSS can also issue network requests through `url()` references and `@import`, so a plugin stylesheet can load external fonts, images, or additional sheets; treat plugin CSS with the same trust expectations as plugin code.

When using the template, update `geolibre-plugin/plugin.json` and `src/geolibre.ts` together so `id`, `name`, and `version` stay in sync. Run `npm run package:geolibre`, then either copy the generated zip into the desktop app data `plugins/` directory, add the template's `geolibre-plugin/` directory in Settings > Plugins for local development, or host the `geolibre-plugin/` directory and add its `plugin.json` URL.

Plugin marketplace

The Settings menu's **Manage Plugins** entry opens a standalone dialog (modeled on QGIS's plugin manager) with **All**, **Installed**, **Not installed**, **Upgradeable**, and **Settings** sections. The first four list curated registry plugins so users can install, update, and uninstall them without hand-entering manifest URLs; the Settings section installs a plugin from a local `.zip` and manages additional local plugin directories and manual manifest URLs. Actions apply immediately (install/uninstall/update are live; uninstall asks for confirmation). It is a thin layer over the manifest-URL loader above: installing an entry records its manifest URL in the plugin manifest URL list, and the existing loader fetches and registers it. It introduces no new trust path.

The registry is JSON, fetched from the primary policy's `plugins.registryUrl`, then the legacy `VITE_GEOLIBRE_PLUGIN_REGISTRY_URL` setting, or by default the hosted registry at `https://plugins.geolibre.app/plugin-registry.json` (the [opengeos/geolibre-plugins](#) repo, published to GitHub Pages with CORS enabled). It is an array, or an object with a `plugins` array, of entries:

```
{
  "version": 1,
  "plugins": [
    {
      "id": "example-plugin",
      "name": "Example Plugin",
      "version": "1.0.0",
      "description": "Optional short description",
    }
  ]
}
```



```

    "author": "Example Author",
    "homepage": "https://github.com/example/example-plugin",
    "manifestUrl": "https://example.com/example-plugin/plugin.json",
    "categories": ["Example"],
    "minGeoLibreVersion": "1.0.0",
    "publishableSettings": ["search"],
    "bundleSha256": "3f5c...(64 hex characters)"
  }
]
}

```

`id`, `name`, `version`, and `manifestUrl` are required; the rest are optional. A relative `manifestUrl` is resolved against the registry location, so a plugin hosted alongside the registry (e.g. `sample/plugin.json`) can be listed with a relative path. `minGeoLibreVersion` gates installation against the running app version. `publishableSettings` is optional and lets a plugin's project state (`getProjectState()`) survive "Strip credentials" and shared or exported projects. By default an external plugin's whole state is dropped there and counted as credential-bearing, because it can hold anything. List the top-level state keys that are safe to publish (`["search", "filters"]`), or use `true` to keep the whole state. The declaration is reviewed with the registry entry and is never read from a project file. What is kept is still scrubbed for credential-named fields and credentialed URLs, and a registry entry cannot widen a built-in plugin's list. Keep secrets out of those keys; use `app.credentials` for them.

`bundleSha256` is optional: the lowercase hex SHA-256 of the published bundle, computed the way `computePluginBundleHash` in `plugin-integrity.ts` does (SHA-256 of the entry, SHA-256 of the style or of an empty string, then SHA-256 of the two digests). The hosted registry generates it for every plugin it serves. When an entry has one, installing it from the marketplace or a `?plugin=` deep link pins that hash before the URL is installed, so the first load checks the downloaded code against the reviewed hash instead of trusting whatever the URL serves first; a mismatch is held back like any changed bundle. The Update action likewise refuses a download that doesn't match, before evaluating it. An entry without `bundleSha256` keeps the trust-on-first-use pin.

The registry's maintainers can also pull plugins through `blocklist.json`, published next to the registry (`https://plugins.geolibre.app/blocklist.json` for the hosted one, or `blocklist.json` beside a configured `registryUrl`). GeoLibre fetches it once per session before loading external plugins and caches it for offline starts. An entry with only an `id` blocks every version of that plugin from every external source (registry, manifest URL, zip, plugin directory): installing and loading are refused through the same policy gate as deployment-policy `blocked`, with the entry's `reason` shown. An entry that also has a `bundleSha256` blocks just that bundle, from every external source: a URL plugin, zip archive or plugin directory with exactly that code isn't loaded or installed, and an Update to it is refused, all before the code runs. Bundled drop-ins are never blocked. If the blocklist can't be fetched (or returns 404 after a list was cached), the cached copy applies; a registry that has never published one blocks nothing.

Curate the registry and host plugin bundles in the [opengeos/geolibre-plugins](https://github.com/opengeos/geolibre-plugins) repo, which ships a `sample/` template.

Externally loaded plugins (registry installs, zips, manifest URLs, and bundled drop-ins) are listed together in a **Plugins → Installed** submenu at the bottom of the Plugins menu, sorted alphabetically by display name, rather than mixed into the built-in entries. The submenu appears only when at least one external plugin is loaded and ends with a **Manage Plugins...** shortcut. A plugin that also registers its own top-level menu (see [Toolbar menus](#)) keeps its Installed entry too.

Uninstalling prompts for confirmation, then unregisters the plugin at runtime (deactivating any active map control) so the Plugins menu updates without a reload. When a registry entry advertises a newer `version` than the loaded plugin, the marketplace shows an Update action that re-fetches the manifest URL and re-registers the published version in place; the new version is fetched and validated before the old one is removed, so a failed update leaves the installed plugin intact.

Assistant tools

Plugins can expose the same action to their panel and the AI Assistant. External plugins can use `app.registerAssistantToolSpec` with a plain JSON Schema and a callback, without importing or bundling `@strands-agents/sdk` or `zod`:

```
let disposeTool;

export default {
  id: "city-loader",
  name: "City loader",
  version: "1.0.0",
  activate(app) {
    // Share this function with the panel's Add button.
    async function addCities(input) {
      if (
        !input ||
        typeof input !== "object" ||
        typeof input.name !== "string"
      ) {
        throw new Error("name must be a string");
      }
      const response = await fetch(
        "https://raw.githubusercontent.com/opengeos/leafmap/master/examples/data/us_cities.geojson"
      );
      if (!response.ok) throw new Error(`Download failed: ${response.status}`);
      const data = await response.json();
      const layerId = app.addGeoJsonLayer(input.name, data);
      return { layerId, featureCount: data.features.length };
    }
    disposeTool = app.registerAssistantToolSpec?.({
      name: "add_cities",
      description: "Add US cities to the map as a GeoJSON layer",
      inputSchema: {
        type: "object",
        properties: { name: { type: "string" } },
        required: ["name"],
        additionalProperties: false,
      },
      callback: addCities,
    });
  },
  deactivate() {
    disposeTool?.();
    disposeTool = undefined;
  }
};
```

```
    },  
  };
```

JSON Schema input is not automatically validated. The callback must validate model-supplied input before performing an action. Return JSON-serializable data or a promise of it; a callback returning `undefined` produces `null`. Thrown errors become assistant tool error results. Omitting `inputSchema` describes an empty object with no additional properties.

Built-in plugins that already import the SDK can instead call

`app.registerAssistantTool(tool({ name, description, inputSchema, callback })),` using the SDK's `tool()` factory with a zod schema for automatic validation. Both entry points use one registry and return an unregister function. External plugins should feature-detect these optional methods for older hosts.

The host injects the plugin owner, ignoring any owner argument supplied by a plugin. Model-facing names are `plugin_<owner-length>_<plugin-id>_<tool-name>`. Names and plugin IDs use letters, digits, underscores, or hyphens, and the full name must fit within 64 characters. Names that differ only in case or underscore/hyphen spelling are rejected if they collide. Registering the same name for the same owner replaces it; an older disposer cannot remove the replacement. Tools are removed on deactivation, failed activation, and plugin removal, and stale asynchronous registrations after teardown are ignored.

Register from `activate` (or from the app it hands you, including asynchronously). A tool lives for exactly one activation, so the app passed to the other lifecycle callbacks — `applyProjectState`, `setMapControlPosition`, `handleUrlParameters`, `deactivate` — omits both methods: those run for inactive plugins, whose registrations no cleanup path would reach.

The assistant refreshes its tools before the next prompt while retaining its conversation history. Plugin callbacks execute plugin-authored code, like a panel button; they should use the app API to update layers and other app state.

Progressive tool disclosure

While active plugins register at most 12 tools in total, every plugin tool is sent to the model in full on every request. Past that, the assistant stops sending plugin tool schemas up front, so the tool list does not grow with every installed plugin. The system prompt instead lists each plugin tool under a `Plugin tools:` heading, grouped by plugin, with its name and a one-line summary (the first sentence of the first line of its `description`, capped at 160 characters). The model loads the tools it needs with the host tool `load_plugin_tools`, passing exact tool names or a plugin id to load all of that plugin's tools. Loaded tools become callable in the same turn and stay loaded for the rest of the conversation. GeoLibre's own tools are always sent.

Because the summary is all the model sees before it loads a tool, **open each `description` with a sentence that says what the tool does and which data it covers**, so tools with the same shape in different plugins or domains (for example `query_*_database`) can be told apart without their schemas. Guidance registered with `registerAssistantGuidance` is still sent in full and can name your tools; the model loads them before calling them.

Assistant guidance

A tool's description is read only after the model has already decided which tool to call; that decision is driven by the system prompt. When a plugin's tools need rules about *when* to use them (for example, "call `get_pm25_ranking` directly, never as a table function inside `run_sql` "), register that text as guidance and the host appends it to the assistant's system prompt:

```
let disposeGuidance;

export default {
  id: "air-quality",
  name: "Air quality",
  version: "1.0.0",
  activate(app) {
    // ...register tools as above...
    disposeGuidance = app.registerAssistantGuidance?.(
      [
        "For PM2.5 questions, call plugin_11_air-quality_get_pm25_ranking directly",
        "with its own arguments. Never wrap it in run_sql or use it as a FROM clause;",
        "it is not a SQL table function.",
      ].join(" ")
    );
  },
  deactivate() {
    disposeGuidance?.();
    disposeGuidance = undefined;
  },
};
```

Guidance is appended under a `Plugin guidance:` heading after GeoLibre's own prompt, in registration order, each block labelled `[plugin <id>]` with the plugin that registered it, and never replaces or edits the host text. The heading tells the model the text only governs when and how to call the plugin's own tools and that the host guidelines still apply. The text itself is not filtered: like plugin code, it is trusted once the plugin is loaded, so only install plugins you trust. It must be a non-empty string of at most 4000 characters; identical text from the same plugin replaces the earlier registration instead of repeating it. Like tools, guidance is activation-only: the host injects the plugin owner, ignores any owner argument a plugin supplies, removes the text on deactivation, failed activation, and plugin removal, and the app handed to the other lifecycle callbacks omits the method. The assistant recomposes its system prompt together with its tools before the next prompt, keeping the conversation history. Feature-detect the method for older hosts.

Map renderers

The host exposes `app.getMapRenderer()` to read the current primary renderer. Engine declarations are enforced by the plugin manager for activation, URL parameters, project restoration, and delayed control registration, as well as by the Plugins menu and command palette. Renderer changes suspend unsupported plugins and retain their saved settings and activation for the return trip. Compatible plugins remount their controls on the replacement renderer.

Reading the viewport

A catalog or service browser that narrows its search to what the user can see must read the extent through `app.getViewBounds()`, not `app.getMap()?.getBounds()`. `getMap()` is null on the globe, so the second form yields no bounds there — and a plugin that reads "no bounds" as "no filter" then searches the whole world while its "current view only" checkbox stays ticked, which is exactly the silent success an `engines` declaration is meant to rule out. `getViewBounds` answers from whichever engine is primary and unwraps an antimeridian crossing (east > 180), as `MapExtent` does everywhere else in the app.

`getViewBounds` has its own `null`: no map mounted yet, the globe mid-morph between scene modes, or a camera pointed away from Earth. Do not read that as "no filter" — widening the search is the same lie in a second place. Refuse the search and say the extent is unavailable, as the ArcGIS Hub panel does.

A frontend test scans every plugin that declares Cesium support, follows its relative imports so a plugin split across a subdirectory is covered too, and fails on a `getMap()`-routed bounds read — chained or split across two statements. A call that deliberately branches on `getMap()` being null carries an `engine-audit-allow: getMap-bounds` comment on its own line or just above it, with the reason — the opt-out is scoped to that call, so a second bounds read elsewhere in the same module still reports.

A plugin that drives the renderer directly branches on which handle is non-null: `app.getMap()` on MapLibre, `app.getCesiumScene()` on the globe. The Cesium handle carries the `@cesium/engine` namespace alongside the live widget, scene, camera, and clock, so a plugin constructs Cesium objects (`SunLight`, `JulianDate`, `Cartesian3`) without importing the engine itself — which is what keeps Cesium off the 2D boot path. Its `primary` flag distinguishes the primary map area from a grid pane; the built-in environment plugins bind to the primary map only, as they do on MapLibre. The Sun simulation (native `SunLight`, globe lighting, and the scene clock), Atmospheric Effects (the sky box, `SkyAtmosphere` hue/saturation/brightness shifts, and the background colour), the Flight Simulator (`camera.setView` each frame with navigation inputs suspended), and the Clouds / Precipitation overlays (store tile layers) are the reference implementations of this pattern. A plugin bound to the globe must restore any scene state it changes on `deactivate`, because both engines are rebuilt on a renderer swap and the host re-activates compatible plugins against the new one.

Sampling raster values

`app.readRasterWindow(layerId, options)` reads one band of a raster layer over a geographic window, downsampled to a small grid. It is what the viewport stretch uses: pair it with `getViewBounds()` to compute a rescale range from the pixels actually on screen, rather than from the whole scene.

It resolves `null` when the layer is unavailable: missing, or not a raster the control has loaded. Bounds that do not intersect the raster are **not** `null`. That is a reading with an empty `values`, so check the length rather than only the null. It reads from the nearest suitable overview for the requested `width / height`, so a 32x32 window over a continent is a cheap batched read rather than one request per sample.

`values` is row-major, and the two ways a cell can be unusable are different. Unreadable tiles come back as `NaN`, but **NoData pixels keep their sentinel value**, and a sentinel like `-9999` is perfectly finite. Filtering on `Number.isFinite` alone therefore leaves the fill in and drags a min/max stretch down to it. Compare against `reading.nodata` as well:

```
const bounds = app.getViewBounds?();
if (!bounds) return; // no map mounted, or the globe is mid-morph
const reading = await app.readRasterWindow?.(layerId, {
  bounds,
  band: 1,
  width: 32,
  height: 32,
  signal: controller.signal,
});
const values =
  reading?.values.filter(
    (value) => Number.isFinite(value) && value !== reading.nodata
  ) ?? [];
if (values.length === 0) return; // window missed the raster entirely
```

Pass a `signal` for anything driven by camera movement and abort the previous read when a new one starts, so a fast pan does not queue a backlog of reads whose results land out of order.

Supporting the Mapbox renderer

The [Mapbox renderer](#) draws with Mapbox GL JS, whose runtime API is the Style Spec surface MapLibre grew out of: sources and style layers (`addSource`, `addLayer`, `setPaintProperty`, `setLayoutProperty`, `setFilter`, `getSource`, `getLayer`, `getStyle`, `moveLayer`), images (`addImage`, `updateImage`), the camera (`getBounds`, `getZoom`, `easeTo`, `jumpTo`, `fitBounds`, `project`, `unproject`), events (`on` / `off` / `once`), the DOM (`getCanvas`, `getContainer`) and picking (`queryRenderedFeatures`). `IControl` is the same contract, and `app.addMapControl` mounts a MapLibre-typed control on the Mapbox map through an adapter that aliases the `.maplibregl-ctrl-*` corner classes; the docked right-panel bridge the Web Services panels use mounts through the same door. A plugin that stays on that surface declares `engines: ["maplibre", "mapbox"]` and users can toggle it from the Plugins menu on either 2D engine; the plugin manager re-activates it against the new map on a renderer swap.

`app.getMap()` is `null` on Mapbox by design, so such a plugin must not reach the map through it alone — that is the same silent no-op the globe rule above guards against. Built-in plugins read the map through `getStyleMap(app)` (`packages/plugins/src/plugins/style-map.ts`): the MapLibre map when there is one, else the Mapbox map through MapLibre's types (the cast the engine itself applies

when hosting a control). External plugins do the equivalent with `app.getMap?.() ?? app.getMapboxMap?.()`. The cast is honest only for the shared surface; a mapbox-gl map has none of MapLibre's extensions:

- `addProtocol` / `removeProtocol` — `pmtiles://`, COG and other custom tile protocols do not load, which is why the COG raster control stays MapLibre-only (Overture Maps instead asks `maplibre-gl-overture-maps` for plain `.pmtiles` URLs, which mapbox-gl 3.30+ reads through its own tile provider). `setTransformRequest` — Mapbox takes `transformRequest` only at construction, so GeoLens private rasters (whose API key is injected per request) are MapLibre-only. Custom `CustomLayerInterface` layers (`capabilities.customLayers` is false on Mapbox). The terrain camera helpers (`calculateCameraOptionsFromCameraLngLatAltRotation`, `getCenterClampedToGround`): Mapbox kept the free camera those replaced, so the Flight Simulator flies it there through `setFreeCameraOptions` and a `MercatorCoordinate` carrying the altitude. The `transform` / `_camera` internals some upstream controls read.
- `getProjection()` differs in shape: `{ type: "globe" }` on MapLibre, `{ name: "globe" }` on Mapbox; `setProjection` takes `{ type }` on MapLibre and a name string (or `{ name }`) on Mapbox. Read both.
- MapLibre's `Popup` and `Marker` classes imported from `maplibre-gl` do not work on a mapbox-gl map: their update path reads `map._camera.transform` and throws on the first move. A plugin that needs markers on both engines positions a DOM element through `map.project` instead, as the Elements panel does. When an upstream library insists on constructing the engine's own classes, `app.getMapboxGl()` hands out the mapbox-gl namespace: the Geo Editor feeds its `Marker` / `LngLatBounds` to Geoman's map adapter and its `Popup` to `maplibre-gl-geo-editor`'s `createPopup` option (`geo-editor-mapbox.ts`), Street View feeds its `Marker` to `maplibre-gl-streetview`'s `createMarker` option, Layer Swipe feeds its `Map` to `maplibre-gl-swipe`'s `createMap` option, which builds the clipped comparison pane, and GeoAgent hands `maplibre-gl-geoagent`'s `mapEngine` option the whole namespace (`geoagent-map-engine.ts`) — not a narrowed subset, because its `run_maplibre_script` tool passes it straight to the script it runs. The pattern upstream is the same each time: the library keeps the element and its styling and takes only the engine class that positions it. A plugin that constructs a *second* Mapbox map must also pass `app.getMapboxAccessToken()` in its constructor options: mapbox-gl reads its token from the global `mapboxgl.accessToken` unless handed one, and GeoLibre sets it per map, so a second map built without it renders nothing and logs every frame.

The same frontend audit that scans Cesium-capable plugins scans every plugin declaring Mapbox support, follows its relative imports, and fails on a read through `app.getMap()` that does not fall back to `getMapboxMap` on the same line (mark a deliberate MapLibre-detection branch with `engine-audit-allow: getMap-mapbox`) and on any of the MapLibre-only members above (`engine-audit-allow: maplibre-only` for a call behind a runtime engine check). Both opt-outs are scoped to the call, on its line or just above it.

Store layers work the same way on both 2D engines. A raster or WMS layer a control created natively and mirrored into the store as an external native layer (`externalNativeLayer: true` with `metadata.sourceId` and `nativeLayerIds`, as the Web Services, basemap, Esri Wayback and USGS LiDAR index controls do) is adopted by the Mapbox engine under those native ids — it is not drawn

twice, store visibility/opacity/removal apply to the control's layer, and a style reload rebuilds it, exactly as MapLibre's layer-sync does. A store layer the engine cannot compile from its `source` — a `registerExternalNativeLayer` record with no drawable source, or a kind whose pixels only the plugin can produce (`time-slider`, `timelapse`, `openaerialmap-footprints`) — counts as plugin-owned (`isMapboxPluginLayer` in `packages/map/src/mapbox-layers.ts`): the engine leaves drawing to the plugin and only mirrors the store's visibility and opacity onto the plugin's `nativeLayerIds` through the shared paint builders, honouring `metadata.controlOwnsPaint`.

What a MapLibre control gets on the globe

`app.addMapControl` works under Cesium too. The globe mounts the control's DOM in the same four `.maplibregl-ctrl-{top,bottom}-{left,right}` corner containers over its canvas — so the scoped CSS in `index.css` keeps applying — and hands `onAdd` a MapLibre-shaped facade over the Cesium scene rather than a real `Map`. The facade answers:

- `getContainer`, `getCanvas`, `isStyleLoaded`, and the `Evented` methods (`on` / `off` / `once` / `fire`). `getContainer` returns the sized canvas parent so controls can anchor their panels. Camera `movestart`, `move`, and `moveend`, canvas `resize`, and geographic mouse events reach subscriptions; pointer events over space are omitted because they have no ground location. The host removes these subscriptions when the globe is destroyed.
- `getCenter`, `getZoom`, `getBearing`, `getPitch` from the store's map view, and `jumpTo` / `flyTo` / `easeTo` by writing it back.
- `project`, `unproject`, and `getBounds` from the live scene: a coordinate is projected on the terrain surface, a screen point is picked against terrain then the ellipsoid, and the bounds come from the camera's view rectangle. A coordinate the scene cannot place reads as off-screen window coordinates, a screen point that misses the globe unprojects to the view centre, and a camera with no bounded view rectangle reports the whole world — the same shapes MapLibre's globe projection answers with, so a control keeps running instead of throwing mid-render.
- `setStyle(url)`, routed to the project basemap.

Everything that paints through the Mapbox Style Spec, including `addSource`, `removeSource`, `addLayer`, `removeLayer`, `setPaintProperty`, `setLayoutProperty`, `getStyle` — **throws**. That is the honest boundary: a control that draws its own map layers has no globe representation, and a silent no-op would leave it reporting success while nothing appears. `addMapControl` catches the throw and returns `false`, so a control that trips it fails to mount rather than taking plugin activation down with it. A plugin whose control needs those methods should keep the default `engines`: `["maplibre"]` and, if the globe matters, add a Cesium branch through `app.getCesiumScene()`.

<https://geolibre.app/collaboration/>

Real-time collaboration (live-synced sessions)

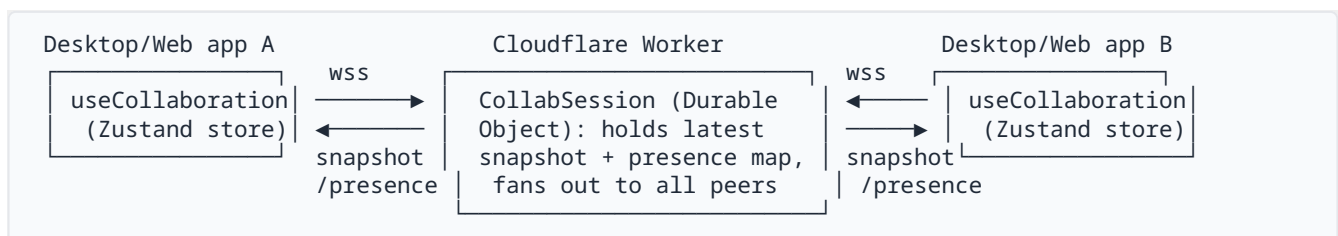
Status: **experimental MVP** (issue [#307](#)). Disabled unless `VITE_GEOLIBRE_COLLAB_URL` is configured.

GeoLibre's project sharing is otherwise snapshot-based (upload to share.geolibre.app). This feature adds a **live** mode: several people open the same session and see each other's layer/style/view edits in real time, with presence cursors and viewport indicators. It targets classrooms, workshops, and small teams.

What syncs

- **Project state** — layers, layer groups, styles, basemap, and the map view (camera). Broadcast as whole-project snapshots.
- **Presence** — each participant's live cursor position and viewport rectangle, plus a name + color. Presence is ephemeral and never persisted.
- **Chat** — short text messages with an optional attached map coordinate ([#754](#)). Bounded recent history is relayed to late joiners; never written to a project.
- **Permissions** — the host's per-participant view-only / can-edit overrides ([#754](#)), broadcast on the participant list as `editOverride`.
- **Comments** — map comment add / reply / resolve / delete actions, relayed as `comment-mutation` frames and applied to each peer's store.

Architecture



There is **one centralized relay** (a Cloudflare Durable Object), not a P2P mesh. The DO holds the latest project snapshot so a late joiner is bootstrapped immediately, and fans every message out to the other connected sockets.

Why a Durable Object relay (and not CRDT/WebRTC)

The MVP deliberately picks the simplest thing that works:

- The store is already the single source of truth, and `serializeProject / parseProject` already produce a validated, normalized wire format. `useEmbedBridge` already broadcasts exactly this over `postMessage`. The collaboration adapter is that same pattern over a WebSocket.
- A **whole-snapshot, last-write-wins** model is trivially consistent: the last snapshot the relay sees wins, full stop. Mutation-level merging would need per-field clocks; a CRDT (Yjs/Automerger) would add a sizeable client bundle and a second source of truth alongside Zustand.
- The relay builds directly on the existing `workers/viewer` Cloudflare setup.

CRDT / per-action mutation transport is the documented **v2** path (see Limitations).

Sync protocol

All frames are JSON, and each direction is a discriminated union on `type` (`ClientMessage` and `ServerMessage`). The relays' types live in `packages/collab-core/src/protocol.ts` (`@geolibre/collab-core/protocol`, with `project` typed as `unknown`); the frontend keeps a parallel copy in `apps/geolibre-desktop/src/lib/collab-protocol.ts` with `project` typed as `GeoLibreProject`. Keep the `type` discriminants and field names in sync.

Client → server:

type	payload	notes
join	<code>displayName</code> , <code>color</code> , <code>hostToken?</code> , <code>inviteToken?</code> , <code>identityToken?</code>	first frame after connect; the relay assigns the <code>clientId</code> (returned in <code>welcome</code>)
snapshot	<code>project</code> , <code>rev</code>	a debounced project push; co-editors only
presence	<code>cursor?</code> , <code>view?</code>	throttled cursor / viewport
set-mode	<code>mode</code>	host only
set-participant-mode	<code>clientId</code> , <code>canEdit</code>	host only; pin one guest to can-edit / view-only (#754; persisted per participant key, so it survives a reconnect only for identity/invite joins — see <i>Moderation</i>)
mint-invite	<code>role</code> , <code>maxUses?</code>	host only; mint a co-edit or view-only invite link
revoke-invite	<code>token</code>	host only; invalidate an active invite link
set-session-config	<code>requireIdentity</code>	host only; mandate signed-in identity to join

type	payload	notes
set-layer-locks	lockedLayerIds	host only; mark specific layer IDs read-only
kick-participant	clientId, reason?	host only; disconnect a participant
block-participant	clientId, reason?	host only; disconnect and bar that participant key from rejoining (see the caveat under <i>Moderation</i>)
chat	text, coordinate?	a chat message, with an optional attached map coordinate (#754)
comment-mutation	action	add / reply / toggle-resolve / delete a map comment; validated and rate-limited by the relay

Server → client:

type	payload	notes
welcome	clientId, role, mode, participants[], snapshot \ null, presence, chat[], rev, requireIdentity, identitySupported, lockedLayerIds, invites[]	sent once on join; the late-joiner bootstrap
snapshot	project, origin, rev	fan-out of a peer's snapshot
presence	clientId, cursor?, view?	fan-out of a peer's presence
participants	participants[]	on join / leave / role / permission change; each carries editOverride and identity
mode	mode	host changed the session mode
invite-created	invite	fan-out of a new invite link created by host
invite-revoked	token	fan-out when host revokes an invite link
session-config	requireIdentity	broadcast when session identity policy changes
layer-locks	lockedLayerIds	broadcast when host locks/unlocks layer IDs
kicked	reason	sent to a participant who was kicked or blocked by the host
chat	message	fan-out of a chat message (echoed to the sender, so order is server-authoritative) (#754)

type	payload	notes
comment - mutation	action	fan-out of a peer's comment change
error	code, message	forbidden, too-large, bad-message, not-found, identity-required, identity-unavailable, or layer-locked

Echo / feedback-loop prevention

The adapter caches `lastAppliedContent` (the serialized project string). Before applying an inbound snapshot it sets `lastAppliedContent` to the post-normalization string, then applies via `loadProject`. The store subscription that `loadProject` triggers re-serializes to an identical string and is suppressed, so a remote apply is never re-broadcast — the exact trick `useEmbedBridge` uses with `lastPostedContent`. Frames whose `origin` is our own `clientId` are also ignored defensively (the relay already excludes the sender).

Undo interaction

Remote snapshots are applied through `loadProject`, which ends with `clearHistory()`. This keeps remote edits out of the local undo stack — but it also means **a collaborator's edit clears your undo history**. That is an accepted MVP limitation; a coalesced-history option is a v2 item.

Durable Object (workers/collab)

- `POST /sessions` — host creates a session: generates a short base32 code, mints a host token, stores `{ mode, hostToken, requireIdentity }`, returns `{ sessionId, hostToken, mode, requireIdentity }` to the host only. A disallowed origin gets `403` and a rate-limited caller `429` (see the operator note under *Chat*).
- `GET /sessions/:id/ws` — WebSocket upgrade, routed to `env.COLLAB_SESSION.get(idFromName(id))`.
- `GET /sessions/:id/log` / `DELETE /sessions/:id/log` — host-only download or clear of the session log (`Authorization: Bearer <hostToken>`).
- `GET /health` (also `/`) — `{ ok, service, identitySupported }`.

`CollabSession` uses the **WebSocket Hibernation API** so idle sessions evict from memory while keeping sockets open. Per-socket participant metadata is kept via `ws.serializeAttachment()` (survives hibernation). Durable storage holds the `latestSnapshot`, a monotonic `rev`, the `mode`, the `hostToken`, and a bounded `chat` log; presence and per-participant permission overrides are kept on the in-memory / per-socket attachment. Server-side enforcement: a `snapshot` from a guest who cannot edit (session `view-only`, or a host-set per-participant view-only override) is dropped with an `error: forbidden`; `set-mode` and `set-participant-mode` require the host token. Oversized snapshots (> 10 MB by default) are rejected with `error: too-large`; the cap sits under Cloudflare's

32 MiB ceiling on a received WebSocket message. Hosted snapshots are stored in chunks across rows of a SQLite table, because a Durable Object caps both a key/value entry and a single SQLite string at 2 MB — well under what portable GeoJSON from local files and external plugins needs. An empty session is reclaimed after a TTL via a storage alarm, which drops the chunks with the rest of the database.

Frontend

- `lib/collab-protocol.ts` — the frontend copy of the message types.
- `lib/collab-client.ts` — WebSocket transport, `resolveCollabBaseUrl()` (wss/loopback validation, returns `null` when unset), exponential-backoff reconnect.
- `hooks/useCollaboration.ts` — orchestration: subscribes to the store (debounced, deduped snapshot push for co-editors), reads `map mousemove` (throttled) and `moveend` for presence, routes inbound frames, and exposes start/join/leave/set-mode actions. Inert no-op when `resolveCollabBaseUrl()` is `null`.
- `lib/build-project-snapshot.ts` — the shared `buildProjectSnapshot()` lifted from `useEmbedBridge` so the bridge and the adapter share one definition.
- Store: an ephemeral `collaboration slice (packages/core)`, excluded from the project file (never read by `projectFromStore`) and from undo history (never added to `partialize`).
- `components/layout/RemoteCursorsOverlay.tsx` — renders remote cursors as MapLibre Markers and viewport rectangles as a dedicated GeoJSON line layer.
- `components/layout/CollaborateDialog.tsx` + a flag-gated `TopToolbar` entry.
- `components/layout/CollaborationStatusBadge.tsx` — the persistent on-canvas badge that hosts the participant roster (with the host's per-participant permission toggles) and the chat drawer (#754). `useCollaboration` is owned by `DesktopShell` and passed to both the dialog and this badge, so they share one socket. `participantCanEdit()` (in `lib/collab-protocol.ts`) is the shared effective-permission helper used by the relay-mirrored client UI.

Identity & permissions (MVP)

Anonymous. The host starts a session and shares a code/link; joiners pick a display name and a color. The host chooses the session **mode**:

- **view-only** — guests can watch and see presence, but their snapshot pushes are rejected server-side.
- **co-edit** — anyone with the link can edit.

Per-participant overrides (#754). Beyond the session-wide mode, the host can pin an individual guest with `set-participant-mode { clientId, canEdit }`. The relay records the override on that socket's attachment (so it survives a hibernation wake) and re-broadcasts the participant list with an `editOverride` field (`true` / `false`, or `null` to follow the session mode). Effective edit permission, computed identically on the client and the relay, is: the host always edits; otherwise the override wins; otherwise the session mode applies. A guest pinned to view-only has their `snapshot` pushes

rejected with `error: forbidden`, exactly like the session-wide view-only path. An override is also persisted against the guest's **participant key**, so it is reapplied when they rejoin — but only for identity- and invite-based joins, for the same reason a block is (see *Moderation* below): an anonymous guest arrives with a fresh `clientId` and therefore a fresh key, so their override reverts to the session default on reconnect. The host roster surfaces a per-guest toggle; other participants see each guest's current permission read-only.

The host token (returned only to the creator) gates `set-mode` and `set-participant-mode`, so a guest can't escalate the session or another guest. Codes are unguessable and sessions auto-expire. The relay assigns each participant's `clientId` server-side (the client-supplied value is ignored) so one participant can't claim another's identity, and it validates the `color` to a hex value before storing/broadcasting it.

Signed-in identity (optional)

A joiner may present an `identityToken`. It is **not** self-reported JSON: the relay verifies an HMAC-SHA256 signature over the token's payload segment before it will populate `participant.identity`, so a client cannot mint an account, impersonate another user, or wear the roster's verified badge. The format is

```
<base64url(payloadJSON)>.<base64url(hmacSha256(base64url(payloadJSON)))>
```

with payload claims `{ userId, username, provider?, exp? }`. `verifyIdentityToken` in `@geolibre/collab-core` is the single implementation both relays call, and `signIdentityToken` alongside it mints one.

Identity is **off unless a deployment opts in** by setting the signing secret shared with its issuer — `COLLAB_IDENTITY_SECRET` (a Worker secret for `workers/collab`, an environment variable for `workers/collab-node`). With no secret configured:

- every `identityToken` verifies to null, so all joiners are anonymous;
- `requireIdentity` cannot be turned on — `POST /sessions` answers `400` and `set-session-config` answers `error: identity-unavailable`, rather than arming a gate that would reject every guest including the host's own collaborators;
- `GET /health` and every `welcome` report `identitySupported: false`, which is how the Collaborate dialog decides to hide the "require a signed-in account" option entirely.

GeoLibre itself ships no sign-in flow yet, so the stock deployment leaves the secret unset and the whole identity path dormant. The protocol and relay support are in place for a deployment that fronts the relay with its own issuer.

Moderation

`kick-participant` disconnects a participant. `block-participant` also records their **participant key** so the relay refuses the next join. That key is `user:<userId>` for a verified identity, `invite:<token>` for an invite-based join, and otherwise `anon:<clientId>` — where `clientId` is minted fresh on every join.

The same key backs a persisted per-participant override, so both carry the same caveat: a block is durable only for identity- and invite-based joins. **An anonymous guest can rejoin simply by reconnecting**, since they arrive with a new `clientId` and therefore a new key. Blocking is a moderation convenience, not a security boundary; a session that needs an enforceable ban has to require an invite or a signed-in identity.

Chat (#754)

A lightweight, in-session text channel. A `chat { text, coordinate? }` frame is validated server-side (non-empty, trimmed, capped at 2000 chars; the coordinate runs through the same finite-number sanitizer as presence) and fanned out to **every** participant including the sender, so the relay's ordering is the single source of truth (clients render from the echo rather than optimistically). Each message carries a server-assigned `id` and `ts`. The relay keeps a bounded, persisted history (last 50 messages) so a late joiner (or a post-hibernation welcome) sees the recent conversation via the `welcome.chat[]` bootstrap. Chat is open to everyone in the session, including view-only guests; only project edits are gated. A message can attach the sender's current map center as a clickable coordinate that recenters the recipient's map. Chat lives on the on-canvas status badge so it is reachable while working on the map; it is ephemeral and never written to a project file.

Operator note: `POST /sessions` validates the request `Origin` (or `Referer`) against `ALLOWED_ORIGINS` via `isAllowedOrigin` (defaults to the hosted origins (`geolibre.app`, `web.geolibre.app`, its legacy `viewer.geolibre.app` alias, `studio.geolibre.app`, and `collab.geolibre.app`), single-label HTTPS deployment hosts under `*.geolibre-preview.pages.dev`, loopback hosts (`localhost`, `*.localhost`, and `127.0.0.1`), and `tauri://localhost`). Nested or custom-port preview hosts and look-alike domains are rejected; the shared `opengeos.org` GitHub Pages preview origin is deliberately not trusted) as browser-origin filtering and defense-in-depth (not authentication or a general server-side access gate) and enforces a per-IP `checkRateLimit` (10 requests / 60 s). `Access-Control-Allow-Origin: *` is still sent on responses so non-browser clients (e.g. Tauri) are not blocked by CORS; non-browser clients may omit these headers and remain supported, unless a verifiable credential requirement is added. Configure `ALLOWED_ORIGINS` (comma-separated list) to restrict which browser origins may create sessions in production.

Feature flag

Set `VITE_GEOLIBRE_COLLAB_URL` to the relay base (e.g. `wss://collab.geolibre.app`, or `ws://127.0.0.1:8787` for `wrangler dev`). When unset, the hook is inert and all collaboration UI is hidden, so production builds ship the feature dark. The Tauri CSP `connect-src` must list the wss host (the existing `https:` directive does **not** authorize `wss:`).

In the Docker image the same setting is available at **container runtime** as `-e GEOLIBRE_COLLAB_URL=...`: the entrypoint validates it, writes it into `geolibre-runtime-config.js`, and `resolveCollabBaseUrl()` prefers that over the build-time variable — so a prebuilt image can be pointed at a self-hosted relay without a rebuild. A value that is not `wss://` (or `ws://` on loopback) fails the container boot rather than silently leaving collaboration dark. The entrypoint also substitutes the relay's origin into the nginx CSP's `connect-src` (`__GEOLIBRE_COLLAB_CONNECT_SRC__` in `docker/nginx.conf`), since that directive has no bare `wss:` — so unlike the desktop build below, the web/Docker path needs no manual CSP edit. See [Run with Docker](#).

Self-hosting note: the desktop CSP pins `wss://collab.geolibre.app` (plus `ws://localhost/127.0.0.1` for dev). Pointing the desktop build at a different relay means updating `connect-src` in `apps/geolibre-desktop/src-tauri/tauri.conf.json` and rebuilding — the CSP and the `VITE_GEOLIBRE_COLLAB_URL` flag are independent knobs. The web build inherits the page's CSP instead, so it only needs the env var.

Deploying the relay (`collab.geolibre.app`)

Two hosts implement the same protocol and import the same permission/validation core:

- `workers/collab` is the Cloudflare Durable Object used by the hosted service.
- `workers/collab-node` is the self-hostable Node/SQLite relay. It is included as `geolibre-collab` in the root `docker-compose.yml`; its persistent database is mounted at `/data/collab.sqlite`. Configure `COLLAB_MAX_SNAPSHOT_BYTES` (relay default 10,000,000; the compose file defaults it to 1,000,000) and `COLLAB_IDLE_TTL_MS` (default two hours) when needed.

Both relay implementations accept `COLLAB_MAX_SNAPSHOT_BYTES` as a positive integer number of bytes. For the Cloudflare relay, set it as a Worker variable, for example:

```
[vars]
COLLAB_MAX_SNAPSHOT_BYTES = "10000000"
```

The configured value must remain below the deployment platform's WebSocket message-size limit.

The hosted relay deploys to Cloudflare Workers the same way as `workers/viewer`:

- **CI:** `.github/workflows/deploy-collab.yml` deploys on any push to `main` that touches `workers/collab/**` (or via manual `workflow_dispatch`). It reuses the existing `CLOUDFLARE_API_TOKEN` / `CLOUDFLARE_ACCOUNT_ID` repo secrets — the token needs the **Workers Scripts Write** permission (Cloudflare's "Edit Cloudflare Workers" template includes it). Deploying the Durable Object is part of the same script upload, so no separate Durable Objects permission is needed.
- **Manual:** `cd workers/collab && npx wrangler deploy`.

`wrangler.toml` already declares the `collab.geolibre.app` custom-domain route and the SQLite Durable Object migration, so the first deploy provisions DNS, TLS, and the DO class automatically — no manual Cloudflare dashboard steps. SQLite-backed Durable Objects are available on the free Workers plan.

Once the relay is live, point the app at it by setting `VITE_GEOLIBRE_COLLAB_URL=wss://collab.geolibre.app` in the web/Pages build environment. Until that env var is set, the feature stays dark.

Limitations / v2

- **Last-write-wins:** simultaneous co-edits race; the last debounced snapshot wins and the slower edit is overwritten. Presence helps users avoid colliding.
- **Payload size:** layers can embed `FeatureCollection`s. `projectFromStore` already strips redundant `geojson` for URL-backed layers, but a large in-memory/local-file layer can exceed the snapshot cap (`COLLAB_MAX_SNAPSHOT_BYTES`: 10 MB by default for the relay, 1 MB in the Docker Compose setup) and is rejected with `error: too-large` (share via URL instead). v2: diff / chunked layer sync.
- **Undo:** a remote apply clears local undo (see above).
- v2 directions: per-action mutation or CRDT transport, coalesced remote-apply history, richer permission/identity (tie to `share.geolibre.app` accounts).

Testing

Automated:

- `npm run test:worker` typechecks both relays and runs the Node relay's socket integration suite.
- `npm run test:frontend` runs `tests/collab-protocol.test.ts` (protocol round-trip including the `set-participant-mode` / `chat` frames, `resolveCollabBaseUrl` validation, echo-suppression logic, and the `participantCanEdit` effective-permission helper), plus the shared relay conformance suite.

Testing the full feature locally

Collaboration is dark until `VITE_GEOLIBRE_COLLAB_URL` points at a running relay, so local testing has two parts: run the relay, then run the app against it.

1. **Start a relay** in one terminal. For the Cloudflare implementation:

```
cd workers/collab && npx wrangler dev --port 8787 --local
# → Ready on http://localhost:8787
```

Or run the self-hostable Node implementation:

```
npm run build -w geolibre-collab-node
npm start -w geolibre-collab-node
```

1. **Start the app pointing at that relay** in another terminal:

```
VITE_GEOLIBRE_COLLAB_URL=ws://127.0.0.1:8787 npm run dev
# → http://localhost:5173
```

Or put `VITE_GEOLIBRE_COLLAB_URL=ws://127.0.0.1:8787` in `apps/geolibre-desktop/.env.local` so you don't repeat it. With the variable unset the Collaborate menu item stays hidden — that is the feature flag working. (For the desktop shell use `npm run tauri:dev` with the same variable; the Tauri CSP already allows `ws://127.0.0.1:*` / `ws://localhost:*`.)

1. **Open two independent windows** at `http://localhost:5173` — a normal window plus an incognito window works well so they don't share state.
2. **Drive a session:**
3. Window A: **Project → Collaborate...**, enter a name, pick a color, **Start session** (choose *Anyone can edit*). Copy the session code or the share link.
4. Window B: open the share link directly (the Collaborate dialog auto-opens with the code prefilled — just enter a name and **Join**), or open **Project → Collaborate...** and paste the code.
5. Verify: B immediately sees A's existing layers; adding/removing a layer, changing a style, or panning in A reflects in B within ~300 ms; each window shows the other's live **cursor** and a dashed **viewport rectangle**; toggling A (the host) to *view-only* blocks B's edits.
6. Verify Parts 3 & 4 (#754) via the on-canvas status badge (bottom-left, click to expand): A (the host) can flip B between **Can edit** and **View-only** per-participant in the roster, and either window can send a **chat** message (optionally attaching the current map center, which the other side can click to recenter).

Relay-only smoke test (no UI): with `wrangler dev` running, POST to `http://127.0.0.1:8787/sessions` to mint a code, then open a WebSocket to `ws://127.0.0.1:8787/sessions/<code>/ws` and exchange `join` / `snapshot` / `presence` frames — the quickest way to confirm the relay independent of the front end.

<https://geolibre.app/ui-profiles/>

UI Profiles & Data Source Filtering

GeoLibre can hide data sources, web services, and plugins to simplify the interface for beginners or to standardize a deployment across a team. Hiding is **non-destructive** — nothing is removed, and any item can be re-enabled at any time. Profile preferences are stored locally in the browser/app and never travel inside a saved `.geolibre.json` project.

Looking to lock a deployment down?

Profiles declutter; the user can undo them. To pin what a deployment is *permitted* to do — a kiosk, a classroom instance — see [Deployment Capabilities](#), which is never surfaced in the UI. The two are independent and can be combined.

For users

Onboarding

By default GeoLibre starts on the **Advanced** interface — everything visible — and does not show a first-launch welcome dialog. Choose a simpler experience level at any time from **Settings → Interface** (see below):

- **Beginner** — only the essential data sources and tools.
- **Intermediate** — common data sources, services, and plugins.
- **Advanced** — everything GeoLibre offers (the default).

An administrator can pre-configure the interface in the primary `deployment.json` policy, or use the legacy `admin-profile.json` fallback described below.

Settings → Interface

The interface is controlled by a single four-state selector:

- **Beginner / Intermediate / Advanced** are developer-curated presets. Selecting one applies its layout immediately. **Advanced** reveals everything, so it is the full, unrestricted interface.
- **Custom** activates automatically the moment you hand-edit any item below; it is a status, not a button you click.

Open **Settings → Interface** to:

- Pick an **experience level**, which fills the checklists from each item's complexity and takes effect at once.

- Check or uncheck individual **data sources**, **plugins**, whole **menus** (Project, Edit, View, Add Data, Processing, Controls, Plugins, Help), and the items within the Project, Edit, View, Processing, Controls, Settings, and Help menus. Editing any item switches the selector to **Custom**.

The **Settings** menu itself, and its Language / Layout / Interface entries, are always shown so the profile UI can never be hidden away.

For administrators

Use the `interface` section in the primary deployment policy. For example:

```
{
  "version": 1,
  "interface": {
    "enabled": true,
    "level": "intermediate",
    "lock": true,
    "hiddenDataSources": ["postgres", "video"],
    "hiddenPlugins": ["maplibre-gl-geoagent"]
  }
}
```

The `interface` field table below describes this section. Provision it at `<base>/deployment.json` for `web/embed`, the `desktop config directory`, or as a `Docker source policy`.

Legacy: `admin-profile.json` — still honoured

The legacy file remains a fallback when the selected deployment policy has no non-empty, valid `interface` section. Its unwrapped format is:

```
{
  "enabled": true,
  "level": "intermediate",
  "lock": true,
  "hiddenDataSources": ["postgres", "video"],
  "hiddenPlugins": ["maplibre-gl-geoagent"]
}
```

For `web/embed`, serve it from the application root. On desktop, the original file remains at `<app_config_dir>/admin-profile.json`; see the standard [Tauri config paths](#). A present desktop file is authoritative even if malformed, and does not fall back to the bundled copy. Only an absent file or a failed read/command selects the web copy.

Interface fields

The primary `interface` object and the legacy `admin-profile.json` file share these profile fields; the legacy file stores them at top level.

Field	Type	Meaning
<code>enabled</code>	boolean	Whether filtering is active. Defaults to <code>true</code> .
<code>level</code>	<code>"beginner" \ "intermediate" \ "advanced"</code>	Seeds the hidden lists from each item's tier. Optional.
<code>lock</code>	boolean	When <code>true</code> , users cannot change the profile from Settings. To release the lock on the next launch, remove or clear <code>interface.lock</code> in the selected deployment policy, or the top-level <code>lock</code> in the legacy profile.
<code>hiddenDataSources</code>	string[]	Explicit data-source ids to hide. Overrides the preset when present.
<code>hiddenPlugins</code>	string[]	Explicit plugin ids to hide. Overrides the preset when present.
<code>hiddenMenus</code>	string[]	Top-level menu ids to hide (<code>project</code> , <code>edit</code> , <code>view</code> , <code>addData</code> , <code>processing</code> , <code>controls</code> , <code>plugins</code> , <code>help</code>).
<code>hiddenMenuItems</code>	string[]	Menu-item ids to hide (e.g. <code>processing.raster</code> , <code>help.diagnostics</code> , <code>controls.minimap</code>).

Data-source ids are the catalog ids in `apps/geolibre-desktop/src/lib/ui-profile.ts` (e.g. `vector`, `xyz`, `mbtiles`, `postgres`). Plugin ids are the stable ids defined in `packages/plugins/src/plugins/*` and `packages/plugins/src/plugin-ids.ts` (e.g. `maplibre-gl-geoagent`). Menu and menu-item ids are the catalog ids in the same `ui-profile.ts` (`TOP_LEVEL_MENUS`, `MENU_ITEM_CATALOG`).

When a `level` preset is active, external/bundled drop-in plugins (which load asynchronously after startup) are folded into the hidden set as they appear, so a beginner profile keeps hiding advanced plugins even when they load late.

<https://geolibre.app/deployment-capabilities/>

Deployment Capabilities

A deployment can pin what the app it serves is allowed to do — a read-only kiosk, a classroom instance, and everything between that and the full app.

Capabilities are coarse on purpose. Each one names a whole class of action ("may add data at all"), not an individual menu item, so a locked-down deployment cannot be defeated by one item somebody forgot to list.

Client gates are not a general authorization boundary

Withholding a capability removes the affordance: the menu is not rendered, the command palette does not list or run the action, the keyboard shortcut does nothing, and the embed API refuses the command. These client gates do **not** stop someone with browser devtools.

The Docker container also enforces its final deployment policy at nginx: selected `/sidecar/` route families require `processing:run` or `data:add`, and `/ai` is disabled unless the final policy enables AI and the approved proxy environment is configured. See the exact route table and startup rules in [Self-Hosting](#). This enforcement is **container-only**: it does not restrict browser WASM engines, desktop processing, or a separately hosted sidecar.

Keep Basic Auth or a real auth proxy and `GEOLIBRE_CONVERSION_ROOTS` in place. Container policy enforcement is not user authentication, and capabilities remain the interface half of the story outside those routes.

Not the same as UI Profiles

[UI Profiles](#) also hide menus and items, and the two run independently. The difference is who decides:

	UI Profiles	Deployment capabilities
Purpose	Reduce clutter for the audience	Pin what the deployment permits
Set by	The user, primary policy <code>interface</code> , or legacy admin profile	Primary runtime policy; legacy runtime/build environment fallbacks
Reversible in the app	Yes, from Settings → Interface (unless <code>lock</code> is set)	No — never surfaced in the UI
Granularity	Individual items, data sources, plugins	Whole capabilities

Where both apply, the capability is checked first. An action the deployment withheld is never on offer, whatever the profile says.

The capabilities

Capability	Grants
<code>project:edit</code>	Authoring the project: New, Open, Open Recent, Import, Project History, Save, Save As, Duplicate, Save as Template, Collaborate, StoryMap; Undo/Redo (the menu items and the Ctrl/Cmd+Z, Ctrl/Cmd+Shift+Z, and Ctrl+Y shortcuts); Export Selection; adding a review comment; the embed API's <code>loadProject</code> .
<code>data:add</code>	Bringing data in: the whole Add Data menu, dragging a file onto the map (browser and desktop), and the embed API's <code>addLayer</code> and <code>addData</code> .
<code>processing:run</code>	The whole Processing menu — Whitebox, SQL, Python, the AI assistant, geocoding, Model Builder, conversion/vector/raster tools — and the embed API's <code>openTool</code> .
<code>export:data</code>	Getting data or a rendering back out: Share, Export HTML, Print, Print Layout, Offline Basemap, and the embed API's <code>exportImage</code> .
<code>plugins:install</code>	The Plugins menu, plugin-registered toolbar menus, plugin items in the built-in menus, activating or deactivating a plugin, and the plugin marketplace ("Manage plugins").
<code>settings:manage</code>	The Settings dialog and the Style Manager.

Anything not listed is unprivileged and stays available in every configuration: panning and zooming, the View and Controls menus, layer visibility and ordering, identify, the selection tools, and Help.

Configuring it

Runtime (recommended)

Set `capabilities` in `deployment.json`, or use the Docker runtime override:

```
docker run --rm -p 8080:80 \
  -e GEOLIBRE_CAPABILITIES=data:add,export:data \
  ghcr.io/opengeos/geolibre:latest
```

Runtime policy support ships in the published image from v3.3.0 on; see [Deployment Policy](#). `none` grants nothing with the environment variable; JSON uses `"capabilities": []` for the same empty grant. Omitted sources grant the default full set.

Legacy input — `VITE_GEOLIBRE_CAPABILITIES` still honoured

`VITE_GEOLIBRE_CAPABILITIES` is a legacy build-time fallback:

```
VITE_GEOLIBRE_CAPABILITIES="data:add,processing:run,export:data" npm run build
```

On a prebuilt client, runtime deployment policy takes precedence, followed by `window.__GEOLIBRE_DEPLOYMENT_ENV__`, then this build-time value. The legacy parser drops unknown tokens; an unrecognized-only nonblank value grants none, and blank means unset. These rules do not describe JSON policy parsing or Docker's strict source validation.

```
VITE_GEOLIBRE_CAPABILITIES=none npm run build
VITE_GEOLIBRE_CAPABILITIES="export:data" npm run build
VITE_GEOLIBRE_CAPABILITIES="project:edit,data:add,processing:run,export:data" npm run build
```

The examples above are legacy build-time configuration, still honoured; they do not configure Docker nginx route enforcement.

Client policy loading can fall back

Web loading waits at most 3 seconds. A missing or late policy uses the runtime environment fallback, then the legacy build environment, then the default full grant. Client gates and browser WASM are not access control; use Docker nginx policy enforcement or protect APIs independently.

Runtime examples

A kiosk or exhibit terminal, with no optional capabilities:

```
docker run --rm -p 8080:80 -e GEOLIBRE_CAPABILITIES=none ghcr.io/opengeos/geolibre:latest
```

Allow visitors to export an image:

```
docker run --rm -p 8080:80 -e GEOLIBRE_CAPABILITIES=export:data ghcr.io/opengeos/geolibre:latest
```

A classroom instance with project authoring, data and processing tools, and export, but no plugin installs or settings:

```
docker run --rm -p 8080:80 \
  -e GEOLIBRE_CAPABILITIES=project:edit,data:add,processing:run,export:data \
  ghcr.io/opengeos/geolibre:latest
```

An embedded map can use an empty grant to refuse commands such as `loadProject`, `addLayer`, `addData`, `openTool`, and `exportImage`, while `setView`, `highlightFeature`, and `layer-visibility` commands remain available.

Embed API behavior

A denied command rejects rather than silently doing nothing, so the host page can tell the difference between "refused" and "no effect":

```
await map.addData("https://example.com/data.geojson");  
// Error: Missing data:add capability
```

See [Embedding & Sharing](#) for the full command list. The embed origin allowlist (`GEOLIBRE_EMBED_ORIGINS`) and capabilities are independent: the allowlist decides *who* may send commands, capabilities decide *which* commands exist.

Related pages

- [Deployment Policy](#) — primary policy and Docker enforcement
- [Self-Hosting](#) — exact Docker server-side route guards
- [UI Profiles](#) — non-destructive interface filtering
- [Embedding & Sharing](#) — embed API and origin allowlist
- [Getting Started](#) — container configuration

<https://geolibre.app/deployment-policy/>

Deployment Policy

`deployment.json` is a single, versioned file that describes the client-facing settings of a GeoLibre deployment: which capabilities users have, which interface elements are visible, which plugins may load, the curated service library, sharing endpoints, and branding.

Loading

The web and Jupyter builds fetch `deployment.json` from the app's base URL (`<base>/deployment.json`) before the first render. Desktop reads its `config-dir` file first (see [Desktop](#)); only an absent file or a failed read falls back to that web URL. The selected policy is resolved before the first render. No policy applies when the selected file is absent (404 or an HTML fallback page), not JSON, or of an unknown `version`. An unreachable web file or a fetch that takes more than 3 seconds also yields no policy. In those cases the app uses the next client configuration source.

Every build ships a `deployment.json` stub containing `null`, which means no policy, so static hosts answer the startup fetch instead of logging a 404. To apply a policy on a static host, replace that file in the published output; on Docker the entrypoint overwrites it on every boot. On desktop the stub is the bundled web fallback, so a `config-dir` file still takes precedence.

The recommended deployment input is a versioned `deployment.json`. On Docker, the entrypoint validates the mounted source, applies nonblank environment overrides field by field, and writes the generated public policy on every boot (see [Docker](#)). The client then resolves settings in this order: final policy, `window.__GEOLIBRE_DEPLOYMENT_ENV__`, then build environment.

Availability: Runtime policy delivery, desktop `config-dir` loading, and Docker enforcement first shipped in v3.3.0. Older Docker images and desktop releases ignore `deployment.json`; use v3.3.0 or later (the `ghcr.io/opengeos/geolibre:latest` image is rebuilt from `main`).

The previous `admin-profile.json` and `VITE_GEOLIBRE_CAPABILITIES` inputs are **legacy, still honoured**. They remain fallbacks where the primary policy does not specify a usable section; this documentation does not remove them.

What each section does today:

- `capabilities` restricts the app; `[]` grants none, and omitting it leaves the legacy `VITE_GEOLIBRE_CAPABILITIES` (or the default full grant) in force.
- A non-empty `interface` object replaces the legacy `admin-profile.json` whole; fields are not merged. An omitted, empty (`{}`), or invalid `interface` leaves the legacy profile eligible.
- `plugins.registryUrl` sets the plugin registry. `allowed`, `blocked` and `sideload` gate external plugin loads and installs; `defaultActive` seeds activation in fresh projects.
- `services`, `sharing`, `geolens` and `branding.appName` override the matching `GEOLIBRE_*` deployment settings. `sharing.embedOrigins: []` turns the embed API off.

- `branding.welcome: false` suppresses the first-launch wizard.
- `ai.enabled: true` points the assistant at the same-origin `/ai` proxy; `false` removes any operator-configured AI proxy (a provider a user enters in Settings is unaffected). `ai.model` picks the proxy's model.

The client parser is lenient and section-based. Invalid sections, including capabilities with unknown names, are dropped with a warning; other valid sections survive. Capabilities then fall back to runtime or build environment, or the default full grant. Docker source validation is strict and fails boot before overrides can repair an invalid input file.

```
{ "version": 1, "capabilities": [ "data:add", "export:data" ], "interface":
  { "enabled": true, "level": "intermediate", "lock": true } }
```

For capability details and Docker route enforcement, see [Deployment Capabilities](#) and [Self-Hosting: container policy enforcement](#).

Example

```
{
  "version": 1,
  "capabilities": [ "project:edit", "data:add", "processing:run", "export:data", "plugins:install", "settings:manage" ],
  "interface": {
    "enabled": true,
    "level": "intermediate",
    "lock": true,
    "hiddenDataSources": [ "arcgis" ],
    "hiddenPlugins": [ "plugin-a" ],
    "hiddenMenus": [ "help" ],
    "hiddenMenuItems": [ "project.print" ]
  },
  "plugins": {
    "registryUrl": "https://plugins.example.com/registry.json",
    "allowed": [ "acme-tools" ],
    "blocked": [ "bad-plugin" ],
    "sideload": false,
    "defaultActive": [ "acme-tools" ]
  },
  "services": {
    "builtins": true,
    "catalog": [
      {
        "id": "city-wms",
        "name": "City WMS",
        "kind": "wms",
        "category": "Municipal",
        "fields": { "url": "https://maps.example.com/wms", "version": "1.3.0", "opacity": 0.8, "transparent": true }
      }
    ]
  },
  "sharing": {
    "shareUrl": "https://projects.example.com",
    "collabUrl": "wss://relay.example.com",
    "embedOrigins": [ "https://portal.example.com" ]
  },
  "geolens": { "url": "same-origin" },
  "ai": { "enabled": true, "model": "gpt-5-mini" },
}
```

```
"branding": { "appName": "Acme Maps", "welcome": false }
}
```

Point your editor at the schema for completion and validation by adding `"$schema": "https://raw.githubusercontent.com/opengeos/GeoLibre/main/schema/deployment.schema.json"`. GeoLibre ignores `$schema`.

Field reference

Every section and every field is optional except `version`. An absent section means "not specified": the next source in the precedence chain applies.

Top level

Field	Type	Meaning
<code>version</code>	<code>1</code>	Policy format version. Documents with any other version are ignored.

capabilities

An array of the capability names from [Deployment Capabilities](#): `project:edit`, `data:add`, `processing:run`, `export:data`, `plugins:install`, `settings:manage`. Omit it to grant all capabilities; `[]` grants none.

interface

Field	Type	Meaning
<code>enabled</code>	boolean	Whether UI profile filtering is active (default true).
<code>level</code>	<code>beginner</code> <code>intermediate</code> <code>advanced</code>	Experience-level preset that seeds the hidden lists.
<code>lock</code>	boolean	Prevent users changing the profile in Settings.
<code>hiddenDataSources</code> , <code>hiddenPlugins</code> , <code>hiddenMenus</code> , <code>hiddenMenuItems</code>	string[]	Explicit hidden ids, overriding the preset.

plugins

Field	Type	Meaning
<code>registryUrl</code>	string	Plugin marketplace registry URL, absolute or relative to the app.
<code>allowed</code>	string[]	External plugin ids allowed to load. Omit for any; <code>[]</code> for none.
<code>blocked</code>	string[]	External plugin ids never loaded.
<code>sideload</code>	boolean	Allow installing from a manifest URL, zip, directory or project file (default true).

Field	Type	Meaning
defaultActive	string[]	Plugin ids active in a fresh project.

services

Field	Type	Meaning
builtins	boolean	<code>false</code> hides the built-in starter services.
catalog	object[]	Curated entries: <code>id</code> , <code>name</code> , <code>kind</code> (<code>wms</code> , <code>wfs</code> , <code>wmts</code> , <code>xyz</code> , <code>arcgis</code> , <code>csw</code>), optional <code>category</code> , and non-empty <code>fields</code> (string, number or boolean values).

sharing

Field	Type	Meaning
shareUrl	string	Projects server URL (<code>https://...</code> ; plain <code>http://</code> only on <code>localhost / 127.0.0.1</code>), or <code>off</code> to remove Share and the Gallery.
collabUrl	string	Live collaboration relay (<code>wss://...</code> ; plain <code>ws://</code> only on loopback).
embedOrigins	string[]	Origins allowed to drive a framed app (<code>https://host</code>), or <code>*</code> for any.

geolens

Field	Type	Meaning
url	string	Default GeoLens server (<code>https://...</code> ; plain <code>http://</code> only on loopback in the container), <code>same-origin</code> , or <code>off</code> .

ai

Field	Type	Meaning
enabled	boolean	Expose the same-origin AI assistant route (default false).
model	string	Default assistant model id.

branding

Field	Type	Meaning
appName	string	App name in the toolbar and tab title, at most 60 characters.
welcome	boolean	<code>false</code> skips the first-launch welcome wizard.

Omitted versus empty

For `capabilities` and `plugins.allowed`, leaving the field out and writing `[]` mean opposite things. Omitted means "no restriction from this file"; `[]` means "nothing is granted/allowed".

Plugin precedence

An id in `blocked` is never loaded, even if it is also in `allowed`. When `allowed` is present, any non-bundled external id not in it is not loaded.

The evaluator checks `sideload` permission first, then `blocked`, then `allowed`. Bundled drop-ins under `public/plugins/` are exempt from `allowed` and `sideload`, but still honor `blocked`. Built-in plugins are not external plugins and are not governed by these load restrictions.

For URL plugins, GeoLibre must fetch `plugin.json` to learn the id. A denied id never has its entry or stylesheet fetched or its code imported. Denials appear as external plugin load issues.

With `sideload: false`, manifest URL, zip and directory controls disappear and programmatic installs refuse. Project-supplied manifest URLs produce no trust prompt and cannot be trusted into settings. Previously installed URLs stay in settings, but only URLs recognized by permitted entries in the current registry may load (plus bundled drop-ins). If that registry is unavailable, installed URLs fail closed; bundled drop-ins can still load. Existing file-installed archives and additional directories cannot load. Each skipped configured directory is reported as an external plugin load issue without reading its contents. Each settings-triggered or forced scan checks current registry membership before reusing a previous load. A previously loaded URL that is no longer approved is unloaded, including when the registry is unavailable. Its installed URL and integrity pin remain until the user uninstalls it; registry approval returning does not silently trust changed code. Denied registry entries are hidden in the marketplace, but their installed source URLs remain removable in Settings.

`defaultActive` marks permitted, loaded external plugins for activation in a fresh project. It does not override a saved project's active plugin list and does not allow a denied plugin to load. URL updates preserve these deployment defaults for later fresh projects. Without a `plugins` section, existing loading and activation behavior is unchanged.

Plugin policy is client-side enforcement only, not a server security boundary: a modified client can bypass it. It does not provide signing or sandboxing.

Precedence between sources

The order, highest first, is:

1. final `deployment.json` policy (including container environment overrides)
2. runtime environment (`window.__GEOLIBRE_DEPLOYMENT_ENV__`)
3. build-time environment

The legacy `admin-profile.json` is still honoured for the interface only when the selected policy has no non-empty, valid `interface` section.

Versioning

New fields are additive and keep `version: 1`. A document with any other version is ignored with a console warning.

Validation

The client parser is lenient and works section by section:

- A section with any invalid field, or any unknown key inside it, is dropped whole with a console warning. The other sections still apply.
- An unknown capability name drops the entire `capabilities` section, so the deployment falls back to environment settings or defaults (it does not grant nothing). Check the console if a restriction seems missing.
- Unknown top-level keys are ignored with one warning.
- Non-JSON content or a non-object is ignored silently.

The container validates strictly: it rejects unknown keys, duplicate or invalid ids, service ids that collide after trimming, and numeric service field values beyond the safe-integer range, none of which JSON Schema alone can all express.

Desktop

Place `deployment.json` in Tauri's `<app_config_dir>` (see [UI Profiles](#) for the legacy profile locations). For the standard `org.geolibre.desktop` application identifier, the paths are:

OS	Policy path
Linux	<code>\$XDG_CONFIG_HOME/org.geolibre.desktop/deployment.json</code> , or <code>~/.config/org.geolibre.desktop/deployment.json</code> when <code>XDG_CONFIG_HOME</code> is unset
macOS	<code>~/Library/Application Support/org.geolibre.desktop/deployment.json</code>
Windows	<code>%APPDATA%\org.geolibre.desktop\deployment.json</code> (normally <code>C:\Users\<user>\AppData\Roaming\org.geolibre.desktop\deployment.json</code>)

These follow Tauri's [app config directory](#). Sandboxed installations may resolve the directory inside their sandbox; custom builds with a different application identifier use that identifier instead.

The `read_deployment_policy` command returns raw UTF-8 text, or `null` when the file is absent. A leading UTF-8 BOM is accepted by the parser. The selected policy is applied before the first render, including capability restrictions, without rebuilding the app. Restart GeoLibre after changing the file; changes are not watched.

An existing config-dir file is authoritative, even if empty, malformed or of an unsupported version: it yields no policy rather than falling back to a bundled or web file. Config-dir and web policies are never merged. When the file is absent, GeoLibre fetches `<base>/deployment.json` instead. Other read errors (including permission failures) produce a console warning in every build and fall back to that same web file. Unsupported versions also warn in every build. With no file in either location, existing behavior is unchanged.

A non-empty, valid policy `interface` replaces the legacy profile whole; an absent, empty, or dropped invalid `interface` leaves the legacy profile eligible. The config directory is user-writable: this is desktop provisioning, not a security boundary or server-side enforcement.

Docker

The image writes the generated public file to `/usr/share/nginx/html/deployment.json` on every boot, served at `/deployment.json` with `Cache-Control: no-store`, `Content-Type: application/json` and `X-Content-Type-Options: nosniff`. The source file is selected by `GEOLIBRE_DEPLOYMENT_FILE` (for example `/etc/geolibre/deployment.json`); mount it read-only there, never over the generated output. With no source or overrides, the image writes `{"version":1}`, which adds no capability restriction.

The source file is strictly validated before any environment overrides are applied. Nonblank `GEOLIBRE_*` values then override their corresponding fields; blank values count as unset. Invalid input stops boot and cannot be repaired by an override.

```
docker run --rm -p 8080:80 \
  -v "$PWD/deployment.json:/etc/geolibre/deployment.json:ro" \
  -e GEOLIBRE_DEPLOYMENT_FILE=/etc/geolibre/deployment.json \
  -e GEOLIBRE_CAPABILITIES=data:add,export:data \
  ghcr.io/opengeos/geolibre:latest
```

Configuration changes take effect on the next container start; they do not require rebuilding the image.

Variable	Policy field
<code>GEOLIBRE_CAPABILITIES</code>	<code>capabilities</code> : comma-separated names, or <code>none</code> for no grants
<code>GEOLIBRE_SERVICES_FILE</code>	<code>services.catalog</code>
<code>GEOLIBRE_BUILTIN_SERVICES=off</code>	<code>services.builtins</code> : <code>false</code>
<code>GEOLIBRE_SHARE_URL</code>	<code>sharing.shareUrl</code>
<code>GEOLIBRE_COLLAB_URL</code>	<code>sharing.collabUrl</code>

Variable	Policy field
GEOLIBRE_EMBED_ORIGINS	sharing.embedOrigins
GEOLIBRE_GEOLENS_URL	geolens.url
GEOLIBRE_APP_NAME	branding.appName : whitespace collapsed, cut to 60 characters
GEOLIBRE_AI_URL / GEOLIBRE_AI_MODEL	ai.enabled: true / ai.model

The boot log has one `Deployment policy: <path> from <VAR> = <value>` line per override. Tokens never appear in it, and a query string or fragment on a URL is replaced with `[redacted]`.

Invalid input stops the boot with an `ERROR:` line that names the JSON path (for example `ERROR: GEOLIBRE_DEPLOYMENT_FILE capabilities[1] must be one of: ...`), so nginx never starts with a weaker policy than you asked for. A file with `ai.enabled: true` also needs `GEOLIBRE_AI_URL`, `GEOLIBRE_AI_PROXY_URL` and `GEOLIBRE_AI_PROXY_TOKEN`, otherwise the boot fails.

`GEOLIBRE_CAPABILITIES` overrides the final policy's client grants at container generation time. The generated policy is what nginx enforces; build-time client grants alone do not configure nginx.

Public file, no secrets

`deployment.json` is served to every browser. Never put secrets in it. The AI proxy URL and token, the sidecar token, trusted proxies, Basic Auth and CSP stay in server environment variables and files.

Client hiding is not enforcement

Hiding or removing an interface element does not stop someone with browser devtools. Nginx enforcement applies only to the bundled Docker routes; it does not restrict desktop processing, browser WASM, separately exposed sidecars, or plugin execution. See [Deployment Capabilities](#) and [Self-Hosting](#).

<https://geolibre.app/i18n/>

Internationalization (i18n)

GeoLibre's interface is translatable. UI strings live in JSON message catalogs rather than being hard-coded in components, so the app can be shipped in any language by dropping in a catalog. The layer is built on [react-i18next](#) and lives in `apps/geolibre-desktop/src/i18n/`.

This is an **incremental** effort: high-traffic surfaces (the top toolbar and menus, the Settings dialog) are translated first, and untranslated strings fall back to English until they are migrated. Adding a new language never requires touching component code.

How it works

```
apps/geolibre-desktop/src/i18n/
├── index.ts           # i18next init, catalog auto-discovery, language detection
├── languages.ts      # friendly language names + resolution helpers
├── i18next.d.ts      # types t() keys against the English catalog
└── locales/
    ├── en.json       # the baseline English catalog (source of truth)
    ├── zh.json       # Simplified Chinese
    ├── es.json       # Spanish
    ├── fr.json       # French
    ├── de.json       # German
    ├── pt.json       # Portuguese
    ├── it.json       # Italian
    ├── nl.json       # Dutch
    ├── ja.json       # Japanese
    ├── ko.json       # Korean
    ├── ru.json       # Russian
    ├── tr.json       # Turkish
    ├── id.json       # Indonesian
    ├── hi.json       # Hindi
    ├── th.json       # Thai
    ├── vi.json       # Vietnamese
    ├── az.json       # Azerbaijani
    ├── ka.json       # Georgian
    ├── ar.json       # Arabic (right-to-left)
    ├── he.json       # Hebrew (right-to-left)
    └── fa.json       # Persian (right-to-left)
```

The non-English catalogs were seeded with machine-assisted translations and are open to native-speaker review and correction (PRs welcome).

- **en.json is the source of truth.** Every key used by `t()` must exist here; `i18next.d.ts` types the `t()` function against it, so a missing or misspelled key is a compile error (`npm run typecheck`).
- **Catalogs are auto-discovered.** `index.ts` picks up every `locales/*.json` via `import.meta.glob`, so adding `locales/fr.json` is all it takes to make French selectable — no code changes.
- **English is bundled; other locales are lazily loaded.** `en.json` ships in the app chunk (it is the fallback and the type source). Every other catalog is a separate chunk (`i18n-locale-<code>`) imported only when it becomes the active language — the initial locale is awaited before the

first render (`i18nReady` in `index.ts`), and switching languages (`useLanguage.ts`) loads the target catalog before applying it. Fully translated, the non-English catalogs total several MB, so shipping them all at boot is avoided. The service worker keeps these chunks out of the app-shell precache and CacheFirst-caches each on first use (see `maximumFileSizeToCacheInBytes` / `HEAVY_PRECACHE_IGNORES` in `vite.config.ts`): the active language works offline after its first online load, and English works offline unconditionally.

- **Bundled locales may lag behind** `en.json`. Any key missing from a non-English catalog falls back to `en` at runtime (`fallbackLng: "en"`), and the catalogs are filled in before each release. See [Keeping catalogs complete](#).

Language detection

On startup the initial language is resolved in priority order (`getInitialLanguage` in `index.ts`):

1. the `?locale=` or `?lang=` **query parameter** (for embeds — consistent with the existing `?theme=` / `?maponly` params),
2. the language **persisted in desktop settings** (`language` field),
3. the browser's preferred languages (`navigator.languages`),
4. the default, `en`.

Only languages that ship a catalog are honored; anything else falls through to the next rule. The in-app selector (the Settings menu → Language, or the Language section of the Settings dialog) changes the language live and records the choice so it survives reloads.

The `?locale` parameter makes embeds language-aware, e.g.:

```
https://geolibre.app/?maponly&locale=es
```

Using translations in components

```
import { useTranslation } from "react-i18next";

function Example() {
  const { t } = useTranslation();
  return <button aria-label={t("common.cancel")}>{t("common.save")}</button>;
}
```

- **Interpolation:** `t("settings.env.removeAria", { name })` against a catalog value like `"Remove {{name}}"`.
- **Pluralization:** define `key_one` / `key_other` and call `t("settings.env.variablesCount", { count })`.
- **Rich text (links, bold):** use the `<Trans>` component with named `components`, e.g. a catalog value of `"... at <tokenLink>share.geolibre.app/settings</tokenLink>."` rendered with `<Trans i18nKey="..." components={{ tokenLink: <a href="..." /> }} />`.

- **Module-scope constants** can't call `t()` (no hook in scope). Store a stable **catalog key** on the constant instead of the English string and resolve it with `t(item.labelKey)` at the render site. See `SECTION_ITEMS` in `SettingsDialog.tsx` and the command/menu arrays in `toolbar/constants.ts`.

Do **not** translate: developer logs (`console.*`), URLs, store IDs, file-format names/extensions, or persisted data values.

Adding a new language

1. Copy `apps/geolibre-desktop/src/i18n/locales/en.json` to `locales/<code>.json` (e.g. `fr.json`, `pt-BR.json`) and translate the values. Aim to translate every key; any you leave out fall back to English at runtime, and the catalog test reports them rather than failing. Keep a value in English only where English is the language's own convention (product names, file formats, units).
2. Use your language's CLDR plural categories for plural keys: keep only the forms your language has (e.g. Indonesian/Japanese/Korean/Chinese/Thai drop `_one` and keep `_other`; Russian adds `_few` / `_many`; Arabic carries all six forms). The catalog test compares plural-normalized keys, so a locale may carry more or fewer plural forms than `en` without failing.
3. If the code isn't already in `LANGUAGE_NAMES` (`languages.ts`), add an entry so the selector shows a friendly native name. (Optional — it falls back to the raw code.)
4. That's it. The language appears in **Settings → Language** (and the Language section of the Settings dialog) and works with `?locale=<code>`.

Keeping catalogs complete

`tests/i18n-catalogs.test.ts` (part of `npm run test:frontend`) checks every bundled catalog against `en.json`:

- **No extra keys.** A key absent from `en.json` is a typo or a leftover.
- **Same placeholders.** Each translation keeps the English string's `{{placeholders}}` and `<tag>` markup.
- **Missing keys are reported, not enforced.** The test logs each locale's missing keys. Plural keys are compared by their base, so a locale carries the plural forms its language needs (`_other` only for `zh/ja/ko/id/th/vi`; `_two` for `he`; `_few` / `_many` for `ru`; all six for `ar`) rather than `en`'s `_one` / `_other` pair.

So a change that adds a `t()` key, or a `plugin.<id>.*` key a built-in plugin resolves through `app.translate`, only needs to add it to `en.json`. The other catalogs are filled in before each release with the `geolibre-i18n` audit. When you do add translations, add the keys without reordering the catalogs (several have drifted from `en.json`'s order, and a reordering diff hides the real change).

Right-to-left languages

Arabic (`ar`), Hebrew (`he`) and Persian (`fa`) ship as right-to-left catalogs, and the layout direction is wired so further RTL locales are additive:

- **Direction registry.** `RTL_LANGUAGES` in `languages.ts` lists the right-to-left base subtags (`ar` , `he` , `fa` , `ur`); `languageDirection(code)` resolves any tag the same way `resolveLanguage` does (`ar-SA` → `rtl` , `unknown` → `ltr`).
- **Document mirroring.** `index.ts` subscribes to i18next's `languageChanged` event — registered before `init` , so it also fires for the initial language — and sets `document.documentElement.lang` and `.dir` . With `dir="rtl"` the flex-based shell mirrors automatically: Layers on the right, Style on the left.
- **Radix direction.** `App.tsx` wraps the shell in `DirectionProvider` (re-exported from `@geolibre/ui`), so menus and submenus open toward the reading direction and arrow keys, sliders, and tabs follow the mirrored order.
- **Logical utilities.** Components use Tailwind's logical spacing/position utilities (`ms-* / me-*` , `ps-* / pe-*` , `text-start / text-end` , `border-s / border-e` , `start-* / end-*`) instead of the physical `ml-* / left-*` forms, so styles flip with the document direction. Use the logical forms in new code, and don't mix a physical and a logical utility for the same side on one element — `tailwind-merge` treats them as distinct groups, so both would apply. Symmetric centering (`left-1/2 + -translate-x-1/2`), map-anchored overlays, and JS-computed pixel offsets intentionally stay physical.
- **The map chrome stays ltr.** MapLibre anchors its controls with physical CSS, so `index.css` pins `.maplibregl-control-container { direction: ltr }` — the map chrome is identical in every language. The rest of the `.maplibregl-map` subtree is deliberately left free to mirror, so app overlays portalled into the map container (e.g. the story-map presenter) still follow the RTL layout. Arabic text in popups still shapes correctly through the Unicode bidi algorithm, and basemap label shaping in the interactive map is handled by MapLibre's built-in bidi/shaping (6.11+); no RTL text plugin is registered there, because the legacy plugin mis-joins Kurdish letters. The exported story-map HTML pins an older CDN `maplibre-gl` and still loads the plugin.

Adding another right-to-left language is the same recipe as any language, plus its base subtag in `RTL_LANGUAGES` if it isn't already listed.

Keeping formatting locale-aware

Number, date, and coordinate formatting should continue to use the runtime locale via `Intl` APIs (e.g. `new Intl.NumberFormat(undefined, ...) / Intl.DateTimeFormat`) rather than hard-coded formatting, so values render in the user's regional conventions.

Processing tool metadata

Tool names, descriptions, group labels, parameter labels/help text and select option labels live in processing registries that have no i18n access. The Processing dialogs (Vector / Network / Spatial statistics / Raster / Batch tools, the Whitebox toolbox, and the Model Builder palette) render them through `apps/geolibre-desktop/src/lib/processing-tool-i18n.ts`, which resolves `processing.toolMeta.<catalog>.<toolId>...` and falls back to the registry's own string. The catalog name (`vector` | `network` | `statistics` | `raster` | `whitebox`) is part of every key because tool ids repeat across registries — `reproject` is both a vector tool and a raster tool. Grouping labels share one namespace, `processing.toolGroup.<slug>`, because the same label ("Analysis") appears in more than one registry.

Because the fallback is the registry string itself, **English is correct with no catalog entries at all**. For the four small bundled registries, `en.json` carries the keys translators work from: `tests/i18n-catalogs.test.ts` rejects a key in another bundled locale that has no English counterpart. That baseline is generated:

```
npm run i18n:tools      # regenerate the en.json baseline from the registries
npm run i18n:tools:check # fail if it is out of date (runs in `npm run ci`)
```

Re-run it after adding or renaming a tool, parameter, or select option, and commit the result. Only `processing.toolMeta` and `processing.toolGroup` in `en.json` are rewritten; the other locales are never touched.

Whitebox tool names and parameter help text use `processing.toolMeta.whitebox.<toolId>...`, but their hundreds of descriptions are **not bundled in any locale**. Whitebox supplies the English fallback at runtime; optional translations are versioned language packs hosted in opengeos/geolibre-language-packs. Users can download the active locale's pack from Settings → Language or import a compatible JSON file. Packs are validated before being merged and persist in an IndexedDB database separate from projects and desktop settings.

The v1 pack format accepts only these roots:

- `processing.toolMeta.whitebox`
- `processing.whitebox.categories`
- `processing.whitebox.menuTool`
- `processing.whitebox.menuSubcategory`

Official packs resolve from `https://languages.geolibre.app/v1/whitebox/<locale>.json`. A self-hosted build may set `VITE_LANGUAGE_PACK_BASE_URL` to an internal mirror. Local file import continues to work without network access.

The Model Builder palette is the one place that mixes the two, and its group headings need care: `groupModelTools` keys groups on the raw label, so a heading has no provider of its own, and the WASM catalog ships a `terrain` category (lowercase) that slugs onto the raster registry's `Terrain`. Translating both would render two identically-labelled headings. `translateModelToolGroup` gates on the group's tools instead — a heading is translated only when at least one of its tools comes from

the vector registry. Use it for palette headings rather than `translateToolGroup` directly. Note that `modelProviderCatalog` now returns `"whitebox"` rather than `null` for Whitebox tools, so "has an owned catalog" is no longer the same test: a Whitebox-only group's heading stays raw catalog text even when an optional Whitebox language pack is installed.

Plugin-facing surfaces

Plugin display names resolve through `toolbar.plugin.<pluginId>`, and every surface that shows one (the Plugins menu, the command palette, Settings → Interface, Manage Plugins) goes through `lib/plugin-display-name.ts`, so a plugin cannot read translated in one menu and English in the next. A plugin with no catalog entry falls back to its registered name, which is right for the brand/ proper-noun plugins (Mapillary, NASA Earthdata, ...) and for externally installed plugins the host cannot know at build time.

A plugin's **own** UI is plain DOM and cannot use `useTranslation`. The app API gives it `getLocale()`, `onLocaleChange(listener)`, `translate(key, defaultValue, params)` and `registerTranslations(resources)` instead, and toolbar menu labels and panel titles accept getter functions so they can call `translate` and be re-read after a language change. See [Following the app language](#) in the plugin API reference.

Built-in plugins keep their strings in the bundled catalogs under `plugin.<pluginId>.*` and resolve them with `createPluginTranslator(app, id)` from `@geolibre/plugins`, which prefixes the key and interpolates the English fallback. A docked panel's header uses `pluginDisplayTitle(app, id, name)`, which reads `toolbar.plugin.<id>`, so the header matches the Plugins menu entry that opened it. Some older built-ins instead receive their labels from the host through a `set<Plugin>Labels(...)` export that `usePluginLabelSync` calls on every language change; both routes end in the same catalogs.

Plain DOM a plugin builds once (a button, a status line) does not re-render on a language change. Subscribe with `app.onLocaleChange` and re-set its text, as the GeoAgent Earth Engine **Sign in** button does.

Errors thrown from plugin code

Non-UI plugin modules (the ArcGIS layer loader, Earth Engine sign-in, the route recorder) have no `t()`, and their messages reach the user wherever a caller shows `error.message`: a dialog, a toast, or an upstream control that renders it as is. Throw a `LocalizedError` from `@geolibre/plugins` instead of a plain `Error`:

```
throw new LocalizedError(
  "arcgisService.errors.serviceRequestFailed",
  "ArcGIS service request failed with {{status}}.",
  { status: response.status },
  { cause },
);
```

It translates itself when constructed, through the translator the host registers once with `setLocalizedErrorTranslator` (in `hooks/usePlugins.ts`), so no display site needs changing. The English stays on `error.englishMessage`. For an error class that also carries server-supplied text, build just the GeoLibre-authored part with `localizedMessage(key, english, params)`. Leave a server's own message (an ArcGIS error envelope, a Google OAuth error) as it is. Add the key to `en.json` with the same English: `tests/localized-error.test.ts` scans `packages/plugins/src` for every call and fails when a key is missing or its English differs.

Keys derived from a registry

The Deck.gl Layer dialog shows names from the layer registry (`deckgl-viz/registry.ts`), which has no i18n access. `lib/deck-viz-i18n.ts` derives keys from the registry's ids (`addData.deckViz.kinds.<kind>.label|description`, `addData.deckViz.categories.<category>`, `addData.deckViz.roles.<roleKey>`) and falls back to the registry's English. `tests/deck-viz-i18n.test.ts` fails when `en.json` and the registry disagree, so update both together.

Known limitations

Some plugins wrap **third-party MapLibre controls** that render their own UI (e.g. the Esri Wayback plugin's `maplibre-gl-esri-wayback` control, with labels such as "Only versions with local changes"). Those strings live inside the external library and never pass through GeoLibre's `react-i18next` pipeline, so they stay English regardless of the selected language. Translating them requires an upstream change to the library (or a fork); it can't be done from a locale catalog here.

<https://geolibre.app/python/>

Python package (Jupyter)

`pypi` `v3.3.0`  `Open in Colab` `conda-forge` `v3.3.0` `recipe` `geolibre`

GeoLibre ships a Python package, `geolibre`, that embeds the full GeoLibre app inside a Jupyter notebook cell as an `anywidget`, with a `leaflet`-style API.

The widget loads the complete GeoLibre app (menus, panels, processing tools) in an `iframe`. State syncs both ways through a single `.geolibre.json` project, so data you add from Python appears in the UI, and edits you make in the UI (panning, zooming, adding layers) are readable back from Python.

Install

```
pip install geolibre
```

Or with conda from [conda-forge](#):

```
conda install -c conda-forge geolibre
```

Optional extras provide GeoPandas/Shapely support for GeoDataFrames and local vector files, plus `xarray`/`rioxarray`/`rasterio`/`rio-tiler` support for in-memory rasters:

```
pip install "geolibre[all]"      # both
pip install "geolibre[vector]"  # GeoPandas/Shapely only
pip install "geolibre[raster]"  # xarray/rioxarray/rasterio/rio-tiler only
```

`[all]` is the union of the two, so it now installs `rasterio` (and `GDAL` with it); use `[vector]` to keep an existing vector-only environment as light as before.

The optional `[all]` extra is pip-only. If you installed via conda, add it with `pip install "geolibre[all]"` inside the same environment.

To embed the map in a [Dash](#) app, install the `dash` extra and use `from geolibre import DashMap`:

```
pip install "geolibre[dash]"
```

Quickstart

```
from geolibre import Map

m = Map(center=(-100, 40), zoom=4)
m.add_geojson("https://example.com/data.geojson", name="Data")
m
```

The full GeoLibre UI renders in the cell. Add more data and drive the view:

```
m.add_tile_layer(
    "https://tile.openstreetmap.org/{z}/{x}/{y}.png",
    name="OpenStreetMap",
    attribution="(c) OpenStreetMap contributors",
)
m.add_cog("https://example.com/dem.tif", name="DEM", colormap="terrain")
m.add_basemap("dark")
m.set_center(-120, 47, zoom=8)
```

Google Earth Engine layers are optional and need `pip install earthengine-api` plus credentials (`ee.Authenticate()` once, then a Google Cloud project):

```
import ee

ee.Authenticate() # once per machine
ee.Initialize(project="your-google-cloud-project")
m.add_ee_layer(ee.Image("USGS/SRTMGL1_003"), {"min": 0, "max": 3000}, name="SRTM")
```

`add_ee_layer` evaluates the Earth Engine object in the kernel and adds the resulting tile URL as a raster layer (ImageCollections are mosaicked, vector objects are styled into raster tiles — for those, `vis_params` takes `ee.FeatureCollection.style()` keys such as `color`, `fillColor`, `width`, and `pointSize`, not image keys). That URL is tied to an Earth Engine map id that expires, so a saved project may need the Earth Engine layer regenerated when it is reopened. The result is a plain raster tile layer, not one of the live layers the app's own Earth Engine panel manages.

`add_raster` / `add_cog` also accept a **local** GeoTIFF path on the kernel host: the file is served by the bundled localhost server so the app can read it. This works directly in local Jupyter and VS Code. In Google Colab, where the kernel proxy does not preserve the byte-range semantics required by browser COG rendering, GeoLibre renders local rasters as PNG XYZ tiles in the kernel instead. JupyterHub can route the COG through the kernel port when `jupyter-server-proxy` is available. A static-server-extension-only deployment cannot expose kernel files, so pass a hosted URL there. The served URL is session-scoped, so a project saved with a local raster will not restore it when reopened later — pass a hosted URL for durable projects.

Install `geolibre[raster]` to visualize an in-memory xarray object. Spatial dimensions named `lon` / `lat` or `longitude` / `latitude` default to EPSG:4326; otherwise provide CRS and dimension names explicitly. Dataset variables become bands unless one is selected:

```
m.add_raster(data_array, name="Temperature", colormap="viridis")
m.add_raster(
    dataset,
    name="Temperature",
    array_args={"variable": "temperature", "isel": {"time": 0}},
)
```

The temporary Cloud-Optimized GeoTIFF backing an xarray layer is removed when the widget is closed, so xarray layers have the same session-only limitation as local files. Locally the browser reads that COG directly; in Colab rio-tiler renders it into ordinary PNG XYZ tiles to avoid Colab's

incompatible byte-range proxy behavior. Call `m.close()` when you are done to remove it promptly; otherwise it is removed when the `Map` is garbage collected or when the kernel exits normally. A kernel that is killed outright leaves the file behind in the system temp directory.

Add markers and data-driven symbology without precomputing styles:

```
m.add_marker(-122.4, 37.8, properties={"name": "San Francisco"})
m.add_marker_cluster([(-122.4, 37.8), (-122.3, 37.9), (-122.5, 37.7)])
m.add_heatmap([(-122.4, 37.8), (-122.3, 37.9)], radius=35)
m.add_csv("cities.csv", x="longitude", y="latitude", name="Cities")
m.add_choropleth(
    "https://example.com/counties.geojson",
    column="population",
    colormap="blues",
    scheme="quantile",
)
```

Add a legend, a colorbar, and a swipe (split-map) comparison:

```
# The app's legend panel (Controls > Legend), built from each layer's style.
m.set_map_legend("Population", position="bottom-right")

# A built-in land-cover legend, or your own {label: color} dict.
m.add_legend(builtin="nlcd")
m.add_legend(legend_dict={"Water": "#0000ff", "Land": "#00ff00"})

# A colorbar for a continuous raster.
m.add_colorbar(colormap="terrain", vmin=0, vmax=4000, label="Elevation", units="m")

# Compare two layers (or a layer against the basemap) with a swipe slider.
before = m.add_cog("https://example.com/before.tif", name="Before")
after = m.add_cog("https://example.com/after.tif", name="After")
m.split_map(before, after)
```

Two-way sync

Because the project syncs both ways, you can pan or zoom the map in the UI and then read the live state back from Python:

```
proj = m.to_project()
proj["mapView"]["center"] # reflects the live UI view
[layer["name"] for layer in proj["layers"]]
```

Save and reload projects, fully interchangeable with the desktop and web apps:

```
m.save_project("my-map.geolibre.json")

m2 = Map()
m2.load_project("my-map.geolibre.json")
m2
```


`to_project()`, `save_project()`, and `to_html()` redact credentials — API keys, tokens, authenticated request headers, secret environment variables (every row not marked `"secret": false`), geocoder keys, and credential URL parameters — so anything you serialize, commit, or share is safe by default. Pass `keep_credentials=True` to `to_project()` or `save_project()` for a trusted local file that must keep working without re-entering them:

```
m.save_project("private.geolibre.json", keep_credentials=True)
```

The `project` trait itself is unredacted, so live two-way sync with the widget keeps authenticated layers rendering.

Map options

```
Map(
    center=(-100, 40),    # [lng, lat]
    zoom=4,
    basemap="dark",       # a basemap name or a MapLibre style URL
    height="800px",
    layout="embed",       # "embed" (compact UI), "full" (desktop UI), or "maponly"
    theme="light",        # "light" or "dark"
)
```

Interactive scripting

Beyond adding data, the widget can **query the live app and react to it** — the same surface as the in-app [Python Console](#). These calls round-trip to the running map, so the map must be displayed first (show `m` in a cell), then run the queries in a later cell.

```
m.get_center()           # live [lng, lat], reflecting UI pans/zooms
m.get_bounds()           # [west, south, east, north]
m.fly_to(-122.4, 37.8, zoom=10)
m.identify(-122.4, 37.8) # features at a point, like clicking the map

# Layer objects: read and mutate layers, read their features
for layer in m.layers:
    layer.opacity = 0.6
features = m.layers[0].get_features() # list of GeoJSON Feature objects

# Run a processing algorithm; result layers are added to the map
m.list_algorithms()
m.run_algorithm("buffer", {"layer": layer_id, "distance": 1000})

# Read back what the user selected or drew on the map
m.get_selected_features() # the clicked feature(s) as Feature objects
m.get_drawn_features()    # features sketched with the Geo Editor
m.user_rois               # the drawn ROIs as a GeoJSON FeatureCollection
m.get_drawn_features(as_gdf=True) # the same, as a GeoDataFrame (needs GeoPandas)

png = m.to_image()        # PNG bytes (or m.to_image("map.png"))
m.to_html("map.html")     # standalone HTML embedding the live project
```

React to user interaction with event callbacks:

```
m.on_click(lambda e: print("clicked", e["lngLat"]))
m.on_selection_change(lambda e: print("selected", e))
m.on_layer_change(lambda e: print("layers", e["layerIds"]))
```

Blocking queries

Interactive queries block the kernel until the app replies (via `jupyter_ui_poll`, installed automatically). Pass `timeout=` for slow calls, e.g. `m.run_algorithm(..., timeout=300)`.

API reference

Interactive queries, events & processing

Method	Description
<code>get_view()</code> / <code>get_center()</code> / <code>get_bounds()</code>	Read the live camera / center / viewport bounds.
<code>fly_to(lng, lat, zoom=, bearing=, pitch=, duration=)</code>	Animate the camera.
<code>fit_bounds([w, s, e, n])</code>	Fit the camera to a bounding box.
<code>zoom_to_bounds([w, s, e, n])</code> / <code>zoom_to_layer(layer)</code>	Leafmap-style view helpers; layers may be addressed by id, name, or handle.
<code>identify(lng, lat, layer_id=None)</code>	Query rendered features at a point.
<code>get_features(layer_id)</code>	A layer's features as <code>Feature</code> objects.
<code>get_selected_features(as_gdf=False)</code>	The feature(s) selected in the app, as <code>Feature</code> objects (or a <code>GeoDataFrame</code>).
<code>get_drawn_features(as_gdf=False)</code> / <code>user_rois</code>	Features drawn with the Geo Editor; <code>user_rois</code> returns them as a <code>FeatureCollection</code> .
<code>layers</code> / <code>get_layer(id)</code>	<code>Layer</code> handles (read state; set <code>name</code> / <code>visible</code> / <code>opacity</code> , <code>set_style</code> , <code>get_features</code> , <code>zoom_to</code> , <code>remove</code>).
<code>layer_names</code> / <code>find_layer(name)</code> / <code>find_layer_index(name)</code>	Inspect layers by display name.
<code>set_layer_visibility(layer, visible)</code> / <code>set_layer_opacity(layer, opacity)</code>	Change a layer by id, name, or handle.
<code>list_algorithms()</code>	Available processing algorithms (<code>id</code> , <code>parameters</code> , ...).
<code>run_algorithm(id, parameters=None, timeout=)</code>	Run an algorithm; returns <code>{logs, resultLayerIds}</code> .
<code>run_model_builder(graph, timeout=)</code>	Run a serialized Model Builder graph in the displayed app.

Method	Description
<code>list_whitebox_tools() / run_whitebox_tool(tool_id, parameters=None, timeout=)</code>	List, or run in the browser via WASM, the bundled Whitebox tools; <code>run_whitebox_tool</code> returns <code>{logs, resultLayerIds}</code> .
<code>to_image(path=None, timeout=)</code>	Capture the map as PNG bytes, or write to <code>path</code> .
<code>to_html(path=None, title=, width=, height=, app_url=)</code>	Export a standalone HTML page that embeds the current project (credentials redacted) in this map's <code>layout</code> and <code>theme</code> ; returns the HTML or writes to <code>path</code> .
<code>on(event, cb) / on_click / on_selection_change / on_layer_change</code>	Register event callbacks; returns an unsubscribe function.
<code>request(method, params=None, timeout=)</code>	Low-level command primitive behind the methods above.

Data, view & projects

Method	Description
<code>Map(center, zoom, basemap=, renderer=, height=, layout=, theme=, server_proxy=)</code>	Create a map.
<code>add_geojson(data, name=, **style)</code>	Add GeoJSON from a dict, file path, URL, JSON string, or GeoDataFrame.
<code>add_gdf(gdf, name=, column=None, **style)</code>	Add a GeoDataFrame, optionally as a choropleth.
<code>add_csv(data, x="longitude", y="latitude", name=, **style) / add_xy_data(...)</code>	Add points from a CSV path, URL, text, DataFrame, or row mappings.
<code>add_marker(lng, lat, name=, properties=, color=, opacity=, radius=, stroke_color=, stroke_width=, shape=, size=, icon=, **style)</code>	Add a single point marker (<code>properties</code> appear on click).
<code>add_markers(points, name=, color=, opacity=, radius=, stroke_color=, stroke_width=, shape=, size=, icon=, **style)</code>	Add point markers from <code>(lng, lat)</code> pairs, <code>{lng/lon/x, lat/y, ...}</code> dicts, GeoJSON, or a GeoDataFrame.

Method	Description
<code>add_circle_markers(points, name=, radius=, **style)</code>	Add circle markers with an explicit <code>radius</code> .
<code>add_marker_cluster(points, name=, cluster_radius=, cluster_max_zoom=, **style)</code>	Add clustered point markers.
<code>add_heatmap(points, name=, radius=, intensity=, color_ramp=, weight_field=, **style)</code>	Add point data using the density heatmap renderer, optionally weighted by a numeric field.
<code>add_choropleth(data, column, name=, class_count=, colormap=, scheme=, **style)</code>	Add a GeoJSON layer with graduated symbology computed from a numeric <code>column</code> .
<code>add_data(data, column=None, name=, **kwargs)</code>	Add data; a choropleth when <code>column</code> is given, else a plain GeoJSON layer (leafmap parity).
<code>add_vector(data, name=, render_mode=, data_format=, source_layer=, **style)</code>	Add a vector dataset from a URL (GeoParquet, FlatGeobuf, zipped Shapefile, GeoJSON, ...) or a local file (read via GeoPandas and inlined).
<code>add_geoparquet(data, name=, **style)</code>	Add a GeoParquet dataset (URL or local file).
<code>add_flatgeobuf(data, name=, **style)</code>	Add a FlatGeobuf dataset (URL or local file).
<code>add_shp(data, name=, **style)</code>	Add a Shapefile (zipped URL or local <code>.shp</code>).
<code>add_kml(data, name=, **style) /</code> <code>add_gpkg(data, name=, layer=None, **style)</code>	Add KML/KMZ or GeoPackage data.
<code>add_polyline(polyline, name="Polyline", precision=5, unescape=False, **style)</code>	Add an Encoded Polyline layer from a string or list of strings (precision 5 or 6).
<code>add_vector_tiles(url, name=, source_layers=, source_layer=, **style)</code>	Add a vector tile layer from a TileJSON endpoint.

Method	Description
<code>add_pmtiles(url, name=, tile_type=, source_layers=, **style)</code>	Add a PMTiles archive (vector or raster).
<code>add_tile_layer(url, name=, tile_size=, attribution=, bounds=, **style)</code>	Add a raster XYZ tile layer.
<code>add_ee_layer(ee_object, vis_params=, name=, shown=, opacity=)</code>	Add an authenticated Google Earth Engine object as raster tiles (needs <code>earthengine-api</code>).
<code>add_wms(endpoint, layers, name=, styles=, image_format=, transparent=, tile_size=, version=, crs=, bounds=, **style)</code>	Add a WMS layer (GetMap, tiled raster). <code>bounds</code> is <code>[west, south, east, north]</code> , needed for zoom-to-layer. <code>crs</code> defaults to <code>EPSG:3857</code> ; for a server without Web Mercator pass a CRS it lists: preferably a geographic one (<code>EPSG:4326</code> , <code>EPSG:4258</code> , <code>EPSG:6706</code> , or <code>CRS:84</code> with <code>version="1.3.0"</code>), otherwise a projected <code>EPSG:<code></code> such as <code>EPSG:25832</code> , which the desktop app warps from its EPSG tables. Only the desktop app redraws those tiles into Web Mercator: the web build and <code>to_html</code> pages still send the Web Mercator BBOX, so such a layer stays blank there.
<code>add_wmts(url, name=, tile_size=, bounds=, **style)</code>	Add a WMTS layer from a tile URL template.
<code>add_wfs(endpoint, type_name, name=, version=, output_format=, srs_name=, max_features=, **style)</code>	Add a WFS layer (GetFeature GeoJSON, fetched and inlined).
<code>add_cog(url, name=, bands=, colormap=, rescale=, **style)</code>	Add a Cloud Optimized GeoTIFF (URL or a kernel-side local GeoTIFF path).
<code>add_raster(source, name=, url=, bands=, colormap=, rescale=, array_args=, **style)</code>	Add a COG/GeoTIFF URL or path, or an xarray DataArray/Dataset (xarray needs <code>geolibre[raster]</code>).
<code>add_lidar(url, name=None, **style)</code>	Add a LAS, LAZ, COPC or EPT point cloud by URL (COPC/EPT stream by level of detail).
<code>point_cloud_annotations()</code>	Read the point labels, instance ids, custom classes, 3D boxes and 3D vectors saved by the app's point cloud annotator . Labels and instance ids are keyed by source URL, then node key and point index; <code>geolibre.project.apply_point_labels</code> writes a whole-file source's labels onto <code>laspy</code> classification.
<code>prelabel_point_cloud(url, input_file, tool="ground", only_unclassified=True)</code>	Run the annotator's Whitebox pre-label on a local copy of a LiDAR layer and save the changed classes as its labels (<code>ground</code> or <code>ground-vegetation</code> ; needs <code>geolibre[pointcloud]</code>).

Method	Description
<code>write_labeled_point_cloud(url, input_file, output_file)</code>	Write a local copy of a LiDAR layer's LAS/LAZ/COPC file with the saved labels and instance ids applied, streaming files larger than memory (needs <code>geolibre[pointcloud]</code>).
<code>set_point_cloud_classes(classes)</code>	Define the annotator's custom classes, e.g. <code>[{"code": 64, "name": "Car", "color": "#e11d48"}]</code> (codes 19-255); the LiDAR layer draws them in their colour and names them in its legend.
<code>add_3d_tiles(url=None, name=, ion_asset_id=, altitude_offset=, request_headers=, **style)</code>	Add a 3D Tiles <code>tileset.json</code> URL, or a Cesium Ion tileset by asset id (3D globe only).
<code>add_cesium_ion(asset_id, name=, kind="3d-tiles", altitude_offset=, **style)</code>	Add a Cesium Ion asset by id: a 3D Tiles tileset or (<code>kind="imagery"</code>) an imagery layer. Renders on the 3D globe, with the app's Ion token.
<code>add_czml(url=None, name=, data=, source_path=, **style)</code>	Add a CZML (Cesium Language) dynamic 3D scene by URL or inline packets: orbits, vehicle tracks, moving models. Renders on the 3D globe, which follows the document's clock.
<code>add_cesium_kml(url=None, name=, data=, source_path=, **style)</code>	Native KML/KMZ on the globe with document styles and overlays. Supply a URL, inline XML, or a KMZ data URL; use <code>add_kml</code> for vector conversion.
<code>add_video(urls, coordinates, name=, **style)</code>	Add a georeferenced video (four <code>[lng, lat]</code> corners).
<code>add_basemap(basemap) / set_basemap(basemap)</code>	Set the background basemap.
<code>split_map(left_layers=None, right_layers=None, orientation=, position=, control_position=)</code>	Add a swipe (split-map) comparison slider between two layer sets.
<code>set_map_legend(title=None, position=, group_by_layer=, visible=, collapsed=)</code>	Show the map legend (Controls → Legend), whose rows come from each visible layer's symbology. One per map; calling it again updates it.
<code>add_legend(title=None, legend_dict=, labels=, colors=, builtin=, position=, shape=)</code>	Add a legend from a <code>{label: color}</code> dict, parallel <code>labels / colors</code> , or a builtin preset (<code>"nlcd"</code> , <code>"esa_worldcover"</code>).

Method	Description
<code>add_colorbar(colormap=, vmin=, vmax=, label=, units=, colors=, orientation=, position=)</code>	Add a colorbar for a continuous raster, from a named colormap or custom <code>colors</code> .
<code>add_colormap(colormap, vmin=, vmax=, label=, **kwargs)</code>	Add a colorbar from a named colormap (leafmap-style alias of <code>add_colorbar</code>).
<code>set_center(lng, lat, zoom=None)</code>	Center (and optionally zoom) the map.
<code>set_center_zoom(lng, lat, zoom=None)</code>	Alias of <code>set_center</code> (leafmap compatibility).
<code>set_zoom(zoom) / set_bearing(bearing) / set_pitch(pitch) / fit_project_bounds(bounds)</code>	Persist camera changes without requiring the widget to be displayed.
<code>center / zoom / bearing / pitch / basemap / name</code>	Read persisted project and camera state; <code>name</code> is writable.
<code>rename_layer(layer, name) / move_layer(layer, index) / duplicate_layer(layer, name=) / show_layer(layer) / hide_layer(layer)</code>	Manage layers by id, name, or <code>Layer</code> handle.
<code>layer_properties(layer) / column_values(layer, column) / describe()</code>	Inspect inlined data and summarize a project without a browser round trip.
<code>remove_layer(layer_id) / clear_layers()</code>	Remove one layer by id, name, or handle, or remove all layers.
<code>set_popup(layer, fields=None, click=, hover=, title=, title_expression=, body_expression=, show_feature_id=, max_width=, image_height=, tooltip=, merge=False)</code>	Choose what a click popup shows for a layer, and how each value is formatted.
<code>set_tooltip(layer, fields=True) / clear_popup(layer)</code>	Turn a hover tooltip on (or off), or drop the popup config and restore the default popup.

Method	Description
<pre>set_layer_metadata(layer, merge=False, title=, abstract=, keywords=, license=, attribution=, contact=, lineage=, temporal_extent=, links=) / clear_layer_metadata(layer)</pre>	Describe a layer for catalogs (the app's Metadata dialog, its STAC Item export and GeoParquet export metadata). Emails, dates (ISO 8601) and link URLs are validated. <code>Layer.descriptive_metadata</code> reads it back.
<pre>set_layer_filter(layer, expression)</pre>	Hide a layer's features that do not match a boolean MapLibre expression, e.g. <code>[">=", ["get", "pop"], 100000]</code> (the filter Select by Expression → Filter layer saves). <code>None</code> clears it.
<pre>set_labels(layer, field=None, expression=None, enabled=None, **options)</pre>	Label features from a property or a text expression. Options: <code>placement</code> , <code>size</code> , <code>color</code> , <code>halo_color</code> , <code>halo_width</code> , <code>min_zoom</code> , <code>max_zoom</code> , <code>allow_overlap</code> , <code>anchor</code> , <code>offset_x</code> , <code>offset_y</code> , <code>rotation</code> , <code>max_width</code> , <code>transform</code> , <code>number_format</code> , <code>number_decimals</code> , <code>number_locale</code> , <code>dedupe</code> , and the data-defined <code>size_expression</code> , <code>color_expression</code> , <code>opacity_expression</code> , <code>visibility_expression</code> , <code>priority_expression</code> . Settings left out keep their values.
<pre>set_plugin_state(plugin_id, state=None, position=, activate=True, allow_unknown=False, clear=False)</pre>	Store a plugin's saved project state (the blob it restores on open) for a built-in id in <code>geolibre.project.PLUGIN_STATE_IDS</code> , or an external plugin's id with <code>allow_unknown=True</code> . <code>state</code> must be plain JSON; omit it to change only <code>position</code> / <code>activate</code> . <code>clear=True</code> removes the stored state and leaves activation alone.
<pre>set_story_map(**settings)</pre>	Set the story map 's <code>title</code> , <code>subtitle</code> , <code>byline</code> , <code>footer</code> , <code>theme</code> , <code>show_markers</code> , <code>marker_color</code> , <code>inset</code> , <code>inset_position</code> , <code>hide_chapter_nav</code> , <code>start_slide</code> , <code>end_slide</code> .
<pre>add_story_chapter(title, description=, center=, zoom=, pitch=, bearing=, image=, alignment=, hidden=, map_animation=, rotate_animation=, on_enter=, on_exit=, index=)</pre>	Add a story chapter; camera values left out come from the saved view. <code>on_enter</code> / <code>on_exit</code> fade layers: <code>[{"layer": "Cities", "opacity": 1, "duration": 800}]</code> .
<pre>move_story_chapter(chapter, index) / remove_story_chapter(chapter)</pre>	Reorder or drop a chapter by id, title, or 0-based index.
<pre>add_bookmark(name, center=, zoom=, pitch=, bearing=, folder=, visible_layers=)</pre>	Save a view to the Bookmarks panel; camera values left out come from the saved view. <code>folder</code> is a folder id or name (a new name creates it); <code>visible_layers</code> lists the layers (ids, names, or handles) to show when it is opened.
<pre>remove_bookmark(bookmark)</pre>	Drop a bookmark by id, name, or 0-based index.

Method	Description
<code>set_identify(layer="all")</code>	Arm the Identify tool so a click opens the popup: on one layer (id, name, or handle), on a list of layers, on every visible layer ("all"), or off (None).
<code>show_control(name, visible=True) / hide_control(name)</code>	Show or hide a toolbar panel (bookmark , search , measure , minimap , print) or a built-in map control (navigation , fullscreen , compass , geolocate , globe , scale , attribution , logo).
<code>set_projection(projection) / projection</code>	Draw the map as a "globe" (the default) or flat "mercator" map; saved in the project.
<code>to_project(keep_credentials=False)</code>	Return the current project as a dict, credentials redacted unless <code>keep_credentials=True</code> .
<code>load_project(src)</code>	Replace the project from a dict, JSON string, or <code>.geolibre.json</code> path.
<code>save_project(path, keep_credentials=False)</code>	Write the current project to a <code>.geolibre.json</code> file, credentials redacted unless <code>keep_credentials=True</code> .

Style keyword arguments (for example `fillColor`, `strokeColor`, `strokeWidth`, `circleRadius`) map to the GeoLibre [layer style fields](#).

Marker symbology

`add_marker` and `add_markers` take the common point-symbology settings as named arguments, so you do not have to know the underlying style keys:

```
m.add_markers(points, color="#e11d48", radius=8, stroke_color="#ffffff", stroke_width=2)
m.add_markers(points, shape="pin", color="#e11d48", size=32)
m.add_markers(points, icon='<svg viewBox="0 0 24 24">...</svg>', size=28)
```

A point layer draws two ways. By default it is a MapLibre circle sized by `radius`. Passing `shape`, `size`, or `icon` switches it to a **marker sprite**: one of `circle`, `square`, `triangle`, `diamond`, `star`, `cross`, `pin`, or `custom` (which needs `icon`, raw SVG markup or a data URL).

The two modes take different settings, and mixing them is an error rather than a silent no-op. A sprite layer replaces the circle layer outright and draws its own white halo, so `opacity`, `radius`, `stroke_color`, and `stroke_width` are circle-only and are rejected when `shape` / `size` / `icon` is also given — use `size` for a sprite's size. A sprite's `color` must be a hex color, because the sprite baker accepts nothing else and would otherwise fall back to the default blue in silence.

Where a named argument and its underlying style key are both passed (`add_markers(pts, radius=8, circleRadius=20)`), the raw style key wins — it is the low-level escape hatch.

Popups and tooltips

Without any configuration, clicking a feature while Identify is armed shows the layer name and every visible property, and there is no hover tooltip. Every `add_*` method that takes style overrides accepts `popup=` and `tooltip=` to change that (the exception is `add_ee_layer`, which has a fixed signature), and `set_popup` / `set_tooltip` / `clear_popup` change it on a layer that already exists.

```

m.add_markers(
    points,
    popup={
        "title": "name",          # heading, instead of the layer name
        "fields": [
            {"field": "name", "label": "Site"},
            {"field": "photo", "kind": "image", "label": "Photo"},
            {"field": "url", "kind": "link", "link_label": "Read more"},
            {"field": "pop", "kind": "number", "thousands": True, "suffix": " people"},
            {"field": "surveyed", "kind": "date", "date_format": "datetime"},
        ],
    },
    tooltip="name",
)

m.set_popup("Sites", ["name", "pop"], title="name") # replace the config
m.set_tooltip("Sites", ["name"])                  # add a hover tip
m.set_popup("Sites", click=False)                  # no popup on click
m.clear_popup("Sites")                             # back to the default

```

Popups open only while the Identify tool is armed, which in the app is the Identify button on a layer or the "Identify visible layers" button in the Layers panel header. `set_identify` arms it from Python, and can run before the map is displayed. Identify covers one layer, a list of layers, or every visible layer, and hover tooltips pause while it is armed:

```

m.set_identify()          # every visible layer
m.set_identify("Sites")   # one layer
m.set_identify(["Sites", "Roads"]) # only these layers
m.set_identify(None)      # off

m.show_control("bookmark") # open the Bookmarks panel
m.show_control("search")   # open the place search box
m.hide_control("globe")    # hide the globe/flat toggle button
m.set_projection("mercator") # draw a flat map instead of a globe

```

`save_project`, `to_project` and `to_html` save the Identify target and the controls in the project's `interaction` block, so the saved file or HTML page opens the same way, and `load_project` reads them back. `set_projection` is saved as a project preference.

`popup=` also accepts shorter forms: a single property name (`popup="name"`), a list of names (`popup=["name", "pop"]`), a list of field mappings, or `False` to suppress the click popup. The full mapping form above takes the same keys as `set_popup` — `fields`, `click`, `hover`, `title`, `title_expression`, `body_expression`, `show_feature_id`, `max_width`, `image_height`, `tooltip` — and rejects a key it does not know, so a misspelling is an error rather than a setting that quietly does nothing. Those keys belong *inside* `popup=`; passed to `add_markers` directly they would be taken for style keys. The two sizes are the exception: `popup_max_width=` and `popup_image_height=` work as top-level arguments on every `add_*` method too, for when a wider popup is the only change you want. `tooltip=` takes a property name, a list of names, `True` to put every configured popup field in the tip, or `False` to turn it off.

A field's `kind` decides how the value renders:

kind	Renders as
auto (default)	Text, except an inline base64 raster data URL, which becomes a thumbnail.

kind	Renders as
text	Text, with <code>prefix</code> / <code>suffix</code> applied.
number	A localized number; <code>decimals</code> , <code>thousands</code> , <code>prefix</code> , <code>suffix</code> .
date	A localized date; <code>date_format</code> is <code>date</code> , <code>datetime</code> , <code>time</code> , <code>iso</code> , or <code>year</code> .
link	An <code>http(s)</code> value becomes a link, labelled <code>link_label</code> .
image	An <code>http(s)</code> value or inline base64 raster data URL becomes a thumbnail.

Sizing the popup and its pictures

The popup is 520 px wide by default (420 px when it carries a picture, which it lets you drag wider), and a picture inside it draws at most `min(50vh, 420px)` tall. `max_width` and `image_height` change both, in CSS pixels:

```
m.add_markers(
    sites,
    shape="pin", color="#e11d48", size=32,
    popup={
        "title": "name",
        "max_width": 640,      # 288-1200; the viewport still caps it
        "image_height": 420,   # 40-1200
        "fields": [
            {"field": "photo", "kind": "image", "label": "Photo"},
            {"field": "url", "kind": "link", "link_label": "Read more"},
        ],
    },
    tooltip="name",
)

# The same two settings as top-level arguments, on any add_* method:
m.add_markers(sites, popup=["name", "photo"], popup_max_width=640, popup_image_height=420)

# Or on a layer that already exists (merge=True keeps the fields):
m.set_popup("Sites", max_width=640, image_height=420, merge=True)
```

A thumbnail keeps its aspect ratio, so a landscape photo needs the **width** to grow before it can use the extra height — raise `max_width` alongside `image_height`. A size outside the range the app renders is an error rather than a value silently clamped on the map, and clicking a picture still opens it full-size in a lightbox whatever the popup's own size is.

Two rules worth knowing before you port a popup from another library:

- **Raw HTML in a property is not rendered as markup, and neither is Markdown.** A popup value that arrives from a GeoJSON file is untrusted, so it is written as text rather than parsed. Use `kind="image"` and `kind="link"` for pictures and links, and `body_expression` (a [MapLibre expression](#), as JSON text) when you want a composed sentence instead of a table: `body_expression='["concat", ["get", "name"], " – ", ["get", "county"], " County"]'`. The one exception is a KML `description` property, whose known markup is sanitized and rendered, so a converted KML keeps its description card.

- **A tooltip needs fields flagged for hover, and flagging them narrows the click popup.** The two share one field list: the tooltip shows the entries flagged for hover, and the click popup shows *every* entry — but only falls back to "all visible properties" while that list is empty. So

```
m.add_markers(points, tooltip="name")    # click popup now shows ONLY name
```

because naming a tooltip field creates the list. To keep the full click popup, list the fields you want on click as well, and flag one for hover:

```
m.add_markers(points, popup=["name", "pop", "county"], tooltip="name")
```

A tooltip that could never show anything is an error rather than a tip that silently never appears — including one whose only flagged field is an `image`, since images are dropped from tooltips (their value is a URL, which would become the whole tip). The click popup still shows the picture.

Use in marimo

`marimo` can render GeoLibre's anywidget, but its browser may not be able to reach the random `127.0.0.1` port where GeoLibre normally serves the bundled app. The symptom is an iframe displaying `127.0.0.1 refused to connect`. Point the widget at GeoLibre's hosted app before displaying it:

```
from geolibre import Map

m = Map(center=(-100, 40), zoom=4)
m._app_url = "https://web.geolibre.app/"
m.add_basemap("dark")
m.add_vector(
    "https://data.source.coop/giswqs/opengeos/world_cities.geojson",
    name="World cities",
)
m
```

This uses the same project-sync bridge as the regular widget; only the app's location changes. Set `_app_url` before returning `m` from the cell so the iframe uses the hosted URL on its first render.

The example uses `add_vector()` so the hosted browser app fetches the remote GeoJSON directly. `add_geojson(url)` instead downloads and inlines the file in Python, which can fail when a data host rejects Python's HTTP client.

Because the hosted app cannot access files exposed by the kernel's temporary localhost server, use hosted URLs for rasters and other sources the browser loads directly. Local GeoJSON, CSV, and vector files that GeoLibre reads in Python and inlines into the project continue to work. The `_app_url` attribute is currently an internal compatibility workaround rather than a public constructor option.

Privacy: The widget sends its synchronized project, including any inlined local data, to the origin in `_app_url` through `window.postMessage`. Use only a trusted app URL, or host the GeoLibre app yourself, when working with sensitive data.

How it works

The wheel bundles the GeoLibre web build. At import time the package starts a small localhost static server that serves the bundled app; the widget renders that app in an iframe and exchanges the project over `window.postMessage`. Adding data from Python rewrites the synced project and pushes it into the app; UI edits flow back the same way.

Environment support

The interactive widget works in **local Jupyter, VS Code, Google Colab, JupyterHub / remote servers, and marimo**:

- **Local Jupyter / VS Code** - the app is served directly from localhost.
- **Google Colab** - routes through Colab's built-in port proxy (`google.colab.kernel.proxyPort`) automatically.
- **JupyterHub** (including managed/shared hubs, detected at runtime via `JUPYTERHUB_SERVICE_PREFIX`) - the front-end probes two same-origin routes and uses whichever is live, so a host needs only **one** of them:
 - the Jupyter Server extension bundled with `geolibre` , mounted at `{base_url}geolibre/app/` on the notebook server's own origin. It is enabled automatically on `pip install geolibre` and needs no `jupyter-server-proxy` and no extra port, so it works on locked-down hubs that block raw-port proxying -- but it only registers after the Jupyter server restarts, since it loads from a startup config drop-in.
 - `jupyter-server-proxy` at `{base_url}proxy/{port}/` , which reaches the kernel's localhost bundle in the **running** server with no restart, wherever `jupyter-server-proxy` is installed.
- **Other remote servers** (Binder, remote JupyterLab over SSH/network) - pass `Map(server_proxy=True)` to use that same dual-route remote path.
- **marimo** - use the hosted app URL shown in [Use in marimo](#); Jupyter's proxy and server-extension routes are not available in marimo.

Set `Map(server_proxy=False)` to force the direct localhost path. If the app fails to load on a hub, either install `jupyter-server-proxy` , or confirm the extension is enabled with `jupyter server extension list` (look for `geolibre`; run `jupyter server extension enable geolibre` if absent) and **restart** the Jupyter server so the extension loads.

URL fetching

`add_geojson(url)` , `add_csv(url)` / `add_xy_data(url)` , and `add_wfs()` fetch the URL from the **kernel**, following redirects, so a notebook can reach any host the kernel can. Every hop is checked, and a URL that resolves to a non-public address (private, loopback, or link-local — including cloud metadata endpoints such as `169.254.169.254`) is refused; responses are capped at 50 MB. Tile and service layers are fetched by the **browser** instead, so those can still point at a local server. Do not load untrusted `.geolibre.json` projects or URLs on a shared/multi-tenant kernel.

MCP server

The same package ships an **MCP** server that authors `.geolibre.json` projects from an AI client, with no notebook and no running app involved:

```
pip install "geolibre[mcp]"
geolibre-mcp --root ~/maps
```

It builds projects through the same builders this package uses, so anything it writes opens in the widget (and in the desktop and web apps) unchanged. See [MCP server](#) for the tool list and client configuration.

Building from source

The package lives in `python/`. The bundled app is produced from the monorepo with:

```
npm run build:embed      # builds the app and stages it into the wheel
python -m build           # builds the wheel
python -m twine upload dist/* # upload to PyPI
pip install -e python     # editable install for development
```

Changes to the Python code are picked up on kernel restart. Changes to the app (TypeScript) require re-running `npm run build:embed` and restarting the kernel.

Rendering engines and mixed pane layouts

```
m = Map(renderer="cesium", center=(-100, 40), zoom=4)
m.set_map_layout(1, 2, view_kinds=["cesium", "maplibre"], sync_view=True)
pane_id = m.project["secondaryMapView"][0]["id"]
m.set_renderer("cesium", pane_id=pane_id)
assert m.get_renderer() == "cesium"
```

Renderer choices are `"maplibre"`, `"mapbox"`, `"cesium"`, and `"arcgis"`. Omitting `pane_id` targets the primary map. Grid dimensions are 1–4; `view_kinds` contains one renderer per pane, primary first. Existing pane IDs, cameras, and visibility overrides survive layout resizing. Save the project normally to preserve `primaryRenderer` and each secondary pane's `viewKind`.

`DashMap(renderer="cesium")` selects the same initial renderer; Dash callbacks can update these fields through `project`.

<https://geolibre.app/r/>

R package

CRAN **0.2.0**  **R-CMD-check** **passing**

The `geolibre` R package embeds the full GeoLibre map in RStudio, Quarto, R Markdown, and Shiny through `htmlwidgets`. It creates standard `.geolibre.json` projects, so maps can move between R and the GeoLibre web or desktop application.

[View the R package website](#)[Open the interactive example](#)[Browse the source](#)

Installation

Install the released version from [CRAN](#):

```
install.packages("geolibre")
```

Or the development version from GitHub:

```
install.packages("pak")
pak::pak("opengeos/geolibre-r")
```

Quick start

Create an interactive map from GeoJSON and set its initial camera:

```
library(geolibre)

places <- list(
  type = "FeatureCollection",
  features = list(list(
    type = "Feature",
    properties = list(name = "Washington, DC"),
    geometry = list(
      type = "Point",
      coordinates = c(-77.0369, 38.9072)
    )
  ))
)

geolibre(map_only = TRUE) |>
  add_geojson(
    places,
    name = "Places",
    style = list(fillColor = "#dc2626", circleRadius = 8)
  ) |>
  set_view(center = c(-77.0369, 38.9072), zoom = 10)
```

The widget is responsive in the RStudio Viewer and rendered HTML documents. Its map loads the hosted GeoLibre application, so viewing it requires an internet connection unless `app_url` points to a self-hosted deployment.

Spatial data with `sf`

`add_sf()` accepts an `sf` object and transforms it to WGS 84 before passing GeoJSON to the browser:

```
library(sf)

nc <- st_read(system.file("shape/nc.shp", package = "sf"), quiet = TRUE)

geolibre() |>
  add_sf(nc, name = "North Carolina counties")
```

The `sf` package is optional. On Linux, installing it may require system libraries such as GDAL, GEOS, PROJ, and UDUNITS.

Remote rasters

GeoLibre reads remote Cloud Optimized GeoTIFFs directly in the browser. The server hosting the raster must support CORS and HTTP range requests.

```
geolibre() |>
  add_raster(
    "https://example.org/visual.tif",
    name = "Satellite image",
    bands = c(1, 2, 3)
  )
```

Project files

Save the widget state as a portable GeoLibre project and restore it later:

```
map <- geolibre() |> add_geojson(places)
save_project(map, "example.geolibre.json")

restored <- geolibre(load_project("example.geolibre.json"))
```

The saved project can also be opened in GeoLibre Web or GeoLibre Desktop. In a Shiny application, browser-side changes are available through `input$<outputId>_project`.

Shiny

Use `geolibreOutput()` and `renderGeolibre()` to render a map. A proxy updates the live widget without rebuilding the output:

```
library(shiny)
library(geolibre)

ui <- fluidPage(
  geolibreOutput("map", height = "80vh"),
  actionButton("reset", "Reset view")
)

server <- function(input, output, session) {
  map <- reactiveVal(geolibre(map_only = TRUE))
  output$map <- renderGeolibre(map())

  observeEvent(input$reset, {
    next_map <- set_view(map(), center = c(0, 20), zoom = 2)
    map(next_map)
    update_geolibre(geolibre_proxy("map"), next_map)
  })
}

shinyApp(ui, server)
```

Self-hosting

Set a custom GeoLibre application URL for one widget or for the R session:

```
geolibre(app_url = "https://gis.example.org/")
options(geolibre.app_url = "https://gis.example.org/")
```

The deployment must expose the GeoLibre web app and its `?embed=1` project bridge. See the [package reference](#) for the complete API.

<https://geolibre.app/mcp/>

MCP server

GeoLibre ships an [MCP](#) server that authors GeoLibre projects. Point an MCP client (Claude Desktop, Claude Code, or any other) at it and you can ask for a map in words: the server writes a real `.geolibre.json` project you open in the desktop app, the web app, or the `geolibre` Jupyter widget, and can export it as a standalone HTML page.

The file tools are **headless**. They need no browser, no running GeoLibre instance, and no bundled web build. They build project files with the same [project builders](#) the Python package uses, so a project they write is byte-for-byte the kind the app already loads.

The `live_*` tools are the other half. They speak the desktop Notebook relay ([Notebook panel](#)), so a command from an MCP client moves the map already open in GeoLibre Desktop. Open **Processing → Jupyter Notebook** once per app launch; the panel can be closed after that. Web and JupyterLite have no relay, so those tools have nothing to attach to there.

Install

The MCP SDK is an optional extra:

```
pip install "geolibre[mcp]"
```

Run it

```
geolibre-mcp --root ~/maps
```

The server speaks MCP over stdio, which is what desktop clients spawn. The `--root` flag is repeatable, and `GEOLIBRE_MCP_ROOTS` (: -separated, ; on Windows) does the same job from the environment. With neither set, the workspace is the current directory. `--transport` switches from the default `stdio` to `streamable-http` or `sse` for clients that connect over HTTP.

Client configuration

Claude Desktop (`claude_desktop_config.json`) and most other clients take the same shape:

```
{
  "mcpServers": {
    "geolibre": {
      "command": "geolibre-mcp",
      "args": ["--root", "/Users/you/maps"]
    }
  }
}
```

For Claude Code:

```
claude mcp add geolibre -- geolibre-mcp --root ~/maps
```

If `geolibre-mcp` is not on the client's `PATH` (common when it was installed into a virtualenv), give the interpreter instead:

```
{
  "mcpServers": {
    "geolibre": {
      "command": "/path/to/venv/bin/python",
      "args": ["-m", "geolibre.mcp", "--root", "/Users/you/maps"]
    }
  }
}
```

The workspace

Every path in every tool call is resolved against the allowed roots before the server touches it, mirroring `GEOLIBRE_CONVERSION_ROOTS` in the [sidecar](#). Paths outside them are refused, and so is a symlink inside a root that points out of it. Relative paths resolve against the first root, so a client can say `city.geolibre.json` without knowing the host layout.

Three more guards on writes: the server only writes files ending in `.json` (projects) or `.html` (exports) — a bare `.json` with no name is refused too — it refuses to replace an existing file unless the call passes `overwrite`, and a tool that edits an existing project first checks the file actually is one, so an unrelated `package.json` sitting inside a root cannot be rewritten as a map.

Give it a directory meant for maps, not your home directory.

Tools

Project lifecycle

Tool	What it does
<code>create_project</code>	Write a new, empty project with a name, center, zoom, and basemap.
<code>describe_project</code>	Summarize the camera, basemap, layers, and map controls. Inlined feature data is reported as a count, never echoed back.
<code>get_point_cloud_annotation_s</code>	Per point cloud, how many points the app's annotator relabelled into each class and how many points each instance holds, plus the custom classes, 3D vectors and saved 3D boxes (with status and attributes).
<code>set_point_cloud_classes</code>	Define the annotator's custom classes (codes 19-255, name, <code>#rrggbb</code> color); keeps existing labels and boxes.

Tool	What it does
<code>prelabel_point_cloud</code>	Run the app's Whitebox pre-label classifiers on a local copy of a LiDAR layer and save the changed classes as its labels (needs <code>geolibre[pointcloud]</code>).
<code>write_labeled_point_cloud</code>	Write a local copy of a LiDAR layer's LAS/LAZ/COPC file with the project's labels and instance ids applied (needs <code>geolibre[pointcloud]</code>).
<code>list_catalog</code>	List the named basemaps, color ramps, legend presets, and the plugin ids <code>set_plugin_state</code> accepts, plus the active workspace roots.

Adding layers

Tool	For
<code>add_geojson_layer</code>	Vector data inlined into the project, from a URL, a workspace file, or literal GeoJSON. Self-contained, and the only kind <code>classify_layer</code> can style.
<code>add_vector_layer</code>	A large remote FlatGeobuf / GeoParquet / GeoJSON read in place.
<code>add_raster_layer</code>	A Cloud Optimized GeoTIFF, with band, colormap, and rescale options.
<code>add_tile_layer</code>	A raster XYZ tile template.
<code>add_tiles_layer</code>	PMTiles archives and vector tile services.
<code>add_ogc_layer</code>	WMS and WMTS endpoints.
<code>add_lidar_layer</code>	LAS/LAZ/COPC/EPT point clouds by URL.
<code>add_3d_tiles_layer</code>	OGC 3D Tiles tilesets, by URL or Cesium Ion asset id.
<code>add_cesium_ion_layer</code>	Cesium Ion assets (tileset or imagery) by id; rendered by the 3D globe only.
<code>add_czml_layer</code>	A CZML (Cesium Language) dynamic 3D scene, by URL or inline packets; rendered by the 3D globe only.
<code>add_cesium_kml_layer</code>	Native globe KML/KMZ from a URL, inline XML, or KMZ data URL, preserving document styles and overlays.

Editing

Tool	What it does
<code>update_layer</code>	Rename, show/hide, set opacity, or reorder.
<code>remove_layer</code>	Drop a layer.
<code>style_layer</code>	Merge style keys (<code>fillColor</code> , <code>strokeWidth</code> , <code>circleRadius</code> , ...).

Tool	What it does
<code>set_layer_popup</code>	Choose the fields a click popup shows, their labels and formats, its width and image height, and an optional hover tooltip.
<code>set_layer_metadata</code>	Describe a layer for catalogs (title, abstract, keywords, license, attribution, contact, lineage, temporal extent, links): what the app's Metadata dialog edits and exports as a STAC Item.
<code>classify_layer</code>	Build a graduated choropleth from a numeric column.
<code>list_layer_properties</code>	List a layer's feature properties with sample values.
<code>set_layer_filter</code>	Hide the features that do not match a boolean MapLibre expression (the saved filter Select by Expression → Filter layer writes); omit the expression to clear it.
<code>set_labels</code>	Label features from a property or a text expression, with size, colors, halo, zoom range, anchor, and the rarer settings under <code>options</code> . Settings left out keep their values.

Layers are addressed by id **or** by display name, so a client can work from what `describe_project` showed it without tracking UUIDs.

Framing and decoration

Tool	What it does
<code>set_view</code>	Set center, zoom, bearing, and pitch, or pass a <code>bbox</code> to frame an area.
<code>set_basemap</code>	Switch the background style.
<code>set_map_legend</code>	Show the map legend, built from the layers' own symbology.
<code>add_legend</code>	Add a legend from a preset, a <code>{label: color}</code> map, or paired lists.
<code>add_colorbar</code>	Add a colorbar for continuous data.
<code>add_swipe</code>	Configure the split-map comparison slider.

Plugin state and story maps

Tool	What it does
<code>set_plugin_state</code>	Store a plugin's saved project state (the blob it restores on open, such as the Time Slider's timeline). Built-in ids come from <code>list_catalog</code> ; an external plugin's id needs <code>allow_unknown</code> .
<code>set_story_map</code>	Set a story map's title block, theme, markers, inset, and start/end slides.
<code>add_story_chapter</code>	Add a chapter with its camera, text, image, alignment, animation, and layer fades. A camera value left out comes from the project's saved view.
<code>move_story_chapter</code> / <code>remove_story_chapter</code>	Reorder or drop a chapter by id, title, or index.

Tool	What it does
<code>add_bookmark</code>	Add a saved view to the Bookmarks panel, optionally in a folder (a new folder name creates it) and with the layers to show when it is opened. A camera value left out comes from the project's saved view.
<code>remove_bookmark</code>	Drop a bookmark by id, name, or index.

The shapes match what the app reads (`parseProject` in `@geolibre/core`); a round-trip test loads a project these tools wrote through it and checks nothing is dropped or rewritten.

Live desktop map

Tool	What it does
<code>live_status</code>	Report whether a GeoLibre Desktop window is listening.
<code>live_list_layers</code>	List the open map's layers.
<code>live_fly_to</code> / <code>live_fit_bounds</code> / <code>live_zoom_to_layer</code>	Move the camera.
<code>live_set_basemap</code>	Switch the basemap.
<code>live_add_geojson</code> / <code>live_remove_layer</code>	Add or remove a layer.
<code>live_set_visibility</code> / <code>live_set_opacity</code> / <code>live_set_style</code>	Change a layer.
<code>live_list_algorithms</code>	List the app's processing algorithms and their parameters, optionally filtered by <code>query</code> .
<code>live_run_algorithm</code>	Run an algorithm on the open map; returns its logs and the ids of the result layers it added.

Processing is live-only. The algorithm catalog and the engines that run it (Turf.js, DuckDB-WASM) live in the app, so there is nothing a file tool could run. The relay waits about five seconds for a result: a longer run keeps going in the app, and the tool reports that it did not finish in time instead of its result layer ids, so check `live_list_layers` before running it again.

Export

`export_html` writes a standalone page that embeds the hosted GeoLibre viewer and injects the project into it, so the recipient needs no install. Credentials are stripped from the project on the way out. Layers pointing at local files will not load for anyone else, so use hosted URLs for a shareable export.

app_url is a trust boundary

`export_html`'s optional `app_url` names the viewer the exported page embeds, and the page posts the project to exactly that origin (it must be an `http / https` URL). It exists so you can pin a self-hosted deployment.

Treat it as a destination, not a cosmetic setting: whoever opens the exported file hands the project's contents — inlined GeoJSON, layer URLs, the camera — to that origin. Credentials are already stripped, so this is not a key leak, but the rest of the project still travels.

This matters because the caller here is a model, which may be acting on content it has read. If an exported page points somewhere you did not choose, that is worth a second look. Only accept an `app_url` you intended.

Notes and limits

- **set_view with a bbox is approximate.** A saved project stores a center and zoom, and the app applies those verbatim on load rather than fitting a stored bbox. The server therefore resolves the box to a camera itself, using an assumed map-pane size, and lands within roughly half a zoom level of what the app's own "zoom to layer" would pick. Pass `center` and `zoom` when you need exact framing.
- **Inlined GeoJSON is capped at 50 MB**, and a project file the server reads at 256 MB. Past those, use `add_vector_layer` or a tiled source.
- **Remote fetches are checked:** a URL whose host resolves to a private, loopback, or link-local address is refused, on every redirect hop as well as the first request, so a crafted URL cannot reach a cloud metadata endpoint.
- **File tools write a project.** `live_*` **tools move the open desktop map.** A live edit is in the session until you save it from the app. The relay is loopback-only and refuses redirects, so the Jupyter token stays on the machine. The same scripting surface backs the [Python widget](#) and the [embed API](#); those remain how a host page or a notebook drives a map without MCP.

Driving the server from an agent

GeoLibre ships an [agent skill](#) that teaches an external AI agent how to use these tools well: which `add_*_layer` tool fits which data, what order to call things in, and the limits above. Install it from `skills/geolibre/` in the repository.

Under the hood

The tools are thin wrappers over `geolibre.authoring`, a widget-free module of operations on project dicts (add/remove/restyle a layer, move the camera, compose the map controls). `geolibre.Map` delegates to the same module, so the notebook widget and the MCP server cannot drift apart in how they build a project.

Renderer and pane authoring

Use `set_renderer(path, "cesium")` to open a project on the globe. `set_map_layout(path, 1, 2, view_kinds=["cesium", "maplibre"])` creates a mixed grid and returns the secondary panes, with their IDs, as `secondaryMapViewes`. Pass one as `pane_id` to `set_renderer` to change only that pane. Camera tools continue to use longitude/latitude and the shared zoom, bearing, and pitch convention. The accepted renderer names are `maplibre`, `mapbox`, `cesium`, and `arcgis`.

<https://geolibre.app/agent-skill/>

Agent skill

GeoLibre ships an **agent skill** — a `SKILL.md` that teaches an external AI coding agent (Claude Code, Claude Desktop, or any runtime that loads skills) how to build GeoLibre maps on your behalf.

It lives in the repository at `skills/geolibre/`.

What it is for

GeoLibre already has an in-app [AI Assistant](#), which chats with your data *inside* the running app and applies changes through the app's own store. The skill covers the other direction: an agent working **outside** GeoLibre — in your terminal, your editor, or a notebook — that should produce a GeoLibre map as its output.

	AI Assistant	Agent skill
Runs	Inside the app	In your agent, outside the app
Acts on	The live map you are looking at	A <code>.geolibre.json</code> file on disk
Needs	A configured AI provider	An agent that loads skills
Output	Edits you can undo with Ctrl/Cmd + Z	A project file and/or a standalone HTML page

They compose: an agent authors the project, you open it in GeoLibre and keep working with the Assistant. The MCP server's `live_*` tools are a third path: they move the map already open in Desktop, through the Notebook relay. See [MCP server](#).

What it teaches

- **Which entry point to use** — the [MCP server](#) for a chat session, the [Python package](#) for notebooks and scripts, hand-written [project JSON](#) when nothing is installed, and the [embed API](#) for a live instance in a page.
- **The authoring workflow** — create a project, add layers, frame the camera, style or classify, add a legend or colorbar, export a shareable HTML page.
- **The rules that are easy to get wrong** — that `classify_layer` needs inlined GeoJSON, that a local file path does not travel to a recipient, that a `bbox` resolves to a camera approximately, that basemap and color-ramp names should come from `list_catalog` rather than a guess, and that `export_html`'s `app_url` is a trust boundary.

Install

```
git clone --depth 1 https://github.com/opegeos/GeoLibre /tmp/geolibre
mkdir -p ~/.claude/skills
cp -r /tmp/geolibre/skills/geolibre ~/.claude/skills/
```

Use `~/.claude/skills/` inside a project instead of `~/.claude/skills/` to scope it to one repository. Start a new agent session afterwards; the skill loads itself when a request matches.

For the shortest path, install the MCP server the skill prefers:

```
pip install "geolibre[mcp]"
claude mcp add geolibre -- geolibre-mcp --root ~/maps
```

The skill still works without it — it falls back to the Python API and to writing project JSON directly.

Other agent runtimes

`SKILL.md` is plain Markdown with YAML frontmatter, so most agent runtimes that support skills can load it as-is. If yours has no skill loader, paste the file into your system prompt or `AGENTS.md` — it is written to stand on its own, and its `references/` files are only read when a task needs them.

<https://geolibre.app/notebook/>

Notebook panel

GeoLibre can run a Jupyter notebook **beside the map**, in a resizable, collapsible panel (open it from **Processing → Jupyter Notebook**). It works in both the web and desktop builds, with a different Python runtime behind the same `<iframe>` panel:

- **Web build → JupyterLite**. A full Jupyter UI running entirely in the browser on a [Pyodide](#) (WebAssembly) kernel — no server. Install extra packages on demand from a cell with `%pip install geopandas`.
- **Desktop build → a real JupyterLab server**. The app launches `jupyter lab` from a uv-managed Python environment (the same mechanism as the FastAPI sidecar) on `127.0.0.1:8766`, token-authenticated, and embeds it. This gives full CPython with the native geospatial stack (geopandas, rasterio, GDAL, ...). The Mac App Store build is the exception: it cannot spawn the server, so it embeds the JupyterLite site like the web build.

This is distinct from the [geolibre Python package](#), which does the inverse — embedding the *whole GeoLibre app* inside a notebook cell.

Scripting the map from a cell

A lightweight `geolibre` client is preloaded into the notebook kernel — on web it is bundled into JupyterLite's filesystem, so **no `pip install` is needed**; on desktop it is placed on the kernel's import path. Just import it:

```
import geolibre

m = geolibre.connect()          # or geolibre.Map()
m.fly_to(-122.4, 37.8, zoom=11) # animate the live map in the left pane
layer_id = m.add_geojson(
    gdf,
    name="My layer",
    fillColor="#facc15",
    strokeColor="#d97706",
) # GeoDataFrame, dict, or JSON string; returns an id on desktop
m.get_layer(layer_id)
m.list_layers()
m.fit_bounds([-123, 37, -122, 38])
m.set_basemap("https://.../style.json")
```

Most mutation calls are **fire-and-forget**. On desktop, `add_geojson` uses the relay's correlated request/reply path and returns the new layer id. The same path exposes `list_layers()`, which returns one dict per live layer with `id`, `name`, `type`, `visible`, and `opacity`, and `get_layer(layer_id)`, which returns one matching layer or raises `ValueError`. The id can be passed directly to `set_visibility`, `set_opacity`, `set_style`, `remove_layer`, or `zoom_to_layer`.

All commands fan out to every GeoLibre window connected to the same Jupyter server. For `add_geojson`, `list_layers`, and `get_layer`, the first correlated reply is returned to the notebook and later replies are ignored. Because each window maintains its own map state, use one connected window when chaining a returned layer id into later read-back calls. This behavior is only visible if you attach multiple GeoLibre windows to one Jupyter server.

`add_geojson` returns `None` instead of an id in two situations, and never fails outright in either:

- **Nothing received the command** (no window connected, or the relay itself unreachable). It behaves like every other mutation — the layer goes out over the display transport with a `GeoLibreNotConnectedWarning`.
- **A window took it but did not answer within 5s**, which a large `FeatureCollection` can do. You get a `GeoLibreTimeoutWarning`; the layer is still being added, so it is deliberately *not* re-sent (that would add it twice) — find it with `list_layers()`.

The read-back calls have nothing to return in either situation, so they raise `GeoLibreNotConnectedError` / `GeoLibreTimeoutError` (both `RuntimeError`) instead.

Synchronous read-back is desktop-only. JupyterLite uses browser `postMessage`; blocking its Python call would also block the browser event loop that must deliver the result. Canonical client source: `backend/geolibre_server/notebook_client.py`.

Driving the map from an external client (VS Code, ...)

The desktop app's JupyterLab server is a normal, token-authenticated Jupyter server, so you can attach any Jupyter client to it and keep your own editor — and the map commands above still work. Open the Notebook panel once (that is what starts the server), then use its **link button** in the panel header to copy the connection URL and paste it into your client (in VS Code: *Jupyter: Specify Jupyter Server for Connections* → *Existing*).

This works because commands travel over a **relay** on the Jupyter server rather than depending on how the notebook is being *displayed*:

- `backend/geolibre_server/geolibre_server/jupyter_relay.py` is a Jupyter Server extension (enabled from `jupyter_server_config.py`) exposing `POST .../geolibre/relay/command`, a `.../geolibre/relay/socket` `WebSocket`, and `GET .../geolibre/relay/status`. At load it publishes `GEOLIBRE_RELAY_URL` and `GEOLIBRE_RELAY_TOKEN` into the server's environment, which **kernels inherit** — that is how `import geolibre` finds the map with no configuration.
- The app subscribes to that socket for its whole lifetime (`useJupyterRelay`), reconnecting with backoff, and runs each command against the same `createScriptingHandlers` surface.
- `useNotebookBridge` remains the `postMessage` path for the **embedded** panel and for web (JupyterLite), where the notebook page really is the app's `iframe`.

The POST answers with how many app windows received the command, so a disconnected session is reported instead of silently doing nothing:

```
geolibre.is_connected() # False -> no GeoLibre window is listening
```

When a command cannot be delivered the client raises a `GeoLibreNotConnectedWarning` pointing at your own line. Promote it to an error with:

```
import warnings, geolibre
warnings.simplefilter("error", geolibre.GeoLibreNotConnectedWarning)
```

Only the desktop server has the relay: on web (JupyterLite) there is no server to attach an external client to.

Theme

On the **web build** the notebook follows the app's light/dark theme live (no reload). JupyterLite is built with `exposeAppInBrowser` (see `apps/geolibre-desktop/jupyterlite/jupyter-lite.json`), which puts its app object on `window.jupyterapp`; since the JupyterLite iframe is same-origin, `useNotebookThemeSync` reaches in and runs the built-in `apputils:change-theme` command whenever the app theme changes. On the **desktop build** the JupyterLab server is a different origin, so this cross-origin call is a no-op and the notebook keeps its own theme (syncing it there would need a small JupyterLab extension — a future enhancement).

Building the web JupyterLite site

The web build embeds a self-hosted JupyterLite site under `apps/geolibre-desktop/public/jupyterlite/`. It is **not** committed (≈ 70 MB) and is generated by a best-effort prebuild step that needs the `jupyter lite` CLI:

```
pip install -r apps/geolibre-desktop/jupyterlite/requirements.txt
npm run build:jupyterlite
```

Both `npm run dev` and `npm run build` run this automatically:

- `npm run dev` builds it **once** on first run via a `predev` hook (`--if-missing`), then is instant on subsequent runs.
- `npm run build` always rebuilds it (via `prebuild`) so a changed client/config is picked up.

Both **skip with a warning** when `jupyter lite` is not installed, so a Node-only build still succeeds. Be aware of what that produces: the panel does *not* show a "not built" message. Its iframe asks for `/jupyterlite/lab/index.html`, and anything that answers an unknown path with `index.html` — Tauri's asset resolver, or a static host with an SPA fallback such as `try_files $uri /index.html` — hands the panel a **second copy of GeoLibre** instead (GeoLibre#1851, GeoLibre#1658). Install the deps above and rebuild.

Two builds therefore treat a missing CLI as fatal rather than skippable, because they serve the site and cannot degrade: the Mac App Store build, and any build that sets `GEOLIBRE_JUPYTERLITE_REQUIRED=1` (the Docker image does). The Mac App Store build cannot spawn a JupyterLab server, so its Notebook panel embeds the JupyterLite site instead. Every other desktop (Tauri) dev and build path skips it entirely (they use the real JupyterLab server), so the static site never bloats the installer.

The Docker image builds and serves the site, and gives `/jupyterlite/` its own block in `docker/nginx.conf`:

- Its own CSP — JupyterLab bootstraps from an inline `<script>`, which the app's policy forbids, so under the app policy the site loads its HTML and then never boots.
- No SPA fallback (`try_files $uri $uri/ =404`), so in this image the failure above surfaces as a plain **404** rather than as the duplicated app. The duplicate remains what Tauri and SPA-fallback hosts produce.
- The long-lived immutable cache policy for the content-hashed assets, which the prefix match would otherwise take away from them.

Build config lives in `apps/geolibre-desktop/jupyterlite/` (a `jupyter_lite_config.json`, a `jupyter-lite.json`, and the build `requirements.txt`). The build stages the kernel client and the starter `Welcome.ipynb` into its `files/` from their canonical copies under `backend/geolibre_server/` (`notebook_client.py`, `notebook_examples/Welcome.ipynb`). The generated directory is excluded from the PWA precache (see `pwaPlugin` in `apps/geolibre-desktop/vite.config.ts`).

Desktop server

- Backend extra: `notebook` in `backend/geolibre_server/pyproject.toml` (`jupyterlab`, `jupyter-server`), synced into its own uv project environment so it never disturbs the sidecar's env.
- Launcher / lifecycle: `start_jupyter_server` / `stop_jupyter_server` Tauri commands in `apps/geolibre-desktop/src-tauri/src/lib.rs` (mirrors the sidecar launcher), with the TS wrapper in `apps/geolibre-desktop/src/lib/jupyter.ts`.
- Framing: `backend/geolibre_server/jupyter_server_config.py` relaxes Content-Security-Policy: `frame-ancestors` to the Tauri webview / loopback origins so the app can embed the server; the Tauri CSP (`tauri.conf.json`) adds the loopback origins to `frame-src` / `child-src`.
- Map-command relay: the same config enables the `geolibre_server.jupyter_relay` server extension (see above). Its WebSocket accepts only the app's own origins and every endpoint requires the server's per-launch token, so a command can only come from something that already has kernel-execution rights there.

<https://geolibre.app/roadmap/>

GeoLibre Roadmap

v0.1: Map viewer and GeoJSON

- [x] Tauri + React + MapLibre shell
- [x] GeoJSON load, layer panel, style panel
- [x] Attribute table (basic)
- [x] Processing UI with local algorithms
- [x] Plugin interface + sample plugins

v0.2: Project persistence

- [x] `.geolibre.json` save/open
- [x] In-session recent project tracking
- [x] Feature highlight from attribute table
- [x] Optional zoom to selected feature
- [x] Recent projects UI and persistence

v0.3: Cloud-native formats

- [x] GeoParquet import through DuckDB-WASM
- [x] FlatGeobuf import through DuckDB-WASM and URL-based Components plugin panel
- [x] PMTiles through Components plugin
- [x] COG and GeoTIFF raster rendering
- [x] Zoom to layer for GeoJSON and source-bounds-aware layer types

v0.4: DuckDB Spatial

- [x] DuckDB-WASM integration
- [x] `INSTALL spatial / LOAD spatial`
- [x] Shapefile, KMZ/KML, GeoPackage, GeoParquet, FlatGeobuf, GML, and related vector import paths

v0.5: Advanced Add Data and plugin-backed layers

- [x] Add Data dialogs for XYZ, WMS, vector files, GeoJSON URLs, vector tiles, raster tile templates, COG and GeoTIFF rasters, MBTiles, and ArcGIS layers

- [x] MapLibre Components plugin with FlatGeobuf, PMTiles, Zarr, LiDAR, and Gaussian splat panels
- [x] Desktop MBTiles metadata and tile reads through Tauri commands
- [x] Plugin control position controls in the Plugins menu
- [x] Layer control integration for GeoLibre-managed layers

v0.6: Project access, web embeds, and expanded integrations

- [x] Persistent recent projects with desktop file recents and URL-backed web recents
- [x] Separate Open Project from File and Open Project from URL flows
- [x] Browser demo query options for compact layout, icon-only toolbar, and hidden panels
- [x] PostgreSQL layer workflow through desktop Martin server integration
- [x] STAC search workflow for adding catalog-backed raster layers
- [x] Esri Wayback, GeoAgent, GeoEditor, Street View, and Swipe plugin integrations

v0.7: Add Data expansion, identify, settings, and processing

- [x] GPX loading from URL or local file, with selectable waypoint, track, and route layers
- [x] Delimited text loading from URL or local file using longitude and latitude fields
- [x] WFS GetFeature loading through the Add Data dialog
- [x] WMS GetFeatureInfo identify support with hardened popup handling
- [x] Whitebox toolbox backed by a managed Python sidecar
- [x] Inline attribute editing, horizontal table scrolling, and scrollable identify popups
- [x] Settings dialog for map preferences and runtime environment variables
- [x] Plugin state persistence in project files
- [x] Default GeoJSON sample URL and larger identify popup
- [x] Local raster file loading fix
- [x] Large-file pre-commit guard

v0.8: Viewer, desktop packaging, plugins, and dynamic layers

- [x] Cloudflare Worker viewer served from `web.geolibre.app`
- [x] Browser demo links updated to the production viewer
- [x] GPX drag-and-drop split into named waypoint, track, and route layers
- [x] Vector layers reprojected to EPSG:4326 on load
- [x] Desktop About dialog update check
- [x] Dynamic external plugin zip loading from the app data plugins directory
- [x] Safe fallback for `crypto.randomUUID` in non-secure contexts
- [x] External plugin manifest support with `plugin.json`
- [x] 3D Tiles layer support through `maplibre-gl-3d-tiles`
- [x] 3D Tiles restoration when reopening projects
- [x] GeoParquet panel DuckDB startup fix

- [x] MSIX desktop packaging and cleaner build output
- [x] External native GeoJSON layers registered from local directories
- [x] Raster basemaps registered as external native layers
- [x] Text marker labels rendered on GeoJSON layers
- [x] Manual and automatic refresh for WFS and GeoJSON URL layers
- [x] Multiple DuckDB SQL query-result layers
- [x] Desktop diagnostics panel and improved diagnostics/status bar contrast
- [x] Toolbar toggles for Colorbar, Legend, and HTML panels

v0.9: Data integrations, processing, and menu reorganization

- [x] SQL Workspace for running DuckDB Spatial SQL against loaded layers, local files, and remote URLs, with sample queries, query history, and adding results to the map or exporting them
- [x] Planetary Computer panel for browsing and loading STAC data
- [x] Earth Engine panel for browsing and loading datasets
- [x] Overture Maps plugin for loading Overture data themes
- [x] Time Slider plugin for animating time series raster and vector data, powered by `maplibre-gl-time-slider`
- [x] Web Services menu with four federal data plugins
- [x] Add Raster Layer powered by the `maplibre-gl-raster` plugin
- [x] Add Vector Layer powered by the `maplibre-gl-vector` plugin
- [x] Identify, selection, and attribute table support for DuckDB layers
- [x] Conversion menu under Processing for Vector to GeoParquet/FlatGeobuf/PMTiles, CSV to GeoParquet, and Raster to COG, backed by a hardened conversion sidecar with a path allowlist
- [x] Vector menu under Processing with common geometry tools (buffer, centroids, convex hull, dissolve, bounding box, simplify, clip, intersection, difference, union) running client-side with Turf.js, plus an optional GeoPandas sidecar engine
- [x] Raster menu under Processing with common raster tools (hillshade, slope, aspect, reproject, resample, clip by extent, clip by mask layer, polygonize, contour) backed by a rasterio sidecar, path in and path out
- [x] Drag and drop vector and GeoTIFF/COG raster files onto the map to add them as layers
- [x] Whitebox batch tools run against a selected input directory
- [x] Controls menu with Measure, Bookmark, Minimap, and View State tools
- [x] Print menu backed by `PrintControl`
- [x] Project menu consolidating New, Open, Save, and Save As
- [x] Layout settings with per-panel visibility toggles
- [x] Insert before dropdown for placing layers in the stack
- [x] Component panels persisted and controls reset on new project
- [x] Plugins can declare and handle URL query parameters
- [x] `maponly` query parameter for chrome-free map embeds
- [x] `theme` query parameter to set the initial light/dark theme for embeds
- [x] Docker support for the browser app

- [x] `VITE_DUCKDB_SPATIAL_EXTENSION_PATH` for offline spatial extension loading

v1.0: Processing pipelines, external plugin system, and stable prototype

- [x] GDAL / Rasterio / GeoPandas pipelines
- [x] Buffer, reproject, and export GeoJSON processing tools
- [x] Expanded WhiteboxTools coverage
- [x] External plugin package distribution workflow
- [x] Plugin marketplace / registry design (see [Plugin marketplace and registry](#))
- [x] Plugin marketplace MVP: curated registry plus browse and install UI
- [x] Plugin update (in-place re-fetch) and uninstall with confirmation
- [x] Project menu Share action that uploads to [share.geolibre.app](#) using a personal API token
- [x] Python package (`geolibre`) for Jupyter notebooks: embeds the full app as an [anywidget](#) with a leafmap-style API (`add_geojson` , `add_tile_layer` , `add_cog`) and two-way `.geolibre.json` project sync
- [x] Performance tuning and test suite
- [x] Cross-platform installers
- [x] Documentation and tutorials

v1.1: Vector styling, attribute table management, and atmosphere effects

- [x] In-browser GeoPandas engine for the Vector tools via Pyodide (no server, same results as the optional sidecar)
- [x] Host `deck.gl` exposed to external plugins via `app.getDeckGL()` , so plugins render on the shared instance instead of bundling their own copy
- [x] Style panel support for Add Vector Layer (`maplibre-gl-vector`) layers, including single, categorized, graduated, and expression symbology applied to the control's native layers
- [x] Rename layers from the layer panel by double-clicking the name or from the layer actions menu
- [x] Open attribute table and Export actions added to the layer actions menu
- [x] Manual and automatic (timed) refresh extended to Add Vector Layer URL layers
- [x] Attribute table column management: rename, delete, hide/show, and reorder fields, persisted with the project
- [x] Atmospheric Effects plugin: deep-space backdrop, parallax starfield, comets, and a globe atmosphere halo at low zoom (toggled from the Controls menu)
- [x] conda-forge install instructions and a video tutorial in the documentation
- [x] MIT license
- [x] CSP allowance for `cdn.jsdelivr.net` so DuckDB-WASM loads its bundles in the browser build

v1.2: New data sources, attribute analytics, routing, and platform polish

- [x] OpenStreetMap PBF file loading parsed in-browser with osmix
- [x] Cloud-Optimized NetCDF/HDF layers loaded via kerchunk references
- [x] Authenticated 3D Tiles tilesets via custom request headers
- [x] Georeferenced video overlay layers
- [x] Deck.gl Layer builder for composing deck.gl overlays from uploaded files and remote URLs
- [x] In-browser PostGIS SQL engine via PGLite, alongside the DuckDB Spatial SQL Workspace
- [x] Attribute table add-field and field-calculator tools
- [x] Attribute table Charts panel (histogram, scatter, bar, line, box)
- [x] Spatial join added to the Vector tools
- [x] Select by value and Select by location tools
- [x] H3 tools to create hexagonal grids and bin points into H3 cells
- [x] Point heatmap renderer and clustering, including for Add Vector Layer point layers
- [x] Directions plugin for interactive routing via `maplibre-gl-directions`, with a one-time privacy notice before enabling it
- [x] Undo/redo for layer and style operations
- [x] Command palette (`Ctrl / Cmd + K`) and global keyboard shortcuts with a `?` cheat sheet
- [x] Print layout composer with PNG and PDF export
- [x] Installable, offline-capable Progressive Web App (PWA) build
- [x] Internationalization framework (react-i18next) with extracted string catalogs and a `?` `locale / ?lang` embed language parameter
- [x] Accessibility pass with axe checks across key screens
- [x] App, section, and plugin React error boundaries
- [x] Playwright end-to-end smoke tests and a CI job
- [x] Expanded Python `Map` API covering more Add Data layer types
- [x] CDN-loaded PGLite/PostGIS to shrink the Jupyter wheel and the desktop binary

v1.3: Analysis depth, real-time collaboration, story maps, scripting, and an AI assistant

- [x] Spatial Statistics toolbox under Processing
- [x] Vector tools: Smooth, Regular grid, and Voronoi/Delaunay
- [x] IDW / kriging interpolation (point layer → continuous raster surface)
- [x] Attribute (table) join vector tool, joining a table's fields by a matching key
- [x] Raster analysis tools: zonal statistics, raster calculator, reclassify, mosaic, and focal statistics
- [x] Client-side raster processing fallback that runs in the browser when the Python sidecar is unavailable
- [x] Single-band pseudocolor with classification and RGB band combination for raster styling
- [x] Batch run and model/pipeline chaining for processing tools
- [x] Network analysis: isochrones, service areas, and origin-destination cost matrices
- [x] Collapsible layer groups/folders in the layer panel

- [x] glTF/GLB 3D model layers placed at coordinates
- [x] Client-side vector tiling for large local vector layers
- [x] Warning before loading very large vector files
- [x] Transparent rewrite of public S3, GCS, and Azure cloud-storage URLs in SQL queries
- [x] Shapefile and GeoPackage export
- [x] Apache Sedona as an additional SQL Workspace engine
- [x] Batch geocoding and reverse geocoding tools, with a multi-provider geocoding abstraction
- [x] Real-time multi-user collaboration (MVP) backed by a Cloudflare Worker
- [x] Scroll-driven story map builder, presenter, and standalone HTML export
- [x] User-editable legend for the print layout
- [x] Field statistics summary panel in the attribute table
- [x] AI Segmentation toolbox via [segment-geospatial](#) (SamGeo) and Meta's SAM 3, proxied through the sidecar to a separate `samgeo-api` model server
- [x] Natural-language GIS assistant (Strands agent) that turns plain-English requests into auditable, undoable GeoLibre operations
- [x] Python automation API and an in-app Python Console
- [x] Python package: local raster, marker/cluster, and choropleth APIs; `split_map`, `add_legend`, and `add_colorbar` helpers; typed read-back of selected/drawn features; and `to_html` export
- [x] Homebrew Cask packaging for macOS
- [x] Native Android app from the same codebase via Tauri v2 mobile, with a CI workflow that builds signed, per-ABI release APKs (~40 MB) — see [Android](#)
- [x] `isMobile()` feature-gating that hides desktop-process tools (Whitebox, Raster, Conversion, AI Segmentation, PostgreSQL/Martin) on Android so nothing is shown that cannot run
- [x] Responsive, touch-friendly mobile layout: Layers/Style panels overlay the map as slide-over sheets on phones, pointer-event (touch) panel resizing, and safe-area insets so the toolbar clears the system status bar
- [x] Download Offline Area tool that pre-caches the current map view's basemap tiles into the service-worker cache
- [x] Service-worker caching of the CDN-loaded Pyodide and PGLite/PostGIS engines so browser SQL and Python keep working offline after first use

v1.4: Jupyter beside the map, spectral indices, georeferencing, and field collection

- [x] Resizable, collapsible Notebook panel docked beside the map: the web build embeds a self-hosted JupyterLite site (in-browser Pyodide kernel) and the desktop build launches a uv-managed JupyterLab server, both seeded with a runnable Welcome tour. Notebook cells drive the live map through the shared scripting bridge via an auto-loaded `geolibre` client, and the JupyterLite theme follows the app theme — see [Notebook Panel](#)
- [x] Spectral Index toolbox under Processing → Raster (NDVI, GNDVI, NDWI, NDMI, NDBI, NBR, EVI, and SAVI) with Sentinel-2, Landsat 8-9, NAIP, and custom band layouts and a reflectance-scale knob, evaluated client-side via `geotiff.js` or on the rasterio sidecar through the existing raster calculator

- [x] Raster Georeferencer (Processing → Georeferencing): pin a non-georeferenced image to the map with ground control points, using a least-squares affine fit and per-GCP and RMS residuals, added as a corner-pinned overlay that persists in the project and works offline
- [x] Field Collection tool (Controls menu) for capturing point, line, and polygon observations with a per-layer custom form (text/number/date/choice fields and an optional inline photo), placed by device GPS or by tapping the map, with a floating quick-open control; captures are written to a tagged GeoJSON layer that flows into the attribute table, export, and offline use
- [x] Runtime overrides for `VITE_PYODIDE_INDEX_URL` and `VITE_DUCKDB_SPATIAL_EXTENSION_PATH` through the existing runtime-environment system, so air-gapped or corporate deployments can point Pyodide and the DuckDB Spatial extension at internal mirrors without rebuilding the app

v1.5: Dashboards, in-browser Whitebox, map navigation history, and signed macOS installers

- [x] Dashboard panel of chart widgets that summarizes the loaded layers at a glance, with configurable charts and a collapsible layout that docks alongside the map
- [x] Whitebox toolbox now runs entirely in the browser through a WebAssembly runtime with raster I/O, so its tools (and GeoLibre's own WASM raster tools, now surfaced in the same toolbox) work with no Python sidecar
- [x] Print layout composer gained an explicit map-scale input, a title block with editable title and footer, page-size controls, and a custom print extent, with more reliable preview rendering for production-quality PNG and PDF map exports
- [x] Time Slider can bind existing vector layers already on the map to the timeline, animating their time-series attributes without re-importing the data
- [x] New View menu with viewport history navigation (step back and forward through previous map extents), a reset pitch and bearing control with a rotation indicator, a distinct north arrow in place of the ambiguous compass, a lock indicator when map bounds are restricted, and a refined default set of top-right map controls
- [x] Saved service library for web-service layers, so frequently used WMS, WFS, XYZ, and ArcGIS endpoints can be stored once and re-added with a click rather than re-entered each session
- [x] Add Data dialog is now fully internationalized and accepts comma decimal separators and drag-and-dropped CSV files for coordinate data
- [x] Inspect COG pixel values directly from the Identify icon, plus reversed and custom raster color ramps, the full colormap list, and a styling panel you can reopen from the layer actions menu
- [x] Customizable UI profiles that tailor which menus, panels, and data sources are visible, so a deployment can present a focused subset of the app to its users (see [UI Profiles](#))
- [x] Bookmarks now capture the active layers alongside the camera, with selectable export, a resizable and reorderable panel, and a save-as name prompt
- [x] Sequential route (directions) network tool under Processing for computing routes through an ordered set of waypoints

- [x] Protomaps basemaps in the New project dialog, support for stacking multiple raster basemaps, and basemap element visibility presets in the layer control
- [x] Spinning Globe panel under Atmospheric Effects and customizable atmosphere halo and deep-space colors for the globe view
- [x] Notebook panel can split the workspace 50/50 with the map and auto-collapse the Style panel for more room, and the attribute table gains a column explorer for finding and toggling fields in wide tables
- [x] USGS LiDAR plugin replaces the previous LiDAR Viewer for browsing and loading USGS 3DEP point-cloud data, imported KML and KMZ layers honor their embedded symbology, and vector strokes can be sized in scale-proportional meters
- [x] macOS desktop installers are now signed with an Apple Developer ID certificate and notarized by Apple, so they open without a Gatekeeper workaround, with a generator for submitting GeoLibre to the official Homebrew cask (see [Downloads](#))

v1.6: Multi-map layouts, advanced symbology and labeling, and plugin zip install

- [x] Multi-map grid that splits the workspace into a grid of map views with synchronized camera movement, so you can compare basemaps, layers, or time steps side by side
- [x] Advanced vector symbology with a rule-based renderer (filter-driven style rules), proportional symbols, fill patterns, and a built-in marker library for richer point and polygon styling
- [x] Label engine that labels vector features by any attribute, with placement and styling controls
- [x] Install external plugins from an uploaded zip on both desktop and web, alongside the existing manifest-URL and bundled drop-in paths, with the Manage Plugins list now sorted alphabetically
- [x] New vector analysis tools under Processing → Vector for movement, space-time, and cell-coverage analysis
- [x] Sample-data dropdowns standardized across every Add Data dialog, so each upstream-backed panel offers ready-to-load example datasets
- [x] Search places box in the Layers panel footer for geocoding to a location without leaving the panel
- [x] Accent color schemes beyond light and dark, so the UI theme can be tinted to a chosen accent color
- [x] Swap the core basemap by double-clicking it in the layer panel
- [x] Story map presenter gained a Reset button and auto-collapses the side panels while presenting for a cleaner full-screen story
- [x] Progressive Share setup that separates the website and local token steps for a clearer first-time configuration
- [x] Interactive sidecar help banners for the Whitebox toolbox and AI Segmentation that guide you when the Python sidecar is not running or fails to start

- [x] Guided update workflow with a startup update check, update preferences, and clearer status colors
- [x] Time Slider defaults an unspecified end date to the current date
- [x] Windows Package Manager (winget) packaging as `OpenGeos.GeoLibre`, so the app can be installed and updated through winget
- [x] [Microsoft Store](#) listing, so Windows users can install the signed, auto-updating build directly from the Store (see [Downloads](#))

v1.7: Plugin UI host API, color ramp previews, and category-browsed Whitebox tools

- [x] Plugin UI host API that lets plugins register first-class right-sidebar panels, toolbar menus, and floating panels that dock beside the built-in Style panel instead of emulating an overlay, with external plugin toolbar menus now placed after the Help menu (see [Plugin API](#))
- [x] Color ramp gradient swatches in both the vector Style panel and the raster Color ramp picker, so you can see each colormap's gradient inline (on the trigger and beside every option) while choosing rather than picking from a plain list of names
- [x] Richer vector labeling with ArcGIS-style controls (anchor, X/Y offset, rotation, wrap width, and letter case), a Duplicate labels option, and unique/concatenate modes that collapse points stacked at the same coordinate into a single deduplicated label
- [x] Whitebox toolbox is now browsable by category directly in the Processing menu, with nested subcategory submenus, GeoLibre's own WASM tools grouped under their own subheading, an offline-bundled tool catalog for restricted environments, and tools that open the dialog preselected and scrolled into view
- [x] On-canvas collaboration session-status badge and roster (a pulsing live dot, connected-participant count, and an expandable client list that announces joins and leaves), plus a clear "Go to map and collaborate" button so the host has a non-destructive way back to the map
- [x] Welcome wizard is suppressed for embeds: project deep links (`?url=`) skip onboarding automatically, and a new `?welcome=0` parameter lets any embed opt out

v1.8: Camera tours, live story maps, standalone HTML export, and map annotations

- [x] Record an animated camera tour to video straight from the Controls menu, with a clearer keyframe layout, per-keyframe recapture, a two-step save, and the ability to save and reload the entire tour setup as a JSON file
- [x] Story Map plugin can now compose its chapters directly on the live map instead of a separate editor, and generates a printable PDF handout of the finished story
- [x] Export a project to a single standalone interactive HTML file that runs offline with no server, plus a project gallery for browsing and opening shared projects with one click
- [x] Map annotation layer for drawing text, arrows, and highlights directly on the map, persisted with the project

- [x] Gridlines overlay (renamed from Graticule) draws a coordinate grid with edge labels across the map
- [x] Import geotagged photos as a point layer from their EXIF GPS, with manual placement and drag-to-position for photos that have no embedded coordinates
- [x] GeoRSS feed support in Add Data, loaded from a URL or a local file
- [x] EOX Sentinel-2 cloudless satellite imagery and Openbasiskaart added to the basemap library (Openbasiskaart via basemap-control 0.7.0)
- [x] Right-click context menu on the map for reading coordinates and reaching quick actions without leaving the canvas
- [x] Per-participant permissions and an in-app chat panel for real-time collaboration sessions
- [x] Organize bookmarks into folders for tidier, grouped saved views
- [x] Dedicated AI Providers section in Settings with per-feature provider dropdowns for choosing and configuring AI backends
- [x] Plugin API can now register native raster and tile layers, and offers a shared-rail (replace-style) right-panel dock mode for plugin panels (see [Plugin API](#))
- [x] Time Slider draws a pixel time-series chart for raster stacks, plotting a sampled pixel's value across the timeline
- [x] Colorbar panel gains a stacking direction control, a resizable panel, and a stack-order fix for multiple colorbars
- [x] Persistent mode banner for the Directions and Reverse Geocode tools so the active interaction mode stays visible
- [x] Copy to Clipboard added to the Print Layout composer for pasting the rendered map straight into other apps
- [x] Clearer Set view coordinate input workflow with support for degrees-decimal-minutes (DDM) entry
- [x] Raster paint controls gain a greyscale toggle, a reset action, and numeric inputs, plus info icons explaining layer zoom-visibility controls
- [x] Inline numeric opacity input in the layer control, with a fixed-name notice on the Background layer
- [x] Windows portable zip build, so the desktop app can run without installation

v1.9: CAD import, smarter service discovery, and a docked SQL workspace

- [x] Add CAD drawings (DXF/DWG) as a layer, with a layer picker for choosing which drawing layers to load and a CRS selector for placing the geometry correctly on the map
- [x] WMS and WFS panels now read the service's GetCapabilities document to list the available layers and feature types, so you pick from a populated dropdown instead of typing layer names by hand
- [x] Generic Vector to Vector conversion tool that converts between any supported vector formats by file extension, alongside the existing targeted converters
- [x] SQL Workspace docks as a resizable panel beside the map (rather than a floating window) and gains editor autocomplete for tables, columns, and SQL keywords

- [x] Camera tours gain per-keyframe hold and transition duration controls for finer pacing, plus the ability to save and reload a named tour setup
- [x] Story Map plugin adds a hide-itinerary toggle, subtitle and byline fields on the printable handout, and dedicated start and closing slides for a more polished presentation
- [x] Transparent fill and outline option in the color picker, so features can be styled with no fill or no stroke without leaving the picker
- [x] Plugins can now use the maplibre-gl-raster stack and the projection control, expanding what external plugins can render and configure (see [Plugin API](#))
- [x] Legend populates automatically from a paletted raster's embedded color table, matching the map colors without manual entry
- [x] Website and GitHub links added to the Help menu for quick access to the project home and source

v1.10: I3S scene layers, OGC API vector tiles, and local NetCDF/HDF loading

- [x] Add ArcGIS I3S scene layers (Integrated Mesh and 3D Object layers) as a data source, streamed and rendered in 3D on the shared deck.gl instance through a Tile3DLayer with the I3SLoader
- [x] Add Data now supports OGC API - Tiles vector tile services as a remote source, so standards-based vector tile endpoints can be added alongside XYZ and ArcGIS vector tiles
- [x] Load local HDF5 and NetCDF-4 files directly from disk, extending the NetCDF/HDF workflow beyond Cloud-Optimized references to files already on your machine
- [x] GeoEditor plugin can pull the vector features currently visible in the map view into the editor, so you can start editing what you are looking at without re-importing the source data

v2.0: A 3D globe, planetary mapping, symbology interchange, and editable source layers

- [x] Switch any map pane to a **CesiumJS 3D globe** view that stays camera-synced with the 2D maps and mirrors the layer stack, adding a true photorealistic globe alongside the flat workspace (a Cesium Ion token is optional, and adds terrain plus Ion World Imagery as the fallback for a basemap with no raster form)
- [x] Planetary mapping with the OpenPlanetaryMap basemap set (Mars and the Moon) plus USGS Astrogeology basemaps for Mercury, Venus, the Galilean moons, Titan, Pluto, and Charon reprojected to Web Mercator, a per-project ellipsoid whose radius drives distance/area/scale, and a planet switcher in the Layers panel, plus an expanded EOX Maps catalog and dark-mode-aware basemap theming
- [x] Import and export vector layer symbology as OGC SLD, QGIS QML, and Mapbox GL style JSON, so styles round-trip between GeoLibre, QGIS, and the Mapbox/MapLibre ecosystem
- [x] Edit vector layers and write the changes back to their source, covering GeoPackage and GeoJSON files as well as PostGIS database tables

- [x] New Weather menu with live cloud and precipitation radar overlays (RainViewer), a Clouds overlay in the Controls menu, and a Google Earth-style sun position simulation for realistic lighting
- [x] Richer KML/KMZ support: render GroundOverlay images as map overlays (animated through the Time Slider when time-tagged) and display embedded Collada (.dae) 3D models
- [x] Render vector layers that carry Z coordinates in true 3D rather than flattening them onto the ground plane
- [x] Extract COG, WMS, and XYZ bounding-box subsets directly in the browser, and build a normalized-difference index from any HTTP COG
- [x] New built-in plugins: a Mapillary coverage and street-level image viewer, a Historical Imagery panel, and an Elevation Profile tool, plus a UTM easting/northing grid mode for the Gridlines overlay
- [x] Field Calculator can compute geometry length and area, and the attribute table supports Ctrl- and Shift-click multi-row selection
- [x] Google Earth-style camera-reset keyboard shortcuts, View in Google Maps and View in Google Earth actions, and a double-click terrain control for setting vertical exaggeration
- [x] Import delimited text (CSV) without coordinates as a standalone attribute table
- [x] All 13 locale catalogs completed, with the remaining hardcoded panel and dialog strings migrated to the translation system
- [x] The AI assistant can read provider API keys from OS environment variables, and the desktop diagnostics network log now captures native Tauri HTTP requests and classifies failed `fetch()` errors

v2.1: A data-source Browser panel, route animation, in-browser object detection, and map recording

- [x] A QGIS-style **Browser panel** (Data Source Manager) for exploring and adding data from one place: browse map Services and Recent items, connect to PostGIS databases and browse their schemas and tables, drill into local files, save and reopen Favorites, add a New connection per service kind, and navigate the whole tree from the keyboard
- [x] **Route animation**: animate a marker along any line layer, follow the track in 3D with camera controls, and export the whole animation as an MP4 video
- [x] **In-browser object detection** that runs ONNX/YOLO models directly in the webview, no server or Python required
- [x] Record the map canvas (or a drawn bounding box) to a video file straight from the browser
- [x] A true native-resolution viewer for geotagged photos, so full-detail imagery stays crisp on the map
- [x] Wikipedia knowledge cards: click a place on the map to pull up its Wikipedia summary and info card
- [x] USGS planetary basemaps for nine more celestial bodies, extending the planetary mapping catalog
- [x] New OpenAerialMap plugin for searching open aerial imagery, plus a GEBCO ocean-bathymetry WMS sample with proper attribution

- [x] New Source Cooperative plugin for browsing source.coop open data: search the catalog, walk a product's files, and add GeoParquet, PMTiles, COG, GeoJSON, and FlatGeobuf straight to the map (streamed from the source) or download them
- [x] The Whitebox toolbox is now a floating panel, and Processing subset tools can populate their bounding box from the current map view
- [x] Scale bar switches between metric, imperial, and nautical units
- [x] Bundled plugin drop-ins can set `activeByDefault` in their manifest, deployments can opt out of the welcome dialog, and the app now defaults to the Advanced interface and skips the welcome dialog on first run
- [x] Optional HTTP Basic Auth for the web (Docker) container

v2.2: A styling overhaul, expression-driven fields and labels, print atlas, and browser-native conversions

- [x] A **rule-based renderer** with per-rule symbol properties, scale-dependent visibility, nested rules, and per-rule toggles, so a single layer can carry a full hierarchy of styling rules and hide anything that matches no rule when the else rule is off
- [x] A **Style Manager** that saves reusable symbol, color-ramp, and label presets to a personal library and applies them across projects
- [x] A symbology pack covering inverted-polygon masks, arrow and marker lines, and geometry generators (with attribute-driven centroid size and buffer distance), plus data-driven proportional sizing for marker icons
- [x] **Diagram symbology**: draw pie, donut, and bar charts on features straight from their attributes
- [x] A **data-defined labeling engine** with expression-driven label properties, placement priority, and full control over how labels are drawn
- [x] A shared **Expression Builder** — a function reference, searchable field list, live preview, and reusable variables — wired into filters, labels, styling, and selection
- [x] **Virtual fields**: expression-backed computed columns that update as the underlying data changes, plus a Raster Attribute Table for single-band categorical rasters
- [x] An **attribute form designer** with edit widgets, validation constraints, and conditional field visibility for structured data entry
- [x] **Persistent attribute joins** configured in layer properties, so a table can enrich a layer's features and stick across sessions
- [x] **Select by Expression** and **Select by Location**, plus live query layers backed by DuckDB SQL that re-run as the data updates
- [x] **Atlas / map series** in the Print Layout — generate one page per feature, or a uniform series of pages along a line such as a river or trail — and drop attribute-table and chart blocks onto the page
- [x] Browser-native format conversion for COG, FlatGeobuf, Shapefile, and GeoPackage, and Vector to PMTiles running on a background worker, no Python sidecar required
- [x] **Live GPS tracking**: a moving position marker, a recorded track log, and digitizing new features straight from the GPS feed

- [x] Data quality tools to check validity, fix geometries, and check topology rules, catching and repairing bad geometries before they cause trouble
- [x] A **Processing History** panel that lists every tool run, re-runs it with one click, and copies the equivalent Python code
- [x] Timelapse mode now animates EOX Sentinel-2 cloudless annual basemaps and NASA GIBS providers (Landsat/WELD and MODIS land cover) with a provider picker and legend
- [x] Desktop gains OS trust store and mTLS client-certificate support for native HTTP, automatic reload when local files change on disk, and Esri File Geodatabase (`.gdb`) layer support
- [x] New Natural Earth and Source Cooperative data browsers under Plugins > Web Services, including opening or streaming large GeoParquet from Source Cooperative
- [x] Terrain-aware 3D measurements in the Measure tool, optional title/source captions and on-map panel capture (HTML, legend, colorbar) in video recordings, and a new Georgian locale alongside full Arabic right-to-left support

v2.3: An on-map Legend panel, iOS and Google Play, the GeoLens catalog, and space-time hot spots

- [x] An auto-generated **Legend panel** on the map that derives itself from the visible layers' symbology — per-class rows for graduated, categorized, rule-based, and expression styling, gradient bars for heatmaps and continuous raster colormaps, proportional-symbol size ramps, diagram fields, and land-cover labels read from a Raster Attribute Table — with an edit mode for renaming, hiding, and reordering entries, adding a section from a color dictionary, choosing a corner, collapsing sections, resizing the panel, and exporting the rendered legend as JSON, all saved with the project and shared with the Print Layout legend
- [x] **Symbology swatches in the Layers panel**: every row carries a dot, line, square, or image glyph colored from the layer's own styling, dimmed when the layer is hidden, so a tall layer stack reads at a glance
- [x] A new **GeoLens catalog browser** plugin for connecting to a self-hosted [GeoLens](#) server, searching its catalog, and adding datasets as signed vector tiles, OGC API Features GeoJSON, or server-rendered raster tiles, with automatic tile-token refresh and a Metadata link back to each dataset's page
- [x] **iOS support**, scaffolded end to end — Tauri iOS configuration, location permissions, a signing and TestFlight CI workflow, and a publishing guide. See [iOS](#)
- [x] The **Android build is Google Play-ready**: a permanent `org.geolibre.app` package id, API level 36, 16 KB page-size alignment verified in CI, and a signed universal App Bundle published alongside the per-architecture sideload APKs. See [Android](#)
- [x] **Emerging Hot Spot Analysis**: aggregate timestamped points into a space-time cube, run Getis-Ord G_i^* per time slice, and classify every cell as a new, intensifying, persistent, diminishing, sporadic, oscillating, or historical hot or cold spot, entirely in the browser
- [x] The Time Slider animates **mosaic sources** — a MosaicJSON or STAC collection of many COGs per date — rendered as a full spatial mosaic through either the GPU or the WASM engine, with the WASM engine working in globe as well as mercator projection

- [x] **Copy and paste layer styles** between layers from the layer menu, so a set of vectors or rasters can be given one consistent look without restyling each in turn
- [x] **Multiple named AI profiles:** define several provider, model, and credential combinations, mark one as the default, and switch between them from the assistant panel
- [x] **Deep-link any Whitebox tool** with a `?tool=` URL parameter that opens the Processing dialog preselected and pre-fills the tool's form from the remaining query parameters, plus a Copy link button that builds a shareable link from the settings you changed
- [x] Add Data gains a source **CRS field for delimited text**, so CSVs with projected easting and northing columns land in the right place, and a **layer picker for multi-layer GeoPackages**, so a container full of feature tables adds only what you asked for
- [x] The layer metadata dialog reports a raster's real georeferencing read from the GeoTIFF header — CRS and EPSG code, pixel size and extent in CRS units, data type, nodata, compression, tiling, and overviews — in a resizable dialog
- [x] Type a coordinate into the place-search box to fly straight to it, in decimal degrees, DMS, or DDM, with no geocoder round-trip
- [x] A new Random extract vector tool, the active layer now persists with the project, turning on the Terrain control enables 3D relief immediately, and a Logos submenu adds Maptoolkit branding alongside the MapLibre logo

v2.4: STAC and Earthdata catalogs, a free-flight camera, a timeline for tiles and data cubes, and a live embed API

- [x] A new **STAC catalogs browser** plugin that discovers public SpatioTemporal Asset Catalogs from STAC Index, connects to both static catalogs and STAC APIs, searches a collection's items, and adds any visualizable asset — COGs, GeoJSON, and the rest — straight to the map, so working with STAC no longer means leaving the app to hand-copy asset URLs
- [x] A new **Earthdata GIS browser** under Plugins > Web Services that searches NASA's EOSDIS ArcGIS portal and renders its imagery, map, and feature services, and its published web maps, as first-class layers, with the mosaic pixel-size limits that used to return blank tiles handled for you
- [x] A new **Hugging Face** panel that browses the Hub: search datasets or name an account, walk a repo's folders, and add its vector and raster files to the map through the readers that already handle each format — and go the other way, creating a dataset repo and uploading layers to it from inside GeoLibre
- [x] A **flight simulator**: a continuous, interactive free-flight camera you steer over terrain and 3D layers with the keyboard, instead of declaring a destination and watching a scripted `flyTo`
- [x] The **Time Slider now animates tiled data**: vector tiles, PMTiles, and MBTiles can be bound to the timeline and animated over their full extent, with the timestamp field detected from a live tile sample rather than requiring a local copy of the data
- [x] The Time Slider also drives a layer's **own internal time dimension** through a generic temporal adapter, so a data cube such as a Zarr store joins the shared timeline instead of carrying its own bespoke time control, and the binding is saved with the project

- [x] **Zarr gets a real Add Data path:** an Add Zarr Layer dialog for remote stores or a folder on disk, variable and dimension pickers that offer the store's actual coordinate values rather than raw indices, and a Selector field that now applies to a layer already on the map
- [x] **OGC API - Features** collections can be added as vector layers, the JSON-native successor to WFS, from whatever URL you have in hand — a landing page, `/collections`, or a full items URL
- [x] A versioned `postMessage` **API for host pages** that frame GeoLibre: load a project, move the camera, highlight features, and open a processing tool at runtime, with `ready`, `projectLoaded`, `selectionChanged`, `viewChanged`, `toolCompleted`, and `serverFileWritten` events coming back out. Off unless the deployment names its trusted origins. See [Embedding & Sharing](#)
- [x] **Jupyter clients outside the app can drive the map:** attaching VS Code's Jupyter extension, `jupyter console`, or `nbclient` to the desktop app's Jupyter server now makes `fly_to`, `add_geojson`, and the rest work, because the transport follows the kernel's server rather than how the notebook happens to be displayed
- [x] A new **H3 hexagonal grid plugin** that renders the H3 grid over the current view at a chosen resolution, identifies a cell to inspect its index, parent, children, neighbors, and center, and exports the grid or the selection as GeoJSON or CSV — plus typing an **H3 cell index** into the place-search box, as a hexadecimal string or a 64-bit integer, to frame and outline that cell
- [x] **Object detection on geotagged photos:** run the in-browser ONNX/YOLO models over an imported photo layer, not just map imagery
- [x] The AI assistant gains a **model picker in the panel itself**, refreshed model lists for every provider, credentials that survive a provider change, and arrow-key recall of previous prompts
- [x] **Indicator (KPI) tiles in the Dashboard panel:** a big-number widget with count, sum, mean, min, max, or median aggregation and custom prefix and suffix, alongside the existing chart widgets
- [x] **Editing GeoLens datasets in place:** a dataset added as GeoJSON can be redrawn with the GeoEditor or retyped in the attribute table, and the plugin's Edits section shows what changed and writes it back to the GeoLens dataset — added, moved, and deleted features become per-feature `POST / PUT / PATCH / DELETE` calls, with the row ids GeoLens assigns stamped back onto the layer. Private GeoLens rasters also render now, with the API key attached to exactly their tile URLs and nothing else
- [x] Plugins can call `addZarrLayer` to use the Zarr renderer GeoLibre already ships instead of bundling their own, keep their own paint properties on custom layers so the Style panel's sliders actually do something, and expose layer groups of their own
- [x] GeoLibre is on [Google Play](#), so Android users can install the signed, auto-updating build in one tap instead of sideloading an APK (see [Downloads](#))
- [x] GeoLibre Desktop is on the [Mac App Store](#), a sandboxed build of the same app that installs and updates through the Store. It trades the Python sidecar engines, server-backed PostgreSQL/PostGIS layers via martin, the local Jupyter server, Earth Engine sign-in, and external plugin installs for that convenience, so the Homebrew and DMG builds remain the full-feature route (see [Downloads](#))

- [x] Linux **AppImage releases now carry update information and a `.zsync`**, so AppImageUpdate, AppImageLauncher, AppManager, and AM can update the app and transfer only the blocks that changed instead of the whole download
- [x] All 15 non-English locales are back at 100% coverage, Georgian included

v2.5: QGIS and ArcGIS Pro project import, discrete global grids, hyperspectral cubes, and a self-hostable server

- [x] **QGIS project import:** open a `.qgs` or `.qgz` and get its layers, nested groups, group visibility, layer order, styling, and saved map view rebuilt in GeoLibre, with a per-layer report of anything that had to be skipped rather than an all-or-nothing failure. See [Projects](#)
- [x] **ArcGIS Pro project import:** `.aprx` and `.mapx` files are read straight from their CIM JSON without ArcPy, restoring the first 2D map's extent, local vector and GeoTIFF layers, nested groups, visibility, simple symbols, field labels, vector-tile portal items, and cached map services. See [Projects](#)
- [x] **Discrete global grid systems** arrive as three new plugins under Plugins > DGGS — **A5**, **DGGRID**, and **DGGAL** — that render the grid over the current view at a chosen or automatic resolution, identify a cell to read its id, parents, children, neighbors, and center, and add the grid or the selection to the map as a layer or export it
- [x] **DGGS processing tools** to match: a DGGS Generator that fills an extent with cells, DGGS Binning that aggregates a point layer into them, and DGGS Compact that collapses a full set of children into their parent
- [x] **Hyperspectral NetCDF and HDF grids become readable data rather than a single grey band:** local grids are colormapped in the browser from the same catalog the Style panel uses, the camera fits to the layer on add, and a cube gains an RGB band combination picked by wavelength
- [x] **A 3D image cube view** renders a hyperspectral scene as its six exterior faces with draggable slice cuts, so a volume can be read as a volume — windowed and strided reads keep a full EMIT reflectance variable within what the browser can hold
- [x] **Spectral profiles from a click:** identify a pixel on a NetCDF layer to read its value, walk the band axis to chart its spectrum against wavelength, and compare up to six sampled points, each marked on the map as a numbered dot in its chart color, in a floating draggable window that exports as PNG or CSV
- [x] **Sharing, accounts, and live collaboration can now be self-hosted:** the projects and identity contract is written down, with a FastAPI reference implementation and a plain Node collaboration relay sharing one session core, and a single conformance suite runs against both relays so their permission behavior cannot drift. See [Server API](#)
- [x] A published web image can be **pointed at that server at runtime** through `GEOLIBRE_SHARE_URL` and `GEOLIBRE_COLLAB_URL`, instead of forking the image to rebuild it, with `off` removing Share and the Project Gallery entirely and a malformed value refusing to boot rather than quietly uploading projects to the public hosted service

- [x] **Smart styling on add**: each new vector layer takes the next unused color from a qualitative palette with sizing that follows its dominant geometry, so a freshly loaded stack is legible before the Style panel is ever opened, and a dismissible **Style suggestions** strip offers up to three one-click renderers derived from the layer's own attributes
- [x] **Vector-tile layers can be classified by their tile attributes**: PMTiles, MBTiles, and remote vector tiles carry no local features, so the Style panel now samples what the viewport has already loaded to fill in the field list and the classifier's values, and 3D extrusion defaults to a real height property
- [x] **Quick analysis** from the map's right-click menu and a layer's actions menu: buffer the clicked point at three distances or drive and walk isochrones from it, and buffer, centroid, convex hull, or bounding-box a whole layer, with defaults filled in and every run landing in Processing History, re-runnable and copyable as Python
- [x] An **Elements panel** that lists the map's annotations — text, arrows, rectangle, ellipse and freehand highlights, pin markers, sticky notes, and placed images — so each one is managed from a list instead of hunted for on the canvas, with a placed image optionally pinned to an extent so it scales with the view. See [Annotations and the Elements panel](#)
- [x] A **Layer Library**: save a fully configured layer, with its source, style, labels, filters, joins, virtual fields, and attribute form, then re-add it to any later project in one click from the Browser panel's My Data section, and export the library as a JSON bundle to share with a team
- [x] **Project duplication, templates, and open-a-copy**: duplicate a project, save one as a reusable template that keeps the basemap, groups, styles, legend, widgets, and layout while stripping the data, and open a copy of anything in the gallery
- [x] **Live layer connections persist with the project**: the refresh cadence, last-synchronized time, last error, and on-failure policy for WFS, GeoJSON URL, and Add Vector Layer URL layers are saved as a `connection` record, so a reopened project keeps refreshing on schedule and the Layers panel shows each live layer's status
- [x] **Open-data catalog browsers for ArcGIS Hub, Socrata, and CKAN** (Humanitarian Data Exchange) under Plugins > Web Services: search public dataset catalogs by keyword, restrict an ArcGIS Hub search to the current map area, page through results, and add a dataset to the map or download it
- [x] **Geocode a CSV at import time**: the Delimited Text panel gains an Addresses mode that concatenates the columns you pick and geocodes each row through the project's provider, keeping unmatched rows as null-geometry features so they stay visible and fixable in the attribute table rather than being silently dropped
- [x] A **CartoCiudad (IGN España) geocoding provider** for forward and reverse geocoding against Spain's official address data, with no API key
- [x] **KML and KMZ layer export**, so a styled layer can go back out to Google Earth, and **KML Super-Overlay import**, which serves a `NetworkLink`-driven tile pyramid to the map instead of loading the whole thing at once
- [x] A **Dashboard selector widget** that turns a categorical field into a set of chips and cross-filters every other widget bound to the same layer, in single- or multi-select mode
- [x] **GPS Tracking reads an external NMEA receiver** over Web Serial or Web Bluetooth, with a baud-rate picker and a live sentence and fix counter, alongside the device's own geolocation. See [GPS tracking](#)

- [x] The Processing dialog stops asking users to translate on their behalf: distance parameters on WGS84 vector layers get a **metric unit picker** that converts to the degrees the tool actually reads, EPSG parameters get a **searchable CRS catalog** grouped into Geographic and Projected, and Download OSM Vector's four boundary numbers become one **Area of interest** control with Use map extent and Draw on map
- [x] **GeoLibre Desktop is on the Mac App Store**, a sandboxed build of the same app that installs and updates through the Store. See [Downloads](#)
- [x] **GeoLibre is on the App Store** as a native iPhone and iPad app, built from the same codebase via Tauri v2 mobile and approved by App Store review, so iOS users install and update it in one tap. See [iOS](#)
- [x] The `geolibre` Python package gains **leafmap-parity helpers**: local raster, marker and cluster, and choropleth layers, `split_map`, `add_legend`, and `add_colorbar`, typed read-back of selected and drawn features, and `to_html` export. See [Python package](#)
- [x] **Thai** joins the shipped locales, bringing the catalogs to 16 languages besides English
- [x] Vector export honors the fields you hid in the attribute table, plugins can read a pixel off a native Zarr layer with `queryZarrLayer`, layers move into a group as a whole selection, and the print extent can be drawn by touch

v2.6: Data and style deep links, an MCP server for AI clients, COG-backed terrain, and a design system of its own

- [x] **Deep links can carry the data itself**: `?data=` opens a hosted GeoJSON, GeoParquet, COG, PMTiles, a REST endpoint returning a FeatureCollection, or a ZIP of GeoJSON files without publishing a project file first, `?style=` applies vector or raster symbology beside it, and a compact GeoLibre style export makes that style file easy to produce from a layer already on the map. See [Embedding & Sharing](#)
- [x] **ArcGIS MapServer and ImageServer join Add Data**: the two service types that serve rendered images load as ordinary raster layers, so opacity, the Style panel's brightness, contrast and saturation controls, reordering, and project save and reload all work on them, drawing tiles from the service's own fused cache where it was built on the standard Web Mercator scheme and from `/export` or `/exportImage` otherwise. Sublayers are browsed and picked from the service's own list rather than typed in as ids, and an ImageServer rendering rule is one field away
- [x] **A Cloud Optimized GeoTIFF can drive 3D terrain**: terrain was locked to the global AWS Terrarium tiles, so higher-resolution local elevation could not shape the hillshade. Terrain settings now accept a single-band EPSG:3857 or EPSG:4326 COG over HTTP or from a local file, read in ranges and encoded to Terrarium tiles through a custom MapLibre protocol
- [x] **Regional basemaps, starting with China**: GeoLibre's defaults are hosted outside mainland China with no presence inside it, so from there the picker offered nothing that loads. A collapsed **Regional** section in both the New Project and Change Basemap panels adds five keyless Chinese basemaps (高德地图, 高德卫星, 高德混合, 腾讯地图, 腾讯深色), with the same providers plus Tianditu in the Basemaps control plugin, and the section is driven by a table in core so a second region is an entry rather than another dialog change

- [x] **The basemap you picked is the one you come back to:** an empty startup workspace seeds from your last selected basemap, while a project's own basemap stays authoritative and planetary bodies keep their ellipsoid behavior
- [x] **Collaboration snapshots are portable:** a guest could join a session and see an empty map, because snapshots kept local file references and control-managed vector data the collaborator cannot read. Shared layers now embed their features, a host seeds the first snapshot instead of waiting to make an edit, guests arrive at the host's viewport, and the session Copy button yields a joinable URL. See [Collaboration](#)
- [x] **Map comments become navigable:** clicking a pin on the map reveals, highlights, and scrolls to its card, the panel ships collapsed on the Style rail so it is discoverable, **C** places a new comment from the command palette, and comments are saved with the project. See [Review comments](#)
- [x] **An MCP server for authoring projects,** `geolibre-mcp`: a headless stdio server that lets an AI client write real `.geolibre.json` files with no browser and no running app, composing them from the same builders the Python package already uses, so anything it writes opens unchanged in the desktop app, the web app, and the Jupyter widget. Every read and write is confined to the roots given by `--root` or `GEOLIBRE_MCP_ROOTS`. See [MCP server](#)
- [x] **The Python package manages layers and moves the camera without a browser round trip:** reorder, duplicate, rename, and remove layers, read attribute values, and frame the map as plain project mutations on `Map` and `Layer`, with the authoring helpers exported for scripts that never display a widget, and credentials swept out of anything a notebook cell might print back. See [Python package](#)
- [x] **Whitebox tools run from the Jupyter widget:** `list_whitebox_tools` and `run_whitebox_tool` reach the bundled WASM catalog over the same channel the camera and the client-side algorithms already use, resolving `Layer` handles to layer ids and adding both vector and raster outputs back to the map, so terrain and raster work no longer means leaving the notebook for the UI
- [x] **Topological polygon digitizing,** so a polygon drawn against its neighbor shares that edge instead of leaving a sliver or an overlap behind
- [x] **Print Layout gains an intersecting page filter,** so an attribute-table block can list every feature the page touches rather than only the ones that fall wholly inside it, correct across the antimeridian and for a view panned several world copies from home
- [x] **Story map PDF handouts get clickable location markers:** each chapter page carries a pin drawn with its tip on the chapter camera's centre coordinate and a PDF link that opens that coordinate in Google Maps, taking the story's marker color, so a reader of the printed handout can navigate to the place
- [x] **A design system of the app's own, in place of stock shadcn:** self-hosted IBM Plex Sans and IBM Plex Mono so the interface reads the same everywhere instead of Segoe UI on Windows, Roboto on Linux, and SF on macOS; a real dark-mode elevation ladder where panels, dialogs, and menus had all shared one near-identical color; theme-aware shadows that were effectively invisible in dark mode; and a single map-glass treatment for chrome floating over the map, replacing eleven drifted copies of it

- [x] **Embeds can lead with the map:** a `panels=collapsed` layout keeps the Layers and Style icon rails reachable while starting both panels collapsed, and the top toolbar can be hidden outright for a focused viewer, giving a middle ground between panels open and panels gone. See [Embedding & Sharing](#)
- [x] **An optional Clerk access gate** for hosted deployments that need individual sign-in, with an optional waitlist screen, loaded on demand and kept out of the default PWA precache. The gate is decided by the build target rather than by a client-controlled query parameter, and public, native, and embedded builds are unchanged. See [Getting started](#)
- [x] **A no-external-CDN build** for deployments that cannot load from untrusted third-party hosts: `GEOLIBRE_NO_EXTERNAL_CDN=1` strips the GeoLibre-controlled CDN references, vendors the PGlite and CereusDB engines into the build rather than dropping them, and makes the few features that genuinely need a remote host say so up front instead of failing at the end of a run. See [Self-hosting](#)
- [x] **External plugins can query layers read-only**, reading the features a layer holds through the host rather than re-fetching and re-parsing the source themselves. See [Plugin API](#)
- [x] **Persian and Vietnamese join the shipped locales**, bringing the catalogs to 18 languages besides English, with Persian set in Vazirmatn and only that font's arabic subset shipped, so no other locale pays for it
- [x] Selecting a layer no longer pops the Style panel open over the map; it expands from an explicit **Open Style panel** item in the layer's menu instead

v2.7: A visual Model Builder, STAC assets of every kind, a Chrome data opener, and selection by drawing

- [x] **Processing workflows are drawn on a canvas**, ArcGIS Model Builder style: drop tools as nodes, wire an output into the next tool's input, and save the graph as a model that re-runs as one job, with the whole graph validated (cycles, missing connections, invented parameters, a string where a number belongs) before anything executes
- [x] **The AI assistant can author a model from a plain-English description** and open it on the canvas for review before it runs, drawing on the same palette as the canvas itself, so a raster chain such as fill depressions to flow accumulation to extract streams is a valid model. It asks before replacing unsaved canvas work or a run still in flight. See [AI assistant](#)
- [x] **A model copies out as a runnable Python script**, so a workflow built by pointing and clicking continues in the Notebook panel or JupyterLite rather than stopping at the canvas
- [x] **The STAC browser adds every asset type the app can draw:** PMTiles archives, GeoParquet tables, and Zarr stores with a variable picker join the COGs it already loaded, an Icechunk repository is read through its own manifest and keyed by branch rather than by repository alone, and Azure-hosted assets are signed at add time so a private Planetary Computer container opens like a public one
- [x] **A static STAC catalog can be browsed as a tree**, not only searched: walk its collections and sub-collections, open a folder to read it, and start a search from whatever you picked, flying to that collection's own extent

- [x] **Select features by drawing on the map:** QGIS-style click, rectangle, polygon, freehand, and radius gestures, with Shift and Alt combining the result into the existing selection and Clear selection sitting alongside them, so picking a handful of features means pointing at them instead of describing them in an expression
- [x] **Open data in GeoLibre,** a [Chrome extension](#) on the Chrome Web Store, collects the dataset links and map services on the page you are viewing (WMS, WMTS, WFS, OGC API - Features, ArcGIS Feature Services, XYZ/TMS, and vector tiles, including services detected inside embedded maps and raw responses) and opens the ones you pick together on one GeoLibre map
- [x] **Apache Iceberg tables load as vector layers,** read in-browser through DuckDB's `iceberg` and `spatial` extensions from a table's metadata location or an attached REST catalog, reporting the table's true row count from the manifest before anything is scanned, taking the geometry column's CRS from the schema, and letting an optional SQL box decide which rows are rendered
- [x] **Three vector tools the client-side suite was missing:** Merge layers, with a multi-layer parameter picker that unites attribute schemas and can record each feature's source layer; Extract vertices, which turns every vertex into a point carrying its part and vertex index; and Points along geometry, which walks lines and polygon boundaries geodesically at a fixed interval
- [x] **Encoded polylines become a first-class format:** a codec for Google and OSRM precision 5 and Valhalla and Mapbox precision 6, with an interactive preview and coordinate inspector in Add Data, batch parsing with custom delimiters, processing tools, layer export at either precision, and matching support in the Python package
- [x] **An activity log for shared projects and collaboration sessions:** a project owner can read who opened and edited a shared project, and a session host can download the session log, bounded in both entry count and stored bytes and read through a bearer token rather than a URL query. See [Server API](#)
- [x] **Automatic editor tracking on a vector layer:** created-by, created-at, edited-by, and edited-at fields fill themselves in as features are added and changed, configured per layer and saved with the project. See [Project format](#)
- [x] **Building massing authoring:** sketch a footprint, give it a height, and get an extruded mass, with the extrusion style managed for you as massing features come and go and ordinary polygon sketches left flat
- [x] **The Elevation Profile plugin profiles the line features you have selected,** so a route already on the map is charted without redrawing it by hand
- [x] **Plugins and processing tools are translatable:** plugin display names resolve through one place, so a plugin no longer reads translated in the Plugins menu and English in the command palette, and every registry tool name, description, group label, parameter, and select option is generated into the English catalog for translators to work from. See [Internationalization](#)
- [x] **A PostGIS table with more than one geometry column stops being a guess:** every registered geometry column is offered in Add Data, validated against the PostGIS catalogs, and carried through read, save, and refresh

- [x] **A shared link can carry presentation settings:** `settingsUrl=` loads language, layout, accent theme, and UI profile before the first render, restricted to presentation fields so a link cannot inject credentials, plugins, or local paths, and lasting for that page only instead of replacing what the user has saved. See [Embedding & Sharing](#)
- [x] **Geometry generators become attribute-driven:** derived centroids scale with a field as proportional symbols, and each feature's buffer takes its distance from a column instead of one typed number, following QGIS's data-defined overrides
- [x] **Excel workbooks import as point layers** from their X and Y columns, with a worksheet picker
- [x] **The Layers panel search box searches the map's own data:** the typed query is matched against the attributes of the loaded vector layers alongside the geocoder, running locally so data matches appear before the network answers, grouped by layer above the place results and ranked exact match first, then prefix, then contains, with picking a feature selecting it on its layer and flying to it with the attribute table's highlight
- [x] **A composite score builder for suitability and index analysis:** normalize two or more numeric fields (min-max, z-score, rank, or quantile), weight them, combine them with a weighted arithmetic or geometric mean, and write one index (0-100 by default, 0-1 on request) that the layer is immediately styled by, with how a feature missing a value is treated made an explicit choice rather than a silent default
- [x] A Style panel button on every layer card, field statistics scoped to the selected features, USGS LiDAR clipped and downloaded as COPC, ArcGIS ImageServer raster functions browsed from the service's own list, and `?data=` accepting several URLs at once

v2.8: A popup designer, quick filters, per-layer permissions, and the packages on npm

- [x] **An author decides what a viewer sees on click and hover:** a per-layer Popup section in the Style panel picks which fields the Identify popup shows, in what order, under what labels, with what number, date, link, and image formatting, what titles it, and whether a hover tooltip follows the pointer — so a shared map, a `layout=viewer` embed, or a story-map chapter stops showing join artifacts, editor-tracking columns, and internal ids to its readers. See [Project format](#)
- [x] **Quick filters answer a layer with controls instead of an expression:** pick a field and GeoLibre profiles its values to offer the matching control — checkboxes with per-value counts for a categorical field, a range slider with typed bounds for a numeric one, a date range for a timestamp, a text match otherwise — with the control state saved, not the compiled expression, so a filter can always be reopened and changed. A filtered layer is flagged with a funnel icon and cleared in one action, the controls render in the read-only viewer layout, and a host page's `setFilter`, the Time Slider's window, and a user's own filter now narrow a layer together rather than clobbering each other
- [x] **Per-layer capabilities:** a layer can be marked read-only or restricted to creating, updating, or deleting features, and both the UI and the backend enforce it, so a reference layer in a shared project cannot be edited by the people it was shared with

- [x] **Per-layer cartographic blend modes:** overlaying color or imagery on a hillshade takes Multiply rather than reduced opacity, which dilutes both layers instead of combining them. MapLibre exposes no per-layer blend API, so the render loop is wrapped to set the GPU blend state while one layer draws, blending in the right z-position of the same canvas, with fills and lines composited as one surface and labels left out so place names stay legible
- [x] `@geolibre/core` and `@geolibre/map` **publish to npm**, so another application can build on GeoLibre's store, project schema, and MapLibre layer-sync engine without vendoring the monorepo. See [Maintenance](#)
- [x] **An agent skill for driving GeoLibre from outside the app:** `skills/geolibre/` is a SKILL.md plus progressive-disclosure references for the MCP tool surface, the Python API, hand-written project JSON, and the basemap/ramp/format catalog, so an external agent knows which entry point to reach for and in what order to call it. Its copy is guarded by a test against the real registries. See [Agent skill](#)
- [x] **Interactive SamGeo segmentation**, a dockable SAM3 workflow with text, point, box, and automatic modes, reprojecting results into WGS84 so the generated polygons land where the imagery is
- [x] **The Python package speaks Earth Engine and xarray:** `add_ee_layer` turns an authenticated Earth Engine image, collection, feature collection, or feature into a restorable tile layer geemap-style, and a `DataArray` or `Dataset` is materialized as a session-scoped COG that renders directly in the browser. See [Python API](#)
- [x] **Every shipped locale is complete again**, three times over: the Whitebox toolbox's 1066 tool names, 45 subcategory labels, and unlocked tool parameters are generated into the catalogs and translated into Simplified Chinese, the SamGeo panel, Whitebox job strings, and the language-pack manager are filled in across all 18 locales, and a later sweep closed the 136 keys that had drifted out of every non-English catalog. See [Internationalization](#)
- [x] **A tiled archive opens as a group of its source layers**, so a multi-layer PMTiles file lands as one folder of named, individually styleable layers instead of a single opaque row
- [x] **Basemaps are previewed, not just listed:** each raster entry shows a z=2 tile and each keyless style entry shows its background color and then an offscreen MapLibre snapshot, so a basemap can be told apart before it is applied
- [x] **A configurable startup map view** in Settings, so a deployment or a personal install opens where its users work rather than on the world
- [x] **GeoEditor sketches export as a project layer**, carrying their style, opacity, and visibility, so drawn work leaves the editor without being redrawn or re-imported
- [x] **KML folder structure survives import:** placemarks that sit inside a Folder become one layer per folder, the rest stay merged per file, and the import frames the combined extent
- [x] Release downloads are mirrored to GitHub Pages and `web.geolibre.app` moved there, the Model Builder panel maximizes across the map canvas, the annotation toolbar collapses, hosted-site analytics became opt-in at build time (and never ship in the desktop, Jupyter, Docker, or fork builds — see [Privacy](#)), and the docs gained a [Video Tutorials](#) page and a user guide checked against the running app

v2.9: A tokenless 3D globe, kiosk and classroom deployments, docked service panels, and keyless global elevation

- [x] **The 3D globe no longer needs a Cesium Ion token**, and it finally draws the project's own basemap. The pane used to hide itself unless a token was configured and then ignored the basemap entirely, so a project saved with a globe reopened as a flat map and a dark-themed project kept a bright globe in the corner. Planetary and regional basemaps carry over as raster tile sets, GeoLibre's vector styles map to a keyless raster basemap of matching tone, and the blank basemap draws a bare ellipsoid. A token is now an upgrade rather than a gate: it adds Cesium World Terrain and Ion world imagery
- [x] **The two panes finally agree on scale**: the globe camera aims at the terrain surface rather than the ellipsoid, so a view over high ground stops rendering more than twice as close as its 2D twin. The globe also builds a bare `CesiumWidget` and imports `@cesium/engine` directly instead of the `cesium` barrel, dropping the ten widgets it only switched off again and the Knockout dependency that came with them, and it now has automated coverage for the first time
- [x] **Kiosk and classroom deployments can lock the app down**: a capability model in `@geolibre/core` defines what a build is allowed to do, and the UI, the command palette, the keyboard shortcut layer, and the embed API all enforce it, so a denied action cannot be reached from a menu, a shortcut, or a host page. Denials explain themselves inline rather than silently disabling a control, and per-layer permissions from v2.8 now sit under one model. Processing, sidecar tools, project editing, data loading, plugins, and settings each gate independently
- [x] **Elevation data without an account**: a keyless global DEM downloader mosaics and clips the public AWS Terrain Tiles to the current map view or a bounding box drawn on the map, straight from Processing > GeoLibre Toolbox > Raster, with no API key to obtain and in-flight downloads cancelled when the tool closes
- [x] **Web Services plugins dock like everything else**. The first five moved into the shared dockable panel host, keeping their MapLibre control lifecycle and layer sync, so a catalog browser resizes, collapses, and sits alongside the Layers and Style panels rather than floating over the map
- [x] **Two new open-data catalogs** under Plugins > Web Services: **Planet Open Data**, which opens Planet's disaster releases through the STAC browser with the public catalog preselected, and **Vantor Open Data**, a STAC explorer for pre-event and post-event imagery that filters by event and phase, draws a bounding box, and picks its own COG rendering engine (GPU, WebAssembly, or TiTiler). Catalog plugins that share panel state are now mutually exclusive, and a failed switch restores the plugin it displaced
- [x] **Identify answers for every visible layer at once**, folding vector features and raster pixel values from all visible queryable layers into one grouped, expandable popup, instead of making you pick a layer first and click again
- [x] **Annotation layers are fully editable**: an Elements panel lists what a layer holds, image overlay corners stay synchronized when an annotation moves, and the style controls shown are the ones that apply to the selected element

- [x] **GeoParquet's geo metadata is read in full** — `version`, `primary_column`, `encoding`, `geometry_types`, `bbox`, `covering`, and `edges` — alongside the CRS a Parquet 2.0 file records only on its geometry column's GEOMETRY/GEOGRAPHY logical type, which GeoLibre previously could not see at all. A 3D `bbox` is now read minima-first rather than mistaking its ymax for an elevation, and a lon/lat table without geometry is recognized
- [x] **GeoLibre embeds in a Plotly Dash app** through a new `DashMap` component, which pushes project changes into a mounted iframe after the ready handshake the way the Jupyter widget does. See [Python API](#)
- [x] **Projects open the way files do**: double-clicking a `.geolibre.json` in the desktop file manager opens it through a file association (with native project events on macOS), and dropping one on the map switches projects, keeping the unsaved-work prompt and reporting a bad file as a project error rather than a failed layer
- [x] **Background controls say what they do**, and a Blank background takes a custom color that survives a style reload and follows the theme by default, without bleeding into the planetary and regional styles that carry their own
- [x] Shift and Shift+Alt reach click selection again, so add and intersect work across multiple features instead of every click replacing the selection; remote zipped Shapefiles load in the web build; and all 18 shipped locales are complete again after the identify, Vantor, and Cesium features landed. See [Internationalization](#)

v3.0: Cesium as a first-class rendering engine, dynamic 3D scenes on the globe, ArcGIS write-back, and editable SVG print layouts

- [x] **Cesium can be the primary rendering engine, not just a side pane**: View → Rendering engine switches the main map area between MapLibre 2D and Cesium 3D, saved with the project, so a 3D-first workflow no longer builds a split layout it does not want and keeps a MapLibre map it is not looking at. Behind the switch, the whole app now drives the map through a renderer-neutral `MapEngine` interface with `CesiumEngine` as its second implementation, and menus, tools, and plugins are gated on what the active engine can actually do rather than silently doing nothing. See [Architecture](#)
- [x] **The globe carries its own chrome**: Reset view, fullscreen, and a scene-mode picker for 3D, 2D, and Columbus views (the flat modes in Web Mercator to match the project map), a native basemap and terrain gallery with Cesium Ion imagery plus keyless Esri and OpenStreetMap styles, cursor coordinates and elevation in the shared status bar, and controls that move between corners like their 2D counterparts
- [x] **Vector layers look and behave the same on the globe as on the map**: per-feature categorized, graduated, rule-based, expression, proportional, and marker symbology compiled from the same paint expressions the 2D map uses; field and expression labels; point clustering and batched rendering for large point layers; polygon extrusion and true Z elevation instead of clamping everything to the ground; layer filters, the Time Slider clock, and story-map opacity fades; and click popups, hover tooltips, identify, and highlighting through the same popup designer

- [x] **Almost every layer kind reaches the globe:** COG, raster PMTiles, local MBTiles, and native tile fetchers through one protocol bridge; vector tiles, vector PMTiles, and vector MBTiles draped through a hidden MapLibre renderer so their styles match exactly; ArcGIS MapServer, WMTS, and image overlays; and I3S scene layers, LiDAR point clouds, and Gaussian splat tilesets loaded natively by Cesium
- [x] **Cesium-native content beyond parity:** Cesium Ion tilesets and imagery by asset id (with quick picks for Cesium OSM Buildings, Google Photorealistic 3D Tiles, and Asset Depot samples), 3D Tiles classified from the Style panel through a translator to `Cesium3DTileStyle`, time-dynamic **CZML** scenes for orbits, trajectories, and moving entities synchronized with the globe clock, and native **KML/KMZ** that keeps document styling, overlays, and network links. See [Optional 3D globe credentials](#)
- [x] **Tools and plugins follow you onto the globe:** drawing extents and dragging model pins, screenshots, print layouts, video and camera-tour recording that wait for terrain and tiles to settle, an Elevation Profile sampled from the active Cesium terrain, and native globe branches for the Sun simulation, Atmospheric Effects, Flight Simulator, Clouds, and Precipitation plugins. Plugins declare which engines they support, and switching renderers suspends the unsupported ones while keeping their settings. See [Plugin API](#)
- [x] **Edits to an ArcGIS Feature Service go back to the service:** Layer actions → Save edits to ArcGIS service submits additions, attribute and geometry changes, and deletions through `applyEdits`, validates against the service's capabilities and fields, reconciles partial failures and server-assigned ids, and refuses to retry a write whose outcome it cannot confirm
- [x] **Editable SVG print layouts:** a single-page Print Layout exports to SVG, so the title, legend, colorbar, scale bar, north arrow, and other layout elements open as real vectors in Inkscape or Illustrator for final touches
- [x] **Rasters stretch to what is on screen:** an unclassified raster can recompute its display range from the current viewport by min/max, 5th to 95th percentile, or mean ± 2 standard deviations, and keeps doing so as you pan with the Style panel closed. Discrete raster classes also gained their own color and opacity controls, and any loaded COG layer can be picked as the 3D terrain source
- [x] **Select by Expression can filter a layer, not just select from it:** a validated expression becomes a persistent, project-saved layer filter that composes with time, embed, quick, and rule filters on both engines and in style exports. See [Project format](#)
- [x] **New ways to find and reach data:** a **Portolan** catalog browser that opens the public Portolan Registry, an OGC **CSW** catalog search that hands WMS, WFS, and ArcGIS results to prefilled Add Data forms, a **USGS NLDI** panel for flowline tracing, hydrolocation, basin analysis, and navigation to gages, dams, and water-quality sites along the network (see [USGS NLDI](#)), local GLB and glTF model import embedded in the project, and a project gallery that loads more as you scroll
- [x] **A Dimensions plugin** draws CAD-style linear and angular dimension lines whose endpoints snap to vector vertices and stay associated when those vertices are edited
- [x] **Point heatmaps take a color ramp and an attribute weight**, carried through saved styles, style exports, and the automatic legend

- [x] **Links and automation:** coordinate URLs (`?lat=&lon=&zoom=` and compact `?12/40.7/-74.0`) plus Android `geo:` intents open GeoLibre at a place, a `loading=true` parameter exposes screenshot readiness for headless capture (see [Embedding & Sharing](#)), external plugins register AI Assistant tools with plain JSON Schema, and the Python package authors popups, tooltips, marker symbology, and WMS/WMTS bounds (see [Python API](#))
- [x] Field Collection takes multiple photos per observation, numeric labels format with thousands separators, the buffer tool runs inward or across the boundary, saved style presets can be renamed, and the Share dialog warns up front that local-file layers will be missing from a shared map

v3.1: Mapbox and ArcGIS join the rendering engines, a live God's Eye View globe, OAuth sign-in for sharing, and a voice-driven AI Assistant

- [x] **Four interchangeable rendering engines:** alongside MapLibre and Cesium, View → Rendering engine now offers **Mapbox GL JS** and the **ArcGIS Maps SDK for JavaScript** (loaded on demand from a CDN), and the app remembers the last engine across launches. The layer control, default map controls, map-area overlays, GPS, review comments, georeferencing, and sample markers mount on every engine. See [Rendering engines](#)
- [x] **The Mapbox renderer reaches parity:** PMTiles, MBTiles, LiDAR, 3D Tiles, Zarr, Deck.gl layers, 3D models, and DuckDB layers all render on it; marker icons, fill patterns, inverted fills, line decorations, geometry generators, heatmap and cluster renderers, label de-duplication, and data-defined label overrides port across; Identify covers WMS, pixel, and DuckDB layers with hover tips and photo popups; and the Plugins menu, Print Layout atlas, Layer Swipe, Street View, Overture Maps, GeoAgent, Geo Editor, and the Flight Simulator's free camera all run there. See [Mapbox renderer](#)
- [x] **ArcGIS renders in 2D maps and 3D scenes:** native ArcGIS services plus PMTiles, MBTiles, Zarr, and NetCDF grids; 3D scenes with terrain and extrusion, scene zoom and bounds limits, and a nautical scale bar; icons, patterns, and labels on vector tiles; inverted fills, geometry generators, line decorations, and label overrides; raster effects and blend modes; identify popups, hover map tips, geotagged-photo popups, selection, and story-map fades; and the Print Layout atlas. Data plugin controls adapt to the ArcGIS engine. See [ArcGIS renderer](#)
- [x] **A God's Eye View plugin turns the Cesium globe into a live situation room:** USGS earthquakes and Celestrak satellite orbits as live CZML, keyless NASA FIRMS active fires, public transit from Entur and other providers, public CCTV snapshots and traffic cameras from Austin, New South Wales, Ontario, and DriveBC, TomTom live traffic flow, and AIS vessels, with in-plugin API key entry for the feeds that need one. See [Plugins](#)
- [x] **Sign in with OAuth to share and publish:** the self-hostable server API gains an OAuth Authorization Code flow with S256 PKCE and scoped, expiring personal tokens; the web build signs in for project Share and the Gallery, and the desktop app adds OAuth sign-in with session management. Shares take access roles (view, comment, edit), link expiry, password protection, and revocation. See [Server API](#)

- [x] **A faster, more capable AI Assistant:** a voice command mode, a sub-second fast path for simple map commands through TypeSafe, an **OpenRouter** provider, live model discovery for Google, Anthropic, OpenAI, and Bedrock, explicit class breaks for `apply_symbolology` with the applied stops reported back, plugin tools loaded on demand once there are more than twelve, plugin-contributed system-prompt guidance, and semantic search over the Whitebox catalog that ranks exact tool names first. See [AI Assistant](#)
- [x] **New data plugins: Fields of the World** agricultural field boundaries colored by model confidence, **Satellite Embeddings** for browsing and downloading pre-computed foundation-model embeddings such as AlphaEarth and Tessera, **IGN LiDAR HD** for French national point clouds, and an OSM downloader web service
- [x] **Cesium keeps growing:** Zarr layers render on the 3D globe through zarr-cesium, DXF and DWG drawings keep their Z values in 3D, the basemap picker offers Cesium and Mapbox basemaps, and the Ion quick picks add Google Photorealistic 3D Tiles and Asset Depot samples
- [x] **More ways in:** a `?stac=` link opens a STAC catalog or API collection, `?data=` opens LiDAR point clouds, the OGC API - Features form fills its bounding box from the map view, raster TileJSON metadata imports through XYZ sources, and the desktop app draws WMS tiles from servers that do not offer EPSG:3857. A runtime-configured deployment service library lets a hosted build ship its own curated services. See [Adding data](#)
- [x] **Styling and layer refinements:** continuous raster color ramps take opacity classes, popups have a configurable width and image height, a layer group's contents sort A to Z or Z to A, a style can be pasted from the layer menu and the Style panel, and the Buffer tool can dissolve its result
- [x] **For developers and notebooks:** `getLayers` / `onLayersChanged` on `GeoLibreAppAPI`, `moveLayerGroupToGroup` and `listLayerGroups` on the plugin API, a GeoLens connection that is remembered and auto-connects, and a Python API that arms Identify, toggles controls, and sets the projection. See [Plugin API](#) and [Python API](#)
- [x] **Hebrew joins the shipped locales** as a second right-to-left language, and every catalog is complete again after filling 351 missing keys. See [Internationalization](#)

v3.2: Point cloud annotation, private S3 buckets, credentials in the OS keychain, and the GIS portals of every US state

- [x] **Label LiDAR points on the map with the new Point Cloud Annotation plugin:** select points with a brush or polygon, assign ASPRS or custom classes with digit hotkeys and class locks, group points into instances, lift polygon layers such as SAM masks or building footprints onto the cloud, and pre-label with Whitebox LiDAR classifiers running in the browser. Oriented 3D boxes come with top, side, and front views, review status, and attributes, alongside 3D polylines, polygons, and keypoints. A streamed COPC can load an area at full detail, labels are saved with the project, and the result exports as a labelled LAS/LAZ rewrite, NumPy arrays, or GeoJSON and Segments.ai cuboids. See [Point cloud annotation](#)
- [x] **Point cloud labelling from notebooks and agents:** `add_lidar`, annotation readers, class schemas, instances, and headless Whitebox pre-labelling with full-file rewrites are available in the Python package and as MCP tools. See [Python API](#) and [MCP server](#)

- [x] **Private S3 buckets open like public ones:** S3 connections take access keys, named profiles, IAM Identity Center (SSO) sign-in, and IAM roles, with a browsable S3 Browser and SQL credentials masked in the UI. See [Cloud Storage](#)
- [x] **Secrets live in the operating system's credential store on desktop:** API keys, tokens, and credentials saved in project files go to the macOS Keychain, Windows Credential Manager, or Linux Secret Service instead of plain settings, and plugins reach the same store through a new `app.credentials` API. See [Plugin API](#)
- [x] **The GIS portals of the United States in one menu:** new US Federal GIS (24 agencies, from the Census Bureau and NOAA to USGS and FEMA), US State GIS (all 50 states and DC), US Local GIS (about 150 cities, counties, and regional agencies), and Tennessee GIS web services, plus an Ocean Data Platform plugin for HUB Ocean's public ocean datasets. See [Web Services](#)
- [x] **WMS in any projection:** the desktop Add Data dialog lets you choose a WMS layer's CRS, servers that only offer projected CRSs are reprojected into Web Mercator on the fly, and `addWmsLayer` takes an optional `crs`. WFS servers without GeoJSON output load through GML, and ArcGIS MapServer sublayers split into separate layers you can browse in the Browser panel. See [Adding data](#)
- [x] **Reuse styles by layer name:** Project → Export → Export Layer Styles writes every layer's style to one JSON file keyed by layer name, Import Layer Styles restyles matching layers in one undo step, and a Startup setting auto-applies a styles file to each matching layer as it is added. Plugins get `importLayerStyle` to do the same. See [Styling](#)
- [x] **Layer organization:** the on-map layer control shows layer groups, a KML/KMZ file's layers are grouped under a group named after the file, and Add Vector Layer layers convert into native GeoJSON layers for full styling and editing
- [x] **For deployments and sharing:** a `deployment.json` v1 schema with a lenient parser lays the groundwork for one deployment policy file, a deployment can rename the app in the toolbar and tab title, a `?plugin=` deep link activates a built-in plugin, and project owners can transfer ownership of a shared project with deletion protected. See [Deployment Policy](#)
- [x] **More everywhere:** MapLibre-control plugins run on the ArcGIS renderer, LiDAR overlay graphics draw as overlay layers, the Satellite Embeddings plugin visualizes Tessera v1.1, the Python `Map` saves identify and control state, and **Azerbaijani** joins the shipped locales with all 20 catalogs complete. See [Internationalization](#)

v3.3: Line of sight, MGRS grids and coordinates, SQL Server, compare-and-restore project history, and a plugin ecosystem that docks, verifies, and extends the menus (current)

- [x] **Line-of-sight analysis:** pick two points on the map and see an interactive line of sight between them over the terrain
- [x] **MGRS, USNG, and any-EPSG coordinates:** coordinate readouts in MGRS/USNG or any EPSG code, grid-reference search, an MGRS/USNG grid overlay, and grid references in Set View. See [Map tools](#)
- [x] **SQL Server and Azure SQL on desktop:** an Add Data source, an editable data source in the sidebar, save-back of edits, and a Browser panel for tables. See [Adding data](#)

- [x] **Compare project snapshots and restore single layers:** the project history shows what changed between snapshots, lets you bring back one layer instead of the whole project, and says when autosave is paused for a large project. See [Projects](#)
- [x] **Editable layer metadata with STAC Item export:** edit a layer's metadata, export it as a STAC Item, see a service layer's source in its Metadata dialog, and let plugins add metadata
- [x] **Edit from Identify:** Edit geometry and Edit attributes on Identify results, WMS GetFeatureInfo HTML tables read into fields, non-queryable WMS layers skipped, and WMS exceptions shown as errors
- [x] **Export 3D-extruded layers as glTF, OBJ, or STL models** for use in 3D modeling and printing tools
- [x] **An interactive histogram for the raster display stretch**, so you can see the distribution while adjusting the min and max
- [x] **S3 projects:** open projects straight from the S3 Browser, which stays open across project loads and can open at startup. See [Cloud Storage](#)
- [x] **A calmer, clearer interface:** unified toast notifications with a Report issue action, layer-failure and tile-health toasts on the Cesium and ArcGIS engines, a command palette that reaches individual tools, a color vision preview, and starter projects in New Project
- [x] **A plugin ecosystem that docks and verifies:** Historical Imagery, Timelapse, Overture, Basemaps, GeoAgent, Layer Swipe, Elevation Profile, and USGS LiDAR panels dock in the side panel and follow the dark theme; registry installs are verified against the published bundle hash and honor the registry blocklist; plugins can add items to built-in toolbar menus, keep project state through credential stripping, use the host proj4 and WFS APIs, and get a native fetch with a cookie jar on desktop; a `?plugin=` link opens registry plugins behind a trust prompt; external plugins group under Plugins → Installed. A new USGS 3DEP elevation (DEM) plugin joins the catalog. See [Plugin API](#)
- [x] **For Python and AI agents:** `geolibre-mcp` can drive the open desktop map, author filters, labels, plugin state, and story maps, and run live processing. See [MCP server](#) and [Python API](#)
- [x] **For deployments and servers:** `deployment.json` loads before first paint and from the desktop config directory, the Docker image generates a validated `deployment.json` at boot and enforces sidecar, AI, and plugin policy, and the server API gains shares with link role, expiry, and password, per-org OIDC federation and trusted-header sign-in, per-org security policy, and SCIM 2.0 provisioning. See [Deployment Policy](#) and [Server API](#)
- [x] **Complete catalogs, and a zoom preference:** every locale catalog is complete and plugins can ship their own translations, and a Map Preferences option turns off zooming to newly added layers. See [Internationalization](#)

Plugin marketplace and registry (design)

This captures the design for the `v1.0` "Plugin marketplace / registry" item. It builds on the existing external-plugin foundation, the `plugin.json` manifest contract, HTTPS manifest-URL loading, the desktop app data `plugins/` scan, and the bundled `public/plugins/` drop-in mechanism, and it relates to the "External plugin package distribution workflow" item.

Goal

Let users discover, install, update, and remove trusted external plugins from a curated registry without hand-entering manifest URLs, on both the desktop and web builds, while keeping the existing trust model in which plugins are trusted code.

Registry

- A curated, versioned index published as static JSON (for example `registry.json` hosted on `geolibre.app`, or generated from a GitHub repository of submissions). No live backend is required for the MVP.
- Each entry carries `id`, `name`, `version`, `description`, `author`, `homepage`, `manifestUrl`, `categories`, `minGeoLibreVersion`, and optional `screenshots`.
- The index is fetched over HTTPS and cached; entries point at the same `plugin.json` manifests the existing loader already understands.

Browse and install UI

- A standalone Manage Plugins dialog (Settings menu > Manage Plugins), modeled on QGIS, has All / Installed / Not installed / Upgradeable / Settings sections. The four browse sections list registry entries with search and per-entry install, installed, and update states; the Settings section manages plugin sources.
- Install reuses the current external-plugin loader: it resolves the entry's `manifestUrl`, validates it, and registers the plugin.
- Desktop: download the bundle into the app data `plugins/<id>/` directory so it persists and loads on startup through the existing scan.
- Web: record the entry's `manifestUrl` in desktop settings (and, for shared projects, in the project `plugins.manifestUrls`) so it loads on next open.
- Remove drops the recorded manifest URL and unregisters the plugin at runtime (tearing down any active control), so the change takes effect without a restart. (The MVP records manifest URLs rather than downloading bundles; the desktop bundle-download path above is a later enhancement.)

Updates and versioning

- Compare the installed `version` against the registry entry, surface an update available state, and offer a one-click update that re-fetches the bundle.
- Honor `minGeoLibreVersion` so incompatible plugins are flagged, not installed.

Trust and security

- The registry is an allowlist; only curated entries are offered for install.
- HTTPS-only manifests (the existing `isAllowedPluginManifestUrl` rule).
- Explicit user consent on install, because plugin entries execute as trusted code (the desktop CSP permits `blob:` script execution by design).

- The curated registry and explicit install consent are the primary controls.

Relationship to bundled plugins

- Bundled `public/plugins/<id>/` drop-ins remain the zero-config way to ship first-party or private plugins inside a build. The marketplace covers discoverable, user-installed third-party plugins; the two are complementary and share the same `plugin.json` contract and loader.

Phasing

1. Curated static registry plus browse and install through manifest URLs (reuses the current loader; records the manifest URL in settings). **Done.**
2. Version checks, update and removal flows.
3. Submission workflow for third-party authors.

Implementation (phase 1)

The MVP ships in the desktop app and, because the same frontend serves the web build, works in both:

- `apps/geolibre-desktop/src/lib/plugin-registry.ts` fetches and normalizes a registry (`{ "plugins": [ ... ] }`), resolving each entry's `manifestUrl` against the registry location. The registry URL is `VITE_GEOLIBRE_PLUGIN_REGISTRY_URL` or, by default, the hosted registry at `https://plugins.geolibre.app/plugin-registry.json`.
- `apps/geolibre-desktop/src/components/layout/ManagePluginsDialog.tsx` is a standalone dialog (Settings menu > Manage Plugins) with All / Installed / Not installed / Upgradeable / Settings sections: search, install, a confirm step before uninstall, an Update action when a newer version is published, `minGeoLibreVersion` compatibility checks, and inline error handling. The Settings section manages additional local directories and manual manifest URLs. All actions apply immediately (live).
- Installing records the entry's manifest URL in the plugin manifest URL list, so the existing external-plugin loader fetches and registers it. No new trust path is introduced. Uninstalling (after confirmation) unregisters the plugin at runtime — tearing down any active map control — so the Plugins menu updates without a reload. Update re-fetches the manifest URL and re-registers the published version in place, fetching the new version before tearing down the old one so a failed update leaves the installed plugin intact.
- The registry and plugin bundles live in the [opengeos/geolibre-plugins](#) repo, published to GitHub Pages at `plugins.geolibre.app`; it ships a `sample/` template and maintainers add curated entries there.

<https://geolibre.app/contributing/>

Contributing

Thanks for your interest in improving GeoLibre. This guide covers how to set up a development environment, the project layout, the local quality gate, and the pull request workflow. Contributions of all sizes are welcome, from fixing a typo to adding a new processing tool or plugin.

By participating, you agree to keep interactions respectful and constructive.

Ways to Contribute

- **Report a bug or request a feature** by opening an [issue](#). Include steps to reproduce, what you expected, and what happened, plus your OS and whether you hit it in the web or desktop build.
- **Improve the documentation** under `docs/` (this site).
- **Fix a bug or build a feature** in the app or one of the packages.
- **Write a plugin** using the external plugin API (see [Plugins](#) below).

If you plan a large change, open an issue first so we can agree on the approach before you invest time in a pull request.

Prerequisites

- **Node.js** 22 or newer (`.nvmrc` pins the major version, so `nvm use` picks it up)
- **Rust** toolchain ([rustup](#)) for Tauri desktop builds (`rust-toolchain.toml` selects the stable channel CI uses)
- Linux only: `webkit2gtk` and `libayatana-appindicator` (see the [Tauri prerequisites](#))
- **Python** 3.10 or newer for [pre-commit](#) (the commit hooks below) and, if you work on the backend, the conversion sidecar and its tests

Setup

Fork the repository, then clone your fork. Replace `YOUR_GITHUB_USERNAME` with your GitHub username:

```
git clone https://github.com/YOUR_GITHUB_USERNAME/GeoLibre.git
cd GeoLibre
npm install
```

GeoLibre is an npm workspaces monorepo, so a single `npm install` at the root wires up every package. Use npm; the repository tracks `package-lock.json`.

Run it locally

Web build (map in the browser):

```
npm run dev
```

Open <http://localhost:5173>.

Desktop build (Tauri, required for filesystem dialogs, local MBTiles, and local raster reads):

```
npm run tauri:dev
```

Repository layout

```
apps/geolibre-desktop    # Tauri + React app (shell, composition, Tauri I/O)
packages/core            # Domain types, Zustand store, project format
packages/map             # MapLibre integration and layer sync
packages/ui              # Tailwind + shadcn/ui primitives
packages/plugins         # Plugin API and built-in plugins
packages/processing      # Client-side algorithm registry
packages/embed           # Typed iframe embed client (@geolibre/embed)
packages/collab-core     # Shared collaboration code for the collab workers
workers/                 # viewer, collab, collab-node, tiles and ai-proxy workers
backend/geolibre_server  # Optional FastAPI conversion sidecar (Python)
backend/geolibre_server_api # Reference projects and identity API (Python)
python/                  # The geolibre Python package (Jupyter widget, MCP server)
e2e/                     # Playwright end-to-end tests
tests/                   # Frontend unit and component tests
docs/                    # This documentation site (Zensical, mkdocs.yml)
```

See the [Architecture](#) reference for how these fit together, and the [Project Format](#) reference for the saved project schema.

Development workflow

1. Create a feature branch off `main`. Never commit directly to `main`.

```
git switch -c feat/short-description
```

1. Make your change, keeping it focused. Match the style of the surrounding code rather than introducing new patterns.
2. Run the [quality checks](#) and confirm they pass.
3. Commit with a clear message. The history follows a [Conventional Commits](#) style prefix, for example `feat:`, `fix:`, `docs:`, `refactor:`, or `chore:`.
4. Push your branch and open a pull request against `main`. The pull request template asks what changed and why, how you tested it, and which issue it relates to.

Pull requests are reviewed before merging. Automated reviewers may leave inline comments; address them or explain why a suggestion does not apply.

Quality checks

Run the fast TypeScript unit tests while you work:

```
npm run test:frontend
```

For a type check without the Vite build, run `npm run typecheck:fast` (`tsc -b` over the app, no output written besides its incremental cache; it first builds the small `@geolibre/embed` package, whose `dist/` types the app imports). `npm run typecheck` is an alias for the full production build.

Before opening a pull request, run the pre-commit hooks on the files you changed, then the local quality gate:

```
pre-commit run --files path/to/changed.ts path/to/other.tsx # list each file you changed
npm run ci:web      # frontend-only changes: no Rust or Python needed
npm run ci          # the full gate that CI runs
```

Scope pre-commit to your files with `--files` rather than `--all-files`: `--all-files` reformats and re-checks every file in the repository, which is slow and can churn files you did not touch. The local `npm-build` hook still runs the full production build once per invocation; if you already built, add `SKIP=npm-build` in front of the command.

`npm run ci:web` is the quick gate for changes that only touch the web app and its packages: lint, the shared dependency, i18n catalog, gallery and plugin docs checks, `typecheck:fast`, the [test type check](#), and the frontend unit tests. It needs only Node.

`npm run ci` runs the complete gate that mirrors continuous integration, in this order:

Step	Command	Covers
Lint	<code>npm run lint</code>	ESLint over <code>apps/</code> , <code>packages/</code> , <code>workers/</code> and <code>tests/</code>
Shared deps check	<code>npm run check:shared-deps</code>	Dependencies declared by both the app and <code>packages/plugins</code> carry the same range (see Shared dependency ranges)
i18n catalog check	<code>npm run i18n:tools:check</code>	The processing-tool strings in <code>en.json</code> match the tool registries (regenerate with <code>npm run i18n:tools</code>)
Gallery check	<code>npm run gallery:check</code>	<code>docs/gallery.md</code> and the <code>docs/demos.md</code> teaser match <code>scripts/demo-gallery.json</code> (regenerate with <code>npm run gallery</code>)
Plugin docs check	<code>npm run plugins:docs:check</code>	The built-in plugin table in <code>docs/user-guide/plugins.md</code> matches the plugin registry (regenerate with <code>npm run plugins:docs</code>)
Build	<code>npm run build</code>	TypeScript compile (<code>tsc -b</code>) and Vite build

Step	Command	Covers
Frontend tests	<code>npm run test:frontend:coverage</code>	Unit tests under <code>tests/</code> , gated on a coverage floor
Untested modules	<code>npm run check:untested-modules</code>	Source files over 500 lines that no test loads, held by a baseline ratchet
Worker checks	<code>npm run test:worker</code>	Type checks all five workers (<code>viewer</code> , <code>collab</code> , <code>collab-node</code> , <code>tiles</code> , <code>ai-proxy</code>) and runs the <code>collab-node</code> tests
Test type check	<code>npm run typecheck:tests</code>	<code>tsc</code> over <code>tests/</code> , gated on a type-error ratchet
Backend tests	<code>npm run test:backend:coverage</code>	<code>pytest</code> for the Python sidecar, gated on a coverage floor
Docker tests	<code>npm run test:docker</code>	<code>pytest</code> for the container's deployment and sidecar policy scripts under <code>docker/tests</code>
Rust check	<code>npm run check:rust</code>	<code>cargo check</code> for the Tauri shell

The middle of the table is grouped into two scripts, which you can also run on their own: `npm run ci:frontend` (i18n check through worker checks; Node only) and `npm run ci:backend` (backend and Docker tests; needs Python and the npm CLI to run the script, but no `npm install`). CI runs `lint`, `ci:frontend`, `ci:backend` and `check:rust` as separate, parallel jobs in `.github/workflows/ci.yml` rather than one after another, with `check:shared-deps` and `typecheck:tests` running next to `lint` in the "Lint and type check" job, so if you add a step to `npm run ci`, add it to one of those groups and CI picks it up.

On a pull request, `lint` and "Build and test" always run. The other jobs (E2E core, backend, Rust, containers, and the database and OIDC integration jobs) run only when the PR changes a path that feeds them; the `changes` job at the top of `ci.yml` holds those path filters. A single **CI passed** job waits for all of them and fails if any job failed, so it is the one check to watch, and the one branch protection requires. A new CI job must be added to its `needs` list, or its failures will not block a merge.

You only need the toolchains for the areas you touched. A docs-only or frontend-only change does not require Rust or Python (use `npm run ci:web`), though the full `npm run ci` gate does. The backend step needs the sidecar's `test` extra (`pip install -e "backend/geolibre_server[test]"`); without it the vector, raster, SQL and ML tests skip themselves.

Component tests

React panels are tested under the same `node --test` runner, against a [happy-dom](#) DOM with [Testing Library](#), and with no map behind them. The `tests/component-*.test.ts` files are the examples to copy. A component test imports `tests/helpers/dom.ts` first and then loads the component with a dynamic `import()`:


```
import { fireEvent, render, screen, useAppStore } from "../helpers/dom";
import { createElement } from "react";

const { LayerPanel } = await import("../apps/geolibre-desktop/src/components/panels/
LayerPanel");
```

The component has to be imported dynamically. The helper registers loader hooks that let Node load a component the way Vite does: CSS and `?url / ?raw` imports become stubs, `virtual:` modules and `import.meta.env` get stand-ins, and `.tsx` compiles with the automatic JSX runtime. Hooks only apply to modules loaded after they are registered, and a static import is loaded before any module code runs. The helper also installs the DOM and initializes the app's own i18n with `en.json`, so queries use the real English strings. Its `render` wraps the element in the app's i18n, tooltip and direction providers.

Set up state through the store (`useAppStore.setState(...)`) and assert on what the user sees or what lands in the store. After each test the helper unmounts everything and resets the app and desktop-settings stores, `localStorage`, `fetch` and any `stubLayout()`. Some other rules:

- Pass `{ current: null }` for `mapControllerRef`. A panel that needs a live map for a behaviour is a sign that logic belongs in a `lib/` module, where a plain unit test can cover it.
- Wrap store writes made after `render` in `act(...)` so React re-renders before you assert.
- `fetch` rejects by default. Serve a response with `mockFetch(...)`.
- Call `stubLayout()` for virtualized lists such as the attribute table. happy-dom does no layout, so without it every element is 0px tall and the list renders no rows.
- Assert on plain values (`textContent`, `.checked`, `.value`, store state), not on DOM nodes. When an `assert.equal` on a node fails, Node formats the whole happy-dom tree, which looks like a hang. Count query results with `queryAll...().length` instead.

These tests count towards the frontend [coverage floor](#) like any other test, so the first test for a large panel adds that panel to the report.

End-to-end smoke tests

`npm run test:e2e` runs the Playwright suite in `e2e/` against the built web app (it builds, serves it with `vite preview`, and drives a headless Chromium). Install the browser once with `npm install chromium`.

The suite is split into two Playwright projects, which together partition `e2e/` — a plain `npm run test:e2e` still runs every spec exactly once:

Project	Command	What it covers	When it runs
core	<code>npm run test:e2e:core</code>	The app boots and renders a map, plus the shared UI surfaces: layer panel, attribute table, dialogs, drag-and-drop, theme, RTL, accessibility, PWA shell, project save and reopen, the plugin deep-link docs.	Every push, and every PR that changes a path the web build reads, as the E2E core (Playwright) job in <code>ci.yml</code> .

Project	Command	What it covers	When it runs
core - engines	(run by <code>npm run test:e2e:core</code>)	One smoke pass per alternate engine (Cesium, ArcGIS): switch to it, add GeoJSON, identify a feature. Runs after <code>core</code> , on one worker, because a software-rendered 3D view saturates the CPU. The ArcGIS pass loads the SDK from <code>js.arcgis.com</code> .	Every push, and every PR that changes a path the web build reads, as the <code>E2E core</code> (Playwright) job in <code>ci.yml</code> .
features	<code>npm run test:e2e:features</code>	Per-feature integration: Mapbox/Cesium engines, STAC, exports, story maps, the scene graph, plugin install.	Nightly and on demand via <code>e2e-full.yml</code> , sharded 4x — or on a PR labelled <code>full-e2e</code> .

The split is a wall-clock decision, not a judgement about value: the full suite is ~36 min serial on a CI runner and the feature specs are ~27 min of it, which made this job the sole critical path of CI while every other job finished in under 10 minutes. Most feature specs guard a specific shipped regression, so they are still run — just not against every commit.

Add a new spec to `core` only if it would break for every user. The list lives in `CORE_SPECS` in `playwright.config.ts`; anything not named there is automatically part of `features`. If you are touching an area the nightly suite covers, label the PR `full-e2e` to get that check before merging.

Both jobs upload their Playwright report as an artifact on failure.

Each PR's Cloudflare preview is also smoke-tested once it deploys: the `Preview smoke` (Cloudflare) commit status fails if the deployed `/demo/` app never finishes loading its map, logs a console error, or gets a 4xx/5xx from its own host. That catches host and subpath problems that `vite preview at /` cannot. Run it against any deployment with `PREVIEW_URL=<app URL> npm run test:e2e:preview`. The core suite runs the same clean-console check against the local build.

Locally the suite serves the built app on port 4173 and reuses a server already listening there. Set `E2E_PORT` to use another port, for example when a second checkout is already serving its own build on 4173.

Coding conventions

- Automated formatting runs on every commit via pre-commit hooks. `pre-commit.ci` runs the same hooks on every PR (including forks) and auto-commits a `style: auto-format` fix back to the PR branch, so anything the local hooks missed gets fixed automatically. It does not run on push to `main`, so auto-formatted commits never bypass review. Tooling is split by language:
- **Python + Jupyter notebooks** (`.py`, `.ipynb`) — `ruff` formats and lints (`ruff.toml`, line length 100, rules `F / I / W / E`).
- **JS/TS/JSON/CSS/YAML/TOML** — `oxfmt` (npm, NAPI bindings) formats (`.oxfmtc.json`, 2-space indent, double quotes, semicolons, width 100). Note that `oxfmt` also sorts `package.json` keys into its conventional order (e.g. `scripts` before `dependencies`), so expect key reordering, not just whitespace changes, when editing those. `ignorePatterns` in `.oxfmtc.json` keeps `oxfmt` off paths that either belong to another tool or churn on every build: Jupyter notebooks and Python source (owned by `ruff`), lockfiles (`package-lock.json`, `pnpm-lock.yaml`, `yarn.lock`),

generated/vendored bundles (`dist/`, `build/`, `target/`, `node_modules/`, minified files, `.d.ts`), Vite-plugin output (`*.generated.js`, `*.generated.py`, `cesium/`, `jupyterlite/`), and the machine-generated app assets under `apps/geolibre-desktop/public/` (Cesium assets, MapLibre sprite atlases, the Whitebox tool-catalog snapshot, which grows ~64% under oxfmt). The intentionally malformed `e2e/fixtures/malformed.geojson` fixture is also excluded.

- **ESLint** fails on React Hooks rule violations and warns on floating promises, `any`, missing Hook dependencies, and physical Tailwind direction classes (`ml-*`, `left-*`, `text-right`; see [Internationalization](#)). Warnings are held by a [ratchet](#), so a change that adds one fails lint. A `npm run build` typecheck runs too.
- **Notebook outputs are stripped repo-wide on every commit.** The `strip-notebook-outputs` hook runs `nbstripout` over *every tracked* `.ipynb`, not just the notebooks in your commit, so an output that was executed locally cannot ride along in a later, unrelated commit. Because `nbstripout` rewrites in place, this also erases outputs from a notebook you are still iterating on the next time you commit anything — re-run the notebook to get them back, or keep an untracked scratch copy. No `.ipynb` should ever carry outputs in git.
- All text files are normalized to LF via `.gitattributes` (`* text=auto eol=lf`), so line endings are consistent across platforms; `core.autocrlf` is overridden.
- The remaining pre-commit hooks enforce LF line endings (`mixed-line-ending --fix=lf`) and basic whitespace (end-of-file and trailing-whitespace fixers). You rarely need to format by hand — commit once, let the hooks rewrite, then `git add` the fixed files and commit again. Install the hooks after cloning with `pre-commit install`.
- Do not edit files in `node_modules`. If a third-party MapLibre control needs app-specific styling, add a scoped override in `apps/geolibre-desktop/src/index.css` limited to that control's class.
- Keep changes scoped to the package they belong to, and prefer reusing the shared primitives in `packages/ui` and helpers in `packages/core`.

Documentation

The site is built with [Zensical](#), the successor to MkDocs Material from the same team. It reads the existing `mkdocs.yml` configuration directly. To preview your changes locally:

```
python -m pip install -r requirements-docs.txt
zensical serve
```

Open <http://localhost:8000>. When you add a new page, add it to the `nav` in `mkdocs.yml`. The site is built with `zensical build --strict` in CI, so broken links and pages left out of the navigation fail the build. Link to other docs pages with a relative path (for example `architecture.md`), and link to files outside `docs/` with a full GitHub URL.

Images, demos, and sample data

Do not commit screenshots, GIFs, videos, or sample datasets to this repository. Binaries are never really deleted from Git history, so every one of them permanently enlarges the clone for everyone, including CI.

Put them in [opengeos/geolibre-assets](#) instead. It is a plain static host, so any file type works. Anything pushed to that repository's `main` branch is published by GitHub Pages at the matching path under its one hostname, <https://assets.geolibre.app>, usually within a minute:

Repository path	Published URL
<code>images/my-panel.webp</code>	<code>https://assets.geolibre.app/images/my-panel.webp</code>
<code>demos/my-demo.gif</code>	<code>https://assets.geolibre.app/demos/my-demo.gif</code>
<code>data/sample.parquet</code>	<code>https://assets.geolibre.app/data/sample.parquet</code>

Then reference the published URL from your Markdown:

```
! [Raster style panel](https://assets.geolibre.app/images/raster-style-panel.webp)
```

This is the one exception to the "link to files outside `docs/` with a full GitHub URL" rule above: an asset in `geolibre-assets` is referenced by its published `assets.geolibre.app` URL, never by a GitHub blob or raw URL.

A few things worth knowing:

- Use `images/` for stills, `demos/` for animations and screen recordings, `data/` for sample datasets, and `styles/` or `fonts/` for map resources.
- Prefer **WebP** or AVIF for stills, and PMTiles or GeoParquet for data. The screenshots on this site are WebP.
- Keep individual files under 100 MB, GitHub's hard limit.
- Published paths are effectively permanent. Renaming or deleting a file breaks every page already pointing at it, so pick the name once.
- Assets are served with permissive CORS, so the app can fetch them cross-origin.

Sample datasets are still served from another GeoLibre-operated host, `data.geolibre.app`. Those URLs keep working and do not need rewriting; `geolibre-assets` is simply the one you can open a pull request against, so send new assets there.

The only images that belong in this repository are the site chrome under `docs/assets/` — currently just the icon `mkdocs.yml` uses for the logo and favicon. Do not add to that directory; if you find something there that nothing references, it is a leftover, not a precedent.

Plugins and extensions

GeoLibre supports external plugins loaded from a zip, a local directory, or a hosted manifest URL. To build one, start from the [GeoLibre plugin template](#) and follow the [Plugin API](#) contract. You do not need to fork GeoLibre itself to ship a plugin.

Backend sidecar

The optional [FastAPI sidecar](#) powers the server-side conversion tools. The `conversion` extra installs the conversion runtime (DuckDB, rio-cogeo, and friends) and the `dev` extra installs the test runner, so install both when working on it:

```
cd backend/geolibre_server
python -m venv .venv && source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -e ".[conversion,dev]"
geolibre-server
```

For a base, conversion-free run (`pip install -e .` plus a plain `uvicorn` launch), see the [sidecar README](#).

Run its tests with `npm run test:backend` from the repository root, or `python -m pytest` from the backend directory.

Sponsoring

Code is not the only way to contribute. If you or your organization depend on GeoLibre, financial support keeps development, hosting, and app-store distribution going — see [Become a Sponsor](#) for [GitHub Sponsors](#) and [Buy Me a Coffee](#).

License

GeoLibre is released under the [MIT License](#). By contributing, you agree that your contributions are licensed under the same terms.

<https://geolibre.app/maintenance/>

Maintenance Notes

This page collects the repository rules that are easy to break **silently** — hand-written mirrors of unexported upstream constants, coverage and audit gates, publishing layouts, and generated files.

`CLAUDE.md` links here rather than restating any of it.

If you are just getting set up, read [Contributing](#) first.

Dependency bumps that need a manual check

Several features depend on values or DOM contracts that upstream packages do not export, so GeoLibre mirrors them by hand. Drift usually produces **no build error** — the feature just stops working. After bumping any of the packages below (**including Dependabot PRs**), do the listed check and run the frontend suite.

`tests/upstream-contracts.test.ts` checks the **installed** package for the exact shape each mirror below relies on — a class name in the published files, a private field on a real instance, a constant, a snippet of a method body — and its failure message names the section here to read. Mirrors with a dedicated real-package test of their own (named in their section) are not repeated there. Packages that touch the DOM or WebGL at import are checked against their published files' text rather than imported. A test there passing is not the whole check: semantics the shape cannot show (ordering, pixels, network behaviour) still need the manual steps listed. When you add a mirror, add its contract there too.

Patched packages (`patches/`)

`postinstall` applies the `patch-package` patches in `patches/`, and each patch file names the exact version it was made against. A bump that moves a patched package past that version fails `npm install` in CI. Regenerate the patch against the new version (or drop it if upstream fixed the bug), rename the patch file, and update `package-lock.json` in the same PR.

`@carbonplan/zarr-layer` is declared with an exact version (no `^`) in both `apps/geolibre-desktop` and `packages/plugins`. It carried a patch until 0.10.0, which shipped the same non-throwing uniform lookups upstream (the layer vanished at `zoom >= 12` on Mesa GPUs without them). Before bumping it, confirm `createShaderProgram` in its `dist/index.js` still looks up `shift_x`, `shift_y` and `u_worldXOffset` with `gl.getUniformLocation`, not `mustGetUniformLocation` (`tests/upstream-contracts.test.ts` checks this).

`geolibre-wasm` (`packages/processing/package.json`)

- **Processing menu catalog.** `ProcessingMenu.tsx` renders from a checked-in, auto-generated catalog, `apps/geolibre-desktop/src/lib/whitebox-menu-catalog.ts` (do not hand-edit). Run `node scripts/gen-whitebox-menu-catalog.mjs` and commit the result, or new/rename WASM

tools silently miss the menu. The Processing *dialog* lists tools dynamically, so the gap only shows in the menu. Whitebox translations are optional external packs in `opengeos/geolibre-language-packs`, not entries generated into GeoLibre's bundled locale JSON.

- `MAX_VECTOR_PMTILES_ZOOM` (`packages/processing/src/wasm-convert.ts`) mirrors the deepest zoom `vector_to_pmtiles` accepts (18 — past it the tool exits with `validation error: max_zoom must be <= 18`). The cap lives inside the WASM binary and is not exported. If it drifts, the browser's Vector to PMTiles either refuses a zoom the tiler would now accept, or accepts one it will reject after the user has waited. This is **not** the sidecar's cap: freestiler allows 24 (`MAX_PMTILES_ZOOM` in `ConversionDialog.tsx`, mirroring `backend/geolibre_server/geolibre_server/app/conversion.py`), and the dialog validates against whichever engine is about to run. `tests/wasm-convert.test.ts` fails if the mirror drifts.
- `DISTANCE_SEGMENTS` / `NON_DISTANCE_NAMES` (`apps/geolibre-desktop/src/lib/whitebox-distance-params.ts`) decide, by parameter *name*, which Whitebox parameters are ground distances and so get the Processing dialog's metric unit picker (GeoLibre#1540). The segments are generic (`tolerance`, `radius`, `length`, `resolution`), so a tool can carry a matching name that is not a length — `corridor_tolerance` is a 0–1 fraction. Those are safe today only because the picker is confined to tools whose every dataset input is a vector layer, and the colliding names happen to sit on imagery/LiDAR tools; that is a coincidence, not a guarantee. Scan the new catalog for a `double` matching the rule whose description reads as a fraction, ratio, angle or weight, and add it to `NON_DISTANCE_NAMES`. If one is missed, that tool's field offers metres and silently converts a dimensionless number as if it were a distance.

maplibre-gl

- `GLOBE_CONTROL_TOGGLE_SELECTOR` (`packages/map/src/globe-control-toggle.ts`) mirrors the class names MapLibre's own `GlobeControl` puts on its toggle button — `maplibregl-ctrl-globe` and `maplibregl-ctrl-globe-enabled`, swapped on every projection change. `MapCanvas` persists a projection change from a **click** on that button rather than from the `projectiontransition` event, because style initialization and project reconciliation emit that event too and a stale one overwrites the projection of a project that has just loaded. `tests/globe-control-toggle.test.ts` builds a real `GlobeControl` and fails if the mirror stops matching.
- **Per-layer blend modes** (`packages/map/src/layer-blend-modes.ts`) wrap three *unexported* `maplibre-gl` internals, because MapLibre renders every layer into one canvas and ships no per-layer blend API (upstream draft: `maplibre/maplibre-gl-js#8073`). The wrappers are `Painter.prototype.renderLayer` (brackets one layer's draws), `Painter.prototype.useProgram` (tells the layer-opacity composite draw from the draws feeding it), and `Context.prototype.setColorMode` (the single place every draw resolves GL blend state). Fill and line layers additionally get `fill-layer-opacity` / `line-layer-opacity` pinned just under 1 by `style-mapper`, which elects MapLibre 6's render-to-texture composite so a layer blends **as a whole** rather than once per overlapping polygon. `installLayerBlendModes` feature-detects every seam and disables the feature (hiding the Style-panel control) rather than breaking the map, so drift fails *quietly* — which is why `tests/layer-blend-modes.test.ts` asserts the seams and `e2e/blend-modes.spec.ts` asserts real pixels. Run both on a bump.

See [Adding a blend mode](#) before extending the list.

- `DEFAULT_MARKER_OFFSET_Y` (`apps/geolibre-desktop/src/components/storymap/storymap-engine.ts`) mirrors the `-14 px` vertical offset `maplibregl.Marker` applies to its default pin. `createStoryMapMarker` (also reused by Field Collection's capture marker in `apps/geolibre-desktop/src/lib/field-collection-map.ts`) positions that pin by hand on every engine (Mapbox, Cesium, ArcGIS, and MapLibre alike), so if a bump changes the default pin's anchor or offset, story markers drift off their coordinate with no error. `tests/upstream-contracts.test.ts` reads the default offset out of the published bundle and fails if it moves; still play a story on a non-MapLibre renderer if the pin's anchor changed.

`@deck.gl/mapbox` **and** `@deck.gl/maplibre`

`bridgeArcgisDeckControl` (`packages/plugins/src/plugins/arcgis-deck/control-adapter.ts`) reads each overlay's private `_props` field so ArcGIS can transfer its initial layers into a native `ArcgisDeckOverlay` without mounting the MapLibre control. The cast hides upstream changes from TypeScript. After either `deck.gl` package is bumped, run `tests/arcgis-control-adapters.test.ts` and confirm the overlay still exposes `_props` with the initial `DeckProps` object.

`@loaders.gl/tiles` (via `@deck.gl/geo-layers`) — **tile cache cap**

`applyThreeDTilesTilesetMemoryLimit` (`packages/plugins/src/plugins/arcgis-i3s-tiles.ts`) works around `Tileset3D` trimming its tile cache against a `maximumMemoryUsage` field it never copies from the load options, so the cap stays at 32 MB and every camera move evicts the tiles just drawn (issue #2560). The same PR turned `memoryAdjustedScreenSpaceError` off because it ratchets the level of detail down once that cache fills. After bumping `deck.gl` or `loaders.gl`, run `tests/arcgis-i3s-tiles.test.ts`: its real-`Tileset3D` case fails if the field is renamed or no longer starts at 32 MB. If upstream starts honouring the option, the helper can go.

`@maplibre/maplibre-gl-style-spec`

`propertySpecFor` (`packages/core/src/expressions.ts`) fabricates the **unexported** `StylePropertySpecification` shape that `createExpression` uses for expected-result-type enforcement (the Expression Builder's filter → boolean / color checks). The cast hides any contract change from the compiler, so run the frontend suite — the "enforces an expected result type" test in `tests/expressions.test.ts` fails if the shape stops being honored.

`cssColor` (`packages/map/src/feature-style.ts`) turns the `Color` object a compiled colour expression evaluates to into CSS by reading its `toString()` and accepting an `rgba(` or `#` prefix. That format is how the spec's `Color` happens to print, not a documented contract: if a bump changes it, the globe silently paints every classified feature the flat fallback colour rather than failing. The "classifies by a categorical field" test in `tests/cesium-feature-style.test.ts` goes through a real compiled expression, so run the frontend suite after a bump and re-verify a categorized layer on the globe.

`SPEC_DEFAULT_COLOR` (`packages/map/src/mapbox-style-import.ts`) mirrors the spec's default for `fill-color`, `line-color` and `circle-color` — `#000000` for all three — which is the colour a stacked class layer naming none is imported as. It is hard-coded rather than read from the spec because `@geolibre/map` is a published package and does not depend on it (only `packages/core` does). The frontend suite guards both directions: a test in `tests/mapbox-style-import.test.ts` asserts the spec still says `SPEC_DEFAULT_COLOR`, and the behavioural tests beside it assert an imported colourless class renders that colour, so a change to either side fails.

`COLOR_OVERRIDING_PAINT` in the same file mirrors the other half of that lookup: the paint properties that draw the feature themselves, so that the colour default does not apply. The spec encodes it as `line-color's requires: [{ "!": "line-pattern" }]`, which the same test asserts. `fill-pattern` and `line-gradient` are listed for the same reason but the spec does not encode it, so only the `line-pattern` entry is a mirror a test can guard; the other two rest on how MapLibre renders them. A class using any of them is not imported as black — it declines the stack.

Web Services control packages (`packages/plugins/package.json`)

- **Docked panel DOM bridge.** `dockable-map-control.ts` adapts `maplibre-gl-fema-wms`, `maplibre-gl-nasa-earthdata`, `maplibre-gl-enviroatlas`, and `maplibre-gl-national-map` by calling the control lifecycle directly and moving the panel element that `onAdd()` appends to the map container into GeoLibre's native right-panel host. Vantor returns a wrapper instead, so the bridge selects its `.vantor-panel` descendant. The scoped CSS in `index.css` mirrors each package's panel, header, toggle, close, and resize-handle class names; `tests/upstream-contracts.test.ts` lists them per package and fails when a package stops rendering one (or `index.css` stops styling it). After bumping any of these packages, activate every migrated Web Services plugin and verify that its catalog renders, resizing the GeoLibre dock preserves the content, and no vendor panel remains under the map container.

`maplibre-gl-components` (`packages/plugins/package.json`)

- `MAP_PANEL_SELECTOR` (`apps/geolibre-desktop/src/components/layout/RecordVideoDialog.tsx`) mirrors the **rendered** control class names — `maplibre-gl-html-control`, `maplibre-gl-legend`, `maplibre-gl-colorbar` — so Record Video's "Include map panels" option can rasterize those on-map overlays into the recording. These are the display elements, deliberately **not** the `*-gui-control` authoring editors. If a class drifts, the option silently stops burning that panel into the video (or the checkbox never appears) with no build error; `tests/upstream-contracts.test.ts` fails instead.
- **The PMTiles control's layer ids** (`pmtilesControlLayerId` / `pmtilesIdsForSourceLayers` / `pmtilesIdNamesSourceLayer`, `packages/map/src/pmtiles-layer.ts`, read from `layer-sync.ts` and `packages/plugins/src/plugins/components/pmtiles.ts`) mirror an unexported fact about `PMTilesLayerControl`: it names its MapLibre layers `${sourceId}-${name}-${kind}` from the **raw** source-layer name, where `pmtilesVectorLayerId` percent-encodes it. The two agree for every name needing no encoding, so a store layer carrying the control's own ids — the archive kept whole, or a split part, which keeps the ids naming its own source layer — is recognised under the encoded scheme alone until a name holds a `/`, a space or non-ASCII. Then `layer-`

`sync` decides the source layer has no native layer and adds a **second** fill/line/circle trio on top of the control's: drawn twice, and only the control's copy answers the panel. Both schemes are therefore matched, and only ids naming a source layer the store actually holds are kept.

What the user **ticked** is deliberately *not* inferred from those ids: `selectedSourceLayers` is a documented field of the exported `PMTilesLayerControlState` handed to every handler, so `pmtilesLayerOptions` reads it and the compiler checks it — the rules for a stale selection, and for the archive ids the control reuses across a panel close, are written at that function and at `addPMTilesArchive`. A reused id is the one case GeoLibre cannot repair: two archives then name one MapLibre source, the first to sync wins it and the other draws nothing, so `addPMTilesArchive` warns rather than pretending otherwise — while an archive that takes a layer over outright is drawn correctly, keeping the name, folder and styling of the one it replaced. `tests/pmtiles-control-contract.test.ts` drives a **real** control against a real archive, through the real `layeradd` handler into the store, and fails if the id scheme moves or the selection stops reaching the handler.

- **The MeasureControl's** `_panel` and `_sourceId` (`packages/plugins/src/plugins/terrain-measure.ts`, via `measurePanelElement` and `measureSourceId`) are private members that only the control knows: the panel carries GeoLibre's Terrain (3D)/heading sections and the resize styling, and the source id is how `lidar-measure-mirror.ts` finds the geometry it redraws above a LiDAR point cloud (#2533). Both readers warn and fall back to doing nothing on a rename; `tests/upstream-contracts.test.ts` builds a real `MeasureControl` and fails if either field is gone. After a bump, still confirm the Terrain section appears and a measured line shows inside a point cloud.
- **The measure paint** (`MEASURE_LINE_COLOR` / `MEASURE_LINE_WIDTH` / `MEASURE_FILL_COLOR`, `packages/plugins/src/plugins/lidar-measure-mirror.ts`) is passed to the control explicitly rather than left to its defaults, because the deck.gl mirror has to repaint the same geometry in the same colour. Change one form and change the RGBA twin beside it; `tests/lidar-measure-mirror.test.ts` asserts the pair agrees.

`maplibre-gl-basemap-control` (`packages/plugins/package.json`)

`BASEMAP_PANEL_SELECTOR` / `BASEMAP_ROW_SELECTOR` / `BASEMAP_ROW_ID_ATTR` (`packages/plugins/src/plugins/basemap-thumbnails.ts`) mirror the DOM the control renders — `.basemap-control-panel`, `.basemap-control-result`, `data-basemap-id` — which the Basemaps panel's thumbnails hook into to find rows and join each one back to its catalog entry. That package exports only `BasemapControl` / `BasemapDefinition`, so a renamed class fails nothing at build time: the queries stop matching and thumbnails silently stop appearing. `tests/basemap-thumbnails.test.ts` builds a real control and asserts its rendered panel against the mirror.

The same file's `hasUnresolvedPlaceholder` deliberately matches the **complement** of the tile tokens it substitutes rather than mirroring that package's credential placeholders (`{api-key}`, `{aws-region}`), so a new provider's placeholder is skipped instead of being fetched literally. Keep it that way rather than enumerating placeholder names.

maplibre-gl-vector (packages/plugins/package.json)

`MAX_VECTOR_BYTES` (`packages/plugins/src/plugins/remote-file-formats.ts`) mirrors `MAX_REMOTE_FILE_BYTES` , an **internal, unexported** constant in that package (2 GiB — DuckDB-WASM holds remote file sizes in 32 bits). It cannot be imported, so re-check `src/lib/utils/remote.ts` in that package and update the mirror if it moved. If it drifts, the remote-browse panels (Source Cooperative, Hugging Face) silently block GeoParquet the engine could now open, or offer an Add that is certain to fail. Updating the constant is enough: the limit the user is shown is rendered from it, not written into the copy. `tests/upstream-contracts.test.ts` reads the constant out of the published chunk and compares.

`remote-file-formats.ts` is the **single** home for this and the other format/reader/size rules those panels share — a per-panel copy would miss this check, so add new browse panels against that module rather than duplicating it (`source-coop-api.ts` re-exports it under its own names for compatibility).

Adoption (`adoptVectorControlLayers` in `vector-layer-sync.ts` , #2715) hands the control's small GeoJSON-mode layers to GeoLibre, and leans on three things the package does not promise. `getLayerGeoJSON` must return the whole collection for a `renderMode: "geojson"` , `ingestMode: "table"` layer. `removeLayer` must drop the layer's DuckDB table as well as its map source, or adoption keeps two copies of the data. And `KML_ICON_PROPERTY` mirrors the feature property the control's KML/KMZ icon layer filters on (`__geolibre_kml_icon_url`); a layer carrying it stays with the control, so a rename upstream would adopt KML layers and drop their icons. Re-check all three on a bump (the contract test covers only `KML_ICON_PROPERTY`).

maplibre-gl-lidar (packages/plugins/package.json) — half checked by the compiler

The space-effects engine raises the MapLibre canvas to `z-index: 4` so its starfield canvases can sit underneath. That package renders point clouds into an **overlaid** deck.gl canvas which it parks in the canvas container, directly after the map canvas, tagged `DECK_CANVAS_CLASS` (`maplibre-gl-lidar-canvas`). `effectsOverlayCss()` (`packages/plugins/src/plugins/maplibre-effects.ts`) hands that wrapper the canvas's own z-index. Drop the rule and the wrapper falls below the raised canvas, hiding the point cloud outright; drop the re-parenting upstream and the wrapper goes back to covering the Measure/Colorbar/Legend/HTML/Bookmark panels (#2530).

`lidar-measure-mirror.ts` draws the Measure tool's line/polygon into that same overlay (`LidarControl.getDeckOverlay()`) with `depthTest: false` , the trick the plugin's own cross-section line uses to sit above the points. It is added with `addLayer(id, layer, { overlay: true })` (0.21+), which draws it after every point cloud chunk, including chunks that stream in later; the point cloud annotator's highlight, box and vector layers use the same flag. One thing is not compiler checked: the geometry is read from the MapLibre/Mapbox `geojson` source's `_data` field, since neither library exposes a public reader. Losing it costs only the mirror — the measured line goes back to being hidden inside the cloud, which is what #2533 was. `tests/upstream-contracts.test.ts` checks both shapes against the real libraries (a real MapLibre `GeoJSONSource` , and the mapbox-gl bundle's `setData`).

The **class** is a hand-kept copy of the package's `DECK_CANVAS_CLASS`, not an import: `maplibre-gl-lidar` builds into its own lazy chunk, and importing even this one string would pull that chunk onto the startup path. `tests/effects-settings.test.ts` builds its expected selector from the package export, so a rename upstream fails `npm run test:frontend`. The **placement** is not visible to the compiler, so `e2e/lidar-canvas-stacking.spec.ts` mounts the real control and asserts the resulting DOM order and z-indices — run it on a bump (`npx playwright test e2e/lidar-canvas-stacking.spec.ts --project=features`).

The point cloud annotator's **Full detail in view** calls `loadRegion` / `clearPinnedRegion` (0.20+) and decides whether a cloud qualifies without an upstream flag: a streamed COPC is one whose source contains `.copc.` and whose `nodeRanges` carry octree keys rather than "file" (`canLoadFullDetail` in `point-cloud-annotation/index.ts`). Its saved labels also assume a pinned region's nodes keep their octree keys, so a bump that changes node keys or pinning semantics must rerun `e2e/point-cloud-annotation.spec.ts` ("loads the view at full detail").

The annotator's label encoding (delta-varint point index, then a class byte or a varint instance id, raw DEFLATE, base64; `label-store.ts`) is decoded in three places the compiler cannot link: `geolibre.project` (Python reader), `geolibre.pointcloud` (pre-labels and labelled-file rewrites) and the embedded job script in `backend/geolibre_server/geolibre_server/app/pointcloud.py` (/ `pointcloud/apply-labels`, which runs on the conversion runtime and cannot import the Python package). Their tests share app-encoded fixtures (`Y2BkwcP0ahHLfyDgBwA=` for instance ids), so a change to the format must update all three and the fixtures together. The COPC node order they rebuild from the hierarchy (data nodes sorted by chunk offset) must match the order `maplibre-gl-lidar`'s `nodeRanges` index points within a node.

The `?data=` LiDAR deep link leans on two more things the compiler cannot see. `isStreamedLidarUrl` (`apps/geolibre-desktop/src/lib/data-url.ts`) copies the routing at the top of `LidarControl.loadPointCloud` (an `/ept.json` suffix or a `.copc.` anywhere in the URL streams; anything else downloads whole) so that only whole downloads get the size check. If upstream changes that routing, update the copy, or a streamed file gets a needless size check and a downloaded one skips it. `addLidarLayerFromUrl` also relies on `load` firing, and adding the store layer, before `loadPointCloud` resolves; it throws if not. `tests/lidar-url-layer.test.ts` pins the GeoLibre side of both, and `tests/upstream-contracts.test.ts` fails if `loadPointCloud` stops routing on those strings. Re-read `loadPointCloud` on a bump.

The point cloud annotator (`packages/plugins/src/plugins/point-cloud-annotation/`) edits the control's loaded points through the point-editing API the package added in 0.18 (`getPointCloudData`, `refreshPointColors`, `getRenderSettings`, `pauseStreaming` / `resumeStreaming`, `isStreamingLoading`, `DeckOverlay.getViewport`), all called from `lidar-access.ts`. Two things are not compiler checked. Edits write into the arrays `getPointCloudData` returns, which relies on them being the buffers the layers render from (for a streamed cloud, views of the loader's buffers). Saved labels are keyed by `nodeRanges` (`(node key, index - start)`), so a change in how the loaders order a node's points would silently re-apply labels to the wrong points. `ASPRS_CLASSES` in `classes.ts` hand-mirrors the package's unexported `CLASSIFICATION_COLORS`; `tests/point-cloud-annotation.test.ts` reads the real colours back through `ColorSchemeProcessor` and fails on drift. Run `npx playwright test e2e/point-cloud-annotation.spec.ts --project=features` on a bump.

maplibre-gl-layer-control (packages/map/package.json) — private internals

packages/map/src/layer-control-host.ts drives the on-map layer control through its public options and adapter methods (including the layer-group methods added in 0.18.0), but it also reaches into private members the compiler cannot check; LayerControlInternalState in that file lists them:

- panel and the .layer-control-item / .layer-control-checkbox / .layer-control-opacity / .layer-control-name DOM, to mirror store visibility, opacity, and names into an already-built panel in place.
- state.layerStates (same reason) and state.collapsed, read before a rebuild so a control that was open is remounted open.
- basemapLayerIds, seeded when there is no fetchable basemap style URL.
- buildLayerItems(), called to add the Background row on styles with no root basemap layers, and to redraw the rows after a reorder the store refused.

If upstream renames any of these, the panel stops following the store (or a rebuild closes the panel) without an error. tests/upstream-contracts.test.ts builds a real LayerControl and checks every member above and the row class names; it cannot check how a rebuild uses them, so still drive the control in the app on a bump: toggle a grouped layer, reorder inside a group, and hide a group. The fix belongs upstream as public API; this package is ours (opengeos/maplibre-gl-layer-control).

maplibre-gl-splat (packages/plugins/package.json) — private internals

packages/plugins/src/plugins/components/splatting.ts reaches into GaussianSplatControl's private fields, which the compiler cannot check:

- reserveSplattingIds replaces the _layerCounter / _modelCounter instance fields with accessors. Upstream names each asset splat-\${this._layerCounter++} / model-\${this._modelCounter++}, and a new control restarts at 0, so without the accessors a fresh load could take a saved layer's id and overwrite it. Restoring a saved layer under its own id also goes through them.
- recordSplattingPlacements wraps loadSplat / loadModel on the instance and reads _splatLayers / _modelLayers (per-asset longitude/latitude/altitude), _state.rotation / _state.scale and _options.defaultModelRotation, so the store layer carries the placement a restore needs. The restore turns _options.flyTo off while it runs.

If upstream renames those fields or changes how it assigns ids, id reservation and placement restore stop working without an error. tests/splatting-restore.test.ts drives a fake with the same shape, so it will not catch that; tests/upstream-contracts.test.ts builds a real control and checks each field and the splat-\${counter} / model-\${counter} naming and placement recording in loadSplat / loadModel. Better still, upstream an id option and a per-asset placement getter and delete the patching.

@geoman-io/maplibre-geoman-free (packages/plugins/package.json) — change-mode internals

Geoman's change mode removes a vertex on right-click, but only for LineString, Polygon and MultiPolygon. `installMultiLineVertexRemoval` in `packages/plugins/src/plugins/maplibre-geo-editor.ts` adds `MultiLineString` (discussion #2750). It hooks `geoman.actionInstances` and wraps the `edit__change` action's `cutVertex`, reading the `featureData / markerData` payload and calling `fireFeatureUpdatedEvent`. None of that is checked by the compiler. `patch-package` is no help here: the app loads the nested copies under `packages/plugins` and `apps/geolibre-desktop`, not the root one.

If a bump renames the action key or those members, the wrapper silently stops applying and `MultiLineString` vertices go back to logging `EditChange.cutVertex: feature not updated`. `tests/upstream-contracts.test.ts` checks the hook points in both nested copies' `dist/maplibre-geoman.es.js` (text only: Geoman cannot be built without a live map). On a bump, still right-click a `MultiLineString` vertex in Edit mode. If upstream adds `MultiLineString` support, delete the wrapper.

zarr-cesium (packages/map/package.json) — private internals

`packages/map/src/cesium-zarr-imagery.ts` draws Zarr layers on the globe and relies on two things `zarr-cesium` does not export as API:

- `ZarrLayerProvider` only honours named matplotlib colormaps (its documented `colorScale` option is not forwarded in 0.3.3), so GeoLibre writes the layer's ramp to the provider's private `source.colorScale.colors` and calls `source.updateColormapTexture()`. If those move, the globe draws the default ramp and logs one warning.
- `ZARR_DIMENSION_ALIASES` mirrors `zarr-maps-tiling`'s `DIMENSION_ALIASES_DEFAULT` (the names it files under `time / elevation`). Drift sends a selector to the wrong key, so the Time Slider steps nothing.

`tests/cesium-zarr-imagery.test.ts` constructs the real provider and imports the real alias table, so both fail CI on a bump; run it, and check whether upstream now forwards `colorScale` so the private write can go. `zarr-cesium` also imports the `cesium` wrapper, which `apps/geolibre-desktop/vite.config.ts` aliases to `@cesium/engine`: keep that alias if a bump adds more `cesium` imports, or the globe loads a second engine.

maplibre-gl-planetary-computer (packages/plugins/package.json) — private STAC client

The Planetary Computer STAC API sends no CORS headers on `GET /collections` and rejects the preflight a JSON `POST /search` needs, so the library's own `STACClient` fails with "Failed to fetch" in the browser and the Tauri webview. `packages/plugins/src/plugins/planetary-computer-stac.ts` subclasses it: search goes out as a `GET /search`, and the collection list falls back to the bundled `planetary-computer-collections.json` when the live one cannot be read. The subclass reuses the private `fetch / abortController` fields, and `maplibre-planetary-computer.ts` swaps it into the control's private `_stacClient` field before the control is added (collections load in `onAdd`).

If upstream renames those fields, loads collections in its constructor, or adds new POST requests, the panel breaks again without a compiler error. `tests/upstream-contracts.test.ts` checks the real client and control for the private fields, that the constructor loads nothing, and that every non-search read is a GET. Run it and `tests/planetary-computer-stac.test.ts` on a bump. Regenerate the bundled list with `node scripts/gen-planetary-computer-collections.mjs` to pick up new collections. Better still, upstream a GET search and a client/fetch option and delete the swap.

maplibre-gl-raster — stretch and gamma curves

`buildContinuousColormapRgba` (`packages/plugins/src/plugins/raster-symbolology.ts`) mirrors the render pipeline's value adjustments **by hand**, inverted. Value-range opacity paints its alpha into the injected 256-wide colormap texture, so it has to map a texture column back to a data value — and the renderer samples that texture *after* rescale, the stretch curve and gamma. The mirror therefore applies each curve's **inverse**, in the reverse of the renderer's order — gamma first, then the stretch:

- gamma — renderer `pow(x, 1 / max(gamma, 0.0001))`, mirror `pow(t, max(gamma, 0.0001))`
- sqrt stretch — renderer `sqrt(x)`, mirror `t * t`
- log stretch — renderer `log(1 + 99x) / log(1 + 99)`, mirror `(100^t - 1) / 99`

None of it is exported: the curves live in the package's `pushAdjustments` / shader modules, and the strength constant (99) is a hard-coded prop. If upstream reorders the pipeline, changes a curve, or retunes the log strength, nothing fails to build — opacity thresholds just drift off the values the user typed, and only under a non-default stretch or gamma. On a bump, re-read that package's `pushAdjustments` for the forward curves and their order (its own `inverseStretch` helper, used for the histogram ticks, is a second copy of the two stretch inverses above) and run `tests/raster-symbolology.test.ts`. `tests/upstream-contracts.test.ts` reads `pushAdjustments` and the three shader curves out of the published chunk and fails if the order, a curve or the strength changes.

maplibre-gl-raster — checked by the compiler

`GeoLibreCogRenderEngine` (`packages/plugins/src/types.ts`) mirrors the `RenderEngine` union that package exports (`maplibre-gl-raster` | `cog-tiler-wasm` | `titiler`). It is hand-written rather than imported because `types.ts` is the public plugin-API surface and importing there would make that package's types a hard dependency of every external plugin. Unlike the mirrors above this one is checked by the **compiler**: `CogRenderEngineMirrorIsExact` in `packages/plugins/src/plugins/maplibre-raster.ts` asserts both directions of assignability against the real imported type, so a renamed or dropped engine identifier fails `npm run typecheck`. Nothing extra to do on a bump beyond letting the build run.

maplibre-gl-raster — picker data copied to keep it off startup

`apps/geolibre-desktop/src/lib/raster-picker-mirror.ts` is a hand-kept copy of the package's `COLORMAP_OPTIONS`, `NORMALIZED_DIFFERENCE_INDICES`, `CUSTOM_NORMALIZED_DIFFERENCE`, `indexById` and `guessBandForRole`. The Style panel's colormap and spectral index pickers need them as soon as they render, and the package ships as one shared chunk, so importing even one constant put the

whole ~0.35 MB library on the startup path. Everything else GeoLibre takes from the package is async and dynamic-imports it instead; keep new value imports that way. `tests/raster-picker-mirror.test.ts` compares every value and both helpers with the package export, so a colormap added or renamed upstream fails `npm run test:frontend`. Regenerate the copy from the package when it does.

`tauri-plugin-persisted-scope` — private on-disk format

`PersistedScopeState` (`apps/geolibre-desktop/src/tauri/src/lib.rs`) mirrors the plugin's private bincode `Scope` structure so GeoLibre can remove legacy per-photo grants before the plugin synchronously replays them at startup. On a `tauri-plugin-persisted-scope` bump, compare the upstream struct's field order and types against this mirror and run the Rust scope-cleanup tests. Bincode encodes fields positionally, so an upstream layout change is not compiler checked. This mirror is Rust-side, so `tests/upstream-contracts.test.ts` does not cover it.

The `bincode` dependency itself is pinned to the **1.x** line and Dependabot is configured (`.github/dependabot.yml`) to skip its major bumps: the plugin writes the file with bincode 1, so GeoLibre must decode and re-encode it with the same wire format. Only move when

`tauri-plugin-persisted-scope` moves. (bincode 3.0.0 is additionally a deliberately unbuildable release — its whole source is `compile_error!("https://xkcd.com/2347/")`.)

`@tauri-apps/plugin-http` — two upstream behaviors, not APIs

`createNativeSidecarFetch` (`apps/geolibre-desktop/src/lib/sidecar-fetch.ts`) routes Windows sidecar traffic through the plugin's native `fetch` and hardens it with two options whose effect comes from `request`'s implementation rather than from any documented contract. The option *names* are compiler-checked — `NativeFetchInit` is derived from `typeof import("@tauri-apps/plugin-http").fetch`, so a renamed or dropped option fails `npm run typecheck` — but the semantics are not, and both fail silently:

- `maxRedirects: 0` maps to `request::redirect::Policy::none()` (`tauri-plugin-http/src/commands.rs`). Without it the native client follows redirects, and `request` only strips `Authorization` / `Cookie` across hosts, so the per-launch `X-GeoLibre-Token` would be replayed to whatever a 3xx pointed at.
- `proxy: { all: { url, noProxy: "*" } }` is how the sidecar reaches the loopback directly. The plugin has no "disable proxy" switch, but any `ClientBuilder::proxy` call sets `auto_sys_proxy = false` (`request/src/async_impl/client.rs`), and `NoProxy::from_string("*")` matches every host (`hyper-util/src/client/proxy/matcher.rs`), so the supplied proxy never intercepts either. This matters because `request`'s `system-proxy` feature *is* in the resolved graph (confirm with `cargo tree -e features -i request`; the plugin's default `macos-system-configuration` feature pulls it in), and `hyper-util`'s Windows reader copies the registry `ProxyOverride` list verbatim — it never expands the `<local>` token Windows writes for "bypass proxy server for local addresses". Drop this and a corporate-proxied Windows machine sends the sidecar request body and token to the proxy.

`tests/sidecar-fetch.test.ts` pins the options the adapter passes, which catches a careless edit here but cannot exercise the Rust side. On a plugin bump, re-check both behaviors against the sources above; the failure modes are a leaked token and a sidecar that is unreachable only for proxied users, neither of which shows up in CI or on an unproxied dev machine.

`cesium` / `@cesium/engine` / `@cesium/widgets` — runtime assets fetched by URL

The 3D globe pane loads its code from `@cesium/engine` but its runtime assets from the `cesium` wrapper. Both are dependencies and both must move together: `cesium@1.x` pins the matching `@cesium/engine`, and the prebuilt Workers/Assets staged from the wrapper have to match the engine running them. `@cesium/widgets` is a third dependency on the same clock — it depends on `@cesium/engine` itself — and it supplies the globe's home, scene-mode and fullscreen buttons (`packages/map/src/cesium-widget-controls.ts`).

The code imports the engine and the widgets directly rather than through the `cesium` barrel, which re-exports both and defeats tree-shaking — the base-layer picker, geocoder, info box and Knockout all shipped in the lazy chunk even when the pane built a bare `CesiumWidget`. Reverting to `import("cesium")` adds ~340 KB back with no build error and no test failure.

After a bump, check all five — none of these fail the build:

- **Asset copy.** `vite-plugins/copy-cesium-assets.ts` copies `Assets`, `ThirdParty`, `Widgets` and `Workers` out of `cesium/Build/Cesium` into `public/cesium/`, keyed on the installed version. The copy is gitignored, so a stale one is refreshed automatically. A directory *renamed or removed* upstream fails loudly — `cpSync` throws `ENOENT` from `buildStart`, so the dev server and the build both stop. The silent case is the opposite one: a runtime directory *added* upstream is simply not in `RUNTIME_DIRS`, so it is never copied and only surfaces as a 404 when the globe reaches for it.
- `CESIUM_BASE_URL`. `CesiumCanvas.tsx` derives it from the app's `BASE_URL`, not a hardcoded `/cesium`, so a sub-path deploy (the `/demo/` build) still resolves. Cesium reads it at import time; if the Workers 404 the render loop dies with no error boundary.
- **The stylesheets.** `CesiumCanvas.tsx` links four paths by hand (`CESIUM_CSS_PATHS`) rather than the 32 KB `Widgets/widgets.css`: `Widgets/CesiumWidget/CesiumWidget.css` for the canvas sizing, credit container and error panel; `Widgets/shared.css` for the `.cesium-button` chrome both toolbar widgets are built from; and `Widgets/SceneModePicker/SceneModePicker.css` plus `Widgets/FullscreenButton/FullscreenButton.css` for those two widgets. Check every one after a bump — a `<link>` to a path upstream moved 404s silently, and the failure is cosmetic in a way no assertion catches: the globe still mounts, but its canvas stops filling the pane, or a toolbar button loses its size and renders 0x0. (The home button has no stylesheet of its own; `shared.css` is all it needs.)
- **The widget classes and view models.** `cesium-widget-controls.ts` builds `HomeButton`, `SceneModePicker` and `FullscreenButton` directly and writes their tooltips through `viewModel.tooltip` / `tooltip2D` / `tooltip3D` / `tooltipColumbusView`. `index.css` then restyles them through Cesium's own class names (`.cesium-button`, `.cesium-toolbar-button`, `.cesium-sceneModePicker-wrapper`). A renamed observable leaves the English default

in place; a renamed class leaves Cesium's dark-blue chrome on a GeoLibre toolbar. Neither fails the build. Tooltip assertions live in `e2e/cesium-primary-renderer.spec.ts`, alongside control mounting and alignment checks. The fullscreen button is the fragile one: its tooltip is a read-only computed, so the translated string is written onto the element from a `fullscreenchange` listener and survives only because DOM listeners fire in registration order. If a bump makes the widget update its own title differently — batched on a microtask, or bound to another target — the label reverts to English *after the first toggle*, which is why the spec toggles fullscreen and re-reads the title rather than checking it once at mount.

Scene-mode lifetime: the 2D/3D/Columbus picker controls the current `CesiumWidget` only.

Recreating the widget, including switching to MapLibre and back or reopening a project, starts in 3D. The project retains its rendering engine and camera, but does not serialize the Cesium scene mode. Scene-mode persistence is outside the toolbar integration's scope; adding it requires an explicit project/store field and restoration for each Cesium pane.

- **PWA globs.** `**/cesium-*` / `**/Cesium-*` in `vite.config.ts` keep the chunk out of the app-shell precache and `CacheFirst`-cache it instead. A chunk renamed out of that pattern would be precached, adding megabytes to first load.

`e2e/cesium-globe.spec.ts` mounts the real engine keyless and drags the globe, so it catches a broken `CESIUM_BASE_URL` or a dead chunk. `e2e/cesium-primary-renderer.spec.ts` adds the toolbar: it asserts the three buttons mount, carry translated tooltips, and share a right edge, which catches a renamed view-model observable and the 0x0-button case of a missing stylesheet. `tests/upstream-contracts.test.ts` adds the static half: `Build/Cesium`'s directories must equal `RUNTIME_DIRS`, every `CESIUM_CSS_PATHS` file must exist and define each `.cesium-*` class `index.css` restyles, and the view models must still carry the tooltip observables. None of these catches the bundle-size regression — check that in the build output.

ArcGIS Maps SDK for JavaScript — loaded from Esri's CDN

The ArcGIS renderer (`docs/arcgis-renderer.md`) has **no npm dependency**. `packages/map/src/arcgis-sdk.ts` imports the SDK's ES modules from `https://js.arcgis.com/<ARCGIS_SDK_VERSION>/@arcgis/core/...` at runtime and types the surface it uses by hand, so nothing here is checked by the compiler against Esri's declarations. Bumping `ARCGIS_SDK_VERSION` is therefore a manual check, not a Dependabot event:

- **Module paths and default exports.** `SDK_MODULES` in `arcgis-sdk.ts` lists every module the engine loads. `tests/arcgis-renderer.test.ts` only checks the assembly against fakes; probe the real CDN (`curl -sI` each URL returns 200) and mount a pane in a browser — a moved module rejects the whole load and the pane shows the error banner. Also check the deck adapter's lazy imports in `packages/plugins/src/plugins/arcgis-deck/overlay.ts`: `layers/Layer`, `views/2d/layers/BaseLayerViewGL2D`, and `views/3d/webgl/RenderNode`. Mount a `deck.gl` layer in both a 2D `MapView` and a local `SceneView` after a version bump; those imports are outside the engine's module registry.

- **The legacy widgets.** `widgets/Zoom`, `Compass`, `ScaleBar`, `Fullscreen` and `Locate` back the built-in controls. Esri deprecated them in 4.32 in favour of web components and still ships them in 5.x with a console warning each; a release that drops them breaks `setBuiltInControlVisible`. The replacement is the `@arcgis/map-components` CDN build. Attribution already uses the 5.x path: the view draws it while `view.attributionVisible` is on, so the deprecated `Attribution` widget is not loaded.
- **The Compass widget's** `viewModel.reset`. The engine overwrites it so a click levels the pitch as well as the heading, as MapLibre's compass does. That member is not part of the typed surface, and a missing `viewModel` is skipped quietly, so a release that restructures it silently reverts the compass to heading-only. After a bump, tilt a scene and click the compass: both heading and tilt must return to 0.
- **The ESM CDN notice.** The SDK logs "Only use ES modules from ArcGIS CDN for testing" on load; Esri's documented production path is an npm build, which this renderer deliberately avoids (see the size argument in issue #2421). The AMD CDN (`<script src="https://js.arcgis.com/<version>/">`) is the supported alternative if the ESM CDN is ever withdrawn.
- **Hit-test attributes.** `GeoJSONLayer` graphics only carry the fields the renderer reads unless `outFields: ["*"]` is set; identify depends on the compiler's `gl__id` field arriving. `e2e/arcgis-renderer.spec.ts` (opt-in, `ARCgis_API_KEY`) is the check.
- **CSP and caching.** `https://js.arcgis.com/` is allow-listed in `script-src` and `font-src` in `tauri.conf.json` and `docker/nginx.conf`, and cached by the `geolibre-arcgis-sdk` service-worker rule in `vite.config.ts`. The version is in every URL, so a bump mints new cache entries.

Desktop CSP `script-src` allowlist

The desktop CSP (`apps/geolibre-desktop/src-tauri/tauri.conf.json`) does not allow all of `https://cdn.jsdelivr.net/npm/`. It lists one version-pinned path per package the app executes from jsDelivr. That blocks scripts from any other package or version on the CDN; it does not verify what is served at an allowed path, so a compromised release at a pinned path still runs, and CSP stops checking paths once a request is redirected. Each version is owned by something else, so a bump can move the URL the app requests while the CSP keeps the old one. The packaged app then hits a CSP block that `tauri dev` never shows, because `tauri dev` does not apply the CSP. `tests/tauri-csp.test.ts` re-derives every path from its owner and fails when they disagree:

<code>script-src</code> path	Owner (bump this, then update the CSP)	Loaded by
<code>pyodide/v<ver>/full/</code>	<code>PYODIDE_VERSION</code> in <code>pyodide-config.ts</code>	Pyodide's <code>import()</code> of <code>pyodide.asm.js</code> (Python Console)
<code>npm/@electric-sql/pglite@<ver>/, npm/@electric-sql/pglite-postgis@<ver>/</code>	lockfile	<code>pglite-loader.cdn.ts</code> (SQL Workspace → PostGIS)
<code>npm/onnxruntime-web@<ver>/dist/</code>	<code>ORT_VERSION</code> in <code>packages/processing/src/ort.ts</code>	onnxruntime's <code>import()</code> of its wasm glue (object detection, SAM)

script-src path	Owner (bump this, then update the CSP)	Loaded by
<code>npm/@duckdb/duckdb-wasm@<ver>/dist/</code>	lockfile (<code>apps/geolibre-desktop</code>)	<code>maplibre-gl-components</code> ' DuckDB converter: a blob worker that calls <code>importScripts()</code> on the worker URL from <code>getJsDelivrBundles()</code>
<code>npm/@duckdb/duckdb-wasm@1.31.0/</code> , <code>npm/sql.js@1.13.0/dist/</code> , <code>npm/geojson-vt@4.0.2/</code> , <code>npm/vt-pbf@3.1.3/</code>	URLs hard-coded in <code>maplibre-gl-vector</code>	Add Data → Vector Layer (DuckDB, the GeoPackage patch, the MVT fallback)
<code>npm/apache-arrow@...</code> , <code>npm/tslib@...</code> , <code>npm/flatbuffers@...</code> , <code>npm/pbf@...</code> , <code>npm/ieee754@...</code> , <code>npm/@mapbox/point-geometry@...</code> , <code>npm/@mapbox/vector-tile@...</code>	jsDelivr's <code>+/esm</code> builds of the <code>maplibre-gl-vector</code> URLs above	the <code>import</code> statements inside those bundles

Blob workers inherit the page's CSP, so `importScripts()` in one is checked against `script-src`. Workers started from the app's own files (`pyodide-worker.js`, the DuckDB and Whitebox workers) carry no CSP at all: Tauri only attaches the policy header to HTML responses. Wasm, wheels, PGlite data and the CereusDB/gdal3.js binaries are `fetch ed`, so `connect-src` covers them.

When `maplibre-gl-vector` changes a CDN URL, the test's `MAPLIBRE_GL_VECTOR_URLS` assertion fails. Re-derive the transitive `+/esm` paths by listing each new bundle's imports, for example `curl -s https://cdn.jsdelivr.net/npm/vt-pbf@3.1.3+/esm | grep -o '/npm/[^"]*'`, recursing into each result, then update `ESM_TRANSITIVE_PATHS` and the CSP together.

The web build's CSP (the app `location /` in `docker/nginx.conf`) lists exactly the same pinned paths, and the test fails when the two lists disagree, so update both files in the same change. There too the app's own worker files carry no CSP: nginx serves `.js` files from the static-asset location, which sends none. The JupyterLite location allows only `pyodide/` from jsDelivr, unpinned: the site's Pyodide comes from whichever `jupyterlite-pyodide-kernel` release pip resolves when the image is built.

A self-hosted `VITE_PYODIDE_INDEX_URL` mirror needs no CSP change in either build: `pyodide-console.ts` fetches the mirror's entry scripts and runs them from `blob:` URLs, and the vector-tools worker is not under the CSP. A rebuilt app that loads scripts from any other host needs that host's path added to `script-src` in both files.

'unsafe-eval' **stays**. `maplibre-gl-vector` builds its CDN loader with `new Function("url", "return import(url)")` at module scope, and that module is in the startup graph. Without 'unsafe-eval' the packaged app throws an `EvalError` while booting and shows a blank window (verified on the Linux WebKitGTK build). Other features compile strings at runtime as well: the attribute field calculator (`attribute-expression.ts`), the raster calculator (`raster-client.ts`), the AI Assistant's JavaScript tool, the Earth Engine code editor, Emscripten `embind glue` (`maplibre-gl-lidar`, `maplibre-gl-splat`), and `ndarray`, `Ajv` and Mapillary's filter compiler. 'wasm-unsafe-eval' alone is not enough for any of these. Dropping 'unsafe-eval' starts with an upstream `maplibre-gl-vector` change, and then each of these consumers needs a replacement.

`connect-src` **keeps** `https:` **and** `http:`. Users add tile, COG, vector and service URLs from arbitrary hosts (XYZ, WMS/WMTS, ArcGIS, STAC, PMTiles), and some of those hosts are plain-HTTP servers on a LAN, so no fixed host list can cover them. `img-src` keeps `https:` for the same reason.

`httpx` (`backend/geolibre_server_api/pyproject.toml`) — private transport pool

The project's server's protection against identity-provider requests to internal addresses (`geolibre_server_api/egress.py`) needs an `httpcore` network backend, which `httpx.HTTPTransport` does not accept. `GuardedTransport` therefore replaces the private `HTTPTransport._pool` with an `httpcore.ConnectionPool` built on `GuardedBackend`. If `_pool` is renamed, building the transport raises `RuntimeError` at startup. If `httpx` keeps `_pool` but stops sending requests through it, the guard is bypassed with no error. After a bump, check that `HTTPTransport.handle_request` still sends requests through `self._pool`, and run `python -m pytest backend/geolibre_server_api/tests/test_egress.py` (`test_identity_provider_client_refuses_loopback` makes a real connection through the transport).

Shared dependency ranges

`apps/geolibre-desktop` and `packages/plugins` declare many of the same upstream packages. If a bump moves the range in only one of them, npm can no longer hoist one copy and nests a second under the workspace that disagrees: the app bundles two versions, and the contract tests above check one copy while the other runs. Dependabot opens one PR per manifest, so this is easy to merge by accident. `npm run check:shared-deps` (`scripts/check-shared-deps.mjs`, run in the CI "Lint and type check" job) fails when a package declared in both manifests, in any dependency section, carries different ranges. Fix it by giving both the same range and refreshing `package-lock.json` in the same PR.

One shared `three`

The app and its three-based plugins (`maplibre-gl-3d-tiles`, `maplibre-gl-splat`, `maplibre-gl-components`, `deck.gl` mesh layers) share one copy of `three`. Two things keep it that way:

- The upstream `opengeos` packages keep `three` out of their ES `dist` and declare it as a peer (`maplibre-gl-splat` `>= 0.2.9`, `maplibre-gl-3d-tiles` `>= 0.5.11`, `maplibre-gl-components` `>= 0.31.2`). An older release inlines its own `three` again.
- The root `package.json` overrides pins `three` to the app's range, because `@dvt3d/maplibre-three-plugin` declares `three: ^0.178.0` (0.178.x only) and would otherwise pull in a second, older copy. `mapillary-js` also declares an old `three`, but its `dist` bundles its own copy and never imports it, so the override is harmless there. Opening the Mapillary viewer still loads that bundled 0.134 copy, the only remaining "Multiple instances of Three.js" warning.

When bumping `three`, move the override with the app's range, then run `npm ls three`: it should list one version. If npm keeps a stale nested copy, follow the lockfile clean-up in [Dependency updates and the audit allowlist](#). Spark's `dist` embeds `three` inside its sort worker's source string. That copy runs in a Web Worker and never registers as a second main-thread instance.

Adding a blend mode

Do not add a blend mode without checking it in the browser. MapLibre's blend state covers the alpha channel too, and it composites a blended layer as one viewport-filling quad, so any mode that does not reduce to "leave the destination alone" at zero source alpha repaints the whole map. That is what disqualified `darken` (a `MIN` equation erased the entire basemap to transparent black) and `subtract` (a reverse subtract left the canvas at `dstA - srcA`, showing the page through the layer). The shipped list is `BLEND_MODES` in `@geolibre/core`, and both the unit test's blend simulator and the e2e spec pin their exclusion.

Only `fill` and `line` have a `*-layer-opacity` in the style spec, so only they blend as a **whole layer**; `circle` and `fill-extrusion` blend per symbol and visibly double-darken where symbols overlap on screen (measured under Multiply: `rgb(23, 77, 220)` in the overlap vs `rgb(76, 136, 222)` on a single symbol). That is upstream's limitation, documented in [Managing Layers](#); the test "has a layer-level composite for fill and line only" fails if a bump adds one of the missing properties, at which point extend `COMPOSITE_LAYER_TYPES` and `style-mapper` together and drop the caveat.

The Style-panel control (`blendModeControl` in `StylePanel.tsx`, rendered in each of its terminal branches) is gated on `!pluginOwnsPaint && !controlRendersLayer`: blending only reaches layers **GeoLibre itself paints**, so anything a control renders or paints (3D Tiles, Gaussian splats, LiDAR, the COG raster control, and Add Vector Layer, which sets `customLayerType` and `controlOwnsPaint`) is excluded — layer-sync never applies `fillPaint` / `linePaint` to those, so the `*-layer-opacity` that elects the composite never lands and a Blend menu there would silently do nothing. Keep `docs/user-guide/layers.md` and `tests/layer-blend-modes.test.ts` ("the layer kinds the Blend control is offered for") in step with that gate; build the test's mocks the way the real controls build their metadata, or they pass on shapes that never occur.

Coverage floors

The `:coverage` test variants run the same suites and print a coverage summary; CI runs them so every build reports coverage. They are **gated on a floor**: `test:frontend:coverage` fails below 78% lines / 78% branches / 63% functions, and `test:backend:coverage` fails below 55% (`--cov-fail-under`). The floors sit a few points under the current numbers as a **ratchet** — regressions fail CI, and when coverage rises comfortably above a floor, raise the floor to lock in the gain.

The frontend report only counts files a test actually imports, so a module with no test does not appear at all rather than as 0%. That is the part that bites: writing the *first* test for a large untested module reads as a coverage **regression**, because the module and everything it imports enter the denominator at once. GeoLibre#1784 added a test that imported `usePlugins.ts` and so pulled in the

whole built-in plugin registry, 39 files, dropping function coverage 72.90% → 60.36% and reddening `main`. The fix is to test against a leaf module rather than to lower the floor (GeoLibre#1888 extracted `lib/plugin-layer-queries.ts`; `geo-editor-geometry.ts` in `@geolibre/plugins` is the same pattern). Check what a new test *transitively* imports before assuming a coverage drop means the code got worse. The [untested module ratchet](#) below keeps the large modules that no test loads from staying out of sight.

`test:frontend:coverage` runs through `scripts/coverage-check.mjs` rather than calling `node --test` directly. Node still enforces all three floors; the wrapper only re-measures once when **line** coverage alone comes up short with every test passing. Line coverage is nondeterministic on CI (GeoLibre#1889: two runs over byte-identical sources reported 81.82% and 76.47%, 114 of 444 files differing on lines and *none* on branches or functions), and it is not reproducible locally on either Node 22 or 26. Branch and function shortfalls, and any test failure, fail on the spot with no retry, so a real regression still fails fast. `classify()` is exported and covered by `tests/coverage-check.test.ts` — change the retry policy there, not by loosening a floor. If the retry starts firing regularly, fix the measurement instead of widening the mitigation.

The backend coverage run (and `npm run ci`, which calls the `:coverage` variants) needs `pytest-cov` from the backend `dev` extra. Install the `test` extra to run the *full* backend suite — without the optional engines (geopandas/rasterio/sedona/httpx) the vector/raster/SQL/ML tests skip themselves and CI is green but hollow: `pip install -e "backend/geolibre_server[test]"`.

Untested module ratchet

Because the report only counts imported files, a large module that no test loads is invisible to the floors. `npm run check:untested-modules` (`scripts/check-untested-modules.mjs`) makes those visible. It reads the lcov report `test:frontend:coverage` writes to `coverage/frontend.lcov`, which lists every file the suite loaded directly or transitively, and compares it with every source file under `apps/*/src` and `packages/*/src`. Declaration files, `index.*` barrels, locale catalogs, and generated files (an `AUTO-GENERATED`, `@generated`, or `DO NOT EDIT` header, plus the Whitebox menu catalog) do not count. It runs right after the coverage step in `ci:frontend`, so in CI's "Build and test" job and in `npm run ci`; `ci:web` skips it because it does not measure coverage. Run it locally after `npm run test:frontend:coverage`.

Every untested source file over 500 lines must be listed in `scripts/untested-modules-baseline.json` with its line count, and the check fails when:

- **A new file over 500 lines appears that no test loads.** Add a test that imports it, or a leaf module extracted from it (see above for why testing the leaf keeps the coverage floors steady). A renamed baseline file shows up as new: move its entry to the new path.
- **A baseline file grows more than 50 lines past its recorded count** with still no test loading it. Add a test, or put the new code in a tested module.

The baseline only shrinks. When an entry gains a test, drops to 500 lines or fewer, or is deleted, the check prints a hint; `npm run check:untested-modules -- --prune` removes those entries and lowers the counts of entries that shrank, and never adds one. Do that in the same PR so the gain is kept. `--write-baseline` rewrites the file from scratch and exists only to bootstrap it; do not use it to absorb a new untested module.

The same command also prints, without gating, the backend sidecar modules that have no `test_<module>.py` or `test_<module>_*.py` in `backend/geolibre_server/tests`. Some are exercised through other test files (the route tests drive `app/main.py`), so treat the list as a prompt to check, not a count of untested code.

Lint warning ratchet

`npm run lint` passes `--max-warnings` (in the root `package.json`) set to the current warning count, so lint warnings work like the coverage floors: the count can only go down. The warnings come from `react-hooks/exhaustive-deps`, `@typescript-eslint/no-explicit-any`, the type-aware `@typescript-eslint/no-floating-promises` (app, package and worker `src/` only, checked against each file's nearest `tsconfig.json`), and `local/no-physical-tailwind` (`eslint-rules/no-physical-tailwind.mjs`, the right-to-left rule from [Internationalization](#)), and two `eslint-plugin-jsx-a11y` rules (`control-has-associated-label`, `no-static-element-interactions`) on app and package `.tsx`.

`eslint-plugin-jsx-a11y` 6.10 declares ESLint up to 9 as a peer. It runs under ESLint 10, and the `overrides` entry for it in the root `package.json` points its peer at the installed ESLint. When the plugin publishes ESLint 10 support, drop that override. If a future ESLint breaks the plugin, `npm run lint` fails loudly; pin ESLint or disable the two rules rather than reaching for `--legacy-peer-deps`.

- **A PR adds a warning:** fix it. For a floating promise that is a deliberate fire-and-forget, prefix the call with `void` and make sure it handles its own rejection. For a class that must stay physical (a map-anchored overlay, say), add `// eslint-disable-next-line local/no-physical-tailwind -- <why>`. Do not raise the limit.
- **A PR fixes warnings:** lower the limit to the new total that ESLint prints, in the same PR, so the gain is kept.
- **A PR turns on a new rule:** the one time the limit goes up. Raise it by exactly the new rule's count on the code as it is, say so in the PR, and fix those warnings over time like the rest.

Test type-check ratchet

The frontend tests run under `node --import tsx --test`, and `tsx` strips types without checking them, so a test fake that drifts from the interface it stands in for (a renamed property, a new required member, a changed argument) still runs and still passes. `npm run typecheck:tests` closes that gap:

it runs `tsc --noEmit` over three projects in `tests/` and fails when the number of errors in test files rises above `--max-errors` (in the root `package.json`). It runs in CI's "Lint and type check" job and in `npm run ci / ci:web`.

- `tests/tsconfig.json` checks every test against the DOM lib with the Node, React and Vite types and the ambient `.d.ts` files the app and packages compile with. `allowJs` lets tests import the `scripts/*.mjs` they cover with inferred types.
- `tests/tsconfig.workers.json` checks the tests that import `workers/tiles` or `workers/viewer` against `@cloudflare/workers-types`, and `tests/tsconfig.ai-proxy.json` the ones that import `workers/ai-proxy` against that worker's generated `worker-configuration.d.ts`. The Workers runtime types redeclare the DOM globals, so these tests cannot share the main program. A new test that imports worker source goes in the main project's `exclude` **and** one worker project's list; the script fails if an excluded test is in neither worker project.

Only errors in `tests/` count. A test's imports pull product source into the program, and that source is checked by its own workspace's `tsconfig`; an error it reports under the tests' settings (DOM-using `@geolibre/core` code under the Workers types, say) is printed for information and not counted.

The limit works like the [lint warning ratchet](#):

- **A PR adds a type error in a test:** fix it. Build fakes against the real type: spread `DEFAULT_LAYER_STYLE` for a partial `LayerStyle`, use `tests/helpers/layer-fixtures.ts` for a layer, and cast a deliberately partial fake of a large interface (a MapLibre `Map`, a Cesium viewer) once through `unknown` in a small typed helper rather than at every call. Do not raise the limit.
- **A PR fixes type errors:** lower the limit to the new total the script prints, in the same PR.

Run `node scripts/check-test-typecheck.mjs --max-errors 0 --verbose` to list every counted error, or `npx tsc --noEmit -p tests/tsconfig.json` for `tsc`'s own output.

Boot bundle budget

`npm run build` fails when the JS the app entry imports statically (what `index.html` `modulepreloads` before the shell mounts) grows past either budget in `apps/geolibre-desktop/boot-budget.json`, or when it contains Cesium. The check is `bootBundleBudgetPlugin` in `apps/geolibre-desktop/vite.config.ts`.

Budget	Limit	Measured when set
<code>rawBytes</code>	3 MiB, minified	~2.2 MB
<code>gzipBytes</code>	900 KiB, gzip level 9	~680 kB

Gzip is the closer proxy for what a browser downloads; raw is what it parses. Most regressions trip both, because they come from a chunk-grouping change that drags a whole lazy package (Cesium, DuckDB, a `maplibre-gl-*` plugin) onto the boot path. The error lists the largest boot chunks. Fix the grouping in `CODE_SPLITTING_GROUPS / manualChunks` rather than raising a budget; raise one only after confirming the new eager code belongs at boot, and say so in the PR.

`npm run bundle:report` (after a build) writes `bundle-report/bundle-report.md` and `.json`: every JS/WASM asset in `dist/assets` with raw and gzip sizes, the boot set read from the built `index.html` against both budgets, and the size of each top-level `dist` entry. CI's build job uploads it as the `bundle-report` artifact and appends the Markdown to the job summary, so two runs can be compared without rebuilding.

Workers do not share chunks with the main build

Vite bundles each `new Worker(new URL(...))` as a separate build, so anything a worker imports is emitted again even when the main thread has the same module. That once shipped h5wasm (libhdf5, ~4.8 MB) twice, once for the local NetCDF reader and once inside `netcdf-remote.worker.ts`. The main build now emits h5wasm as an explicit chunk and the worker imports it by URL (`apps/geolibre-desktop/vite-plugins/shared-h5wasm.ts`), and the build fails if anything but exactly one `hdf5_h1-*.js` is emitted. A new worker that imports a large library should do the same, or accept the duplicate knowingly; the bundle report makes a second copy easy to spot.

Dependency updates and the audit allowlist

Dependencies are watched two ways: **Dependabot** (`.github/dependabot.yml`) opens grouped weekly update PRs for npm, pip (backend + `python/`), cargo, and Actions, and CI runs `npm run audit:ci` (blocking, in the "Lint and type check" job) plus a non-blocking `pip-audit` of the resolved backend environment (in the "Backend tests" job).

`audit:ci` is `scripts/audit-check.mjs`, a thin wrapper over `npm audit --omit=dev` that still fails on every high/critical advisory *except* the ones listed in its `ALLOWLIST`. The wrapper exists because plain `npm audit` cannot accept a single finding, so one unpatchable transitive advisory reddens every PR until upstream ships a fix — which for an unmaintained leaf package may be never. Only allowlist an advisory when there is **no patched version to upgrade to** and the vulnerable code is **unreachable from a GeoLibre runtime path**, and say why on both counts in the entry. Anything upgradeable gets upgraded instead. Stale entries print a warning rather than failing, since the advisory database is a live service and a transient omission must not redden an unrelated PR.

When the fix is a scoped `overrides` entry (such as `@loaders.gl/compression` → `fflate`), npm 12 accepts it and `npm ls` then checks against the override's range. The lockfile still lists the package's own declared dependency (`fflate: 0.7.4`), though, and npm keeps the old nested copy, so `npm ls` reports it as `invalid`. Delete that package's nested `node_modules/.../<pkg>` entries from `package-lock.json` (and from `node_modules`), then run `npm install` again so it resolves them fresh. Confirm with `npm ls <pkg>` before you commit.

If no other package already installs a copy that satisfies the override, npm does not fetch one. It leaves the package out of the tree and `npm audit` reports 0 vulnerabilities, which looks like a fix but isn't. That is why the `texture-compressor` → `image-size@^2.0.4` override needed a hand-written lock entry: `version`, `resolved`, `integrity`, `license`, `bin` and `engines`, taken from `npm view <pkg>@<version> --json` (npm 12 wraps that output in an array). Check a fresh `npm ci` with `npm ls <pkg>` before you commit. That override patches GHSA-w3rx-r6r6-pgpr and GHSA-5p2g-fcmc-qvqq,

but image-size 2.x takes a buffer rather than a path, so texture-compressor's own CLI can no longer read image sizes. GeoLibre never runs it: only `@loaders.gl/textures'` `encodeImageURLToCompressedTextureURL`, a Node-only encoder, spawns it via `npx`. Drop the override once loaders.gl stops depending on texture-compressor (4.5.x already makes it a peer).

Publishing `@geolibre/core` and `@geolibre/map`

Both are published to npm by `.github/workflows/publish-packages.yml` on each GitHub Release, alongside `@geolibre/embed`. Their checked-in `main / types / exports` point at TypeScript **source**, because that is how the monorepo consumes them: Vite, `tsc` and `tsx` all resolve `./src/index.ts` through the package's own `exports`, so `npm run dev` and `node --import tsx --test tests/<name>.test.ts` need no build step.

The npm tarball ships `dist` instead, and npm cannot express that split on its own: unlike pnpm and Yarn it deliberately **ignores entry fields nested under** `publishConfig` (npm/cli#7586), so a manifest that only states its dist entries there publishes `./src/index.ts` to consumers who never receive `src`. The published entries therefore live under `publishConfig`, and `scripts/prepare-npm-package.mjs` hoists them (and pins the `"*" @geolibre/core` dependency to the release version) just before `npm publish`.

Point those top-level fields at `dist` and every frontend test that imports a `@geolibre/map` subpath fails with `ERR_MODULE_NOT_FOUND`, because `dist` is gitignored and nothing builds it before the suite. `tests/prepare-npm-package.test.ts` guards both halves, including that each published path is one the package's own `tsdown` entries actually emit (`--format esm --dts` writes `<entry>.mjs` and `<entry>.d.mts`, **not** `.d.ts`). The release workflow does build both packages, but a green build proves only that the bundles were written, not that every path the manifest publishes names one of them, so nothing else would notice that drift.

The bundled sidecar lockfile

`backend/geolibre_server/uv.lock` **is committed** (the root `.gitignore` ignores `uv.lock` everywhere else and negates it for this one path). That project is bundled into the desktop installers and launched with `uv run --frozen --project <resource dir> from src-tauri/src/lib.rs` — a directory the user cannot write (`C:\Program Files\...`, `/usr/lib/GeoLibre Desktop/...`). Ship it lockless and `uv` resolves, then tries to *write* `uv.lock` there, fails with "Permission denied" and exits 2 — which reaches the user as "Jupyter server exited before it was ready (exit code: 2)" with the cause invisible.

So: any edit to that `pyproject.toml`'s dependencies must land with a refreshed lock (`uv lock --project backend/geolibre_server`). CI's "Check the bundled sidecar lockfile is in sync" step (`uv lock --check`) fails if they drift.

Credentials must never reach a redistributable build

Anything we hand to someone else — the Jupyter wheel above all — must carry no credential of ours. Three properties of this build make that easy to get wrong, and every guard below blocks one of them.

1. `apps/geolibre-desktop/vite.config.ts` bridges bare shell vars into their `VITE_` names — `GOOGLE_MAPS_API_KEY` → `VITE_GOOGLE_MAPS_API_KEY`, and the same for `MAPBOX_TOKEN` and `CESIUM_TOKEN`. Convenient for local testing; it also means the build machine's shell is build input.
2. Something in the graph reads `import.meta.env` as a **whole object**. Vite cannot tell which keys such a read wants, so it stops replacing per key and inlines the entire `env` record — every `VITE_` var on the build machine — into every chunk that read reaches. Ours was `packages/core/src/runtime-env.ts`; the one we cannot fix is `@clerk/shared's` `getEnvVariable.mjs`, which does `import.meta.env[name]` with a computed name, so the inlined record lands in the `ClerkGate-*.js` chunk.
3. `python/hatch_build.py` skips the JS build when `static/app` already exists and `GEOLIBRE_FORCE_JS_BUILD` is unset. A local `python -m build` therefore packages whatever an earlier `npm run build:embed` left staged, with no JavaScript running at all.

The rules now

- `BUILD_ENV_KEYS` in `vite.config.ts` is the allowlist of `VITE_` names that may reach a bundle. `pruneBuildEnv()` deletes every other `VITE_` var from `process.env` before Vite reads it. Adding a new build-time var means adding it here — otherwise it silently resolves to undefined.
- `CREDENTIAL_ENV_KEYS` is the subset that authenticates as, and bills to, whoever ran the build. In a *redistributable* build these are blanked to `""` (blanked, not deleted, so a `.env` file cannot reintroduce them). A build is redistributable when `GEOLIBRE_EMBED=1` (the Jupyter wheel) or `GEOLIBRE_STRIP_CREDENTIALS=1`. The web deploy is **not** redistributable: it is our own site using our own referrer-restricted keys, and it keeps them.
- Public-by-design identifiers stay in every build: the Clerk *publishable* key, the Auth0 client ID and domain, the GEE OAuth client ID, the GA measurement ID. `publish-python.yml` deliberately injects the GEE client ID into the wheel.
- Prefer `getBuildEnvironment()` from `@geolibre/core` over reading `import.meta.env` yourself. A whole-object read re-opens cause (2) for every chunk it reaches, and nothing in the type system will tell you.

Each stripped credential resolves through `getRuntimeEnvironment()`, which overlays `window.__GEOLIBRE_RUNTIME_ENV__` from Settings → Environment variables. So a wheel user supplies their own token and the affected surfaces degrade as documented: Mapbox prompts in the basemap API-keys view, the 3D globe loses terrain and Ion imagery (the pane itself still works), Protomaps basemaps are hidden.

The scan

`scripts/scan-credentials.mjs` verifies the **output**. A build run by hand can satisfy every rule above and still be wrong, and a third-party asset dropped into the tree can arrive with a key already inside it — neither is visible from the config. It runs in two places:

- `scripts/build-embed.mjs`, before staging `dist-embed` into the Python package.
- `python/hatch_build.py`, before packaging any wheel or sdist — including the stale-assets path (3) above, which no JS-side guard can cover. This one needs no Node.

Both read `scripts/credential-patterns.json`, so the JS and Python scanners cannot drift; the file is force-included into the sdist so an sdist → wheel build is gated too. `tests/credential-scan.test.ts` covers it.

To scan any built directory by hand:

```
node scripts/scan-credentials.mjs apps/geolibre-desktop/dist-embed
```

When the scan fires after a dependency bump

`allowedValueHashes` in `credential-patterns.json` holds the SHA-256 of values that match a pattern but are public by design — currently CesiumJS's built-in default Ion token, which ships inside `cesium` and appears in every build. A Cesium upgrade changes that token, so the guard will fire on the new one.

That is intended. Decode the payload before doing anything: CesiumJS's has `sub: "CesiumJS"` and `iss: "https://api.cesium.com"`. Only once you have confirmed it is the vendor's own token, replace the hash. Never add a hash to silence a finding you have not decoded — the whole point of the list is that it is short and every entry was checked.

Bundled plugin drop-ins under `apps/geolibre-desktop/public/plugins/<id>/` are scanned too. They are third-party build artifacts that are not committed here, so a hardcoded key inside one is fixed in that plugin's own repository, never by allowlisting it. Note that removing such a drop-in does not clean a `dist/` built while it was present: Vite copies `public/` into the output and only clears that output when a build actually runs. Rebuild, or delete the stale directory.

Generated files and cross-file sync

- **The open data gallery.** `gallery.md` and the "Open data showcase" teaser on `demos.md` (between the `<!-- demo-gallery-teaser -->` markers) are generated from `scripts/demo-gallery.json`: the themes, the featured demos, and one entry per demo (slug, title, theme tag, caption, data credit). Edit the JSON, run `npm run gallery`, and commit both files; `npm run gallery:check` (part of `npm run ci`) fails on drift. Each entry's screenshot must already be at `https://assets.geolibre.app/images/<slug>.webp` (opengeos/geolibre-assets), and its `theme` should match the tag on its `share.geolibre.app` project.

- **The built-in plugin reference.** The table on `user-guide/plugins.md` (between the `<!-- plugin-reference:start/end -->` markers) is generated by `scripts/gen-plugin-reference.mjs`. It reads the plugin list and order from `BUILT_IN_PLUGINS` in `usePlugins.ts`, each plugin's `id` and `name` from its definition in `packages/plugins/src`, the display name from `en.json`'s `toolbar.plugin`, the menu location from `PluginsMenu.tsx` and the `WEB_SERVICE_PLUGIN_IDS` / `DGGS_PLUGIN_IDS` groups, and the `?plugin=` link name from the `plugin-deep-link.ts` rules. Only the one-line description (and a `menu` for a plugin `PluginsMenu` hides) is hand-written, in `scripts/plugin-reference.json`. After registering, renaming, or regrouping a built-in plugin, add its entry, run `npm run plugins:docs`, and commit both files. `tests/plugin-reference-docs.test.ts` and `npm run plugins:docs:check` (part of `npm run ci`) fail on drift, and the generator itself fails on a registered plugin with no description.
- **Processing tool metadata.** Names, descriptions, group labels, parameter labels/help and select options live in registries with no i18n access, so the dialogs resolve them through `apps/geolibre-desktop/src/lib/processing-tool-i18n.ts` and fall back to the registry's own English string. For the four small bundled registries, `en.json`'s `processing.toolMeta` / `processing.toolGroup` subtrees are the **generated** baseline translators work from. After adding or renaming one of those tools, parameters, or select options, run `npm run i18n:tools` and commit the result; CI fails on drift. Whitebox's much larger metadata is deliberately absent from all bundled locales and comes from optional, validated packs at `languages.geolibre.app` (or local file import); do not add `processing.toolMeta.whitebox`, `processing.whitebox.categories`, `menuTool`, or `menuSubcategory` back to a bundled locale.
- **The agent skill.** `skills/geolibre/` is a *user-facing* agent skill — a `SKILL.md` plus `references/` that teaches an external AI agent to author `.geolibre.json` projects through `geolibre-mcp`, the Python package, or hand-written JSON. It is not for contributors working on GeoLibre itself. It restates things that live elsewhere: the MCP tool surface (`python/src/geolibre/mcp/server.py`), the basemap/color-ramp/legend catalogs (`python/src/geolibre/basemaps.py`, `color_ramp.py`, `legends.py`), the project schema ([Project Format](#)), and the embed parameters ([Embedding](#)). `python/tests/test_agent_skill.py` guards the parts that can be checked mechanically: every registered MCP tool must appear in the tool reference, every tool and `Map` method the skill names must exist, and the basemap, color-ramp, legend-preset, layer-type, frontmatter, and reference-file lists must match their sources. It cannot check prose, so a changed size cap, limit, or behavioral caveat still has to be carried over by hand — update the skill in the same PR. A stale caveat sends an agent down a path that no longer works, with no failure anywhere.

<https://geolibre.app/citation/>

How to Cite

If you use GeoLibre in your research, teaching, software, or other work, please cite it. Citations help others find the project, recognize the contributors, and support the continued development of free and open-source geospatial software.

DOI [10.5281/zenodo.20785400](https://doi.org/10.5281/zenodo.20785400)

GeoLibre is archived on [Zenodo](#), which mints a Digital Object Identifier (DOI) for every release. The DOI above is the **concept DOI**: it always resolves to the latest version. To cite a specific version instead, open the [Zenodo record](#) and copy the DOI shown for that release.

Recommended citation

Wu, Q. (2026). GeoLibre: A lightweight, cloud-native GIS platform for visualizing, exploring, and analyzing geospatial data. Zenodo. <https://doi.org/10.5281/zenodo.20785400>

BibTeX

```
@software{wu_geolibre,  
  author   = {Wu, Qiusheng},  
  title    =  
  {{GeoLibre: A lightweight, cloud-native GIS platform for visualizing, exploring, and analyzing  
geospatial data}},  
  year     = {2026},  
  publisher = {Zenodo},  
  doi      = {10.5281/zenodo.20785400},  
  url      = {https://doi.org/10.5281/zenodo.20785400}  
}
```

APA

Wu, Q. (2026). GeoLibre: A lightweight, cloud-native GIS platform for visualizing, exploring, and analyzing geospatial data [Computer software]. Zenodo. <https://doi.org/10.5281/zenodo.20785400>

Citation metadata file

The repository ships a `CITATION.cff` file, so you can also use GitHub's **"Cite this repository"** button (in the sidebar of the [repository page](#)) to copy a ready-made APA or BibTeX entry. Reference managers such as Zotero and tools like `cffconvert` can read the same file.

Citing dependencies

GeoLibre builds on many open-source projects. If your work relies heavily on a particular component, please consider citing it as well. See the [Acknowledgements](#) page for the full list of projects that make GeoLibre possible.

<https://geolibre.app/acknowledgements/>

Acknowledgements

GeoLibre stands on the shoulders of the open-source geospatial and web communities. This page recognizes the projects, organizations, and individuals whose work makes GeoLibre possible. If we have inadvertently left someone out, please [open an issue](#) so we can correct it.

Open-source projects

GeoLibre is built almost entirely on free and open-source software. We are grateful to the maintainers and contributors of the following projects (and the many transitive dependencies they rely on).

Mapping and rendering

- [MapLibre GL JS](#) — the core map rendering engine.
- [CesiumJS](#) — the 3D globe rendering engine.
- [deck.gl](#) and the [vis.gl](#) / Open Visualization toolkit — raster, point-cloud, and 3D overlays.
- [Three.js](#) — 3D rendering used by globe and atmosphere effects.
- [PMTiles](#) and [Protomaps](#) — single-file tile archives.
- [OpenFreeMap](#) and [CARTO](#) basemaps, and the broader [OpenStreetMap](#) community whose data underpins them.
- [Overture Maps Foundation](#) — open map data.

Data, formats, and analysis

- [DuckDB](#) and [DuckDB-WASM](#) with the **Spatial** extension — in-browser spatial SQL and vector conversion.
- [Apache Sedona](#) and [PGLite](#) (with PostGIS) — additional in-browser SQL engines.
- [Turf.js](#) — client-side vector geometry tools.
- [GDAL/OGR](#), [GeoTIFF.js](#), [proj4js](#), [shpjs](#), and [FlatGeobuf](#) — geospatial data formats and reprojection.
- [GeoParquet](#) and the [Apache Arrow](#) / Parquet ecosystem.
- [STAC](#) — the SpatioTemporal Asset Catalog specification and tooling.
- [Zarr](#) and [CarbonPlan](#) — cloud-native multidimensional array access.
- [Icechunk](#) and [icechunk-js](#) — transactional storage for Zarr, read in the browser.
- [H3](#) — hexagonal hierarchical spatial indexing.

Python sidecar and notebooks

- [FastAPI](#), [rasterio](#), [GeoPandas](#), [Shapely](#), and [pyproj](#) — the optional Python sidecar.
- [WhiteboxTools](#) — the Whitebox geoprocessing toolbox.
- [segment-geospatial \(SamGeo\)](#) and Meta AI's [Segment Anything](#) — AI segmentation.

- [Jupyter](#), [JupyterLite](#), [Pyodide](#), and [anywidget](#) — the embedded notebook experience.
- [leafmap](#) — the API style and inspiration for the `geolibre` Python package.

App framework and UI

- [Tauri](#) — the cross-platform desktop and mobile shell.
- [React](#), [Vite](#), and [TypeScript](#).
- [Zustand](#) — application state management.
- [Radix UI](#), [shadcn/ui](#), [Tailwind CSS](#), and [Lucide](#) — UI primitives and icons.
- [react-i18next](#) — internationalization.

The **Atmospheric Effects** plugin (deep-space backdrop, parallax starfield, comets, and the globe atmosphere halo) adapts the technique and visual design from [Leonel Dias's](#) article [Globe atmosphere, halo, and comets](#) — the layered Canvas 2D approach, the halo gradient and "screen" blend, the limb-sampling that keeps the halo aligned under pitch, and the starfield/comet parameters.

Community contributors

GeoLibre is shaped by the people who try it, report bugs, request features, and help others get started. Thank you to everyone who has opened an issue, started a discussion, or shared GeoLibre with their community.

- [Ryanphoenix](#) — for many valued contributions to the project, including issue reports, feedback, and improvements.

Beta testers

- [René van der Velde](#), Netherlands — an experienced QGIS practitioner who has championed GeoLibre as an approachable entry-level to mid-tier GIS option and contributed thoughtful, detailed bug reports and feature requests during early testing.

Get involved

Want to see your name here? Contributions of every size are welcome — from filing a well-described bug report to writing a plugin. See the [Contributing guide](#) to get started.

If you would rather support the project financially, see [Become a Sponsor](#).

<https://geolibre.app/privacy/>

Privacy Policy

Last updated: October 6, 2026

GeoLibre Desktop ("GeoLibre", "the app") is an open-source desktop GIS application developed by the OpenGeos community. This policy explains how the app handles your data, and how our websites (geolibre.app and web.geolibre.app) handle yours.

Summary

GeoLibre runs locally on your device. It does not require an account, and the app itself does not collect analytics, telemetry, or usage data. Your geospatial data and projects stay on your device unless you choose to share or export them. If you install plugins from the plugin registry, the registry counts their use anonymously from the requests the app already makes; see [Plugins from the plugin registry](#) below.

Our websites are separate from the app: geolibre.app and web.geolibre.app measure page visits with Google Analytics, as described under [Website analytics](#) below.

Your use of the app is also governed by our [Terms of Service](#).

Data processed locally

The files and projects you open, create, and analyze in GeoLibre (vector and raster datasets, project files, and similar) are processed locally on your computer. GeoLibre does not upload them to the developer or to any server. An optional helper process (a Python component running on your own machine at 127.0.0.1) performs some geoprocessing entirely on-device.

Data sent to third-party services (optional features)

When you use certain optional features, GeoLibre sends requests directly to the relevant third-party service. Those services receive your device's IP address and the request you make, and are governed by their own privacy policies:

- **Basemaps / map tiles:** the current map view is used to request tiles from basemap providers (for example OpenFreeMap and CARTO).
- **Search / geocoding:** the place names or addresses you search are sent to the configured geocoding provider.
- **AI assistant:** if you use it, the prompts you enter, together with metadata about the layers currently loaded in your project (layer names and attribute field names), are sent to the configured AI/LLM provider.

- **Cloud data catalogs:** if you connect to services such as the Microsoft Planetary Computer or Google Earth Engine, your queries are sent to them.
- **Real-time collaboration:** if you join a shared session, your project data (including any GeoJSON from locally-loaded files) is routed through a relay server operated by the OpenGeos project (currently hosted on Cloudflare) and shared with the other participants. The relay holds the latest project snapshot so that later joiners can load the session, and discards it when the session ends.
- **Project sharing:** if you use Project → Share, the project file is uploaded to share.geolibre.app, operated by the OpenGeos project. Vector layers added from local files are embedded in the uploaded file; other local files are not uploaded.

GeoLibre does not control these third-party services; please review their privacy policies for how they handle data.

Plugins from the plugin registry

Settings → Manage Plugins lists plugins from the GeoLibre plugin registry at <https://plugins.geolibre.app>, operated by the OpenGeos project (currently hosted on GitHub Pages and Cloudflare). Opening that dialog downloads the registry, and each time GeoLibre starts it downloads the registry's blocklist (of plugins pulled for safety) and the files of the plugins you installed from it. Like any web server, the registry sees your IP address and which files you request.

The registry uses those requests to show how often each plugin is used, as public counts on its plugin catalog. It records no more than this:

- a count of launches per plugin per day;
- for counting distinct weekly users, a hash of the plugin and your IP address (for IPv6, its /64 network), keyed with a secret that changes every week. The hash can't be turned back into an IP without that secret, can't be matched across weeks or plugins, and is deleted when the week ends; only the weekly total is kept.

No IP address, account, or other information about you or your projects is stored. The app sends nothing extra for this: the counts come only from the plugin downloads it already makes. A deployment that sets its own plugin registry (`plugins.registryUrl` in `deployment.json`) contacts plugins.geolibre.app only for plugin files its registry points to there; those downloads are counted like any other, so a deployment that wants none should host the plugin files itself. Details: <https://plugins.geolibre.app/registry/#usage-statistics>.

Website analytics

This section covers our websites, not the installed app.

The documentation site (<https://geolibre.app>) and the hosted browser app (<https://web.geolibre.app>, and the demo at <https://geolibre.app/demo/>) use Google Analytics 4 to count page visits and see which pages are used. Google receives your IP address, the page you are on, the referring page, and general device/browser information, and sets its own cookies or similar identifiers in your browser. It is governed by [Google's privacy policy](#).

What analytics never sees is the geospatial data you work with: layers, files, project contents, queries, and coordinates are processed in your browser and are not sent to Google. The page address we report is trimmed to the site and path (for example `https://web.geolibre.app/`), with the query string removed, because a GeoLibre link can carry a project URL, an inline dataset, or a collaboration session identifier in its parameters.

You can opt out by blocking `www.googletagmanager.com` with a content blocker or your browser's tracking protection; GeoLibre Web works normally with analytics blocked.

Analytics run only on the sites we host. The desktop app, the Jupyter widget, and the Docker image never load or run them: the measurement ID is supplied at build time and is absent from those builds, so nothing is requested from Google and nothing is reported. A copy you build and host yourself is the same unless you deliberately supply a measurement ID of your own at build time, in which case the analytics are yours, not ours (see [Self-Hosting](#)).

Personal information

GeoLibre does not ask you to create an account and does not collect names, email addresses, or similar identifying information. Network requests for the optional features above necessarily include your device's IP address, which the receiving service may log.

Children

GeoLibre is a professional GIS tool intended for users aged 16 and over. It is not directed at children, and we do not knowingly collect personal data from children.

Your choices

Because GeoLibre stores your data locally, you control it and can delete your projects and files at any time. To avoid sending data to third-party services, do not use the optional online features listed above.

Changes to this policy

We may update this policy from time to time. The current version is always available at <https://geolibre.app/privacy/>.

Contact

Questions can be directed to the project at <https://github.com/opengeos/GeoLibre/issues>.

<https://geolibre.app/terms/>

Terms of Service

Last updated: July 10, 2026

GeoLibre ("the app") is a free, open-source GIS application developed by the OpenGeos community. By downloading, installing, or using GeoLibre — including its desktop, web, and embedded builds — you agree to these Terms of Service ("Terms"). If you do not agree, do not use the app.

The service

GeoLibre lets you view, create, analyze, and share geospatial data. It runs locally on your device and does not require an account. Some optional features connect to third-party services (for example basemap providers, geocoding services, AI/LLM providers, and cloud data catalogs such as Google Earth Engine); those features are described in our [Privacy Policy](#).

License

GeoLibre is open-source software distributed under the MIT License, as published in the project repository at <https://github.com/opengeos/GeoLibre/blob/main/LICENSE>. Your use, modification, and redistribution of the software are governed by that license. These Terms supplement, and do not replace, the software license.

Acceptable use

You agree to use GeoLibre lawfully and responsibly. In particular, you agree not to:

- use the app to violate any applicable law or regulation, or to infringe the rights of others;
- use the app to access, store, or distribute data you are not authorized to use;
- misuse, overload, or attempt to gain unauthorized access to any third-party service that the app connects to, or use such a service in a way that breaches that service's own terms; or
- remove, disable, or circumvent any security or authentication mechanism in the app or the services it connects to.

Google Earth Engine and other third-party services

Optional GeoLibre features let you authenticate with and access third-party services, including **Google Earth Engine**, using your own credentials.

- When you connect to a third-party service, you do so under **your own account** with that provider, and your use is subject to **that provider's terms of service and privacy policy** in addition to these Terms. For Google Earth Engine, this includes the [Earth Engine Terms of Service](#) and the [Google APIs Terms of Service](#).
- GeoLibre uses Google OAuth solely to obtain, on your behalf and with your consent, the access token needed to make Earth Engine requests you initiate. GeoLibre does not sell your data, and does not use Google user data for advertising or for any purpose other than providing the features you request. See the [Privacy Policy](#) for details on what is sent where.
- You are responsible for ensuring you have the necessary rights and quota to use any third-party service through the app, and for complying with that service's usage limits and licensing terms for the data you access.

GeoLibre does not control these third-party services and is not responsible for their availability, content, or conduct.

Your content

GeoLibre does not claim ownership of the data, files, or projects you open or create with it. You retain all rights to your content. Most processing occurs locally on your device. If you enable an online feature, such as real-time collaboration or a third-party integration, applicable data may be transmitted to that service as described in our [Privacy Policy](#). You are responsible for backing up your content and for its lawful use.

Disclaimer of warranties

GeoLibre is provided **"as is" and "as available", without warranties of any kind**, express or implied, including but not limited to warranties of merchantability, fitness for a particular purpose, accuracy, and non-infringement. The developers do not warrant that the app will be uninterrupted, error-free, or free of harmful components, or that any analysis or output it produces is accurate or fit for any particular purpose. You use the app and rely on its output at your own risk.

Limitation of liability

To the maximum extent permitted by law, in no event shall the OpenGeos community, the project maintainers, or contributors be liable for any indirect, incidental, special, consequential, or punitive damages, or for any loss of data, profits, or goodwill, arising out of or in connection with your use of (or inability to use) GeoLibre, even if advised of the possibility of such damages.

Changes to the app and these Terms

GeoLibre is under active development, and features may be added, changed, or removed at any time. We may also update these Terms from time to time. The current version is always available at <https://geolibre.app/terms/>. Your continued use of the app after changes take effect constitutes acceptance of the revised Terms.

Contact

Questions about these Terms can be directed to the project at <https://github.com/opengeos/GeoLibre/issues>.